

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



TS. Nguyễn Thanh Hải

TS. Quách Xuân Trường

ThS. Đoàn Ngọc Phương

BÀI GIẢNG
QUẢN LÝ DỰ ÁN CÔNG NGHỆ THÔNG TIN

Tài liệu lưu hành nội bộ

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

TS. Nguyễn Thanh Hải

TS. Quách Xuân Trường

ThS. Đoàn Ngọc Phương

BÀI GIẢNG

QUẢN LÝ DỰ ÁN CÔNG NGHỆ THÔNG TIN

Thái Nguyên, tháng năm 2024

Mục lục

Mở đầu	7
Chương 1: Giới thiệu về Quản lý Dự án Phần mềm.....	8
1.1 Tổng quan về Quản lý Dự án Phần mềm.....	8
1.1.1 Định nghĩa Quản lý Dự án Phần mềm	8
1.1.2 Sự khác biệt giữa Quản lý Dự án Phần mềm và các loại dự án khác	15
1.1.3 Vai trò của các bên liên quan trong dự án phần mềm	16
1.2 Các Giai đoạn trong Vòng đời Dự án Phần mềm	18
1.2.1. Lập Kế Hoạch	18
1.2.2. Phân Tích Yêu Cầu	20
1.2.3. Thiết Kế	21
1.2.4. Phát Triển.....	22
1.2.5. Kiểm Thử	23
1.2.6. Triển Khai	23
1.2.7. Bảo Trì	26
Câu hỏi ôn tập cuối chương	27
Chương 2: Phương pháp Quản lý dự án	28
2.1 Các phương pháp quản lý dự án phổ biến	28
2.1.1 Mô hình thác nước Waterfall.....	28
2.1.2 Agile.....	30
2.1.3 Lean.....	34
2.1 Ứng dụng các phương pháp quản lý dự án vào thực tiễn	37
2.2.1. Phân tích yêu cầu dự án	37
2.2.2. Lựa chọn phương pháp hợp lý	38
2.2.3. Ứng dụng vào từng dự án phần mềm cụ thể.....	39
Câu hỏi ôn tập cuối chương	43
Chương 3: Lập kế hoạch dự án.....	44
3.1. Phân chia yêu cầu phần mềm thành các nhiệm vụ cụ thể.....	44
3.1.1. Phân tích yêu cầu	44
3.1.2. Xác định nhiệm vụ.....	45

3.2. Các công cụ lập kế hoạch trong xây dựng lịch trình và phân bổ nguồn lực.....	47
3.2.1. PERT.....	48
3.2.2. Gantt chart.....	51
3.3. Quản lý ngân sách dự án.....	52
3.3.1. Dự toán ngân sách.....	52
3.3.2. Điều chỉnh ngân sách khi cần thiết.....	55
Câu hỏi ôn tập cuối chương.....	57
Chương 4: Quản lý rủi ro và chất lượng dự án.....	59
4.1. Quy trình quản lý rủi ro.....	59
4.1.1. Nhận diện rủi ro.....	61
4.1.2. Phân tích rủi ro.....	63
4.1.3. Quản lý rủi ro.....	64
4.1.4. Sử dụng ma trận rủi ro.....	66
4.1.5. Các kỹ thuật phòng ngừa.....	69
4.2. Kiểm soát chất lượng sản phẩm phần mềm.....	71
4.2.1. Kiểm thử thủ công.....	72
4.2.2. Kiểm thử tự động.....	73
Câu hỏi ôn tập cuối chương.....	75
Chương 5: Công cụ và kỹ thuật hỗ trợ.....	76
5.1. Giới thiệu chức năng của các công cụ quản lý dự án hiện đại.....	76
5.1.1. Trello.....	79
5.1.2. Asana.....	81
5.1.3. Jira.....	82
5.1.4. Slack.....	84
5.1.5. GitHub/GitLab.....	85
5.2. Sử dụng các công cụ quản lý dự án.....	87
5.2.1. Theo dõi tiến độ.....	87
5.2.2. Tạo báo cáo tiến độ.....	92
5.2.3. Lập báo cáo chi tiết về dự án.....	95
Câu hỏi ôn tập cuối chương.....	97
Chương 6: Phân tích, báo cáo và kiểm tra dự án.....	98
6.1. KPIs và các chỉ số được sử dụng để đo lường và báo cáo hiệu quả dự án.....	98

6.1.1. Định nghĩa KPIs.....	98
6.1.2. Các chỉ số KPIs phổ biến.....	100
6.2. Xây dựng báo cáo tiến độ và báo cáo kết thúc dự án	111
6.2.1. Nội dung báo cáo tiến độ	111
6.2.2. Nội dung báo cáo kết thúc dự án	117
Câu hỏi ôn tập cuối chương	120
Chương 7: Vai trò và ứng dụng kỹ năng mềm trong quản lý dự án phần mềm	121
7.1. Vai trò của kỹ năng giao tiếp, lãnh đạo, đàm phán và giải quyết xung đột trong quản lý dự án phần mềm.	121
7.1.1. Kỹ năng giao tiếp trong quản lý dự án phần mềm.....	121
7.1.2. Kỹ năng lãnh đạo	124
7.1.3. Kỹ năng đàm phán	128
7.1.4. Kỹ năng giải quyết xung đột.....	131
7.2. Sử dụng kỹ năng mềm để duy trì môi trường làm việc tích cực và hợp tác trong nhóm, thúc đẩy hiệu quả làm việc và xử lý xung đột, đảm bảo tiến độ và chất lượng dự án.	134
7.2.1. Thúc đẩy hiệu quả làm việc	134
7.2.2. Xử lý xung đột	136
7.2.3. Đảm bảo tiến độ và chất lượng dự án	138
Câu hỏi ôn tập cuối chương	143
Chương 8: Quản lý và phân phối kết quả dự án	144
8.1. Quy trình phân phối sản phẩm và kiểm thử trước khi triển khai nhằm đảm bảo phần mềm hoạt động đúng như yêu cầu.	144
8.1.1. Quy trình phân phối sản phẩm.....	144
8.1.2. Kiểm thử trước khi triển khai	146
8.1.3. Đảm bảo phần mềm hoạt động đúng như yêu cầu.....	150
8.2. Quy trình bảo trì.....	153
8.2.1. Lập kế hoạch bảo trì.....	153
8.2.2. Đáp ứng nhu cầu người dùng.....	157
Câu hỏi ôn tập cuối chương	163
Chương 9: Áp dụng kiến thức qua dự án mẫu.....	165
9.1. Lập kế hoạch, triển khai một dự án mẫu.....	165
9.2. Quản lý các giai đoạn của dự án mẫu	169

9.2.1. Tối ưu hóa tiến trình	169
9.2.2. Xử lý các thách thức phát sinh.....	172
9.3. Đánh giá và báo cáo kết quả thông qua dự án mẫu	174
9.3.1. Đánh giá và báo cáo kết quả	175
9.3.2. Cải tiến quy trình cho các dự án thực tế	177
Câu hỏi ôn tập cuối chương	180
Tài liệu tham khảo	181

Mở đầu

Bài giảng Quản lý dự án Công nghệ thông tin được tập thể giảng viên thuộc bộ môn Công nghệ phần mềm biên soạn nhằm phục vụ cho việc giảng dạy của giảng viên và học tập của sinh viên Trường Đại học Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên. Tập bài giảng này được biên soạn theo nội dung đề cương chi tiết học phần giảng Quản lý dự án Công nghệ thông tin ở trình độ đại học.

Với cấu trúc gồm 9 chương, bài giảng sẽ dẫn dắt người học qua các khía cạnh quan trọng như phân tích yêu cầu, lập kế hoạch, triển khai, và quản lý rủi ro, đồng thời cung cấp các ví dụ minh họa và bài tập thực hành để củng cố kiến thức.

Mặc dù tập thể tác giả đã dành nhiều thời gian và công sức để biên soạn, song khó tránh khỏi thiếu sót. Chúng tôi chân thành cảm ơn sự góp ý và đóng góp từ quý thầy cô và các bạn sinh viên, giúp tài liệu ngày càng hoàn thiện và phù hợp hơn với thực tiễn.

Xin trân trọng cảm ơn.

Chương 1: Giới thiệu về Quản lý Dự án Phần mềm

Nội dung chính của chương

Chương này cung cấp cái nhìn tổng quan về quản lý dự án phần mềm, định nghĩa các khái niệm cơ bản và vai trò của các bên liên quan. Sinh viên nắm được các giai đoạn chính trong vòng đời của một dự án phần mềm, tạo nền tảng vững chắc để tìm hiểu sâu hơn về các chủ đề tiếp theo trong môn học.

Mục tiêu cần đạt của chương

- Hiểu khái niệm quản lý dự án phần mềm.
- Phân biệt quản lý dự án phần mềm với các loại dự án khác.
- Hiểu vai trò của các bên liên quan và các giai đoạn trong vòng đời dự án phần mềm.

1.1 Tổng quan về Quản lý Dự án Phần mềm

1.1.1 Định nghĩa Quản lý Dự án Phần mềm

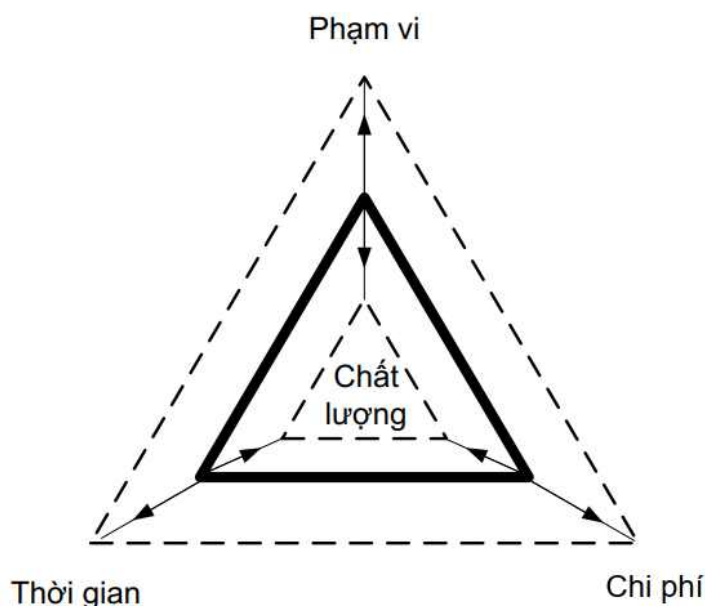
Dự án là gì?

Khái niệm dự án: “*Dự án là một tập hợp các công việc, được thực hiện bởi một nhóm chuyên gia, để đạt được kết quả mong đợi, trong thời gian mong đợi, với ngân sách mong đợi*” (Ngô Trung Việt - Phương pháp quản lý dự án CNTT).

Một dự án là tập hợp các nhiệm vụ được thực hiện bởi một nhóm chuyên gia nhằm tạo ra một sản phẩm hoặc dịch vụ độc nhất. Để hoàn thành các nhiệm vụ này, dự án cần sử dụng các nguồn lực ước tính như thời gian (từ lúc bắt đầu đến khi kết thúc), chi phí (ước tính tài chính), nguồn nhân lực, và các yếu tố khác. Điều đáng chú ý là trong quá trình thực hiện dự án, có thể xuất hiện nhiều biến số không lường trước, ảnh hưởng đến tiến trình và kết quả của dự án.

Công việc dự án khác biệt so với các công việc thường nhật hoặc công việc hành chính trong các cơ quan/tổ chức. Chẳng hạn, việc nghiên cứu và phát triển một loại vắc-xin mới là một dự án. Tuy nhiên, sau khi vắc-xin được cấp phép sử dụng, việc sản xuất các lô vắc-xin trên dây chuyền và phân phối chúng lại là công việc thường xuyên, không phải công việc thuộc phạm vi dự án. Một dự án sẽ kết thúc khi đạt được mục tiêu đã đề ra hoặc bị chấm dứt vì một lý do nhất định. Những lý do phổ biến dẫn đến việc chấm dứt dự

án bao gồm: hết thời gian, cạn kiệt ngân sách, hoặc nhận thấy rằng việc tiếp tục dự án không còn cần thiết nữa.



Hình 1.1. Bộ ba ràng buộc của dự án

Mỗi dự án đều chịu sự ràng buộc bởi nhiều yếu tố khác nhau, nhưng ràng buộc cơ bản và phổ biến nhất là mối quan hệ giữa phạm vi, chi phí và thời gian, được gọi là "*bộ ba ràng buộc*". Ba yếu tố này có mối liên hệ chặt chẽ với nhau, như minh họa trong Hình 1.1. Bất kỳ sự thay đổi nào ở một trong ba yếu tố này đều sẽ tác động đến các yếu tố còn lại hoặc ảnh hưởng đến chất lượng dự án. Chẳng hạn, nếu phạm vi dự án mở rộng, khối lượng công việc tăng lên, kéo theo chi phí và thời gian cũng tăng theo tỷ lệ phần trăm phù hợp (do người quản lý dự án xác định). Ngược lại, nếu phạm vi được thu hẹp, chi phí và thời gian cũng sẽ giảm tương ứng. Tương tự, những điều chỉnh về chi phí hoặc thời gian cũng sẽ ảnh hưởng đến phạm vi và chất lượng của dự án.

Dự án Công nghệ Thông tin là gì?

CNTT = Phần cứng + Phần mềm, sự tích hợp phần cứng, phần mềm và con người.

Dự án CNTT = Dự án liên quan đến phần cứng, phần mềm, và mạng.

Ví dụ: Hệ thống internet banking tại các ngân hàng, cho phép khách hàng thực hiện các giao dịch ngân hàng trực tuyến như chuyển tiền, thanh toán hóa đơn, kiểm tra số dư...

Phân loại dự án CNTT (theo Thư viện Học liệu Mở Việt Nam - Vietnam Open Educational Resources (VOER)).

Dự án CNTT được phân loại từ rất nhỏ, đến cực lớn, như bảng 1.1.

Phân loại	Lập trình viên	Thời gian	Số dòng lệnh (SLOC -Source Line Of Code)
Rất nhỏ	1	1 tháng	500
Nhỏ	1	1-6 tháng	1000-2000
Vừa	2-5	1-2 năm	5000-50000
Lớn	dưới 100	2-3 năm	50000-100000
Rất lớn	dưới 500	4-5 năm	1000000
Cực lớn	trên 500	5-10 năm	trên 1000000

Bảng 1.1. Phân loại dự án CNTT

Quản lý dự án

Quản lý dự án là “*ứng dụng kiến thức, kỹ năng, công cụ và kỹ thuật vào các hoạt động dự án để thỏa mãn các yêu cầu của dự án.*” (PMI, Project Management Body of Knowledge (PMBOK® Guide), 2000, p.6).

Quản lý dự án là một lĩnh vực quan trọng và luôn thu hút sự chú ý lớn từ mọi tổ chức. Không chỉ đóng vai trò cốt lõi đối với những người trực tiếp đảm nhiệm công tác quản lý dự án, lĩnh vực này còn mang lại giá trị thiết thực cho tất cả các thành viên tham gia vào các hoạt động dự án nói riêng cũng như các hoạt động khác của tổ chức nói chung.

Mục tiêu của quản lý dự án

Mục tiêu cơ bản của quản lý dự án nói chung là hoàn thành các công việc dự án theo đúng yêu cầu kỹ thuật và chất lượng, trong phạm vi ngân sách được duyệt và theo đúng tiến độ thời gian cho phép.

Lợi ích của quản lý dự án

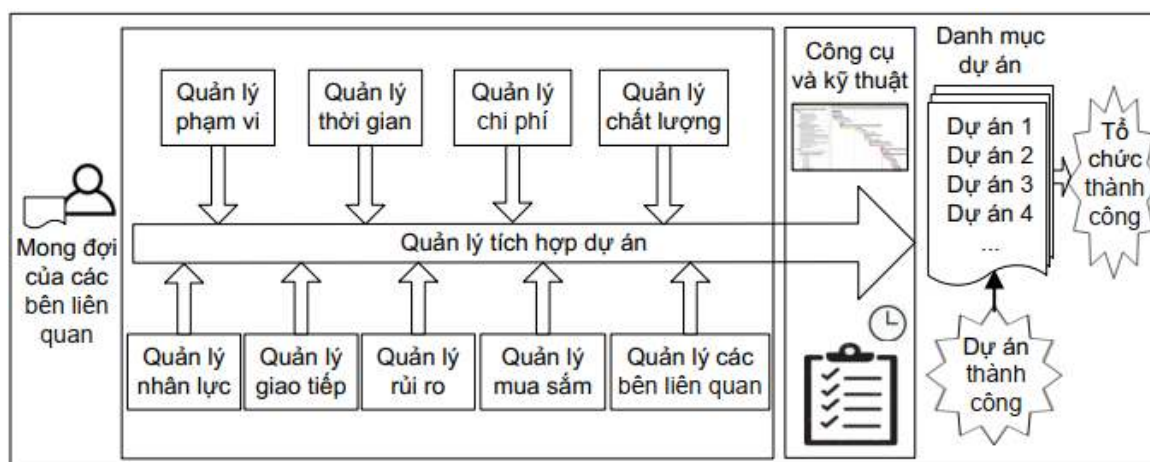
Quản lý dự án đóng vai trò như một công cụ điều phối hiệu quả các nỗ lực cá nhân và tập thể, đòi hỏi sự hợp tác chặt chẽ và tối ưu hóa việc sử dụng các nguồn lực hạn chế. Bản chất của phương pháp này mang lại nhiều lợi ích quan trọng, bao gồm:

- Liên kết các hoạt động: Kết nối tất cả các công việc và hoạt động trong dự án để đảm bảo tính nhất quán và đồng bộ.

- Thúc đẩy giao tiếp: Tạo điều kiện thuận lợi cho sự liên hệ thường xuyên và bền chặt giữa các nhóm quản lý dự án, khách hàng và các nhà cung cấp đầu vào.
- Tăng cường hợp tác và trách nhiệm: Khuyến khích sự hợp tác giữa các thành viên và làm rõ trách nhiệm của từng cá nhân trong dự án.
- Phát hiện và ứng phó kịp thời: Giúp sớm nhận diện các khó khăn, vướng mắc phát sinh và điều chỉnh kịp thời khi có thay đổi hoặc gặp phải điều kiện không lường trước. Đồng thời, hỗ trợ đàm phán giữa các bên liên quan để giải quyết bất đồng một cách hiệu quả.
- Đảm bảo chất lượng sản phẩm và dịch vụ: Đáp ứng kỳ vọng cao của khách hàng thông qua quản lý tốt chất lượng dự án.
- Kiểm soát tài nguyên: Quản lý hiệu quả các nguồn tài chính, thiết bị và nhân sự để đạt được mục tiêu.
- Cải thiện mối quan hệ khách hàng: Xây dựng lòng tin và sự hài lòng của khách hàng thông qua việc đáp ứng tốt yêu cầu và tiến độ.
- Tối ưu hóa thời gian: Giúp rút ngắn thời gian triển khai dự án.
- Giảm chi phí: Tối ưu hóa các nguồn lực để tiết kiệm chi phí.
- Tăng độ tin cậy: Nâng cao uy tín của dự án và tổ chức thông qua việc hoàn thành đúng tiến độ và chất lượng.
- Gia tăng lợi nhuận: Mang lại giá trị tài chính tốt hơn nhờ quản lý hiệu quả.
- Cải thiện năng suất lao động: Tăng hiệu quả làm việc của các thành viên dự án.
- Phối hợp nội bộ tốt hơn: Đảm bảo sự thống nhất và đồng thuận giữa các bộ phận trong tổ chức.
- Nâng cao tinh thần làm việc: Tạo động lực và môi trường làm việc tích cực cho các thành viên dự án.

Hạn chế của quản lý dự án

Mặc dù mang lại nhiều lợi ích, quản lý dự án cũng có những mặt hạn chế nhất định, đòi hỏi phải xem xét kỹ lưỡng để áp dụng hiệu quả bao gồm, mâu thuẫn về chia sẻ nguồn lực trong đơn vị, trách nhiệm của các nhà quản lý trong dự án và vấn đề hậu dự án là những điểm cần khắc phục.



Hình 1.2. Các lĩnh vực trong quản lý dự án

Các lĩnh vực của quản lý dự án

Quản lý dự án bao gồm 10 lĩnh vực chính, như hình 1.2, được mô tả bao gồm:

i. 4 lĩnh vực cốt lõi:

- **Quản lý phạm vi:** Xác định phạm vi dự án, theo dõi và đảm bảo các mục tiêu, nhiệm vụ được thực hiện đúng, xác định công việc nào thuộc phạm vi dự án và cần hoàn thành, đồng thời loại trừ những công việc nằm ngoài phạm vi dự án.
- **Quản lý thời gian:** Lập kế hoạch, phân bổ và theo dõi tiến độ thời gian để đảm bảo dự án hoàn thành đúng hạn. Xác định thời gian cụ thể cho từng nhiệm vụ, thời điểm bắt đầu và kết thúc, cũng như thời gian hoàn thành toàn bộ dự án.
- **Quản lý chi phí:** Ước tính, giám sát chi phí của từng nhiệm vụ và toàn bộ dự án, tổ chức dữ liệu, phân tích và báo cáo thông tin chi phí để đảm bảo ngân sách được sử dụng hiệu quả.
- **Quản lý chất lượng:** Triển khai và giám sát các tiêu chuẩn chất lượng để đảm bảo kết quả dự án đáp ứng kỳ vọng của nhà tài trợ hoặc nhà đầu tư.

ii. 5 lĩnh vực hỗ trợ: Là các phương tiện giúp đạt được mục tiêu dự án, bao gồm:

- **Quản lý nguồn nhân lực:** Hướng dẫn và điều phối sự hợp tác giữa các thành viên dự án, đảm bảo nhân lực được sử dụng hiệu quả để đạt được các mục tiêu đã đề ra.
- **Quản lý thông tin (truyền thông):** Đảm bảo luồng thông tin liên lạc giữa các thành viên, các cấp quản lý và các nhóm liên quan luôn nhanh chóng, chính xác. Trả lời các câu hỏi như: Ai cần thông tin? Ở mức độ chi tiết nào? Báo cáo như thế nào?
- **Quản lý rủi ro:** Xác định các yếu tố rủi ro, sử dụng phương pháp định tính và định lượng để đánh giá mức độ rủi ro, đồng thời xây dựng kế hoạch xử lý và quản lý từng loại rủi ro.

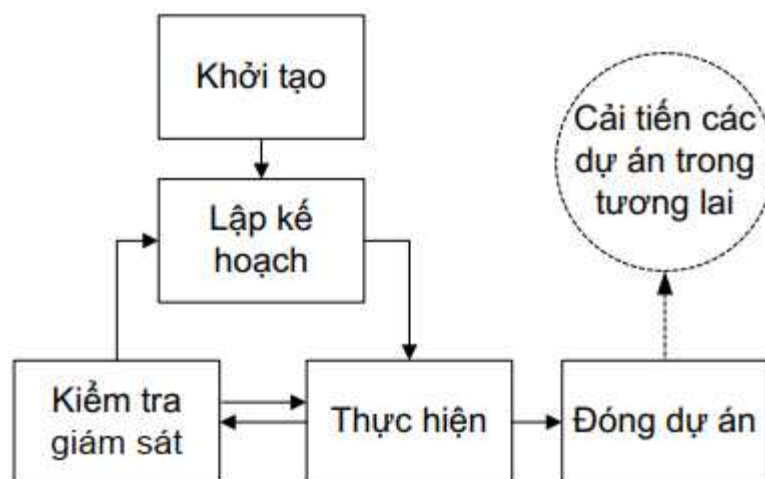
- **Quản lý các bên liên quan của dự án:** bao gồm việc xác định và phân tích nhu cầu của các bên liên quan trong khi quản lý và kiểm soát sự tham gia của họ trong suốt vòng đời của dự án.
- **Quản lý hợp đồng và mua sắm thiết bị:** Lựa chọn nhà cung cấp, đàm phán hợp đồng, quản lý việc mua sắm vật liệu, thiết bị và dịch vụ. Đảm bảo các vấn đề như cung cấp hàng hóa đúng tiến độ, chất lượng và nhu cầu của dự án được giải quyết hiệu quả.

iii. 1 lĩnh vực tích hợp:

- **Quản lý tích hợp dự án:** Kết nối và điều phối tất cả các lĩnh vực trên, đảm bảo sự thống nhất và hiệu quả trong quản lý dự án.

Các lĩnh vực này phối hợp chặt chẽ với nhau để đảm bảo dự án đạt được mục tiêu đề ra một cách hiệu quả và chất lượng.

Một khái niệm quan trọng khác trong quản lý dự án là nhóm quy trình quản lý dự án. Theo PMI (Project Management Institute), nhóm quy trình quản lý dự án là một nhóm hợp lý các quy trình quản lý dự án “đạt được một tập hợp các mục tiêu dự án cụ thể”. Các quy trình quản lý dự án được chia thành năm nhóm bao gồm khởi tạo, lập kế hoạch, thực hiện, giám sát và kết thúc, như hình 1.3. Việc điều chỉnh các nhóm quy trình này sao cho “phù hợp” với từng dự án cụ thể là một yếu tố quan trọng cho sự thành công của dự án.



Hình 1.3. Các nhóm quy trình dự án

Nhóm quy trình quản lý dự án được tổ chức theo quy trình thực hiện dự án từ khởi tạo, lập kế hoạch, thực hiện, giám sát và kết thúc.

Trong đó, *nhóm quy trình khởi tạo* bao gồm các hoạt động xác định, phân công và giao trách nhiệm cho các thành viên liên quan đến dự án hoặc một giai đoạn cụ thể của dự án. Các quy trình khởi tạo được thực hiện trong từng giai đoạn của dự án. Do đó, chúng ta không nên đánh đồng các quy trình quản lý dự án với từng giai đoạn của dự án vì mỗi dự

án có thể có nhiều giai đoạn, nhưng tất cả các giai đoạn của dự án đều có một nhóm các quy trình quản lý.

Nhóm quy trình *lập kế hoạch* bao gồm các quy trình được sử dụng để thiết lập và duy trì kế hoạch công việc nhằm đạt được các mục tiêu của dự án. Một dự án phần mềm có thể bao gồm nhiều loại kế hoạch, chẳng hạn như kế hoạch quản lý phạm vi, thời gian, chi phí, chất lượng, v.v. Mỗi kế hoạch là một phương pháp tiến hành công việc, xác định và xử lý các vấn đề liên quan đến một "lĩnh vực kiến thức" cụ thể trong quản lý dự án. Ví dụ, một người quản lý dự án cần thiết lập một "kế hoạch" để xác định và quản lý các vấn đề phát sinh liên quan đến phạm vi dự án, tạo "lịch trình" và quản lý lịch trình dự án, tạo ước tính chi phí và quản lý chi phí dự án, v.v. Để ứng phó với những thay đổi trong dự án, chúng ta cần thường xuyên xem xét dự án để nắm bắt những thay đổi và cập nhật kế hoạch công việc tương ứng trong quá trình triển khai dự án. Kế hoạch quản lý dự án là tổng hợp tất cả các kế hoạch để quản lý công việc liên quan đến các lĩnh vực kiến thức trong quản lý dự án.

Nhóm quy trình *triển khai* bao gồm các quy trình phải được phối hợp giữa các bên liên quan của dự án để thực hiện các nhiệm vụ dự án khác nhau đã được thiết lập trong các kế hoạch tương ứng của dự án. Đây là các kế hoạch quản lý công việc tạo ra các sản phẩm, dịch vụ hoặc kết quả của dự án. Ví dụ, trong quy trình quản lý phạm vi dự án, chúng ta cần thu thập các yêu cầu, tạo cấu trúc phân tích công việc và đánh giá phạm vi dự án. Trong khi đó, trong nhóm quy trình quản lý lịch trình dự án, nó bao gồm việc xác định các nhiệm vụ dự án, ước tính thời gian còn lại cho mỗi nhiệm vụ, tạo lịch trình dự án và phân tích lịch trình để tìm ra một số tính năng quan trọng của lịch trình dự án như đường dẫn dự án và thực hiện các nhiệm vụ dự án theo lịch trình đã thiết lập, v.v.

Nhóm quy trình *giám sát và kiểm soát* bao gồm các thủ tục đo lường và kiểm soát tiến độ của dự án để đảm bảo nhóm dự án hoàn thành công việc theo đúng kế hoạch. Trong nhóm quy trình này, các nhà quản lý dự án và đối tác dự án đo lường các thông số trạng thái dự án và thực hiện các hành động để ứng phó với các sai lệch so với kế hoạch dự án. Một hành động chung trong quá trình này là báo cáo trạng thái và các vấn đề của dự án cho các bên liên quan để họ có thể thực hiện hành động khắc phục và hành động phù hợp để đưa dự án trở lại đúng hướng.

Nhóm quy trình *đóng dự án* bao gồm các hành động dẫn đến việc chấp nhận chính thức các mục tiêu dự án và đóng dự án. Thông thường, các mục tiêu dự án chính thức được xem xét trong quá trình triển khai dự án. Trong quá trình đóng dự án, chúng tôi xem xét, kiểm tra và yêu cầu phản hồi về bất kỳ mục tiêu dự án nào còn thiếu. Ngoài ra, các hành động quản trị cấp tổ chức được thực hiện để duy trì các nguồn lực của dự án như tài liệu, gói bàn giao, bài học kinh nghiệm từ dự án, hồ sơ chấp nhận mục tiêu dự án, v.v. theo đúng các quy định của quy trình quản lý dự án của tổ chức. Các hành động quản lý này, đặc biệt là các bài học kinh nghiệm từ dự án, giúp tổ chức cải thiện việc quản lý và triển khai dự án trong tương lai.

Quản lý dự án Công nghệ thông tin

Trong giai đoạn đầu, vào những năm 1980, quản lý dự án chủ yếu tập trung vào việc lập kế hoạch lịch trình và phân bổ nguồn lực cần thiết để hỗ trợ ban lãnh đạo trong các lĩnh vực quan trọng như quân sự, công nghệ thông tin (CNTT) và xây dựng. Ngày nay, quản lý dự án không chỉ là lĩnh vực dành riêng cho một nhóm đối tượng mà còn liên quan đến mọi người từ các tầng lớp xã hội và các quốc gia khác nhau. Đây là một nghề chuyên môn đòi hỏi chứng chỉ chuyên môn cao, đồng thời mở ra nhiều cơ hội nghề nghiệp hấp dẫn. Trong bối cảnh chuyển đổi số, sự phát triển của cơ sở hạ tầng phần cứng, mạng lưới truyền thông, phần mềm, cùng với việc tổ chức các nhóm làm việc liên ngành và toàn cầu, đã làm thay đổi hoàn toàn cách thức làm việc truyền thống.

Quản lý dự án phần mềm là quá trình lập kế hoạch, tổ chức, điều phối và kiểm soát các nguồn lực và công việc để đạt được mục tiêu cụ thể của dự án trong thời gian và ngân sách quy định.

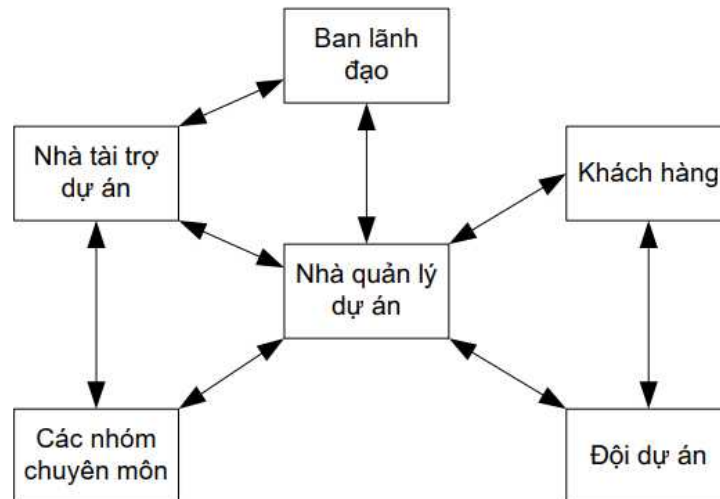
1.1.2 Sự khác biệt giữa Quản lý Dự án Phần mềm và các loại dự án khác

Quản lý dự án phần mềm liên quan đến việc áp dụng các kiến thức, kỹ năng, công cụ và kỹ thuật để đảm bảo rằng sản phẩm phần mềm cuối cùng đáp ứng được yêu cầu và tiêu chuẩn chất lượng. Nó khác với các dự án xây dựng, sản xuất ở tính linh hoạt và khả năng thay đổi yêu cầu. Quản lý dự án phần mềm khác biệt với các loại dự án khác ở các khía cạnh sau:

- **Tính hữu hình:** Sản phẩm của dự án CNTT là vô hình, trong khi sản phẩm của dự án xây dựng là hữu hình. Điều này làm cho việc định nghĩa và đo lường phạm vi, tiến độ trở nên phức tạp.
- **Tính thay đổi:** Yêu cầu của dự án CNTT có thể thay đổi liên tục trong quá trình thực hiện, trong khi yêu cầu của dự án xây dựng thường ổn định hơn.
- **Rủi ro:** Phần mềm thường đối mặt với nhiều rủi ro liên quan đến yêu cầu thay đổi, công nghệ, và khả năng tích hợp. Rủi ro trong dự án CNTT thường liên quan đến công nghệ, chất lượng phần mềm, trong khi rủi ro trong dự án xây dựng thường liên quan đến yếu tố môi trường, vật liệu.
- **Phụ thuộc vào con người:** Chất lượng của sản phẩm phụ thuộc nhiều vào kỹ năng và hiệu suất làm việc của đội ngũ lập trình viên.
- **Tiến hóa:** Các sản phẩm phần mềm có xu hướng thay đổi và phát triển liên tục trong suốt vòng đời của chúng.

1.1.3 Vai trò của các bên liên quan trong dự án phần mềm

Các bên liên quan là những cá nhân hoặc tổ chức có liên quan đến hoặc bị ảnh hưởng bởi các hoạt động và kết quả của một dự án. Ví dụ, trong quá trình xây dựng một ngôi nhà, hàng xóm của chúng ta cũng được coi là bên liên quan vì họ có thể bị ảnh hưởng bởi tiếng ồn, bụi, hoặc các tác động khác từ dự án.



Hình 1.4. Các vai trò chính trong dự án

Đối với một **dự án CNTT hoặc phần mềm**, các bên liên quan đóng vai trò quan trọng trong sự thành công của dự án phần mềm. Các bên liên quan chính bao gồm:

- **Nhà Quản lý dự án (Project Manager):** Chịu trách nhiệm lập kế hoạch, quản lý tiến độ, ngân sách và giao tiếp giữa các bên liên quan.
- **Developer:** Đội ngũ phát triển mã nguồn phần mềm dựa trên yêu cầu và thiết kế đã được phê duyệt.
- **Tester:** Kiểm tra chất lượng sản phẩm, đảm bảo rằng phần mềm hoạt động chính xác và ổn định.
- **Stakeholder:** Bao gồm khách hàng, nhà tài trợ và các bên khác có liên quan, cung cấp yêu cầu và đánh giá kết quả. Nhà đầu tư cung cấp nguồn tài chính cho dự án và mong muốn dự án mang lại giá trị hoặc lợi ích nhất định. Khách hàng là hững người hoặc tổ chức đặt hàng dự án và kỳ vọng sản phẩm hoặc dịch vụ đáp ứng yêu cầu cụ thể. Họ cũng là những người dùng cuối, những cá nhân trực tiếp sử dụng sản phẩm hoặc dịch vụ của dự án và là yếu tố quyết định đến mức độ thành công và hiệu quả của dự án.

Mối quan hệ giữa các bên liên quan này thường phức tạp và đòi hỏi sự quản lý hiệu quả để đảm bảo các yêu cầu, lợi ích và kỳ vọng của họ được đáp ứng, đồng thời giảm thiểu những tác động tiêu cực trong quá trình triển khai dự án. Hình 1.4 minh họa chi tiết các mối quan hệ này.

Người quản lý dự án đóng vai trò là cầu nối duy nhất và chịu trách nhiệm quản lý mọi mối quan hệ với các bên liên quan. Trong suốt vòng đời của dự án, đặc biệt là giai đoạn khởi đầu, nhu cầu và kỳ vọng của các bên liên quan thường rất quan trọng và có thể thay đổi liên tục.

Để đảm bảo thành công trong quản lý dự án, người quản lý dự án cần:

- Phát triển mối quan hệ chặt chẽ với các bên liên quan để xây dựng lòng tin và sự hợp tác.
- Hiểu rõ nhu cầu và kỳ vọng của từng bên liên quan, xác định các ưu tiên và yêu cầu cốt lõi.
- Đáp ứng kịp thời những nhu cầu và kỳ vọng đó thông qua việc giao tiếp hiệu quả, quản lý rủi ro và điều chỉnh kế hoạch khi cần thiết.

Việc duy trì sự liên lạc thường xuyên và tạo điều kiện thuận lợi cho các bên liên quan tham gia vào các quyết định quan trọng sẽ góp phần lớn vào sự thành công của dự án.

Phẩm chất cần thiết đối với một nhà quản lý dự án

- **Thật thà và chính trực (Honesty and Integrity)**

Nhà quản lý dự án cần phải trung thực và giữ vững nguyên tắc đạo đức trong công việc. Sự chính trực giúp họ xây dựng niềm tin với các bên liên quan và tạo ra môi trường làm việc minh bạch, đáng tin cậy.

- **Khả năng ra quyết định (Decision Making Ability)**

Quản lý dự án thường xuyên đối mặt với các tình huống đòi hỏi ra quyết định nhanh chóng và chính xác. Khả năng phân tích vấn đề, cân nhắc lợi ích và rủi ro trước khi đưa ra giải pháp là yếu tố then chốt để đảm bảo thành công của dự án.

- **Hiểu biết các vấn đề về con người (Understanding of Personal Problem)**

Một nhà quản lý giỏi không chỉ tập trung vào công việc mà còn cần quan tâm và hiểu rõ các vấn đề cá nhân của các thành viên trong nhóm. Điều này giúp tạo động lực, duy trì tinh thần làm việc và xử lý kịp thời những khó khăn ảnh hưởng đến hiệu suất làm việc.

- **Tính linh hoạt, đa năng, nhiều tài (Versatility)**

Nhà quản lý dự án cần có khả năng thích nghi với các tình huống thay đổi nhanh chóng và xử lý nhiều nhiệm vụ khác nhau. Sự đa năng giúp họ linh hoạt trong việc điều phối các nguồn lực và giải quyết vấn đề một cách hiệu quả.

Những phẩm chất này không chỉ đảm bảo dự án được quản lý tốt mà còn giúp nhà quản lý xây dựng một đội nhóm mạnh mẽ và đạt được các mục tiêu đã đề ra.

1.2 Các Giai đoạn trong Vòng đời Dự án Phần mềm

1.2.1. Lập Kế Hoạch

Lập kế hoạch là giai đoạn đầu tiên trong vòng đời dự án phần mềm. Giai đoạn này đóng vai trò quan trọng vì nó tạo cơ sở cho tất cả các hoạt động sau này. Một kế hoạch tốt giúp định hướng dự án và đảm bảo rằng các nguồn lực được sử dụng một cách hiệu quả.

Mục tiêu chính của lập kế hoạch là xác định rõ ràng "làm gì, tại sao làm, và làm thế nào để đạt được". Điều này bao gồm việc thiết lập các mục tiêu cụ thể của dự án, đảm bảo rằng tất cả các bên liên quan có cùng hiểu biết về phạm vi công việc, và tạo ra một kế hoạch thực thi chi tiết. Lập kế hoạch là cơ hội để nhóm dự án dự đoán và giải quyết các thách thức tiềm năng trước khi chúng xảy ra.

Các nhiệm vụ chính trong giai đoạn lập kế hoạch

- Xác định mục tiêu dự án:
 - Mục tiêu cần được xác định rõ ràng, cụ thể, đo lường được, khả thi và có thời hạn (SMART).
 - Ví dụ: "Phát triển một ứng dụng quản lý khách hàng với các tính năng theo dõi lịch sử giao dịch, tích hợp thanh toán trực tuyến, và báo cáo doanh thu trong vòng 6 tháng."
- Xác định phạm vi và yêu cầu:

- Phạm vi bao gồm tất cả các công việc cần thực hiện để đạt được mục tiêu.
- Xác định yêu cầu cơ bản của dự án để tránh tình trạng thay đổi không kiểm soát được trong các giai đoạn sau.

- **Lập kế hoạch nguồn lực:**

- Nhân sự: Phân công công việc cho các thành viên trong nhóm, xác định các vai trò và trách nhiệm.

- Tài chính: Dự toán ngân sách cho toàn bộ dự án, bao gồm chi phí phát triển, kiểm thử, triển khai và bảo trì.

- Công cụ: Lựa chọn các công cụ phát triển phần mềm, hệ thống kiểm thử và các nền tảng hỗ trợ cần thiết.

- **Xây dựng lịch trình thực hiện:**

- Sử dụng các công cụ như Gantt Chart hoặc PERT để lập lịch trình chi tiết.

- Phân chia công việc thành các giai đoạn hoặc sprint nhỏ hơn để dễ dàng theo dõi và điều chỉnh.

- **Phân tích và quản lý rủi ro:**

- Xác định các rủi ro tiềm tàng có thể ảnh hưởng đến dự án (ví dụ: chậm tiến độ, thay đổi yêu cầu, thiếu nhân lực).

- Đưa ra các biện pháp phòng ngừa và kế hoạch đối phó nếu rủi ro xảy ra.

Kết quả của giai đoạn lập kế hoạch

Một kế hoạch dự án hoàn chỉnh sẽ bao gồm:

- Mục tiêu rõ ràng: Những gì cần đạt được, khi nào và bằng cách nào.
- Phạm vi và yêu cầu: Các công việc cụ thể cần thực hiện để hoàn thành mục tiêu.
- Nguồn lực: Danh sách các nguồn lực (nhân sự, tài chính, công cụ) cần thiết và cách chúng sẽ được phân bổ.
- Lịch trình: Thời gian hoàn thành từng nhiệm vụ hoặc giai đoạn.
- Kế hoạch quản lý rủi ro: Danh sách các rủi ro, mức độ ưu tiên, và các biện pháp đối phó.

1.2.2. Phân Tích Yêu Cầu

Giai đoạn phân tích yêu cầu đóng vai trò quan trọng trong việc đảm bảo sản phẩm phần mềm đáp ứng được nhu cầu sử dụng thực tế. Đây là giai đoạn mà đội dự án làm việc chặt chẽ với khách hàng và các bên liên quan để hiểu rõ các kỳ vọng, xác định các chức năng cần thiết, và tạo ra một tài liệu đặc tả yêu cầu chi tiết. Phân tích yêu cầu được coi là nền tảng cho các giai đoạn thiết kế, phát triển và kiểm thử.

Mục tiêu của giai đoạn phân tích yêu cầu

- Hiểu đầy đủ các nhu cầu và mong đợi của khách hàng.
- Xác định rõ các yêu cầu chức năng (những gì phần mềm cần làm) và phi chức năng (như hiệu suất, bảo mật, khả năng mở rộng).
- Tạo tài liệu đặc tả yêu cầu để làm cơ sở cho việc thiết kế và phát triển.

Nhiệm vụ của giai đoạn phân tích yêu cầu

- Thu thập và phân tích yêu cầu:
 - **Thu thập yêu cầu:** Được thực hiện thông qua các phương pháp như phỏng vấn, khảo sát, workshop, và nghiên cứu tài liệu hiện có.

Ví dụ: Khi phát triển phần mềm quản lý bán hàng, đội dự án cần phỏng vấn nhân viên bán hàng, quản lý kho, và chủ doanh nghiệp để hiểu rõ cách vận hành hiện tại.

- **Phân tích yêu cầu:** Tổ chức, phân loại và đánh giá các yêu cầu để xác định tính khả thi và mức độ ưu tiên. Đảm bảo rằng các yêu cầu không mâu thuẫn và phù hợp với mục tiêu dự án.
- Xây dựng tài liệu đặc tả yêu cầu (SRS - Software Requirements Specification):

Tài liệu SRS là một bản mô tả chi tiết tất cả các yêu cầu của dự án phần mềm. Nó bao gồm:

- Mô tả chức năng chính của phần mềm.
- Các yêu cầu phi chức năng như tốc độ xử lý, tính bảo mật, và khả năng mở rộng.
- Các ràng buộc kỹ thuật và hệ thống.

Tài liệu này được sử dụng như một tham chiếu cho các giai đoạn sau và là cơ sở để kiểm thử phần mềm.

- Làm rõ các kỳ vọng và giới hạn của dự án:
 - Đảm bảo rằng mọi yêu cầu và mục tiêu của khách hàng đều được hiểu rõ và đồng thuận.
 - Xác định các giới hạn về phạm vi, ngân sách, và thời gian để tránh những hiểu lầm hoặc thay đổi không kiểm soát được.

Kết quả của giai đoạn này là tài liệu đặc tả yêu cầu chi tiết, được xác nhận bởi khách hàng và các bên liên quan.

1.2.3. Thiết Kế

Giai đoạn thiết kế là cầu nối giữa yêu cầu và quá trình phát triển phần mềm. Đây là bước chuyển đổi từ các đặc tả yêu cầu (SRS) thành một bản thiết kế chi tiết, mô tả cách hệ thống sẽ hoạt động và cấu trúc của nó. Thiết kế không chỉ tập trung vào việc đáp ứng các yêu cầu mà còn đảm bảo rằng hệ thống sẽ dễ dàng thực hiện, bảo trì và mở rộng.

Mục tiêu của giai đoạn thiết kế

- Biến các yêu cầu của khách hàng thành một bản thiết kế kỹ thuật cụ thể.
- Đảm bảo rằng thiết kế hệ thống khả thi và hiệu quả.
- Tạo nền tảng vững chắc cho giai đoạn phát triển bằng cách cung cấp hướng dẫn chi tiết cho đội ngũ lập trình.

Nhiệm vụ:

- Thiết kế kiến trúc hệ thống, mô hình dữ liệu, giao diện người dùng và các luồng xử lý.
- Đảm bảo rằng thiết kế có thể thực hiện được và đáp ứng yêu cầu của khách hàng.

Kết quả của giai đoạn thiết kế

- Bản thiết kế hệ thống chi tiết:
 - Là sản phẩm chính của giai đoạn này, cung cấp thông tin rõ ràng về cách hệ thống sẽ được xây dựng.
 - Bao gồm:

- **Kiến trúc hệ thống:** Mô tả các thành phần chính và cách chúng kết nối.
 - **Mô hình dữ liệu:** Các bảng dữ liệu, mối quan hệ và quy tắc ràng buộc.
 - **Giao diện người dùng:** Các mẫu và quy tắc thiết kế giao diện.
 - **Luồng xử lý:** Các sơ đồ mô tả quy trình vận hành.
- Hướng dẫn chi tiết cho đội phát triển:
 - Giúp các lập trình viên hiểu rõ công việc cần làm và cách thực hiện.
 - Giảm thiểu sự không rõ ràng và lỗi trong giai đoạn phát triển.

1.2.4. Phát Triển

Giai đoạn phát triển là bước hiện thực hóa bản thiết kế thành sản phẩm phần mềm thực tế. Đây là một trong những giai đoạn trung tâm của vòng đời dự án, nơi mà đội ngũ lập trình viên chuyển đổi các tài liệu thiết kế chi tiết thành mã nguồn để xây dựng chức năng của phần mềm. Giai đoạn này yêu cầu kỹ năng kỹ thuật cao, sự hợp tác chặt chẽ giữa các thành viên, và khả năng quản lý tốt để đảm bảo chất lượng và tiến độ.

Mục tiêu của giai đoạn phát triển

- Chuyển đổi thiết kế thành mã nguồn có thể thực thi.
- Đảm bảo rằng tất cả các chức năng và yêu cầu đã được triển khai đúng theo tài liệu đặc tả và thiết kế.
- Tạo ra một sản phẩm phần mềm có thể hoạt động cơ bản để chuyển sang giai đoạn kiểm thử.

Nhiệm vụ:

- Viết mã nguồn theo bản thiết kế đã được phê duyệt.
- Tích hợp các thành phần của hệ thống.
- Đảm bảo rằng phần mềm hoạt động cơ bản trước khi kiểm thử.

Kết quả của giai đoạn này là sản phẩm phần mềm ban đầu, sẵn sàng cho giai đoạn kiểm thử.

1.2.5. Kiểm Thử

Kiểm thử là một giai đoạn quan trọng trong vòng đời phát triển phần mềm, tập trung vào việc đánh giá chất lượng của sản phẩm. Giai đoạn này nhằm phát hiện và sửa chữa các lỗi (bug), đảm bảo rằng phần mềm hoạt động đúng theo yêu cầu và ổn định trong môi trường thực tế. Một sản phẩm phần mềm đạt tiêu chuẩn chất lượng là nền tảng cho sự hài lòng của khách hàng và hiệu quả sử dụng.

Mục tiêu của giai đoạn kiểm thử

- Đảm bảo phần mềm đáp ứng đầy đủ các yêu cầu chức năng (functional requirements).
- Đảm bảo rằng phần mềm vận hành tốt dưới các yêu cầu phi chức năng (non-functional requirements) như hiệu suất, bảo mật, và khả năng mở rộng.
- Phát hiện lỗi, đánh giá mức độ nghiêm trọng và sửa chữa trước khi triển khai.

Nhiệm vụ:

- Phát hiện và sửa lỗi trong hệ thống.
- Đảm bảo rằng phần mềm đáp ứng các yêu cầu chức năng và phi chức năng.
- Chuẩn bị sản phẩm cho giai đoạn triển khai.

Kết quả:

- Phần mềm đã qua kiểm thử:
 - Đáp ứng đầy đủ các tiêu chuẩn chất lượng và hoạt động ổn định.
 - Không còn lỗi nghiêm trọng ảnh hưởng đến hoạt động chính của hệ thống.
- Báo cáo kiểm thử:
 - Tài liệu ghi nhận tất cả các lỗi đã được phát hiện và sửa chữa.
 - Bao gồm đánh giá tổng quan về chất lượng của phần mềm và mức độ sẵn sàng để triển khai.

1.2.6. Triển Khai

Giai đoạn triển khai là bước đưa phần mềm từ môi trường phát triển sang môi trường thực tế, nơi nó sẽ được sử dụng bởi khách hàng hoặc người dùng cuối. Đây là một trong

những giai đoạn quan trọng nhất của vòng đời phát triển phần mềm, bởi đây là thời điểm sản phẩm bắt đầu được kiểm chứng hiệu quả và giá trị trong thực tiễn. Giai đoạn này yêu cầu sự chuẩn bị kỹ lưỡng, hỗ trợ tốt từ đội ngũ kỹ thuật và phản hồi nhanh chóng để đảm bảo quá trình triển khai diễn ra suôn sẻ.

Mục tiêu của giai đoạn triển khai

- Đảm bảo rằng phần mềm được cài đặt và cấu hình chính xác trong môi trường thực tế.
- Hỗ trợ người dùng hiệu và sử dụng sản phẩm một cách hiệu quả.
- Xử lý các vấn đề phát sinh nhanh chóng để đảm bảo trải nghiệm mượt mà trong giai đoạn đầu sử dụng.

Nhiệm vụ của giai đoạn triển khai

- Cài đặt và cấu hình phần mềm:
 - Triển khai phần mềm trên cơ sở hạ tầng thực tế, như máy chủ, máy trạm, hoặc thiết bị di động.
 - Cấu hình hệ thống để đảm bảo rằng phần mềm hoạt động đúng trong môi trường sản xuất (production environment).
 - Ví dụ: Cài đặt một ứng dụng quản lý bán hàng trên hệ thống nội bộ của doanh nghiệp và cấu hình kết nối với cơ sở dữ liệu.
- Đào tạo người dùng và cung cấp tài liệu hướng dẫn:
 - Hướng dẫn người dùng cuối cách sử dụng phần mềm thông qua các buổi đào tạo trực tiếp hoặc tài liệu như hướng dẫn sử dụng, video minh họa.
 - Đảm bảo rằng người dùng hiểu được các tính năng chính và cách khai thác tối ưu phần mềm.
- Xử lý các vấn đề phát sinh trong giai đoạn đầu:
 - Xử lý các lỗi nhỏ hoặc sự cố xảy ra do môi trường triển khai hoặc do người dùng chưa quen với sản phẩm.
 - Cung cấp hỗ trợ kỹ thuật để giải quyết các vấn đề trong thời gian nhanh nhất.

Các hoạt động chính trong giai đoạn triển khai

- Chuẩn bị môi trường triển khai:
 - Đảm bảo cơ sở hạ tầng, phần cứng và phần mềm hỗ trợ (middleware) đã sẵn sàng cho quá trình cài đặt.
 - Kiểm tra tính tương thích giữa phần mềm và môi trường sản xuất.
- Triển khai phần mềm:
 - Thực hiện cài đặt phần mềm trên các hệ thống thực tế.
 - Kiểm tra nhanh để đảm bảo rằng tất cả các thành phần đều hoạt động đúng như mong đợi.
- Kiểm tra sau triển khai:
 - Đảm bảo rằng phần mềm không gặp lỗi trong môi trường thực tế và đáp ứng các yêu cầu đã đặt ra.
 - Kiểm tra tương thích với các hệ thống khác (nếu có tích hợp).
- Đào tạo và hỗ trợ người dùng:
 - Tổ chức các buổi đào tạo trực tiếp hoặc trực tuyến để giới thiệu phần mềm.
 - Cung cấp tài liệu hỗ trợ như file PDF hướng dẫn, video minh họa, hoặc các câu hỏi thường gặp (FAQ).
- Theo dõi và xử lý sự cố:
 - Giám sát hoạt động của phần mềm trong thời gian đầu để phát hiện và sửa chữa các vấn đề phát sinh.
 - Thu thập phản hồi từ người dùng để cải thiện sản phẩm.

Kết quả của giai đoạn triển khai

- Phần mềm được sử dụng thành công:
 - Người dùng cuối có thể sử dụng phần mềm mà không gặp lỗi lớn hoặc khó khăn.
 - Phần mềm đáp ứng các yêu cầu vận hành trong môi trường thực tế.

- Người dùng được đào tạo đầy đủ: Người dùng hiểu cách sử dụng phần mềm một cách hiệu quả và tối ưu.
- Hỗ trợ kỹ thuật được duy trì: Đội ngũ hỗ trợ sẵn sàng giải quyết mọi vấn đề hoặc câu hỏi phát sinh trong quá trình sử dụng ban đầu.

1.2.7. Bảo Trì

Bảo trì là giai đoạn cuối cùng trong vòng đời phát triển phần mềm, đóng vai trò quan trọng để đảm bảo rằng sản phẩm phần mềm tiếp tục hoạt động ổn định, hiệu quả, và đáp ứng các nhu cầu thay đổi của người dùng. Đây không chỉ là bước sửa lỗi nhỏ mà còn là quá trình cập nhật và nâng cấp phần mềm để giữ nó phù hợp với môi trường và yêu cầu sử dụng hiện tại.

Mục tiêu của giai đoạn bảo trì

- Đảm bảo phần mềm tiếp tục hoạt động như dự kiến trong môi trường sử dụng thực tế.
- Sửa chữa các lỗi phát sinh sau triển khai và sử dụng.
- Thích ứng với các thay đổi về yêu cầu, công nghệ hoặc môi trường vận hành.
- Cải thiện hiệu suất và trải nghiệm người dùng thông qua nâng cấp và tối ưu hóa.

Nhiệm vụ của giai đoạn bảo trì

- Sửa lỗi phát sinh trong quá trình sử dụng
 - Xử lý các lỗi (bug) chưa phát hiện trong giai đoạn kiểm thử hoặc xuất hiện do thay đổi môi trường sử dụng.
 - Ví dụ: Sửa lỗi hiển thị trên một phiên bản trình duyệt mới hoặc lỗi kết nối với cơ sở dữ liệu.
- Cập nhật phần mềm để phù hợp với yêu cầu mới
 - Thay đổi các tính năng hiện có hoặc thêm mới để đáp ứng nhu cầu của người dùng hoặc thay đổi trong quy trình kinh doanh.
 - Ví dụ: Thêm tính năng hỗ trợ thanh toán điện tử khi xu hướng sử dụng tăng lên.
- Nâng cấp các tính năng để cải thiện hiệu suất và trải nghiệm người dùng

- Tối ưu hóa mã nguồn, cải thiện tốc độ xử lý hoặc tăng khả năng mở rộng của hệ thống.
- Cải tiến giao diện người dùng để thân thiện và dễ sử dụng hơn.

Kết quả của giai đoạn bảo trì

- Phần mềm ổn định và hiệu quả, mượt mà và không có lỗi nghiêm trọng làm gián đoạn trải nghiệm người dùng.
- Phần mềm thích ứng với thay đổi đáp ứng được các yêu cầu mới, công nghệ mới hoặc thay đổi trong quy trình kinh doanh.
- Người dùng hài lòng, phần mềm không chỉ hoạt động tốt mà còn mang lại trải nghiệm tốt hơn thông qua các cải tiến và nâng cấp.

Câu hỏi ôn tập cuối chương

1. Khái niệm quản lý dự án phần mềm là gì? Nêu mục tiêu chính của quản lý dự án phần mềm.
2. Phân biệt giữa quản lý dự án phần mềm và quản lý các loại dự án khác.
3. Vai trò của các bên liên quan trong quản lý dự án phần mềm là gì?
4. Nêu các giai đoạn chính trong vòng đời phát triển phần mềm.
5. Giai đoạn bảo trì trong vòng đời phần mềm bao gồm những nhiệm vụ chính nào?
6. Quản lý phạm vi và quản lý chi phí trong quản lý dự án phần mềm khác nhau như thế nào?
7. Hạn chế phổ biến trong quản lý dự án là gì và cách giảm thiểu những hạn chế này?
8. Kể tên và giải thích 4 lĩnh vực cốt lõi trong quản lý dự án.
9. Lợi ích của việc quản lý dự án hiệu quả là gì?
10. Các yếu tố cần thiết để đảm bảo thành công trong quản lý dự án là gì?

Chương 2: Phương pháp Quản lý dự án

Nội dung chính của chương

Quản lý dự án là quá trình mà nhà quản lý dự án áp dụng các kỹ năng, công cụ, và kiến thức chuyên môn để tổ chức và thực hiện các hoạt động nhằm đạt được các mục tiêu đã đề ra, đồng thời tối ưu hóa việc sử dụng các nguồn lực cần thiết.

Quy trình này bao gồm nhiều bước như xác định mục tiêu cụ thể của dự án, lập kế hoạch chi tiết, phân bổ nhiệm vụ cho các thành viên, giám sát tiến độ thực hiện, cải thiện quy trình dự án, quản lý rủi ro và thúc đẩy sự phối hợp hiệu quả giữa các thành viên trong nhóm. Để đạt được hiệu quả cao trong quản lý dự án, có thể áp dụng các phương pháp chuyên nghiệp và tận dụng các công cụ phần mềm hiện đại được giới thiệu ở chương này.

Mục tiêu cần đạt của chương

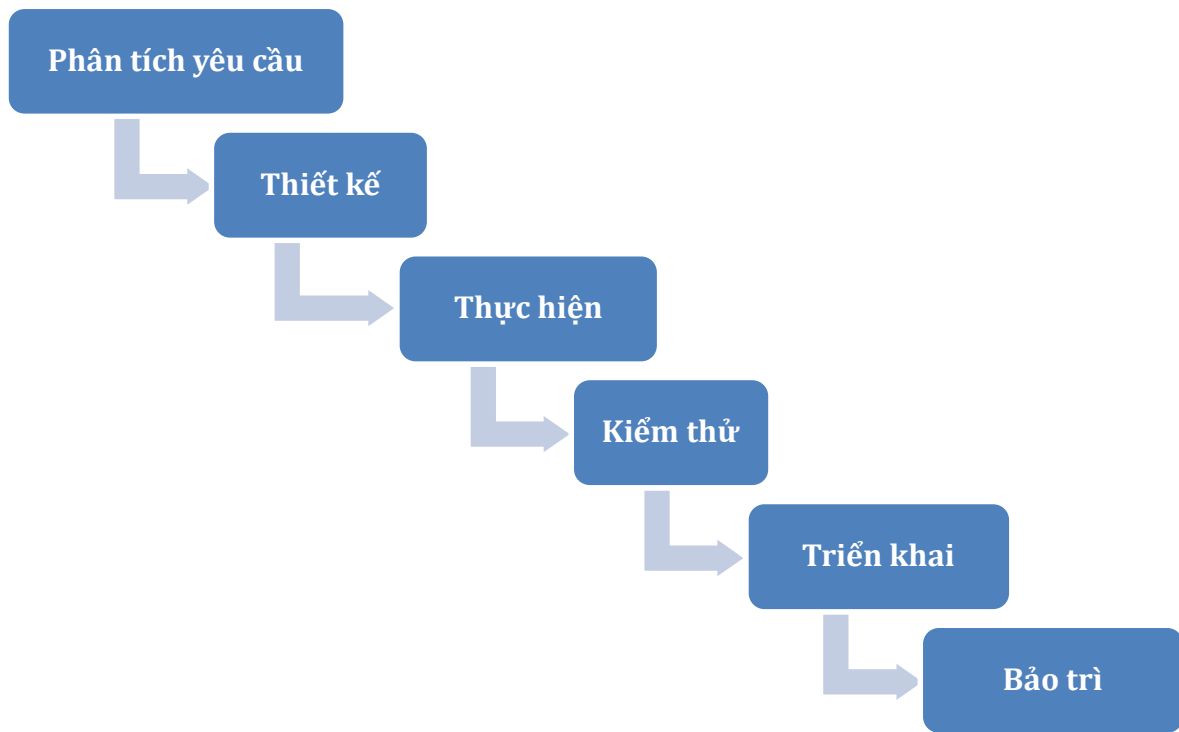
- Hiểu và so sánh các phương pháp quản lý dự án phổ biến.
- Biết lựa chọn phương pháp phù hợp với các tình huống thực tế.

2.1 Các phương pháp quản lý dự án phổ biến

Phương pháp quản lý dự án là tập hợp các quy trình, kỹ thuật, phương pháp và quy tắc được xây dựng và áp dụng để định hướng dự án đạt được các mục tiêu đã đặt ra. Mỗi phương pháp đều mang những đặc điểm riêng, với những ưu và nhược điểm cụ thể. Việc hiểu rõ và áp dụng đúng phương pháp phù hợp không chỉ giúp quản lý dự án hiệu quả hơn mà còn tăng đáng kể khả năng thành công của dự án.

2.1.1 Mô hình thác nước Waterfall

Mô hình thác nước (Waterfall Model) là một trong những mô hình phát triển phần mềm truyền thống và phổ biến nhất. Đây là mô hình tuyến tính, trong đó các giai đoạn phát triển phần mềm diễn ra tuần tự từ trên xuống, giống như nước chảy từ đỉnh thác xuống. Sau khi mục tiêu dự án được xác định, công việc sẽ được phân chia cụ thể cho các nhóm. Mỗi nhóm đảm nhiệm một phần riêng biệt của dự án, và các phần này được triển khai theo quy trình tuần tự, liên tiếp, đảm bảo sự phối hợp nhịp nhàng và liên kết chặt chẽ giữa các bộ phận để hoàn thành dự án. Mỗi giai đoạn phải hoàn thành trước khi chuyển sang giai đoạn tiếp theo, và không có sự quay lại để sửa đổi hay bổ sung các bước đã hoàn thành.



Hình 2.1. Mô hình thác nước

Đặc điểm chính của mô hình thác nước

1. **Tính tuần tự:** Các giai đoạn thực hiện tuần tự theo thứ tự đã định sẵn.
2. **Rõ ràng và dễ quản lý:** Vì mỗi giai đoạn có mục tiêu rõ ràng, nên dễ theo dõi và quản lý tiến độ.
3. **Phù hợp với dự án nhỏ:** Mô hình này phù hợp với các dự án có yêu cầu rõ ràng, ít thay đổi.

Các giai đoạn của mô hình thác nước

Như mô tả trong hình 2.1, mô hình thác nước gồm các giai đoạn:

1. **Phân tích yêu cầu (Requirements analysis):**
 - Thu thập và phân tích yêu cầu từ khách hàng hoặc bên liên quan.
 - Đầu ra là tài liệu yêu cầu phần mềm (SRS - Software Requirement Specification).
2. **Thiết kế (Design):**
 - Xây dựng kiến trúc tổng thể của hệ thống.
 - Định nghĩa cách các thành phần phần mềm tương tác.
 - Thiết kế chi tiết từng module hoặc thành phần của hệ thống.
 - Lập kế hoạch lập trình.

3. Triển khai (Implementation):

- Chuyển các thiết kế thành mã nguồn thực tế.
- Thực hiện và kiểm tra các đơn vị mã nguồn (Unit Testing).

4. Kiểm thử (Testing):

- Kiểm tra toàn bộ hệ thống để phát hiện và sửa lỗi.
- Đảm bảo phần mềm đáp ứng yêu cầu đã đặt ra.

5. Triển khai (Deployment):

- Cài đặt phần mềm vào môi trường thực tế.
- Giao phần mềm cho khách hàng hoặc người sử dụng.

6. Bảo trì (Maintenance):

- Sửa lỗi, cải tiến và cập nhật phần mềm sau khi triển khai.

Ưu điểm của mô hình thác nước

- Rõ ràng, dễ hiểu và dễ thực hiện.
- Tài liệu hóa kỹ lưỡng ở mỗi giai đoạn.
- Phù hợp với các dự án nhỏ, yêu cầu không thay đổi.

Nhược điểm của mô hình thác nước

- Khó khăn khi yêu cầu thay đổi trong quá trình phát triển.
- Không linh hoạt, đặc biệt trong các dự án phức tạp và dài hạn.
- Kiểm thử chỉ được thực hiện ở giai đoạn cuối, có thể dẫn đến chi phí sửa lỗi cao.

Khi nào nên sử dụng mô hình thác nước?

- Khi yêu cầu dự án rõ ràng và ổn định.
- Khi khách hàng có khả năng xác định yêu cầu đầy đủ từ đầu.
- Khi đội phát triển có kinh nghiệm làm việc với mô hình tuyến tính.

Mô hình thác nước là một lựa chọn tốt cho các dự án đơn giản và ít thay đổi. Tuy nhiên, với các dự án lớn, phức tạp và yêu cầu dễ thay đổi, các mô hình phát triển phần mềm linh hoạt, hiệu quả hơn.

2.1.2 Agile

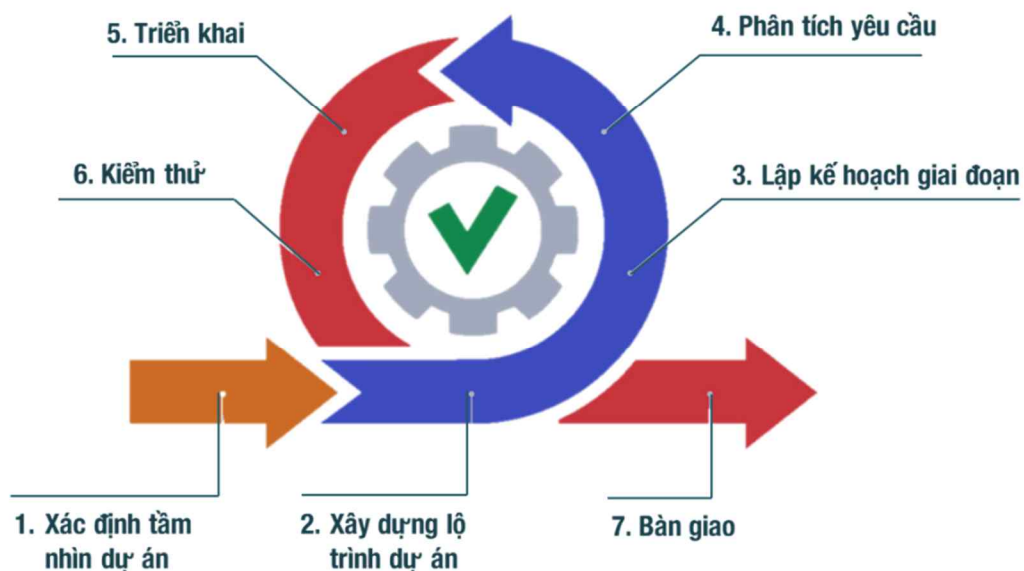
Agile (viết tắt của Agile Software Development) là một mô hình phát triển phần mềm hiện đại, linh hoạt, và lặp đi lặp lại, được thiết kế để thích ứng nhanh với các thay đổi trong yêu cầu dự án. Thay vì theo trình tự cố định như mô hình thác nước, Agile là quá

trình lặp lại nhiều chu kỳ nhỏ, như hình 2.1, trong đó mỗi chu kỳ cung cấp một sản phẩm hoàn chỉnh, có thể sử dụng được.

Đặc điểm chính của mô hình Agile

1. **Tính linh hoạt cao:** Cho phép thay đổi yêu cầu dự án ngay cả trong giai đoạn phát triển.
2. **Chia nhỏ công việc:** Dự án được chia thành các vòng lặp để dễ quản lý và theo dõi.
3. **Tập trung vào người dùng:** Sản phẩm được xây dựng dựa trên phản hồi liên tục từ khách hàng.
4. **Hợp tác nhóm chặt chẽ:** Đội ngũ phát triển, khách hàng và các bên liên quan làm việc sát sao với nhau.
5. **Kiểm tra liên tục:** Mỗi chu kỳ đều bao gồm giai đoạn kiểm tra và đánh giá sản phẩm.

Quy trình Agile



Hình 2.2. Mô hình Agile

Quy trình trong mô hình Agile

Xác định tầm nhìn, xây dựng lộ trình và lập kế hoạch

- Trong những tuần đầu tiên, đội ngũ dự án và khách hàng cùng thảo luận để lập kế hoạch tổng thể cho dự án. Dự án được chia thành các giai đoạn phát triển nhỏ, với thời gian, công sức và lịch trình cụ thể cho từng giai đoạn.
- Trước khi bắt đầu mỗi giai đoạn phát triển, một kế hoạch chi tiết sẽ được xây dựng.
- Các cuộc họp hàng ngày được tổ chức để cập nhật tiến độ và xử lý các vấn đề phát sinh.

Phân tích yêu cầu

- Phân tích yêu cầu không giới hạn trong một thời điểm cố định, mà diễn ra liên tục trong suốt quá trình phát triển.
- Đại diện khách hàng và đội dự án hợp tác chặt chẽ, trao đổi thường xuyên để tìm ra các giải pháp tối ưu trước khi chuyển giao yêu cầu cho đội phát triển.

Triển khai

- Thiết kế sản phẩm được cải tiến liên tục để đáp ứng các yêu cầu thay đổi.
- Mã nguồn là tài sản chung của cả nhóm, và mọi thành viên đều có trách nhiệm sửa lỗi, không phân biệt lỗi do ai gây ra.

Kiểm thử

- Tất cả thành viên trong nhóm đều có trách nhiệm đảm bảo chất lượng sản phẩm.
- Khi phát hiện lỗi, cả nhóm cùng nhau phân tích nguyên nhân và hợp tác để khắc phục.

Bàn giao sản phẩm

- Sản phẩm phần mềm được demo định kỳ (thường hàng tuần) để khách hàng đánh giá và đóng góp ý kiến, đảm bảo sản phẩm đáp ứng đúng nhu cầu.

Ưu điểm của mô hình Agile

- Dễ dàng thích nghi với các thay đổi trong yêu cầu.
- Giảm thiểu rủi ro bằng cách chia nhỏ dự án và kiểm tra liên tục.
- Tăng cường sự tham gia và hợp tác giữa các bên liên quan.

- Cải thiện chất lượng sản phẩm thông qua các vòng phản hồi liên tục.

Nhược điểm của mô hình Agile

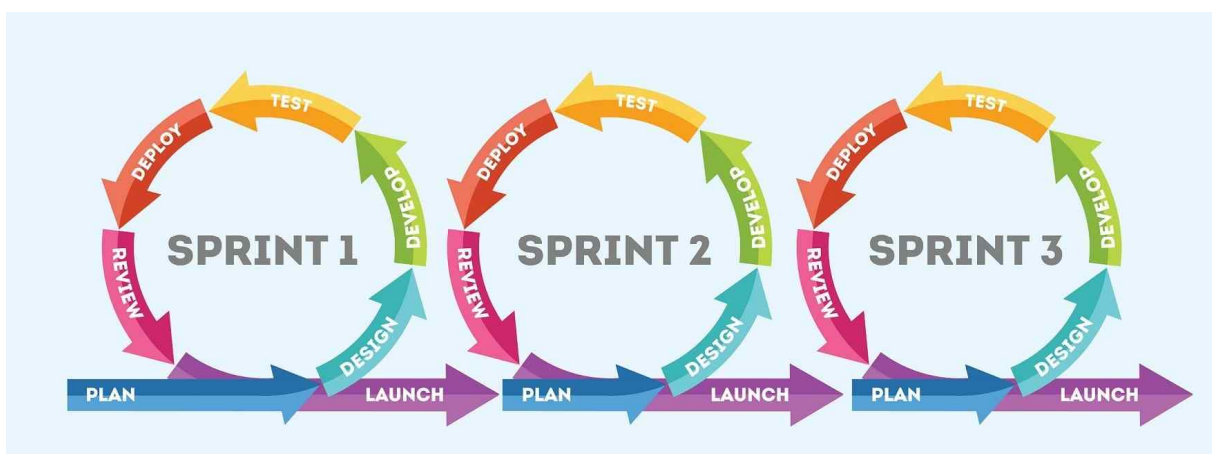
- Đòi hỏi sự tham gia thường xuyên của khách hàng, đôi khi khó đảm bảo.
- Cần đội ngũ phát triển có kỹ năng và khả năng tự quản cao.
- Không phù hợp với các dự án có yêu cầu cố định và rõ ràng ngay từ đầu.
- Có thể gây khó khăn trong việc ước lượng thời gian và chi phí.

Khi nào nên sử dụng mô hình Agile?

- Khi yêu cầu dự án thay đổi thường xuyên.
- Khi sản phẩm cần được đưa ra thị trường nhanh chóng.
- Khi khách hàng muốn tham gia tích cực trong quá trình phát triển.
- Khi đội ngũ phát triển có khả năng làm việc tự chủ và phối hợp tốt.

Mô hình Agile là sự lựa chọn lý tưởng cho các dự án phức tạp, yêu cầu thay đổi liên tục và cần sự linh hoạt cao. Nó giúp cải thiện hiệu quả làm việc nhóm và đảm bảo sản phẩm đáp ứng tối ưu nhu cầu của người dùng.

Có rất nhiều quy trình phát triển phần mềm dựa theo **Agile**, trong đó nổi bật nhất là mô hình **Scrum**, được thiết kế để quản lý các dự án phức tạp và thay đổi thường xuyên. Scrum tập trung vào việc chia dự án thành các chu kỳ ngắn, gọi là **Sprint**



Hình 1.3. Mô hình Scrum

Đặc điểm chính của Scrum

1. **Tính lặp lại (Iterative):** Phát triển phần mềm theo các chu kỳ ngắn, cải thiện dần dần.
2. **Tính gia tăng (Incremental):** Tạo ra sản phẩm hoàn chỉnh từng phần qua mỗi Sprint.
3. **Tính minh bạch (Transparency):** Mọi thành viên đều biết tình trạng dự án thông qua các cuộc họp thường xuyên.
4. **Tập trung vào giá trị:** Ưu tiên phát triển các tính năng mang lại giá trị cao nhất cho khách hàng.
5. **Khả năng thích ứng:** Thích ứng nhanh với các thay đổi trong yêu cầu.

Ưu điểm của Scrum

- Linh hoạt, dễ thích nghi với các thay đổi trong yêu cầu.
- Tăng cường sự tham gia và tương tác của các bên liên quan.
- Cung cấp sản phẩm nhanh chóng và liên tục.
- Khuyến khích sự phối hợp chặt chẽ trong nhóm.

Nhược điểm của Scrum

- Không hiệu quả với các dự án có yêu cầu cố định.
- Đòi hỏi đội ngũ phải tự chủ và có kinh nghiệm.
- Phụ thuộc vào sự tham gia thường xuyên của Product Owner.

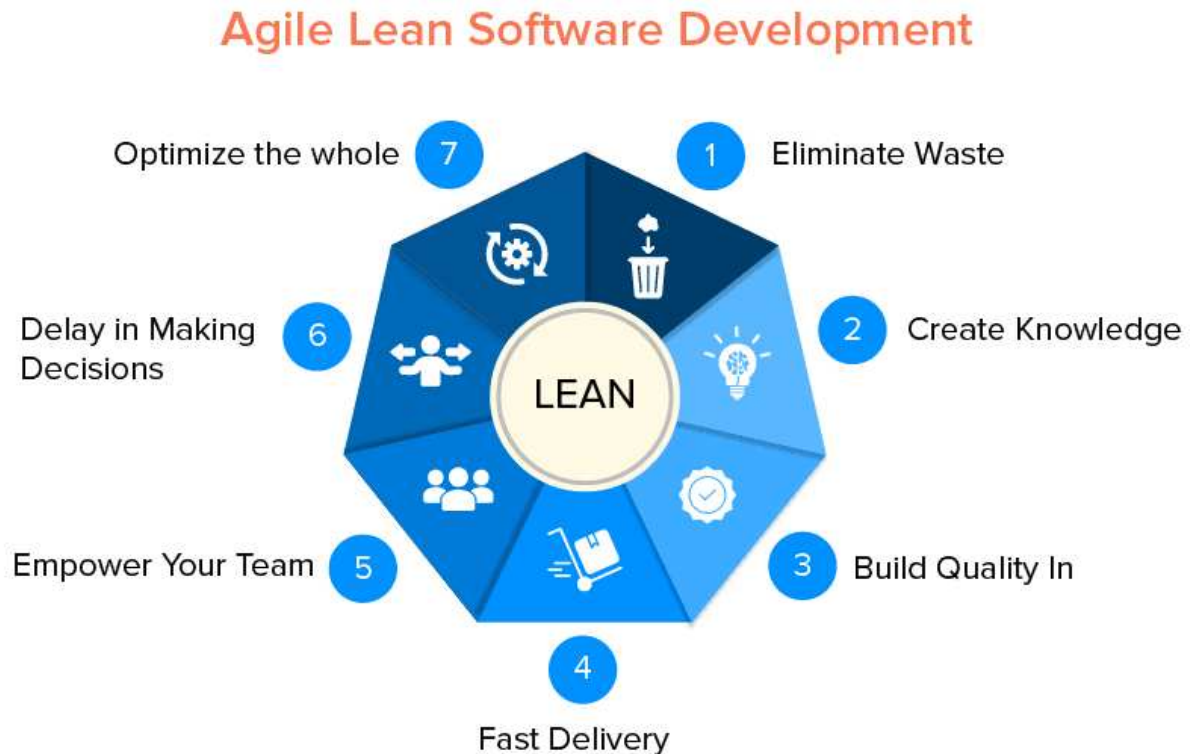
Khi nào nên sử dụng mô hình Scrum

- Khi yêu cầu dự án thay đổi thường xuyên.
- Khi cần phản hồi liên tục từ khách hàng.
- Khi đội ngũ có khả năng tự quản lý và hợp tác tốt.

2.1.3 Lean

Mô hình Lean là một triết lý và phương pháp quản lý tập trung vào việc tối ưu hóa giá trị cho khách hàng bằng cách giảm thiểu lãng phí trong các quy trình phát triển sản phẩm. Lean được lấy cảm hứng từ Hệ thống Sản xuất Toyota (Toyota Production System) và

được áp dụng phổ biến trong lĩnh vực phát triển phần mềm, được biết đến với tên gọi **Lean Software Development**.



Hình 1.4. Mô hình Lean

Đặc điểm chính của mô hình Lean:

1. **Giảm thiểu lãng phí (Eliminate Waste):** Loại bỏ các hoạt động không tạo ra giá trị cho khách hàng, như sản xuất dư thừa, chờ đợi, lỗi sai, hoặc quy trình thừa.
2. **Tăng giá trị cho khách hàng (Build Quality In):** Tập trung vào việc cung cấp đúng sản phẩm, đáp ứng đúng nhu cầu của khách hàng.
3. **Tối ưu hóa quy trình (Optimize the Whole):** Xây dựng quy trình làm việc đồng nhất và hiệu quả từ đầu đến cuối, tránh tối ưu hóa từng phần riêng lẻ.
4. **Phân phối nhanh chóng (Deliver Fast):** Cung cấp sản phẩm nhanh nhất có thể thông qua việc phân phối theo từng phần nhỏ (incremental delivery).
5. **Quyết định dựa trên thông tin (Decide as Late as Possible):** Chỉ ra quyết định khi đã có đủ thông tin cần thiết, tránh lãng phí do các quyết định sai lầm.
6. **Trao quyền cho đội ngũ (Empower the Team):** Khuyến khích đội ngũ tự quản lý, sáng tạo và đóng góp để tăng hiệu quả làm việc.

7. **Cải tiến liên tục (Continuous Improvement):** Không ngừng học hỏi và cải tiến quy trình để nâng cao chất lượng và hiệu suất.

Các bước trong mô hình Lean:

1. **Xác định giá trị (Identify Value):** Xác định những gì khách hàng thực sự cần và tập trung cung cấp các giá trị này.
2. **Xây dựng chuỗi giá trị (Map the Value Stream):** Phân tích và vạch ra toàn bộ quy trình để nhận diện và loại bỏ các hoạt động không cần thiết.
3. **Tạo luồng làm việc liên tục (Create Flow):** Sắp xếp lại quy trình làm việc để đảm bảo mọi bước diễn ra liên tục và không bị gián đoạn.
4. **Phân phối theo nhu cầu (Establish Pull):** Sản xuất hoặc phát triển dựa trên nhu cầu thực tế thay vì dự báo.
5. **Cải tiến liên tục (Seek Perfection):** Liên tục đánh giá và cải thiện quy trình để đạt hiệu quả tối đa.

Ưu điểm của mô hình Lean

- Giảm chi phí và thời gian phát triển.
- Tăng cường giá trị cho khách hàng.
- Quy trình linh hoạt, dễ thích ứng với thay đổi.
- Tăng hiệu quả làm việc của đội ngũ.

Nhược điểm của mô hình Lean

- Đòi hỏi sự thay đổi văn hóa và tư duy trong tổ chức.
- Khó thực hiện nếu không có sự phối hợp chặt chẽ giữa các nhóm.
- Không phù hợp với các dự án có yêu cầu cố định và ít thay đổi.

Khi nào nên sử dụng mô hình Lean

- Khi cần giảm lãng phí và tối ưu hóa quy trình.
- Khi yêu cầu của khách hàng thay đổi thường xuyên.
- Khi đội ngũ có khả năng làm việc tự quản và cải tiến liên tục.

Mô hình Lean là một cách tiếp cận hiệu quả để tối ưu hóa quy trình phát triển và mang lại giá trị cao nhất cho khách hàng, đặc biệt phù hợp với các dự án linh hoạt và yêu cầu thay đổi.

2.1 Ứng dụng các phương pháp quản lý dự án vào thực tiễn

Trong thực tế, việc quản lý dự án đòi hỏi sự kết hợp giữa lý thuyết và các yêu cầu cụ thể của từng dự án. Các mô hình quản lý dự án phổ biến (Waterfall, Agile, Lean, Scrum) đều có những ưu và nhược điểm riêng, và việc lựa chọn mô hình phù hợp cần dựa trên đặc điểm dự án, kỳ vọng của khách hàng, và khả năng của đội ngũ phát triển.

Nguyên tắc chung khi ứng dụng các phương pháp quản lý dự án

1. Hiểu rõ bản chất dự án:

Xác định rõ ràng phạm vi, mục tiêu và yêu cầu của dự án. Điều này giúp định hình cách tiếp cận và lựa chọn phương pháp phù hợp.

2. Đánh giá đặc điểm của nhóm làm việc:

Xem xét năng lực, kinh nghiệm và phong cách làm việc của đội ngũ phát triển.

3. Linh hoạt trong quản lý:

Kết hợp các đặc điểm của nhiều mô hình nếu cần thiết để tối ưu hóa quy trình.

Ví dụ ứng dụng thực tiễn:

- Dự án phát triển phần mềm sử dụng nội bộ với yêu cầu rõ ràng và thời gian cố định có thể phù hợp với **Waterfall**.
- Một ứng dụng khởi nghiệp cần thử nghiệm và thay đổi liên tục sẽ phù hợp với **Agile** hoặc **Scrum**.
- Khi mục tiêu chính là giảm thiểu lãng phí, **Lean** là lựa chọn tối ưu.

2.2.1. Phân tích yêu cầu dự án

Phân tích yêu cầu là bước nền tảng, đảm bảo rằng dự án có một hướng đi đúng đắn ngay từ đầu. Đây là quá trình thu thập, hiểu rõ, và phân loại các yêu cầu từ khách hàng hoặc các bên liên quan.

Các bước phân tích yêu cầu:

1. Thu thập yêu cầu:

Sử dụng các công cụ và phương pháp như phỏng vấn, khảo sát, workshop để nắm bắt yêu cầu từ khách hàng.

2. Xác định yêu cầu chức năng và phi chức năng:

- **Yêu cầu chức năng:** Những gì phần mềm cần thực hiện (tính năng cụ thể).
- **Yêu cầu phi chức năng:** Các yếu tố như hiệu suất, bảo mật, khả năng mở rộng.

3. Ưu tiên hóa yêu cầu:

Sử dụng các kỹ thuật như MoSCoW (Must have, Should have, Could have, Won't have) để phân loại mức độ quan trọng.

4. Tài liệu hóa yêu cầu:

Xây dựng tài liệu SRS (Software Requirements Specification) để làm cơ sở cho việc phát triển.

2.2.2. Lựa chọn phương pháp hợp lý

Dựa trên phân tích yêu cầu, việc lựa chọn phương pháp quản lý dự án phù hợp là một trong những yếu tố quyết định sự thành công của dự án.

Tiêu chí lựa chọn:

1. Tính ổn định của yêu cầu:

- Nếu yêu cầu ổn định, ít thay đổi: **Waterfall**.
- Nếu yêu cầu thay đổi thường xuyên: **Agile** hoặc **Scrum**.

2. Kích thước và độ phức tạp của dự án:

- Dự án lớn, nhiều thành phần: **Scrum** với các đội nhỏ phối hợp tốt.
- Dự án nhỏ, thời gian ngắn: **Lean** hoặc **Agile**.

3. Khách hàng và đội ngũ:

- Nếu khách hàng muốn tham gia sâu vào quá trình phát triển: **Agile**.

- Nếu khách hàng chỉ muốn kết quả cuối cùng: **Waterfall**.

Lời khuyên: Đừng cố gắng áp dụng một phương pháp duy nhất cho mọi dự án. Hãy điều chỉnh linh hoạt dựa trên thực tế.

2.2.3. Ứng dụng vào từng dự án phần mềm cụ thể

Mỗi dự án phần mềm có đặc điểm, phạm vi và yêu cầu khác nhau, vì vậy việc lựa chọn và ứng dụng phương pháp quản lý phù hợp là yếu tố quyết định thành công. Phần này sẽ phân tích chi tiết cách áp dụng từng phương pháp quản lý dự án vào các loại dự án cụ thể thông qua ví dụ minh họa.

1. Dự án phát triển hệ thống quản lý nhân sự, tài chính

Đặc điểm của dự án:

- Yêu cầu rõ ràng, ít thay đổi.
- Cần tích hợp nhiều chức năng liên quan đến nghiệp vụ nội bộ.
- Đòi hỏi tài liệu hóa chi tiết để dễ dàng bảo trì, nâng cấp.

Phương pháp áp dụng: Waterfall

Ứng dụng chi tiết:

1. Xác định yêu cầu:

- Thu thập tất cả các yêu cầu từ các phòng ban (nhân sự, kế toán, kinh doanh).
- Tài liệu hóa yêu cầu chi tiết trong SRS (Software Requirements Specification).

2. Thiết kế hệ thống:

- Thiết kế kiến trúc hệ thống tổng thể: các module như quản lý nhân sự, quản lý tài chính, quản lý kho.
- Lập sơ đồ ERD (Entity-Relationship Diagram) và các tài liệu thiết kế.

3. Lập trình và kiểm thử:

- Xây dựng từng module theo thứ tự ưu tiên.
- Kiểm thử từng phần để đảm bảo tính chính xác và đồng nhất.

4. Triển khai và bảo trì:

- Cài đặt hệ thống trong môi trường thực tế.
- Cung cấp tài liệu hướng dẫn sử dụng và hỗ trợ bảo trì.

2. Dự án xây dựng ứng dụng di động cho startup (ví dụ: ứng dụng đặt đồ ăn)

Đặc điểm của dự án:

- Yêu cầu thay đổi liên tục dựa trên phản hồi người dùng.
- Cần tốc độ phát triển nhanh để chiếm lĩnh thị trường.
- Yêu cầu tính năng đơn giản, dễ mở rộng.

Phương pháp áp dụng: Agile hoặc Scrum

Ứng dụng chi tiết:

1. Lập kế hoạch:

- Phân chia dự án thành các Sprint (2 tuần). Mỗi Sprint tập trung phát triển một nhóm tính năng như: đăng ký người dùng, tìm kiếm nhà hàng, đặt món ăn.
- Xác định Product Backlog: tất cả tính năng mong muốn.

2. Phát triển và kiểm thử:

- Đội phát triển làm việc chặt chẽ với khách hàng. Mỗi Sprint kết thúc bằng một phiên bản có thể sử dụng được.
- Ví dụ: Trong Sprint đầu tiên, nhóm xây dựng tính năng đăng nhập và tìm kiếm. Sprint tiếp theo, thêm tính năng thanh toán và theo dõi đơn hàng.

3. Phản hồi liên tục:

- Sau mỗi Sprint, trình bày sản phẩm cho khách hàng để lấy phản hồi.

- Điều chỉnh các tính năng hoặc thêm yêu cầu mới dựa trên phản hồi từ người dùng cuối.

4. Hoàn thiện:

- Sau nhiều Sprint, ứng dụng sẽ hoàn chỉnh với tất cả các tính năng cần thiết.
- Triển khai ứng dụng lên các nền tảng như App Store hoặc Google Play.

3. Dự án tối ưu hóa quy trình sản xuất (ví dụ: tự động hóa trong nhà máy sản xuất)

Đặc điểm của dự án:

- Mục tiêu chính là giảm thiểu lãng phí và tăng hiệu quả.
- Yêu cầu tính toán chính xác để đảm bảo không gián đoạn sản xuất.
- Cần cải tiến liên tục theo thời gian.

Phương pháp áp dụng: Lean

Ứng dụng chi tiết:

1. Xác định giá trị:

- Xác định các công đoạn trong sản xuất không mang lại giá trị (chẳng hạn thời gian chờ đợi, vận chuyển không cần thiết).
- Mục tiêu là loại bỏ những lãng phí này.

2. Xây dựng chuỗi giá trị:

- Vẽ sơ đồ chuỗi giá trị để phân tích từng bước trong quy trình sản xuất.
- Lên kế hoạch tự động hóa các bước như kiểm kê, đóng gói.

3. Tạo luồng làm việc liên tục:

- Điều chỉnh quy trình sao cho các bước sản xuất diễn ra liên tục, không bị gián đoạn.
- Ví dụ: Sử dụng băng chuyền tự động hóa để thay thế việc vận chuyển thủ công.

4. **Phân phối theo nhu cầu:** Áp dụng hệ thống **Just-In-Time (JIT)** để sản xuất chỉ khi có đơn đặt hàng, tránh tồn kho quá mức.

5. **Cải tiến liên tục:**

- Theo dõi hiệu suất bằng các chỉ số như thời gian hoàn thành, chi phí sản xuất.
- Liên tục điều chỉnh để đạt hiệu quả tối ưu.

4. Dự án xây dựng nền tảng thương mại điện tử lớn (ví dụ: website mua sắm trực tuyến)

Đặc điểm của dự án:

- Quy mô lớn, nhiều đội nhóm tham gia.
- Yêu cầu tích hợp nhiều tính năng như tìm kiếm, thanh toán, quản lý sản phẩm.
- Phải đảm bảo tính ổn định và bảo mật.

Phương pháp áp dụng: Kết hợp Scrum và Lean

Ứng dụng chi tiết:

1. **Phân chia đội nhóm theo Scrum:**

- Mỗi đội đảm nhận một phần của hệ thống (ví dụ: giao diện người dùng, hệ thống thanh toán, cơ sở dữ liệu).
- Mỗi đội hoạt động độc lập với Product Backlog riêng.

2. **Tối ưu hóa quy trình theo Lean:**

- Loại bỏ các công việc dư thừa, tập trung vào các tính năng cốt lõi trong giai đoạn đầu.
- Sử dụng mô hình MVP (Minimum Viable Product) để kiểm tra ý tưởng.

3. **Phát triển và thử nghiệm:**

- Mỗi đội trình bày sản phẩm sau các Sprint để đảm bảo tích hợp tốt.

- Kiểm tra liên tục để đảm bảo hiệu suất và bảo mật.

4. Hoàn thiện và triển khai:

- Sau khi các đội hoàn thành, hệ thống được tích hợp thành một sản phẩm hoàn chỉnh.
- Đảm bảo kiểm thử toàn diện trước khi triển khai.

Như vậy, có thể thấy rằng việc ứng dụng các phương pháp quản lý dự án cụ thể cần được điều chỉnh linh hoạt theo đặc điểm và yêu cầu của từng dự án. Sự thành công không chỉ nằm ở việc chọn đúng phương pháp mà còn ở khả năng áp dụng và phối hợp các phương pháp đó một cách hiệu quả. Sinh viên cần thực hành trên các tình huống thực tế để nắm vững cách ứng dụng trong môi trường làm việc sau này.

Câu hỏi ôn tập cuối chương

1. So sánh mô hình Waterfall với Agile trong quản lý dự án phần mềm.
2. Mô hình Lean được áp dụng như thế nào để giảm thiểu lãng phí trong dự án?
3. Lợi ích chính của việc sử dụng mô hình Scrum trong quản lý dự án là gì?
4. Khi nào nên chọn mô hình Waterfall thay vì Agile hoặc Scrum?
5. Làm thế nào để phân tích và ưu tiên hóa các yêu cầu trong quản lý dự án?
6. Các yếu tố chính cần cân nhắc khi lựa chọn một phương pháp quản lý dự án phù hợp?
7. Lợi ích và thách thức của việc áp dụng phương pháp Agile vào một dự án phần mềm phức tạp là gì?

Chương 3: Lập kế hoạch dự án

Nội dung chính của chương

Nhóm quy trình lập kế hoạch dự án bao gồm các quy trình được sử dụng để thiết lập và duy trì kế hoạch công việc nhằm đạt được các mục tiêu của dự án. Một dự án phần mềm có thể bao gồm nhiều loại kế hoạch, chẳng hạn như kế hoạch quản lý phạm vi, thời gian, chi phí, nguồn lực. Mỗi kế hoạch là một phương pháp tiến hành công việc, xác định và xử lý các vấn đề liên quan đến một "lĩnh vực kiến thức" cụ thể trong quản lý dự án. Chương này sẽ cung cấp những kiến thức và kỹ năng cần thiết để lập một kế hoạch phạm vi yêu cầu, xây dựng lịch trình, phân bổ nguồn lực và ngân sách hiệu quả cho dự án công nghệ thông tin.

Mục tiêu cần đạt của chương

- Sinh viên hiểu cách chia nhỏ yêu cầu và lập kế hoạch dự án chi tiết.
- Sinh viên hiểu cách phân bổ nguồn lực và ngân sách.

3.1. Phân chia yêu cầu phần mềm thành các nhiệm vụ cụ thể

3.1.1. Phân tích yêu cầu

Phân tích yêu cầu là quá trình đầu tiên trong việc chia nhỏ yêu cầu phần mềm. Nó giúp xác định các chức năng chính, làm rõ các mong muốn của khách hàng và tạo cơ sở để chia nhiệm vụ cụ thể, chi tiết.

Phân tích yêu cầu là quá trình tìm hiểu, làm rõ các yêu cầu của khách hàng và chuyển đổi chúng thành tài liệu chi tiết. Quá trình này tập trung vào việc:

- Thu thập thông tin: Tìm hiểu nhu cầu của khách hàng và các bên liên quan (stakeholders).
- Làm rõ yêu cầu: Đảm bảo các yêu cầu được hiểu rõ ràng, không mâu thuẫn hoặc mơ hồ.
- Chuyển đổi yêu cầu: Biến các yêu cầu từ khách hàng thành các tài liệu cụ thể, có thể áp dụng vào thiết kế và phát triển phần mềm.

Mục tiêu của phân tích yêu cầu nhằm đạt được các kết quả:

- **Xác định rõ yêu cầu chức năng (functional):** Các yêu cầu chức năng chính, cụ thể mà phần mềm phải thực hiện (ví dụ: tìm kiếm sách, xuất báo cáo mượn sách).

- **Xác định rõ yêu cầu phi chức năng (non-functional):** Các yêu cầu liên quan đến hiệu suất, bảo mật, giao diện (ví dụ: phần mềm phải hoạt động 24/7, hỗ trợ trên nhiều thiết bị).
- **Đảm bảo sự thống nhất giữa các bên liên quan:** Tất cả các bên (khách hàng, đội phát triển, tester) đều hiểu và đồng ý về yêu cầu.
- **Giúp dễ dàng quản lý và theo dõi tiến độ dự án**

Ví dụ: Khách hàng yêu cầu, “Tôi cần một phần mềm giúp quản lý sách trong thư viện.”
 Kết quả phân tích có thể thu được các yêu cầu chức năng: Tìm kiếm sách, quản lý mượn/trả sách; yêu cầu phi chức năng: Giao diện thân thiện, tốc độ tải trang dưới 3 giây.

Phương pháp phân tích yêu cầu

Thu thập thông tin yêu cầu:

- Tương tác với khách hàng để xác định rõ các mong muốn. Cần chuẩn bị câu hỏi kỹ càng, tập trung vào mục tiêu cụ thể. Ghi lại chi tiết cuộc trao đổi, làm rõ bất kỳ thông tin nào chưa rõ. Một số câu hỏi ví dụ có thể dùng để hỏi khách hàng như: “Bạn muốn hệ thống có những chức năng gì?”; “Ai sẽ là người sử dụng phần mềm này?”. **Lưu ý**, hạn chế ngôn ngữ kỹ thuật, tập trung vào nhu cầu thực tế của khách hàng.
- Ưu tiên hiểu đúng các yêu cầu trước khi bắt đầu lập kế hoạch.

Sử dụng các công cụ hỗ trợ để tài liệu hóa yêu cầu:

- **Sơ đồ Use Case:** Phân tích các chức năng chính. Quá trình tài liệu hóa yêu cầu có thể biểu diễn mối quan hệ giữa các tác nhân (actors) và các trường hợp sử dụng (use cases) hoặc sử dụng sơ đồ ERD (Entity-Relationship Diagram)
- **Bảng phân tích nhiệm vụ:** Là công cụ trực quan để lập danh sách các nhiệm vụ từ yêu cầu.

3.1.2. Xác định nhiệm vụ

Sau khi đã hoàn thành phân tích yêu cầu và làm rõ các mục tiêu dự án, bước tiếp theo trong quá trình lập kế hoạch là chuyển đổi các yêu cầu đó thành danh sách các nhiệm vụ cụ thể (tasks). Đây là một bước quan trọng nhằm tạo nền tảng vững chắc cho việc quản lý tiến độ, phân bổ nguồn lực và giám sát công việc.

Mỗi nhiệm vụ được xác định cần đáp ứng các tiêu chí sau:

- **Cụ thể:** Mô tả rõ ràng nội dung và mục tiêu của nhiệm vụ.
- **Rõ ràng:** Các bước thực hiện phải dễ hiểu, tránh mơ hồ.
- **Khả thi:** Đảm bảo nhiệm vụ có thể hoàn thành với các nguồn lực và thời gian sẵn có. Nhiệm vụ nên được thiết kế để hoàn thành trong một khoảng thời gian hợp lý, giúp quản lý tiến độ hiệu quả hơn.

Ví dụ: Với yêu cầu ***Hỗ trợ tìm kiếm sách trong thư viện***, có thể chia thành các nhiệm vụ cụ thể, gồm Thiết kế giao diện tìm kiếm sách, Lập trình chức năng tìm kiếm trong cơ sở dữ liệu, Kiểm thử chức năng tìm kiếm với các bộ dữ liệu khác nhau.

Tầm quan trọng của việc xác định nhiệm vụ

- **Quản lý tiến độ:** Chia nhỏ nhiệm vụ giúp dễ dàng theo dõi trạng thái công việc, đảm bảo dự án tiến triển đúng kế hoạch.
- **Phân bổ nguồn lực:** Việc xác định nhiệm vụ cụ thể giúp phân bổ đúng người, đúng kỹ năng vào các phần việc phù hợp.
- **Kiểm soát rủi ro:** Dễ dàng phát hiện các vấn đề tiềm tàng ở từng nhiệm vụ nhỏ và điều chỉnh kịp thời.
- **Tăng tính linh hoạt:** Danh sách nhiệm vụ có thể được cập nhật để phù hợp với các thay đổi trong yêu cầu dự án hoặc giải quyết các vấn đề phát sinh.

Lưu ý: Danh sách nhiệm vụ nên được cập nhật định kỳ, đặc biệt khi Yêu cầu dự án thay đổi, phát hiện vấn đề kỹ thuật mới trong quá trình triển khai hoặc phát sinh các ràng buộc mới về nguồn lực hoặc thời gian.

Bảng cấu trúc phân rã công việc (Work breakdown structure -WBS):

WBS là một phương pháp được sử dụng để ***chia các mục tiêu chính*** của dự án thành những ***nhiệm vụ nhỏ hơn*** nhằm đạt được mục tiêu chính.

Cấu trúc và cách xây dựng WBS

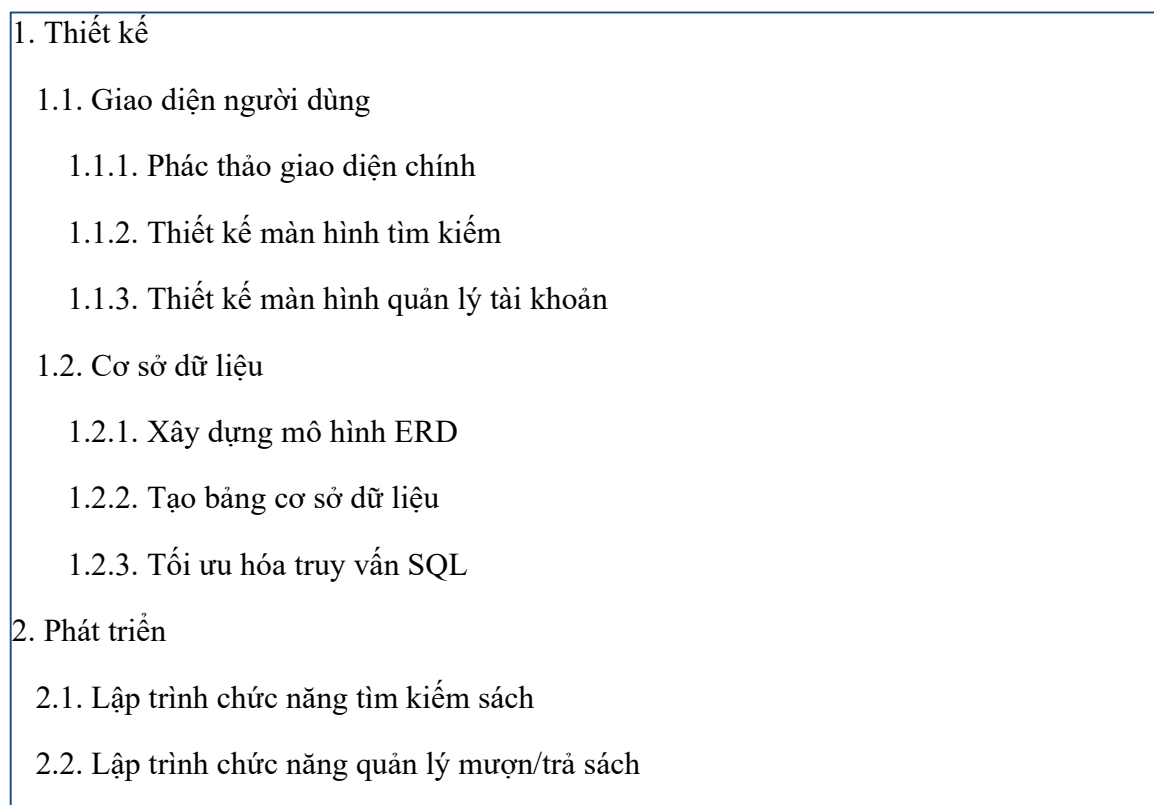
1. **Xác định các giai đoạn chính (Phases):** Giai đoạn là các bước lớn trong dự án, ví dụ: Thiết kế, Phát triển, Kiểm thử, Triển khai.
2. **Chia giai đoạn thành các nhóm công việc lớn (Deliverables):** Ví dụ: Giai đoạn "Thiết kế" có các nhóm công việc lớn như "Thiết kế giao diện người dùng", "Thiết kế cơ sở dữ liệu".
3. **Phân chia thành các nhiệm vụ nhỏ hơn (Tasks):** Chia nhóm công việc lớn thành các nhiệm vụ cụ thể, có thể đo lường được.

Ví dụ: "Thiết kế giao diện người dùng" có các nhiệm vụ nhỏ:

- Phác thảo giao diện chính.
- Thiết kế màn hình tìm kiếm.
- Thiết kế màn hình quản lý tài khoản.

4. Các dạng cấu trúc WBS:

- **Dạng sơ đồ:** WBS dưới dạng sơ đồ Grant hoặc sơ đồ tổ chức thể hiện mối quan hệ theo dạng cây, dễ quan sát.
- **Dạng hợp đồng:** chỉ gồm hai hoặc ba mức đầu tiên, thường dùng để theo dõi các công việc ở mức cao
- **Dạng danh sách:** Đánh số các nhiệm vụ theo hệ thống để dễ theo dõi.



Hình 3.1. Các nhiệm vụ theo dạng danh sách cho bài toán quản lý mượn trả sách

3.2. Các công cụ lập kế hoạch trong xây dựng lịch trình và phân bổ nguồn lực

Việc lập kế hoạch dự án được xác định ở 2 mức: Mức tổng quan và mức chi tiết (WBS).

Mức tổng quan: Tổng quan về kế hoạch dự án là sự mô tả dự án một cách tổng quát về tất cả những nội dung quan trọng nhất của dự án. Nó là tài liệu không thể thiếu khi muốn dự án được phê duyệt, là cơ sở quan trọng cho công tác quản lý và là căn cứ để lập kế hoạch chi tiết của dự án. Nội dung bản Tổng quan của kế hoạch phải gồm các phần sau

1. Kế hoạch đảm bảo chất lượng: Mô tả các chuẩn, các qui trình được sử dụng trong dự án.
2. Kế hoạch thẩm định: Mô tả các phương pháp, nguồn lực, lịch trình thẩm định hệ thống.
3. Kế hoạch bảo trì: Dự tính các yêu cầu về hệ thống, chi phí, nỗ lực cần thiết cho bảo trì.
4. Kế hoạch phát triển đội ngũ: Mô tả kỹ năng và kinh nghiệm của các thành viên trong nhóm dự án sẽ được phát triển như thế nào để quản lý và vận hành dự án sau này.

Mức chi tiết hay cấu trúc phân chia công việc: *Work Breakdown Structure-WBS*.

Giai đoạn này yêu cầu:

1. Xác định các hạng mục (công việc): chia dự án thành các nhóm công việc nhỏ đủ chi tiết để có thể lập kế hoạch, lên lịch biểu.
2. Phân bổ nguồn lực cho các công việc bao gồm con người, cơ sở vật chất, máy móc, tài chính và thời gian.

Khi lập kế hoạch dự án mức chi tiết, phải phân rã toàn bộ dự án thành các đơn vị công việc (WBS), lúc này cần có công cụ để lên lịch biểu và kiểm soát việc thực hiện kế hoạch, để đảm bảo dự án hoàn thành đúng thời hạn. Công cụ để lên lịch biểu và kiểm soát thời gian thường dùng là biểu đồ Gantt và hoặc PERT.

3.2.1. PERT

PERT (Program Evaluation and Review Technique) là một công cụ quản lý dự án dạng biểu đồ mạng giúp lập kế hoạch, phân tích và kiểm soát tiến độ dự án. PERT tập trung vào việc xác định và quản lý thời gian hoàn thành các nhiệm vụ, đặc biệt trong các dự án phức tạp có nhiều nhiệm vụ phụ thuộc lẫn nhau.

Mục đích của PERT

- Ước tính chính xác thời gian hoàn thành dự án.
- Xác định các nhiệm vụ quan trọng và thời gian dự phòng.
- Phân tích và đánh giá các yếu tố ảnh hưởng đến tiến độ dự án.

Các thành phần cơ bản

1. **Nhiệm vụ (Activity):** Công việc cụ thể cần thực hiện.
2. **Sự kiện (Event):** Thời điểm bắt đầu hoặc kết thúc một nhiệm vụ.

3. Thời gian ước tính:

- **Thời gian lạc quan (Optimistic Time - O):** Thời gian ngắn nhất để hoàn thành nhiệm vụ.
- **Thời gian bi quan (Pessimistic Time - P):** Thời gian dài nhất để hoàn thành nhiệm vụ.
- **Thời gian trung bình (có khả năng xảy ra nhất - Most Likely Time - M):** Thời gian thường gặp nhất.

Công thức tính thời gian kỳ vọng:

$$T_E = (O + 4M + P) / 6$$

Trong đó:

- T_E : Thời gian kỳ vọng của nhiệm vụ.
- O, M, P: Thời gian lạc quan, thời gian trung bình, và thời gian bi quan.

Cách xây dựng sơ đồ PERT

Bước 1: Liệt kê các nhiệm vụ và mối quan hệ phụ thuộc

- Xác định tất cả các nhiệm vụ cần thực hiện trong dự án.
- Xác định mối quan hệ giữa các nhiệm vụ (nhiệm vụ nào phải hoàn thành trước khi nhiệm vụ khác bắt đầu).

Nhiệm vụ	Bắt đầu	Kết thúc	Thời gian (T)
A	1	2	3 ngày
B	2	3	2 ngày
C	2	4	4 ngày
D	3	5	5 ngày
E	4	5	6 ngày

Bảng 3.1 Bảng phân chia công việc của dự án

Bước 2: Vẽ sơ đồ mạng PERT

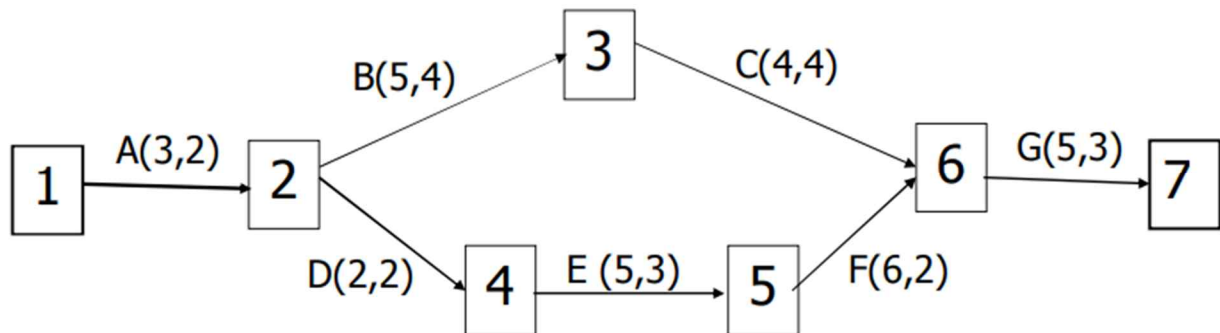
- Mỗi nhiệm vụ được biểu diễn bằng **một cung** (arrow).
- Mỗi sự kiện (bắt đầu/kết thúc nhiệm vụ) được biểu diễn bằng **một nút** (node).

Bước 3: Tính thời gian kỳ vọng cho từng nhiệm vụ

Áp dụng công thức để tính thời gian kỳ vọng cho từng nhiệm vụ.

Bước 4: Xác định đường găng (Critical Path)

- **Đường găng (Critical-Path: đường tới hạn, đường quyết định):** Là chuỗi các nhiệm vụ dài nhất từ điểm bắt đầu đến điểm kết thúc dự án.
- **Thời gian dự phòng (Slack Time):**
 $\text{Slack Time} = \text{Thời gian kết thúc muộn nhất} - \text{Thời gian kết thúc sớm nhất}$



Hình 3.2. Biểu đồ PERT

Ưu điểm và nhược điểm của PERT

1. Ưu điểm

- Giúp nhà quản lý theo dõi sát sao các yếu tố thời gian của dự án.
- Đường găng giúp xác định nhiệm vụ quan trọng cần ưu tiên.
- Hỗ trợ dự đoán thời gian và phát hiện các vấn đề tiềm ẩn.

2. Nhược điểm

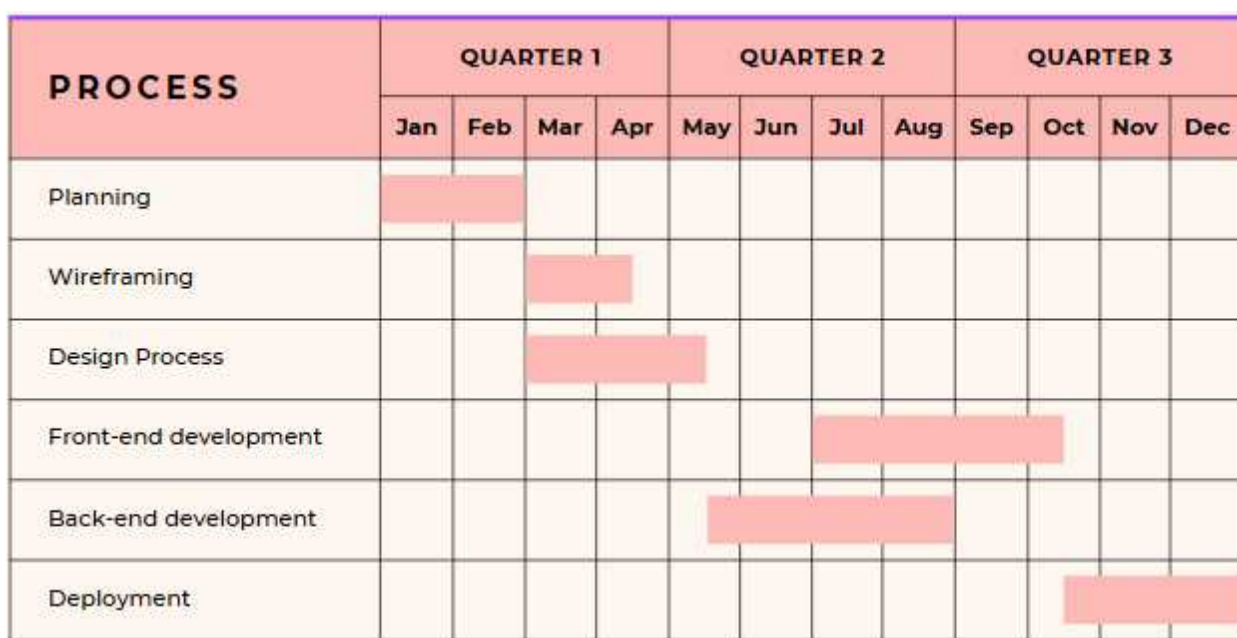
- **Phụ thuộc vào ước tính:** Độ chính xác phụ thuộc vào chất lượng của các thời gian ước tính.
- **Phức tạp:** Đối với các dự án lớn, sơ đồ PERT có thể trở nên phức tạp và khó quản lý.
- **Không tính đến chi phí:** PERT chỉ tập trung vào thời gian, không tích hợp yếu tố chi phí.

3.2.2. Gantt chart

Biểu đồ Gantt là một công cụ quản lý dự án trực quan, biểu diễn các nhiệm vụ của dự án dưới dạng biểu đồ thanh (bar chart). Mỗi thanh biểu diễn một nhiệm vụ, với chiều dài tương ứng với khoảng thời gian thực hiện nhiệm vụ đó.

Mục đích của Gantt Chart

- **Lập kế hoạch:** Xác định thời gian bắt đầu và kết thúc cho từng nhiệm vụ.
- **Theo dõi tiến độ:** Giám sát trạng thái hoàn thành của các nhiệm vụ.
- **Quản lý tài nguyên:** Đảm bảo phân bổ nguồn lực hợp lý.
- **Trực quan hóa:** Cung cấp cái nhìn tổng quan về tiến trình dự án.



Hình 3.3. Biểu đồ Grant

Cấu trúc của Gantt Chart

Cấu trúc của Gantt Chart gồm các thành phần chính sau đây:

- **Trục ngang (Timeline):** Biểu diễn thời gian (ngày, tuần, tháng).
- **Trục dọc (Tasks):** Liệt kê danh sách các nhiệm vụ cần thực hiện.
- **Thanh biểu đồ:** Thể hiện khoảng thời gian thực hiện từng nhiệm vụ.
- **Mốc thời gian (Milestones):** Các sự kiện hoặc điểm mốc quan trọng trong dự án.
- **Phụ thuộc giữa các nhiệm vụ:** Các đường nối thể hiện mối quan hệ giữa các nhiệm vụ.

Ưu điểm và nhược điểm của Gantt Chart

1. Ưu điểm

- Dễ hiểu: Cung cấp cái nhìn trực quan về tiến trình dự án.
- Theo dõi dễ dàng: Xem nhanh trạng thái của từng nhiệm vụ.
- Xác định phụ thuộc: Hiểu rõ mối quan hệ giữa các nhiệm vụ.

2. Nhược điểm

- Khó cập nhật: Khi dự án phức tạp hoặc có nhiều thay đổi.
- Không hiển thị chi tiết: Không thể hiện rõ ràng các yếu tố như rủi ro hoặc chi phí.
- Khả năng mở rộng hạn chế: Biểu đồ có thể trở nên khó quản lý với các dự án lớn.

3.3. Quản lý ngân sách dự án

Ngân sách dự án là một yếu tố quan trọng trong quản lý dự án, giúp xác định và kiểm soát chi phí để đảm bảo dự án được hoàn thành trong phạm vi tài chính đã đề ra. Việc dự toán và điều chỉnh ngân sách đúng cách không chỉ hỗ trợ hoàn thành dự án đúng tiến độ mà còn giảm thiểu rủi ro tài chính.

3.3.1. Dự toán ngân sách

Chi phí hay ngân sách là nguồn lực được tiêu thụ hoặc sử dụng để đạt được mục đích cụ thể. Chi phí thường được thể hiện là số tiền phải trả để có được một hàng hóa hoặc dịch vụ. Vì các dự án tiêu tốn tiền bạc và các nguồn lực tổ chức khác, nên điều quan trọng là người quản lý dự án phải hiểu cách quản lý ngân sách dự án. Quản lý chi phí dự án là quá trình ước tính, phân bổ và kiểm soát chi phí trong một dự án. Giúp dự đoán các chi phí sắp tới để giảm nguy cơ vượt ngân sách. Chi phí dự kiến được tính trong giai đoạn lập kế hoạch của dự án và phải được phê duyệt trước khi bắt đầu dự án. Khi kế hoạch dự án được thực hiện, các chi phí được ghi lại và theo dõi để đảm bảo chi phí nằm trong kế hoạch. Khi dự án hoàn thành, chi phí dự đoán được so sánh với chi phí thực tế, cung cấp chuẩn cho kế hoạch quản lý chi phí và ngân sách dự án trong tương lai.

Dự toán ngân sách là quá trình ước tính chi phí cần thiết để thực hiện các nhiệm vụ và hoàn thành dự án. Ngân sách bao gồm tất cả các khoản chi phí trực tiếp và gián tiếp liên quan đến dự án.

Mục tiêu dự toán ngân sách:

- Xác định chi phí cần thiết cho từng nhiệm vụ trong dự án.
- Phân bổ ngân sách hợp lý để tối ưu hóa nguồn lực.
- Giảm thiểu rủi ro vượt chi phí dự án.

Các phương pháp dự toán ngân sách

Dự toán ngân sách là quá trình ước tính chi phí cần thiết để thực hiện các nhiệm vụ và hoàn thành dự án trong phạm vi tài chính được phê duyệt. Ước tính chi phí là một đầu ra quan trọng trong quản lý chi phí dự án, vì nó cung cấp cơ sở để lập ngân sách và kiểm soát chi tiêu trong suốt quá trình thực hiện dự án. Đây là quá trình xác định các khoản chi phí cần thiết để hoàn thành tất cả các hoạt động trong dự án, từ các nhiệm vụ cụ thể đến các mục tiêu tổng thể. Một ước tính chi phí chính xác không chỉ giúp dự án duy trì trong phạm vi ngân sách mà còn giảm thiểu các rủi ro tài chính tiềm tàng. Ước tính chi phí là một bước quan trọng trong quản lý dự án, giúp đảm bảo các nguồn lực được phân bổ hợp lý, các rủi ro tài chính được giảm thiểu, và mục tiêu dự án được hoàn thành đúng tiến độ.

Để ước tính chi phí, có ba phương pháp chính được sử dụng, mỗi phương pháp phù hợp với các loại dự án và điều kiện khác nhau:

1. Dự toán từ dưới lên (Bottom-up Estimation):

Phương pháp này bắt đầu bằng cách chia nhỏ dự án thành các nhiệm vụ cụ thể nhất có thể, sau đó ước tính chi phí cho từng nhiệm vụ và cộng dồn để ra tổng chi phí. Cách tiếp cận này yêu cầu hiểu biết chi tiết về các công việc cụ thể trong dự án và rất chính xác khi được thực hiện đúng cách. Tuy nhiên, nó có thể tốn nhiều thời gian và công sức do mức độ chi tiết cần thiết.

Ví dụ: Nhiệm vụ "Thiết kế giao diện người dùng" có thể bao gồm chi phí lập trình (200 giờ x 50.000 VNĐ/giờ) và kiểm thử (50 giờ x 35.000 VNĐ/giờ), tổng chi phí là 11.750.000 VNĐ.

2. Dự toán từ trên xuống (Top-down Estimation):

Phương pháp này sử dụng thông tin từ các dự án tương tự trước đây để ước tính tổng chi phí cho toàn bộ dự án. Đây là cách tiếp cận nhanh chóng và hữu ích cho các dự án đã có kinh nghiệm hoặc dữ liệu lịch sử liên quan. Tuy nhiên, nếu dự án có các đặc điểm độc đáo hoặc không rõ ràng, dự toán có thể thiếu chính xác.

Ví dụ: Dựa trên dữ liệu lịch sử, chi phí cho một dự án phần mềm tương tự trước đây là 250.000.000 VNĐ, và con số này được sử dụng làm ước tính ban đầu.

3. Phương pháp tham số (Parametric Estimation):

Đây là phương pháp sử dụng các công thức toán học và dữ liệu lịch sử để ước tính chi phí dựa trên các đặc điểm cụ thể của dự án. Phương pháp này đặc biệt phù hợp cho các dự án lớn, phức tạp hoặc khi có sẵn dữ liệu lịch sử chính xác.

- *Ví dụ:* Để phát triển phần mềm, có thể áp dụng công thức: chi phí = 100.000 VNĐ/giờ x 200 giờ, tổng chi phí là 20.000.000 VNĐ.

Mỗi phương pháp có ưu và nhược điểm riêng, và lựa chọn phương pháp phù hợp sẽ phụ thuộc vào độ chi tiết của dự án, thời gian có sẵn, cũng như mức độ chính xác mong muốn trong dự toán ngân sách.

Cấu trúc ngân sách

Ngân sách dự án bao gồm nhiều khoản chi phí cần được tính toán và phân bổ hợp lý để đảm bảo dự án có thể hoàn thành đúng tiến độ, trong phạm vi tài chính, và đạt được các mục tiêu đã đề ra. Ngân sách dự án thường bao gồm các khoản chi phí:

- **Chi phí nhân sự:** Chi phí nhân sự là khoản tiền dành để trả lương, thưởng, và các chế độ phúc lợi khác cho nhân viên tham gia vào dự án.
- **Chi phí thiết bị:** Chi phí thiết bị là khoản tiền chi trả để mua sắm hoặc thuê các công cụ, máy móc cần thiết cho việc thực hiện dự án.
- **Chi phí vận hành:** Chi phí vận hành bao gồm các khoản liên quan đến hoạt động hàng ngày của dự án, từ vận hành văn phòng đến các tiện ích cần thiết.
- **Chi phí dự phòng:** Chi phí dự phòng là khoản quỹ được dành riêng để đối phó với các rủi ro phát sinh trong quá trình thực hiện dự án. Đây không phải là chi phí cụ thể, nhưng cần thiết để đảm bảo dự án không bị gián đoạn khi gặp sự cố. Các rủi ro này có thể là: rủi ro kỹ thuật gồm các vấn đề như lỗi phần mềm, thay đổi yêu cầu kỹ thuật; chi phí cho rủi ro thời gian như chậm tiến độ do thiếu nhân sự hoặc sự cố không lường trước; rủi ro khi có các thay đổi yêu cầu từ phía khách hàng yêu cầu thêm tính năng hoặc thay đổi thiết kế.

Kiểm soát chi phí là một phần quan trọng trong quản lý dự án, giúp đảm bảo rằng các chi tiêu thực tế luôn nằm trong giới hạn đã được phê duyệt. Điều này bao gồm việc so sánh chi phí thực tế với ước tính ban đầu, theo dõi các chỉ số hiệu suất chi phí, và thực hiện các hành động điều chỉnh khi cần thiết.

Một kế hoạch quản lý chi phí hiệu quả không chỉ giúp kiểm soát chi tiêu mà còn cung cấp thông tin quan trọng để ra quyết định, đảm bảo dự án hoàn thành đúng tiến độ và đạt được các mục tiêu đã đề ra.

3.3.2. Điều chỉnh ngân sách khi cần thiết

Điều chỉnh ngân sách là quá trình thay đổi phân bổ tài chính để phù hợp với các yêu cầu mới, thay đổi trong tiến độ hoặc phát sinh rủi ro.

Trong quá trình thực hiện dự án, ngân sách ban đầu được xây dựng dựa trên các giả định về yêu cầu, chi phí và rủi ro. Tuy nhiên, thực tế triển khai thường có những thay đổi bất ngờ đòi hỏi cần phải điều chỉnh ngân sách. Dưới đây là ba trường hợp phổ biến yêu cầu sự điều chỉnh ngân sách trong quản lý dự án.

Khi nào cần điều chỉnh ngân sách?

Thông thường, có ba trường hợp cần điều chỉnh ngân sách cho dự án.

i. Khi có yêu cầu thay đổi từ khách hàng

Yêu cầu thay đổi xảy ra khi khách hàng hoặc các bên liên quan đề nghị thêm hoặc sửa đổi các tính năng trong phạm vi dự án. Những thay đổi này có thể làm tăng khối lượng công việc, yêu cầu thêm nguồn lực hoặc kéo dài thời gian thực hiện.

Ví dụ:

- Khách hàng yêu cầu bổ sung một tính năng mới vào phần mềm, như thêm module báo cáo chi tiết cho phần mềm quản lý nhân sự.
- Yêu cầu nâng cấp giao diện người dùng để tối ưu hóa trải nghiệm người dùng.

Tác động khi có yêu cầu thay đổi từ khách hàng:

- Chi phí lao động tăng do cần thêm thời gian hoặc nhân lực để thực hiện yêu cầu.

- Chi phí thiết bị hoặc phần mềm tăng nếu tính năng mới đòi hỏi công cụ hoặc công nghệ bổ sung.
- Dự án có thể phải kéo dài tiến độ, dẫn đến tăng chi phí vận hành và tài nguyên.

Cách xử lý:

- Xác định chi phí bổ sung cho yêu cầu thay đổi.
- Đàm phán với khách hàng về việc tăng ngân sách hoặc cắt giảm các tính năng ít quan trọng để cân đối.

ii. Khi chi phí tăng ngoài dự kiến

Chi phí tăng là tình huống phổ biến khi giá nguyên vật liệu, lao động hoặc dịch vụ liên quan thay đổi so với dự toán ban đầu. Sự biến động của thị trường hoặc các yếu tố ngoại cảnh như lạm phát cũng có thể gây ra sự gia tăng này.

Ví dụ:

- Giá thuê máy chủ tăng do nhu cầu cao trong thị trường công nghệ thông tin.
- Mức lương nhân viên cần được điều chỉnh vì thị trường lao động thay đổi.

Tác động khi chi phí tăng ngoài dự kiến:

- Ngân sách không còn đủ để chi trả cho các hoạt động dự án đã được lập kế hoạch.
- Phát sinh áp lực phải cắt giảm chi phí ở các hạng mục khác hoặc tăng tổng ngân sách.

Cách xử lý:

- Phân tích các khoản chi phí tăng để xác định mức độ ảnh hưởng.
- Cân nhắc sử dụng chi phí dự phòng hoặc thương lượng với các nhà cung cấp để giảm chi phí.
- Cập nhật ngân sách với sự đồng thuận của các bên liên quan.

iii. Khi có rủi ro phát sinh

Rủi ro phát sinh trong quá trình thực hiện dự án thường xuất hiện dưới nhiều hình thức khác nhau, bao gồm các vấn đề kỹ thuật phức tạp, các sự cố không lường trước được như

lỗi thiết bị, mất dữ liệu, hoặc thay đổi yêu cầu từ phía khách hàng. Bên cạnh đó, tình trạng chậm tiến độ có thể xảy ra ở một hoặc nhiều giai đoạn của dự án do những nguyên nhân như thiếu hụt nguồn lực, sự phối hợp không hiệu quả giữa các nhóm, hoặc tác động từ các yếu tố bên ngoài như biến động thị trường hay thời tiết. Những rủi ro này không chỉ ảnh hưởng đến phạm vi công việc, khiến kế hoạch ban đầu phải điều chỉnh, mà còn tạo áp lực đáng kể lên ngân sách dự án, làm tăng chi phí phát sinh và gây khó khăn trong việc duy trì nguồn lực tài chính ổn định.

Ví dụ:

- Một tính năng chính không hoạt động như dự kiến, dẫn đến phải sửa chữa hoặc thiết kế lại.
- Chậm trễ trong việc giao hàng thiết bị hoặc tài nguyên cần thiết cho dự án.
- Rủi ro pháp lý hoặc thay đổi quy định ảnh hưởng đến cách triển khai dự án.

Tác động khi có rủi ro phát sinh:

Rủi ro phát sinh trong dự án có thể gây ra nhiều hậu quả nghiêm trọng, ảnh hưởng trực tiếp đến tiến độ, phạm vi công việc và chi phí. Những tác động thường gặp bao gồm:

- Chi phí sửa chữa hoặc thay thế các giải pháp không thành công.
- Tăng chi phí lao động do phải làm thêm giờ hoặc thuê chuyên gia bên ngoài.
- Tăng chi phí vận hành do kéo dài tiến độ.

Cách xử lý:

- Kích hoạt ngân sách dự phòng để xử lý các chi phí phát sinh.
- Rà soát kế hoạch và ưu tiên các nhiệm vụ quan trọng để giảm thiểu tác động.
- Báo cáo và trao đổi với các bên liên quan để xác định phương án xử lý tối ưu.

Câu hỏi ôn tập cuối chương

1. Quy trình lập kế hoạch dự án có vai trò gì trong việc quản lý tiến độ và nguồn lực của dự án?
2. Phân tích yêu cầu là gì, và các bước chính trong phân tích yêu cầu bao gồm những gì?
3. Yêu cầu chức năng và yêu cầu phi chức năng khác nhau như thế nào? Hãy đưa ra ví dụ.
4. Danh sách nhiệm vụ cần đảm bảo các tiêu chí nào để quản lý tiến độ hiệu quả?
5. Công cụ PERT được sử dụng trong lập kế hoạch dự án nhằm mục đích gì?

6. Đường găng (Critical Path) trong sơ đồ PERT có ý nghĩa gì đối với quản lý tiến độ dự án?
7. Biểu đồ Gantt giúp ích như thế nào trong việc theo dõi tiến độ và quản lý tài nguyên dự án?
8. Ngân sách dự án bao gồm những khoản chi phí nào? Hãy liệt kê ít nhất ba loại.
9. Sự khác biệt giữa phương pháp dự toán ngân sách từ trên xuống và từ dưới lên là gì?
10. Khi nào cần phải điều chỉnh ngân sách dự án, và điều chỉnh ngân sách có thể xử lý những tình huống nào?

Chương 4: Quản lý rủi ro và chất lượng dự án

Nội dung chính của chương

Chương này tập trung trình bày về quản lý rủi ro và kiểm soát chất lượng trong dự án phần mềm, bắt đầu với việc định nghĩa và nêu vai trò của quản lý rủi ro, đồng thời giới thiệu các loại rủi ro phổ biến như kỹ thuật, tài chính và nhân sự. Tiếp theo, chương đề cập đến quy trình quản lý rủi ro qua các bước nhận diện, phân tích và ưu tiên xử lý rủi ro, kết hợp sử dụng ma trận rủi ro và các kỹ thuật phòng ngừa hiệu quả. Về kiểm soát chất lượng, chương làm rõ vai trò của kiểm thử trong việc đảm bảo phần mềm đáp ứng tiêu chuẩn, so sánh ưu và nhược điểm của kiểm thử thủ công và kiểm thử tự động.

Mục tiêu cần đạt của chương

- Hiểu quy trình quản lý rủi ro trong dự án phần mềm.
- Biết cách nhận diện, phân tích và quản lý rủi ro dự án.
- Sử dụng các công cụ kiểm tra chất lượng, bao gồm kiểm thử thủ công và tự động.

4.1. Quy trình quản lý rủi ro

Trong quá trình thực hiện dự án, các vấn đề phát sinh có thể ảnh hưởng đến tiến độ và kết quả của dự án, đòi hỏi người quản lý phải có các biện pháp ứng phó để giảm thiểu tác động tiêu cực và tối đa hóa tác động tích cực. Những vấn đề này được gọi là rủi ro. *Rủi ro là những sự kiện hoặc điều kiện có thể xảy ra bất ngờ và ảnh hưởng đến dự án theo cả chiều hướng tích cực và tiêu cực.* Rủi ro tích cực, mặc dù hiếm gặp, vẫn cần được quản lý để tận dụng các cơ hội mang lại lợi ích cho dự án. Đối với rủi ro tiêu cực, quá trình quản lý rủi ro cần tập trung vào việc giảm thiểu tác động tiêu cực của chúng đối với dự án.

Quản lý rủi ro trong dự án là một sự kết hợp giữa nghệ thuật và khoa học. Việc nhận diện và ứng phó với rủi ro trong suốt quá trình thực hiện dự án là những mục tiêu quan trọng hàng đầu của dự án. Mặc dù quản lý rủi ro thường bị bỏ qua, nhưng nó lại đóng vai trò quan trọng trong việc nâng cao khả năng thành công của dự án, giúp lựa chọn những dự án phù hợp, xác định rõ phạm vi và phát triển các ước tính thực tế.

Quản lý rủi ro khác với quản lý dự án ở chỗ: quản lý rủi ro là công việc đặc thù, được áp dụng riêng cho từng dự án cụ thể và mang tính phòng ngừa, nhằm giảm thiểu hoặc tận dụng các yếu tố rủi ro. Trong khi đó, quản lý dự án là công việc chung, được thiết kế cho tất cả các dự án và mang tính phản ứng, điều chỉnh dựa trên thực tế diễn ra trong suốt quá trình triển khai dự án.

Kế hoạch quản lý rủi ro là quá trình xác định các hành động cần thiết để quản lý và xử lý rủi ro trong một dự án. Kết quả của quá trình này là một kế hoạch chi tiết về cách thức quản lý các rủi ro có thể phát sinh. Kế hoạch quản lý rủi ro sẽ ghi lại các thủ tục và phương pháp cần thiết để kiểm soát và ứng phó với các rủi ro trong suốt vòng đời của dự án. Để xây dựng kế hoạch này, người quản lý dự án và đội ngũ dự án cần phải tham khảo các tài liệu liên quan đến dự án, các tài liệu dự án trước đây, bài học từ các dự án đã thực hiện và tổ chức các cuộc họp cần thiết để xác định các rủi ro mà dự án có thể gặp phải. Trong kế hoạch quản lý rủi ro, các vai trò và trách nhiệm của các bên liên quan cần được xác định rõ ràng, cùng với các kế hoạch dự phòng, ước tính thay đổi và phân loại các rủi ro. Mức độ chi tiết của kế hoạch quản lý rủi ro sẽ tùy thuộc vào yêu cầu và quy mô của dự án. Một kế hoạch quản lý rủi ro thường bao gồm một số thành phần cơ bản sau:

- **Phân loại rủi ro trong dự án:** là quá trình xác định và phân nhóm các loại rủi ro có thể phát sinh trong suốt quá trình triển khai dự án. Những rủi ro này có thể xuất hiện từ nhiều nguồn khác nhau và ảnh hưởng đến các khía cạnh khác nhau của dự án. Các rủi ro có thể được phân loại theo tính chất như kỹ thuật, tổ chức, hay theo sự tác động của các bên liên quan.

- **Rủi ro kỹ thuật:** thường liên quan đến các yếu tố như công nghệ, phần mềm, phần cứng, hoặc các quy trình kỹ thuật trong dự án. Đây là những yếu tố có thể gây ra sự cố trong việc phát triển hoặc triển khai sản phẩm, dịch vụ mà dự án hướng tới.
- **Rủi ro tổ chức:** bao gồm các vấn đề phát sinh trong quá trình quản lý dự án, như vấn đề về tài chính, nhân sự, hoặc quản lý các nguồn lực. Đây là những yếu tố có thể ảnh hưởng đến việc thực hiện các kế hoạch và đạt được mục tiêu của dự án.
- **Rủi ro từ các bên liên quan:** là những rủi ro xuất phát từ các tác động bên ngoài hoặc sự thay đổi trong yêu cầu, sự hỗ trợ hoặc can thiệp từ các đối tác, khách hàng, nhà cung cấp hoặc các bên có liên quan khác. Những rủi ro này có thể gây ảnh hưởng đến tiến độ, ngân sách hoặc chất lượng của dự án.

- **Bảng phân loại rủi ro:** là công cụ hữu ích giúp liệt kê và phân loại tất cả các rủi ro tiềm ẩn mà dự án có thể gặp phải. Bảng này không chỉ giúp xác định các rủi ro có thể xảy ra mà còn giúp phân biệt các rủi ro cụ thể cho từng loại dự án, từ đó hỗ trợ quá trình đánh giá và quản lý rủi ro hiệu quả hơn. Trong bảng này, các rủi ro thường được ghi nhận với thông tin về nguyên nhân, mức độ nghiêm trọng, và khả năng xảy ra.

- **Khả năng xảy ra và tác động:** là hai yếu tố quan trọng mà người quản lý dự án cần phải xem xét khi đánh giá rủi ro. Đầu tiên, khả năng xảy ra của mỗi rủi ro cần được xác

định, thường là dưới dạng xác suất. Một số rủi ro có khả năng xảy ra rất cao, trong khi một số khác có khả năng xảy ra thấp hơn nhưng lại có thể gây hậu quả nghiêm trọng. Sau khi xác định được khả năng xảy ra, người quản lý dự án cần đánh giá tác động của mỗi rủi ro đối với dự án nếu nó thực sự xảy ra. Tác động này có thể ảnh hưởng đến tiến độ, chi phí, chất lượng, hoặc thậm chí là sự thành công của dự án. Việc đánh giá khả năng xảy ra và tác động của rủi ro giúp người quản lý dự án quyết định mức độ ưu tiên và các biện pháp ứng phó phù hợp với từng loại rủi ro.

Trong quá trình lập kế hoạch rủi ro, cần bao gồm thêm các kế hoạch dự phòng, kế hoạch rút lui và quỹ dự phòng:

- **Kế hoạch dự phòng** (đối phó với các bất ngờ) là những hành động đã được xác định trước mà các thành viên trong dự án sẽ thực hiện nếu một sự kiện rủi ro xảy ra. Kế hoạch này giúp dự án có sự chuẩn bị đối phó khi những tình huống không lường trước xuất hiện.
- **Kế hoạch rút lui** được triển khai đối với những rủi ro có tác động lớn đến các mục tiêu và yêu cầu của dự án. Đây là những biện pháp mà đội ngũ dự án cần thực hiện khi các rủi ro này đe dọa đến khả năng hoàn thành dự án theo kế hoạch.
- **Quỹ dự phòng** (để xử lý những bất ngờ) hay tiền trợ cấp là khoản tiền dự trữ do nhà tài trợ giữ, có thể được sử dụng để giảm thiểu chi phí phát sinh hoặc điều chỉnh lịch trình nếu có sự thay đổi về phạm vi hoặc chất lượng của dự án.

4.1.1. Nhận diện rủi ro

Bước đầu tiên trong quản lý rủi ro là xác định các rủi ro tiềm ẩn đối với dự án. Bước này thường đánh giá rủi ro dựa vào một số nguồn rủi ro chính đối với một dự án phần mềm như chi phí, thời gian, lịch trình, chất lượng, nguồn nhân lực, ước tính công việc, sản phẩm dự án, bên liên quan, cơ sở dữ liệu tổ chức, v.v. Trên thực tế, những gì được coi là rủi ro phụ thuộc vào tổ chức. Đối với cùng một vấn đề, một tổ chức có thể coi đó là rủi ro trong khi một tổ chức khác có thể không coi đó là rủi ro.

Rủi ro là một sự kiện hoặc trạng thái không chắc chắn, nếu xảy ra sẽ có tác động tiêu cực (trong hầu hết các trường hợp) hoặc tích cực (trong những trường hợp rất hiếm) đến mục tiêu của dự án.

Đặc điểm của rủi ro trong một dự án bao gồm:

- Mức độ không chắc chắn, được thể hiện dưới dạng xác suất có giá trị trong khoảng từ 0 đến 1.
- Khả năng quản lý rủi ro, tức là khả năng thực hiện các hành động để kiểm soát và giảm thiểu rủi ro.

- Hậu quả mất mát có thể xảy ra, chẳng hạn như tổn thất tài chính, ảnh hưởng đến sinh mạng, hoặc uy tín của một tổ chức, công ty...
- Mức độ rủi ro được đo lường bằng cách tính tích của xác suất xảy ra rủi ro và hậu quả mất mát tiềm năng.

Nhận biết rủi ro là một quy trình quan trọng trong việc phát hiện và nhận diện các yếu tố có thể gây ảnh hưởng tiêu cực hoặc tạo ra cơ hội trong dự án. Quy trình này tập trung vào việc tìm ra những yếu tố tiềm tàng, đặc biệt là những yếu tố không được dự đoán từ trước nhưng có thể xuất hiện trong suốt quá trình triển khai dự án. Nhận biết rủi ro giúp các nhà quản lý và nhóm dự án hiểu rõ hơn về các tình huống có thể xảy ra, từ đó chuẩn bị những biện pháp phòng ngừa và ứng phó phù hợp. Các công cụ và kỹ thuật nhận biết rủi ro đóng vai trò quan trọng trong việc phát hiện các yếu tố này và giúp đội ngũ dự án có thể đánh giá và xử lý các rủi ro tiềm ẩn.

Một số công cụ và kỹ thuật phổ biến trong việc nhận diện rủi ro bao gồm:

- **Phát huy trí tuệ dân chủ (Brainstorming):** Đây là kỹ thuật khuyến khích sự tham gia của tất cả các thành viên trong nhóm để đưa ra nhiều ý tưởng và quan điểm khác nhau về các nguy cơ hoặc cơ hội tiềm ẩn trong dự án. Quá trình brainstorming giúp tạo ra một danh sách rộng lớn các rủi ro, từ đó có thể phân tích và lựa chọn những yếu tố quan trọng nhất để tập trung vào.
- **Kỹ thuật Delphi:** Đây là một phương pháp thu thập ý kiến từ một nhóm chuyên gia qua các vòng thảo luận ẩn danh. Mỗi chuyên gia sẽ đưa ra nhận xét hoặc dự đoán về các rủi ro có thể xảy ra, và sau đó nhóm sẽ tổng hợp lại các ý kiến này để đạt được sự đồng thuận về các yếu tố rủi ro quan trọng nhất. Kỹ thuật Delphi giúp giảm thiểu sự thiên lệch và mang lại cái nhìn toàn diện từ nhiều chuyên gia khác nhau.
- **Phỏng vấn (Interviewing):** Phỏng vấn là một phương pháp thu thập thông tin từ các cá nhân hoặc nhóm có kinh nghiệm và hiểu biết sâu về dự án. Qua việc trao đổi trực tiếp, người quản lý dự án có thể khám phá những mối nguy tiềm tàng, các yếu tố ảnh hưởng đến dự án mà có thể chưa được nhận diện qua các phương pháp khác.
- **Phân tích SWOT (Strengths – Weaknesses – Opportunities – Threats):** Đây là một công cụ hữu ích trong việc phân tích các yếu tố bên ngoài và bên trong có thể tác động đến dự án. Phân tích SWOT giúp xác định các điểm mạnh và yếu của dự án, cũng như các cơ hội và mối đe dọa từ môi trường bên ngoài. Việc hiểu rõ các yếu tố này giúp nhóm dự án có thể đưa ra các chiến lược ứng phó thích hợp để giảm thiểu rủi ro và tận dụng cơ hội.

Đầu ra của quá trình nhận diện rủi ro là báo cáo rủi ro, bao gồm danh sách các rủi ro đã xác định cùng với các thông tin liên quan, như nguồn gốc, sự kiện rủi ro và các tác động tích cực hoặc tiêu cực đến dự án.

4.1.2. Phân tích rủi ro

Sau khi nhận diện rủi ro, chúng ta sẽ phân tích các rủi ro để chọn ra những rủi ro quan trọng nhất để tiến hành các bước tiếp theo. Chúng ta cần phân tích các rủi ro đó để đưa ra lựa chọn về rủi ro nào cần quản lý, cũng như đưa ra các kế hoạch và hành động cần thực hiện khi những rủi ro đó xảy ra. Mục tiêu của quá trình phân tích là thu thập thông tin đánh giá tầm quan trọng của từng rủi ro, làm cơ sở để đánh giá rủi ro. Đánh giá rủi ro cung cấp cơ sở để lựa chọn những rủi ro quan trọng nhất để lập kế hoạch phòng ngừa và chúng ta không có đủ nguồn lực để giải quyết tất cả các rủi ro. Phân tích rủi ro bao gồm phân tích định tính và định lượng. Phân tích định tính đánh giá khả năng xảy ra và mức độ tác động của từng rủi ro. Trong khi đó, phân tích định lượng là đánh giá chi tiết hơn về giá trị thâm hụt ngân sách, số ngày theo lịch thay đổi so với kế hoạch, v.v. khi rủi ro xảy ra.

a. Phân tích định tính rủi ro:

Phân tích định tính (phân tích tính chất) rủi ro là quá trình đánh giá rủi ro bao gồm việc xác định khả năng xảy ra (thấp, vừa, cao) và mức độ tác động (thường, nghiêm trọng, rất nghiêm trọng) để từ đó xác định quy mô và độ ưu tiên của từng rủi ro. Sau khi đánh giá, các rủi ro sẽ được sắp xếp theo thứ tự ưu tiên, và những rủi ro có thể loại bỏ sẽ được loại bỏ, giúp nhóm quản lý tập trung vào những rủi ro quan trọng nhất và sử dụng hiệu quả các nguồn lực có hạn.

Các công cụ và kỹ thuật định lượng để phân tích rủi ro bao gồm:

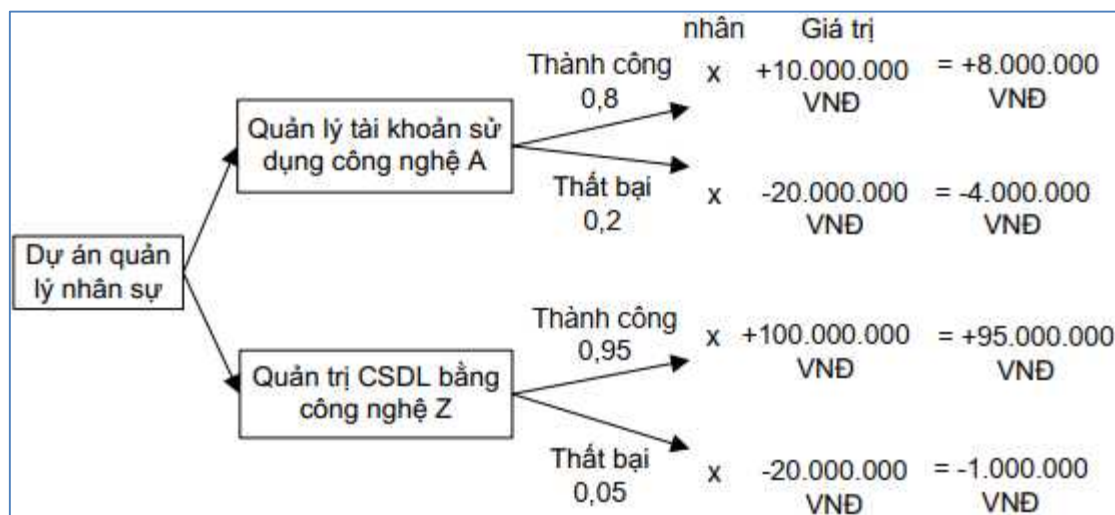
- **Ma trận Xác suất/Tác động (Ma trận rủi ro):** Công cụ này giúp đánh giá mức độ rủi ro bằng cách kết hợp khả năng xảy ra và mức độ tác động của rủi ro.
- **Đánh giá của chuyên gia:** Nhiều công ty sử dụng trực giác và kinh nghiệm của các chuyên gia để nhận diện xu hướng rủi ro trong dự án. Các chuyên gia có thể phân loại các rủi ro thành mức độ cao, vừa, hoặc thấp, sử dụng các kỹ thuật từ đơn giản đến phức tạp.

b. Phân tích định lượng rủi ro

Phân tích định lượng (phân tích mức độ) rủi ro thường được thực hiện sau khi đã hoàn thành phân tích tính chất rủi ro, tuy nhiên, cả hai khâu này có thể được thực hiện đồng thời hoặc riêng biệt, tùy vào đặc thù của dự án. Đối với những dự án có quy mô lớn, phức tạp, và liên quan đến công nghệ tiên tiến, phân tích định lượng rủi ro là cần thiết để đánh giá phạm vi và mức độ ảnh hưởng của các yếu tố rủi ro. Các kỹ thuật chính trong phân tích mức độ rủi ro bao gồm:

- **Phân tích cây quyết định (Decision Tree Analysis):** Phân tích cây quyết định là một phương pháp sử dụng biểu đồ cây để giúp lựa chọn hành động tối ưu trong những tình huống mà kết quả tương lai không thể xác định chắc chắn. Một dạng phổ biến của phân tích cây quyết định là **Phân tích Giá trị Kỳ vọng Tiền tệ (EMV)**, giúp tính toán giá trị kỳ vọng của một quyết định dựa trên xác suất xảy ra các sự kiện rủi ro và giá trị tài chính tương ứng. Phương pháp này cho phép đánh giá các lựa chọn quyết định trong bối cảnh rủi ro không chắc chắn.

Ví dụ: Một dự án có thể sử dụng cây quyết định để đánh giá các lựa chọn về đầu tư dựa trên xác suất thành công hoặc thất bại của các giai đoạn trong dự án, từ đó xác định lựa chọn có giá trị kỳ vọng cao nhất.



Hình 4.1: Cây quyết định và giá trị chi phí dự kiến.

- **Mô phỏng (Simulation):** Mô phỏng sử dụng mô hình của hệ thống để phân tích hành vi dự kiến hoặc hoạt động của hệ thống trong các tình huống khác nhau. Phương pháp mô phỏng **Monte Carlo** là một kỹ thuật phổ biến trong phân tích rủi ro, giúp mô phỏng kết quả của một mô hình nhiều lần để tạo ra một phân bố thống kê về các kết quả có thể xảy ra. Phương pháp này cho phép phân tích một cách toàn diện và đánh giá mức độ rủi ro của các quyết định trong điều kiện không chắc chắn.

Ví dụ: Trong một dự án phát triển phần mềm, mô phỏng Monte Carlo có thể được sử dụng để dự đoán thời gian hoàn thành dự án, với các biến số như độ phức tạp của mã, khả năng phát sinh sự cố, và mức độ hoàn thành công việc theo các kịch bản khác nhau.

4.1.3. Quản lý rủi ro

Sau khi hoàn thành việc đánh giá rủi ro bằng cả phương pháp định tính và định lượng, việc lập kế hoạch hành động để quản lý rủi ro là bước quan trọng tiếp theo. Kế hoạch này không chỉ giúp xác định các biện pháp ứng phó mà còn giúp tối ưu hóa việc sử dụng

nguồn lực, đồng thời giảm thiểu tác động của các rủi ro tiềm ẩn. Nếu không có một kế hoạch quản lý rủi ro rõ ràng, các rủi ro có thể gây ảnh hưởng nghiêm trọng đến tiến độ, chất lượng và ngân sách của dự án. Dưới đây là các biện pháp quản lý rủi ro cần thiết cho những tình huống khác nhau:

Đối với rủi ro có tác động tiêu cực:

- **Tránh rủi ro:** Biện pháp này bao gồm việc thực hiện các hành động nhằm ngăn ngừa rủi ro xảy ra ngay từ đầu. Một trong những cách phổ biến là lựa chọn công nghệ đã được kiểm chứng và quen thuộc với đội ngũ dự án thay vì sử dụng công nghệ mới chưa được thử nghiệm. Ví dụ, nếu một công nghệ mới chưa được triển khai thành công trong các dự án trước đó, việc tiếp tục sử dụng công nghệ cũ hoặc công nghệ đã được xác nhận sẽ giảm thiểu rủi ro. Điều này đặc biệt quan trọng đối với những dự án có yêu cầu khắt khe về tính ổn định và độ tin cậy.
- **Chuyển giao rủi ro:** Đây là biện pháp chuyển giao một phần hoặc toàn bộ rủi ro cho bên thứ ba. Một ví dụ là trong một dự án phần mềm, nếu việc phát triển giao diện người dùng yêu cầu chuyên môn cao nhưng đội ngũ không đủ khả năng, việc thuê một công ty chuyên về thiết kế giao diện người dùng hoặc mua phần mềm bên ngoài có thể giúp giảm bớt rủi ro và tập trung vào các công việc khác. Phương pháp này giúp giảm bớt trách nhiệm và khả năng xảy ra sự cố trong phạm vi dự án của mình.
- **Giảm thiểu rủi ro:** Khi rủi ro không thể loại bỏ hoàn toàn, việc giảm thiểu các tác động tiêu cực là biện pháp hữu hiệu. Một trong những cách thực hiện là chuẩn bị nhân sự thay thế trong trường hợp có sự thay đổi nhân sự không mong muốn. Ví dụ, khi phát hiện một thành viên trong nhóm có xu hướng làm việc độc lập hoặc có kế hoạch nghỉ việc, có thể chuẩn bị sẵn một nhân sự thay thế, để giảm thiểu sự gián đoạn công việc. Ngoài ra, việc chia sẻ công việc giữa các thành viên và đào tạo kỹ năng cho nhân sự mới cũng là một cách giúp giảm thiểu tác động khi có sự thay đổi.
- **Chấp nhận rủi ro:** Đôi khi, một số rủi ro là không thể tránh khỏi hoặc kiểm soát được, và việc chấp nhận chúng là cần thiết. Một ví dụ là trong các dự án phát triển phần mềm di động, khi không thể thử nghiệm tất cả các thiết bị di động trên thị trường. Trong trường hợp này, dự án có thể chấp nhận việc một số thiết bị không hoàn toàn tương thích, và tìm cách đền bù cho khách hàng nếu sự cố xảy ra, chẳng hạn như cung cấp bản cập nhật phần mềm hoặc hỗ trợ kỹ thuật miễn phí. Điều này giúp duy trì sự hài lòng của khách hàng mà không cần phải thực hiện các thay đổi lớn trong kế hoạch dự án.

Đối với rủi ro có tác động tích cực:

- **Tận dụng:** Khi một sự kiện tích cực có thể mang lại lợi ích cho dự án, việc tận dụng cơ hội này là rất quan trọng. Ví dụ, nếu một đối tác cung cấp tài trợ cho dự án, việc sử dụng nguồn tài trợ này để tăng cường tiếp thị hoặc mở rộng đội ngũ chất lượng sẽ giúp thúc đẩy dự án tiến nhanh hơn và đạt được các mục tiêu đề ra. Tận dụng các cơ hội tích cực giúp không chỉ giảm thiểu rủi ro mà còn mở rộng khả năng thành công cho dự án.
- **Quảng bá:** Khi có sự tham gia của các chuyên gia hoặc các nhà chuyên môn, điều này có thể là một cơ hội để tối ưu hóa các hoạt động trong dự án. Việc khuyến khích các chuyên gia công nghệ tham gia có thể cải thiện hiệu quả dự án, từ việc phát triển các tính năng mới đến tối ưu hóa các quy trình trong dự án. Quảng bá sự tham gia của các chuyên gia không chỉ giúp nâng cao giá trị của dự án mà còn làm gia tăng khả năng thành công của nó.
- **Chia sẻ:** Một cách khác để tận dụng rủi ro tích cực là chia sẻ cơ hội thị trường với các đơn vị khác. Ví dụ, hợp tác với các nhà sản xuất phần cứng để phát triển phần mềm tương thích với sản phẩm của họ không chỉ giúp phát triển sản phẩm nhanh hơn mà còn tạo ra các cơ hội thị trường mới cho sản phẩm. Sự hợp tác này có thể làm giảm chi phí phát triển và đồng thời mở rộng khả năng tiếp cận khách hàng tiềm năng.
- **Chấp nhận:** Tương tự như khi chấp nhận rủi ro tiêu cực, đôi khi việc chấp nhận rủi ro tích cực là điều cần thiết. Một ví dụ là khi công việc trong dự án hoàn thành sớm hơn dự kiến hoặc khách hàng phản hồi nhanh chóng, điều này có thể giúp đẩy nhanh tiến độ của dự án mà không cần phải có sự can thiệp thêm. Việc này có thể góp phần vào sự thành công của dự án mà không phải đối mặt với các rủi ro tiềm ẩn, đồng thời tăng sự hài lòng của khách hàng.

Những hành động này giúp dự án không chỉ giảm thiểu các rủi ro tiêu cực mà còn tận dụng tối đa những cơ hội tích cực, từ đó tối ưu hóa kết quả và đảm bảo thành công cho dự án.

4.1.4. Sử dụng ma trận rủi ro

Ma trận rủi ro (hay còn gọi là ma trận xác suất/tác động) là một công cụ quan trọng trong quản lý rủi ro, giúp đánh giá và phân loại các rủi ro dựa trên hai yếu tố chính: **xác suất xảy ra** và **mức độ tác động**. Ma trận này cung cấp một cách tiếp cận trực quan và có hệ thống để xác định ưu tiên xử lý các rủi ro trong dự án, giúp đội ngũ quản lý dự án tập trung vào các yếu tố quan trọng nhất, đồng thời tạo cơ sở cho việc đưa ra quyết định về các biện pháp ứng phó phù hợp.

Cấu trúc Ma trận Rủi ro

Ma trận rủi ro được xây dựng dưới dạng một bảng, trong đó:

- **Trục ngang** đại diện cho **xác suất xảy ra** của rủi ro, thường được phân chia thành ba mức độ: **thấp**, **trung bình**, và **cao**.
- **Trục dọc** thể hiện **mức độ tác động** của rủi ro đối với dự án, được phân loại thành các mức độ như: **ít nghiêm trọng**, **ng nghiêm trọng**, và **rất nghiêm trọng**.

Mỗi ô trong ma trận sẽ chứa các rủi ro được phân loại theo mức độ xác suất và tác động của chúng. Cách thức phân loại này giúp các nhà quản lý dễ dàng đánh giá và xác định các rủi ro cần ưu tiên xử lý, đồng thời tạo ra một bức tranh tổng thể về các yếu tố rủi ro trong dự án.

Mục đích của Ma trận Rủi ro

- **Đánh giá rủi ro:** Ma trận rủi ro giúp xác định rõ các rủi ro có **xác suất** và **tác động** cao, từ đó giúp đội ngũ quản lý tập trung vào những yếu tố quan trọng nhất và có khả năng gây ảnh hưởng lớn nhất đến dự án.
- **Phân loại rủi ro:** Ma trận rủi ro phân chia các rủi ro thành các nhóm, mỗi nhóm có mức độ ưu tiên khác nhau. Điều này giúp đội ngũ quản lý dễ dàng quyết định các biện pháp ứng phó phù hợp.
- **Ưu tiên xử lý:** Ma trận rủi ro giúp xác định rõ ràng các rủi ro cần phải đối mặt ngay lập tức và những rủi ro có thể theo dõi hoặc bỏ qua. Các biện pháp ứng phó sẽ được thiết lập dựa trên mức độ quan trọng của từng rủi ro.

Các bước sử dụng Ma trận Rủi ro

- **Nhận diện các rủi ro tiềm ẩn:** Đây là bước đầu tiên trong quá trình sử dụng ma trận rủi ro. Đội ngũ quản lý dự án cần lập danh sách các rủi ro có thể xảy ra trong dự án, bao gồm cả những yếu tố bên trong và bên ngoài dự án có khả năng ảnh hưởng đến kết quả.
- **Đánh giá xác suất và tác động của từng rủi ro:** Sau khi xác định các rủi ro, mỗi rủi ro sẽ được đánh giá về khả năng xảy ra (xác suất) và mức độ tác động nếu rủi ro đó xảy ra. Đánh giá này có thể được thực hiện dựa trên dữ liệu lịch sử, các chuyên gia trong lĩnh vực, hoặc qua các cuộc thảo luận nhóm.
- **Điền vào ma trận:** Dựa trên các đánh giá về xác suất và tác động, mỗi rủi ro sẽ được xếp vào ô tương ứng trong ma trận rủi ro. Ma trận sẽ cung cấp một cái nhìn trực quan về các rủi ro và mức độ nghiêm trọng của chúng.
- **Lên kế hoạch ứng phó:** Dựa trên vị trí của các rủi ro trong ma trận, đội ngũ quản lý sẽ xác định các biện pháp ứng phó phù hợp. Các rủi ro có xác suất và tác động cao sẽ được ưu tiên xử lý ngay lập tức, trong khi các rủi ro có xác suất thấp hoặc tác động thấp có thể được theo dõi và đánh giá lại sau.

Quy trình ưu tiên xử lý

- **Rủi ro có xác suất và tác động cao:** Những rủi ro này cần được **xử lý ngay lập tức**, vì chúng có thể gây ảnh hưởng nghiêm trọng đến tiến độ, ngân sách hoặc chất lượng của dự án. Cần phải có các biện pháp giảm thiểu hoặc loại bỏ rủi ro ngay từ giai đoạn đầu.
- **Rủi ro có xác suất thấp nhưng tác động cao:** Các rủi ro này có thể cần **biện pháp dự phòng**, như việc chuyển giao rủi ro cho bên thứ ba hoặc chuẩn bị kế hoạch khắc phục để giảm thiểu tác động khi xảy ra.
- **Rủi ro có xác suất và tác động thấp:** Đây là những rủi ro có thể **theo dõi và đánh giá lại** khi cần thiết. Các biện pháp kiểm soát có thể được áp dụng để đảm bảo rằng chúng không phát triển thành những vấn đề lớn hơn trong tương lai.

Lưu ý khi sử dụng Ma trận Rủi ro:

Khi có thay đổi lớn trong yêu cầu dự án, ma trận rủi ro cần được **đánh giá lại và cập nhật** để phản ánh đúng tình hình thực tế của dự án. Việc cập nhật định kỳ và theo dõi các rủi ro giúp duy trì một cái nhìn toàn diện về dự án và hỗ trợ việc ra quyết định kịp thời.

Ví dụ:

Phần sau đây mô tả cách sử dụng ma trận rủi ro trong một dự án phần mềm, có thể bắt đầu quá trình phân tích rủi ro bằng cách nhận diện các rủi ro từ các nguồn khác nhau. Sau đó, mỗi rủi ro sẽ được đánh giá về xác suất và tác động. Các rủi ro sẽ được xếp vào các ô trong ma trận, và dựa vào vị trí của chúng, các biện pháp ứng phó sẽ được xác định. Ma trận rủi ro giúp đội ngũ quản lý dễ dàng tập trung vào những rủi ro quan trọng, và đưa ra các quyết định ứng phó phù hợp để giảm thiểu tác động của chúng.

Rủi ro	Xảy ra	Tác động
Nguồn tài chính giảm đáng kể	Thấp (1)	Rất nghiêm trọng (5)
Không thuê được nhân viên có kỹ năng đúng yêu cầu	Cao (5)	Rất nghiêm trọng (5)
CASE không tích hợp được	Cao (5)	Thường (1)
Môi trường phát triển mới	Thấp (1)	Nghiêm trọng (3)
Phần mềm dùng lại còn nhiều khiếm khuyết	Vừa (3)	Nghiêm trọng (3)

Bảng 4.1. Các rủi ro khi triển khai một dự án mới tại một công ty phần mềm

Áp dụng kỹ thuật ma trận xác suất/ tác động để phân tích tính chất rủi ro như sau:

Sử dụng bảng trọng số để tính điểm rủi ro:

$$\text{Điểm rủi ro} = (\text{xác suất xảy ra}) \times (\text{mức tác động})$$

	Tác động	Thường = 1	Nghiêm trọng = 3	Rất nghiêm trọng = 5
Xác suất	Thấp = 1	1	3	5
	Vừa = 3	3	9	15
	Cao = 5	5	15	25

Bảng 4.2. Bảng tính điểm các rủi ro

Sử dụng bảng tính điểm rủi ro và xếp chúng theo thứ tự giảm dần dựa trên điểm số tính được như sau:

STT	Sắp thứ tự ưu tiên các rủi ro	Điểm rủi ro
1	Không thuê được nhân viên có kỹ năng đúng yêu cầu	25
2	Phần mềm dùng lại còn nhiều khiếm khuyết	9
3	CASE không tích hợp được	5
4	Nguồn tài chính giảm đáng kể	5
5	Môi trường phát triển mới	3

Bảng 4.3. Bảng phân tích rủi ro định tính

Ma trận rủi ro là một công cụ mạnh mẽ giúp các nhà quản lý dự án có thể đánh giá, phân loại và xử lý các rủi ro một cách hệ thống và hiệu quả.

4.1.5. Các kỹ thuật phòng ngừa

Kỹ thuật tối thiểu hóa các mốc milestone được sử dụng nhằm phòng ngừa rủi ro. Đây là phương pháp sử dụng các mục tiêu nhỏ trong kế hoạch hoạt động của dự án, nhằm rút ngắn khoảng cách giữa các mốc thời gian kiểm tra và đánh giá tiến độ. Mục tiêu là giảm thiểu khoảng thời gian giữa các mốc kiểm tra, giúp đội ngũ dự án dễ dàng theo dõi và điều chỉnh khi cần thiết.

Cách thức thực hiện

Kỹ thuật này yêu cầu thiết lập một lịch trình chi tiết, với các mốc kiểm tra (milestones) được xác định ngay từ giai đoạn đầu của dự án. Khoảng cách giữa các mốc kiểm tra được

khuyến cáo là từ 1 đến 2 ngày, tuy nhiên, khoảng cách này cũng có thể dài hơn, từ 1 đến 2 tuần, tùy vào tính chất và yêu cầu của dự án.

Ưu điểm

- **Cải thiện tính trực quan:** Việc rút ngắn các mốc milestone giúp giám đốc dự án dễ dàng theo dõi tiến độ và tình trạng công việc. Điều này cũng giúp việc khôi phục dự án hoặc một phần của dự án trở nên dễ dàng hơn.
- **Giảm rủi ro trượt lịch trình:** Phương pháp này đặc biệt hiệu quả trong việc giảm các rủi ro liên quan đến việc trượt lịch thực hiện công việc, vì nó giúp phát hiện sớm các vấn đề tiềm ẩn và điều chỉnh kịp thời.
- **Ứng dụng cho dự án công nghệ thông tin:** Kỹ thuật này phù hợp với các dự án công nghệ thông tin từ đầu và các dự án phát triển theo mô hình lặp đi lặp lại (iterative), nơi cần có sự điều chỉnh liên tục và theo dõi chặt chẽ.

Nhược điểm: Tăng công sức theo dõi, khi việc theo dõi và kiểm tra thường xuyên có thể tạo ra một khối lượng công việc lớn đối với nhóm quản lý dự án, do phải thường xuyên đánh giá và điều chỉnh tiến độ.

Yêu cầu để thực hiện kỹ thuật:

Để áp dụng kỹ thuật này thành công, giám đốc dự án cần đảm bảo các yêu cầu sau:

- **Lịch trình chi tiết:** Mỗi công việc trong dự án cần có lịch trình thực hiện rõ ràng, chi tiết.
- **Mốc kiểm tra từ giai đoạn đầu:** Các mốc milestone phải được xác định ngay từ đầu và phải được kiểm tra liên tục trong suốt quá trình triển khai dự án.
- **Khoảng cách giữa các mốc kiểm tra:** Khoảng cách giữa các mốc milestone cần được thiết lập sao cho phù hợp với tính chất của dự án (từ 1 đến 2 ngày hoặc có thể lên đến 1-2 tuần).
- **Áp dụng cho các dự án lặp đi lặp lại:** Kỹ thuật này rất hữu ích cho các dự án có phương thức phát triển lặp đi lặp lại, nơi cần sự đánh giá và cải tiến liên tục.
- **Sử dụng mốc kiểm tra nhị phân:** Các mốc milestone cần được xác định theo kiểu nhị phân, tức là có hai trạng thái rõ ràng: "hoàn thành" hoặc "chưa hoàn thành".

Kỹ thuật tối thiểu hóa các mốc milestone là một công cụ mạnh mẽ giúp cải thiện hiệu quả quản lý rủi ro dự án, cải thiện khả năng điều chỉnh dự án kịp thời khi có vấn đề phát sinh.

4.2. Kiểm soát chất lượng sản phẩm phần mềm

Định nghĩa về chất lượng sản phẩm

Tổ chức quốc tế về tiêu chuẩn hoá (ISO - International Standard Organization) định nghĩa *chất lượng là tổng thể các đặc tính và chi tiết của một sản phẩm, trong đó sản phẩm đó phải đáp ứng đầy đủ các yêu cầu và tiêu chuẩn đã được quy định trước.*

Ngoài định nghĩa của ISO, một số chuyên gia khác cũng đưa ra những nguyên tắc cơ bản khi xác định chất lượng, bao gồm:

- **Yêu cầu phù hợp:** Chất lượng của sản phẩm được đánh giá dựa trên khả năng đáp ứng các yêu cầu và tiêu chuẩn đã được đặt ra từ trước. Sản phẩm phải thỏa mãn các yêu cầu về chức năng, tính năng và hiệu suất mà người dùng hoặc khách hàng mong đợi.
- **Tiện lợi cho sử dụng:** Một sản phẩm đạt chất lượng cần phải dễ dàng sử dụng và phù hợp với mục đích sử dụng từ khi nó được sản xuất. Điều này bao gồm tính thân thiện với người dùng và khả năng vận hành hiệu quả trong môi trường thực tế.
- **Chi phí kiểm thử và tích hợp hệ thống:** Trong các dự án công nghệ thông tin, chi phí cho các hoạt động kiểm thử và tích hợp hệ thống chiếm khoảng 29% tổng chi phí của dự án. Đây là tỷ lệ lớn nhất trong các hoạt động khác như thiết kế ban đầu, thiết kế chi tiết, viết mã chương trình và kiểm thử từng chức năng. Điều này cho thấy tầm quan trọng của công tác kiểm thử trong việc đảm bảo chất lượng của sản phẩm cuối cùng, đồng thời cũng phản ánh chi phí lớn liên quan đến quá trình này trong một dự án công nghệ thông tin.

Phần mềm là một sản phẩm trí tuệ, được phát triển bởi con người, và nó là một sản phẩm vô hình, điều này khiến việc đánh giá chất lượng trở nên khó khăn hơn so với các sản phẩm vật lý. Do tính chất phức tạp và trừu tượng của phần mềm, việc đảm bảo chất lượng yêu cầu phải có các biện pháp cụ thể và chủ động để ngăn ngừa các lỗi tiềm năng.

Một lý do quan trọng để quản lý chất lượng phần mềm là vì trong quá trình phát triển, có thể xảy ra rất nhiều vấn đề không lường trước được, từ yêu cầu thay đổi của khách hàng, sự thay đổi về môi trường triển khai, đến các yếu tố kỹ thuật không ổn định. Những điều này có thể ảnh hưởng đến chất lượng của phần mềm nếu không có kế hoạch quản lý và kiểm soát chặt chẽ.

Quản lý chất lượng phần mềm là một chuỗi các hoạt động và phương pháp luận nhằm đảm bảo rằng sản phẩm phần mềm đáp ứng đầy đủ các yêu cầu về chức năng, hiệu suất, bảo mật và độ tin cậy.

Quy trình quản lý chất lượng phần mềm

Quy trình quản lý chất lượng phần mềm bao gồm ba giai đoạn chính, mỗi giai đoạn đóng một vai trò quan trọng trong việc đảm bảo rằng sản phẩm phần mềm đáp ứng các tiêu chuẩn và yêu cầu chất lượng. Các giai đoạn này bao gồm:

1. **Lập kế hoạch chất lượng (Quality Planning):** Đây là giai đoạn đầu tiên trong quy trình quản lý chất lượng, trong đó các tiêu chuẩn chất lượng sẽ được xác định và các phương pháp để đạt được những tiêu chuẩn này sẽ được xây dựng. Giai đoạn này bao gồm việc thiết lập các mục tiêu chất lượng cụ thể cho dự án, định nghĩa các quy trình làm việc cần thiết để đạt được chất lượng mong muốn, và thiết lập các biện pháp kiểm soát để giám sát và đảm bảo rằng các quy trình chất lượng được thực hiện đúng cách trong suốt quá trình phát triển phần mềm.
2. **Đảm bảo chất lượng (Quality Assurance):** Đây là một hoạt động có hệ thống nhằm đánh giá toàn bộ quá trình thực hiện dự án để đảm bảo rằng dự án sẽ đáp ứng các yêu cầu chất lượng. Đảm bảo chất lượng là hoạt động mang tính phòng ngừa, tập trung vào việc ngăn chặn các lỗi hoặc vấn đề có thể phát sinh trong quá trình phát triển phần mềm, thay vì chỉ phát hiện và sửa chữa chúng sau khi chúng xảy ra.
3. **Kiểm soát chất lượng (Quality Control):** Giai đoạn này tập trung vào việc đánh giá các kết quả chất lượng cụ thể của dự án dựa trên các tiêu chuẩn chất lượng đã xác định trước đó. Mục tiêu là xác định các vấn đề ảnh hưởng đến chất lượng và tìm cách cải thiện nó. Trong quá trình kiểm soát chất lượng, các nguyên nhân gây ra các vấn đề chất lượng được phân tích và loại bỏ. Giai đoạn này diễn ra liên tục trong suốt quá trình thực hiện dự án để đảm bảo rằng chất lượng phần mềm luôn được kiểm tra và nâng cao.

Ba giai đoạn này phối hợp với nhau để tạo thành một quy trình quản lý chất lượng toàn diện, đảm bảo rằng sản phẩm phần mềm cuối cùng đáp ứng được các yêu cầu và tiêu chuẩn chất lượng của dự án.

Để đạt được chất lượng sản phẩm mong muốn, kiểm thử phần mềm là một phần không thể thiếu. Kiểm thử phần mềm có thể được thực hiện thông qua hai phương pháp chính: kiểm thử thủ công và kiểm thử tự động. Cả hai phương pháp này đều có những ưu điểm và nhược điểm riêng, và việc lựa chọn phương pháp phụ thuộc vào yêu cầu và tính chất của dự án.

4.2.1. Kiểm thử thủ công

Kiểm thử thủ công là phương pháp mà người kiểm thử thực hiện kiểm tra phần mềm bằng cách thao tác trực tiếp trên hệ thống mà không sử dụng công cụ tự động. Đây là

phương pháp thường được áp dụng trong những trường hợp kiểm thử cần sự sáng tạo hoặc đánh giá trải nghiệm người dùng, chẳng hạn như kiểm thử giao diện người dùng hoặc kiểm thử các tình huống không lường trước được.

Quá trình kiểm thử thủ công:

1. **Xác định các trường hợp kiểm thử:** Người kiểm thử phải xác định các kịch bản kiểm thử, bao gồm các tình huống điển hình và các trường hợp ngoại lệ có thể xảy ra trong quá trình sử dụng phần mềm.
2. **Thực hiện kiểm thử:** Người kiểm thử sẽ nhập dữ liệu vào phần mềm, tương tác với giao diện và các chức năng của hệ thống, ghi nhận kết quả đầu ra và so sánh với kết quả mong đợi.
3. **Ghi nhận và báo cáo lỗi:** Sau khi thực hiện kiểm thử, người kiểm thử sẽ ghi nhận các lỗi phát sinh và báo cáo cho nhóm phát triển để sửa chữa.

Ưu điểm của kiểm thử thủ công:

- **Khả năng đánh giá trải nghiệm người dùng:** Kiểm thử thủ công rất hữu ích trong việc đánh giá tính dễ sử dụng và giao diện của phần mềm, giúp đảm bảo rằng người dùng cuối sẽ có trải nghiệm tốt.
- **Kiểm thử các tình huống phức tạp:** Các tình huống kiểm thử phức tạp hoặc không thể đoán trước có thể được kiểm thử một cách linh hoạt thông qua kiểm thử thủ công.
- **Dễ dàng cho những thay đổi nhỏ:** Kiểm thử thủ công dễ dàng áp dụng khi có các thay đổi nhỏ trong yêu cầu hoặc khi hệ thống không ổn định.

Nhược điểm của kiểm thử thủ công:

- **Tốn thời gian và công sức:** Kiểm thử thủ công yêu cầu nhiều nguồn lực và thời gian, đặc biệt là trong các dự án lớn với nhiều chức năng.
- **Dễ mắc lỗi con người:** Kiểm thử thủ công phụ thuộc vào sự chú ý và kinh nghiệm của người kiểm thử, do đó có thể dẫn đến việc bỏ sót lỗi hoặc đánh giá sai.
- **Khó duy trì và mở rộng:** Khi phần mềm phát triển và có nhiều tính năng mới, việc duy trì và mở rộng các kịch bản kiểm thử thủ công trở nên phức tạp và tốn thời gian.

4.2.2. Kiểm thử tự động

Kiểm thử tự động là phương pháp sử dụng các công cụ và phần mềm để tự động thực hiện các kịch bản kiểm thử. Phương pháp này thường được áp dụng khi phần mềm có các chức năng lặp đi lặp lại hoặc cần thực hiện kiểm thử trên nhiều môi trường khác nhau.

Quá trình kiểm thử tự động:

1. **Xây dựng kịch bản kiểm thử tự động:** Các kịch bản kiểm thử được lập trình sẵn thông qua các công cụ tự động (như Selenium, JUnit, TestNG).
2. **Thực hiện kiểm thử tự động:** Công cụ tự động sẽ thực hiện các kịch bản kiểm thử trên phần mềm, theo dõi kết quả và so sánh với kết quả mong đợi.
3. **Lưu trữ và phân tích kết quả:** Các kết quả của kiểm thử tự động sẽ được ghi lại và phân tích để tìm ra lỗi hoặc sự không phù hợp trong phần mềm.

Ưu điểm của kiểm thử tự động:

- **Tăng hiệu quả và độ chính xác:** Kiểm thử tự động có thể thực hiện hàng nghìn kịch bản kiểm thử trong thời gian ngắn, giúp giảm thiểu sai sót và tăng cường độ chính xác trong việc phát hiện lỗi.
- **Tiết kiệm thời gian và công sức:** Sau khi kịch bản kiểm thử tự động được thiết lập, việc chạy kiểm thử trở nên nhanh chóng và không cần sự can thiệp của con người.
- **Dễ dàng tái sử dụng:** Kịch bản kiểm thử tự động có thể được tái sử dụng trong các lần kiểm thử sau này, đặc biệt là khi phần mềm được cập nhật hoặc thay đổi.
- **Phù hợp với kiểm thử lặp đi lặp lại:** Kiểm thử tự động rất hiệu quả trong việc kiểm thử các tính năng lặp lại, chẳng hạn như chức năng đăng nhập, tìm kiếm, v.v.

Nhược điểm của kiểm thử tự động:

- **Chi phí ban đầu cao:** Việc thiết lập và duy trì kiểm thử tự động đòi hỏi một khoản đầu tư lớn về thời gian và công cụ, đặc biệt trong giai đoạn đầu của dự án.
- **Khó khăn trong việc kiểm thử giao diện người dùng:** Kiểm thử tự động có thể gặp khó khăn trong việc đánh giá các yếu tố cảm quan và trải nghiệm người dùng mà kiểm thử thủ công có thể làm tốt hơn.
- **Yêu cầu duy trì kịch bản kiểm thử:** Khi phần mềm thay đổi, các kịch bản kiểm thử tự động cũng cần được cập nhật để phù hợp với các thay đổi đó, điều này có thể tạo ra một khối lượng công việc đáng kể.

Cả kiểm thử thủ công và kiểm thử tự động đều có vai trò quan trọng trong quá trình kiểm soát chất lượng phần mềm. Kiểm thử thủ công phù hợp với các tình huống phức tạp hoặc không thể tự động hóa, cũng như kiểm tra trải nghiệm người dùng. Trong khi đó, kiểm thử tự động mang lại hiệu quả cao trong việc kiểm tra các chức năng lặp lại hoặc kiểm thử trên nhiều môi trường, giúp tiết kiệm thời gian và công sức cho các nhóm phát triển.

Việc lựa chọn phương pháp kiểm thử phù hợp phụ thuộc vào tính chất của dự án, yêu cầu về thời gian, nguồn lực, và mức độ thay đổi của phần mềm. Một chiến lược kiểm thử hiệu quả thường bao gồm sự kết hợp giữa cả hai phương pháp này để tối ưu hóa chất lượng phần mềm và giảm thiểu rủi ro trong quá trình phát triển.

Câu hỏi ôn tập cuối chương

1. Quản lý rủi ro trong dự án phần mềm có mục tiêu gì?
2. Các bước trong quy trình quản lý rủi ro là gì?
3. Sự khác biệt giữa quản lý rủi ro và quản lý dự án là gì?
4. Ma trận rủi ro giúp đánh giá rủi ro dựa trên yếu tố nào?
5. Tại sao cần phân loại các rủi ro trong một dự án phần mềm?
6. Khi nào cần sử dụng kế hoạch dự phòng trong quản lý rủi ro?
7. Phân tích định tính rủi ro có những yếu tố nào cần đánh giá?
8. Lợi ích của việc sử dụng kiểm thử tự động trong quản lý chất lượng phần mềm là gì?
9. Ma trận rủi ro giúp quản lý dự án ưu tiên xử lý rủi ro như thế nào?
10. Sự khác biệt giữa kiểm thử thủ công và kiểm thử tự động trong việc kiểm soát chất lượng là gì?

Chương 5: Công cụ và kỹ thuật hỗ trợ

Nội dung chính của chương

Chương này cung cấp một cái nhìn tổng quan về các công cụ quản lý dự án hiện đại, đặc biệt là những công cụ phổ biến giúp các nhóm làm việc hiệu quả và quản lý tiến độ dự án một cách tối ưu. Nội dung chương được chia thành hai phần chính. Phần thứ nhất trình bày về những công cụ quản lý dự án phổ biến và chức năng của chúng trong việc hỗ trợ các nhóm làm việc hiệu quả hơn. Phần thứ hai trình bày chi tiết cách sử dụng các công cụ trên để quản lý tiến độ dự án, từ việc theo dõi tiến độ đến lập các báo cáo chi tiết.

Mục tiêu cần đạt của chương

- Hiểu chức năng của các công cụ quản lý dự án hiện đại. Sinh viên sẽ hiểu được các tính năng cơ bản của các công cụ quản lý dự án phổ biến như Trello, Asana, Jira, Slack và GitHub/GitLab.
- Biết cách vận dụng các công cụ để quản lý nhóm, theo dõi tiến độ và báo cáo. Sinh viên sẽ có thể sử dụng các công cụ này để tạo bảng công việc, phân công nhiệm vụ, theo dõi tiến độ công việc, và quản lý các yếu tố khác của dự án như thời gian và chi phí

5.1. Giới thiệu chức năng của các công cụ quản lý dự án hiện đại

Phần mềm quản lý dự án là công cụ hỗ trợ quá trình tổ chức và điều hành các hoạt động liên quan đến dự án hoặc công việc trong một tổ chức. Công cụ này cung cấp các tính năng để lập kế hoạch, dự toán chi phí, phân bổ nguồn lực, và giám sát tiến độ của dự án hoặc công việc. Phần mềm quản lý dự án giúp các doanh nghiệp và nhà quản lý dự án có thể tổ chức, điều phối và giám sát tất cả các khía cạnh của dự án một cách hiệu quả, đảm bảo tính chặt chẽ và đồng bộ trong quá trình triển khai.

Lợi ích của việc sử dụng phần mềm quản trị dự án

Việc sử dụng phần mềm quản trị dự án mang lại nhiều lợi ích quan trọng cho tổ chức và nhóm làm việc, bao gồm:

- **Tăng cường hiệu suất làm việc:** Phần mềm quản trị dự án cung cấp các công cụ và tính năng giúp tối ưu hóa quy trình làm việc, từ lập kế hoạch, phân công công việc cho đến theo dõi tiến độ. Điều này giúp nhóm làm việc hoàn thành công việc hiệu quả hơn và đúng tiến độ.
- **Tăng khả năng điều phối và quản lý:** Phần mềm quản trị dự án hỗ trợ tổ chức và điều phối các nguồn lực, công việc và thời gian một cách hiệu quả. Việc tập trung

thông tin và công cụ quản lý trong một nền tảng giúp nhà quản lý dự án đưa ra quyết định nhanh chóng và chính xác.

- **Tăng tính minh bạch và giao tiếp:** Phần mềm cung cấp một nền tảng để chia sẻ thông tin, tài liệu và tiến độ dự án một cách minh bạch với toàn bộ nhóm làm việc. Điều này cải thiện giao tiếp và đảm bảo mọi thành viên đều có thông tin đầy đủ và kịp thời.
- **Giảm thiểu rủi ro:** Phần mềm giúp nhận diện và quản lý rủi ro hiệu quả hơn. Việc theo dõi tiến độ và phản ứng kịp thời giúp giảm thiểu các vấn đề và rủi ro có thể phát sinh trong quá trình thực hiện dự án.
- **Nâng cao chất lượng dự án:** Bằng cách cung cấp các công cụ phân tích và đánh giá, phần mềm giúp nhà quản lý dự án đưa ra quyết định dựa trên dữ liệu và thông tin chính xác, từ đó nâng cao chất lượng và hiệu suất của dự án.

Có thể nói, phần mềm quản trị dự án giúp tăng cường khả năng quản lý, giao tiếp, chất lượng dự án, giúp giảm thiểu rủi ro trong quá trình thực hiện và hỗ trợ tăng cường hiệu suất làm việc.

Tiêu chí lựa chọn phần mềm quản lý dự án

Việc lựa chọn phần mềm quản lý dự án phù hợp là yếu tố quan trọng để đảm bảo hiệu quả và thành công của dự án. Dưới đây là các tiêu chí chính cần cân nhắc khi lựa chọn phần mềm:

- **Nhu cầu của dự án:**
 - **Kích thước và độ phức tạp của dự án:** Kích thước và độ phức tạp của dự án ảnh hưởng đến các tính năng và chức năng cần có của phần mềm. Ví dụ, một dự án nhỏ chỉ cần các tính năng cơ bản như theo dõi nhiệm vụ và giao tiếp, trong khi các dự án lớn và phức tạp yêu cầu các tính năng như quản lý tài nguyên, lập kế hoạch Gantt, và báo cáo chi tiết.
 - **Ngành nghề:** Một số phần mềm được thiết kế riêng cho các ngành cụ thể, như xây dựng, sản xuất, hoặc phần mềm. Nếu dự án thuộc một ngành nghề cụ thể, hãy tìm kiếm phần mềm có tính năng phù hợp với yêu cầu của ngành đó.
 - **Quy trình làm việc:** Xem xét cách thức làm việc của nhóm để chọn phần mềm phù hợp. Một số phần mềm thích hợp với mô hình thác nước truyền thống, trong khi những phần mềm khác lại hỗ trợ tốt hơn cho mô hình Agile linh hoạt.

- **Tính năng và chức năng:**

- **Theo dõi nhiệm vụ và tiến độ:** Đây là tính năng cơ bản trong mọi phần mềm quản lý dự án. Phần mềm cần cho phép tạo và gán nhiệm vụ, theo dõi tiến độ và cập nhật trạng thái công việc.
- **Giao tiếp và cộng tác:** Phần mềm nên cung cấp nền tảng để chia sẻ thông tin, giao nhiệm vụ, thảo luận và cộng tác giữa các thành viên trong nhóm.
- **Quản lý tài nguyên:** Nếu dự án yêu cầu quản lý nhiều nguồn lực như nhân lực, thiết bị hay vật liệu, hãy chọn phần mềm có tính năng quản lý tài nguyên hiệu quả.
- **Lập kế hoạch và báo cáo:** Phần mềm cần có công cụ để lập kế hoạch, theo dõi tiến độ và tạo báo cáo hiệu suất dự án.
- **Bảo mật:** Đảm bảo phần mềm đáp ứng các yêu cầu bảo mật, đặc biệt khi làm việc với dữ liệu nhạy cảm.

- **Dễ sử dụng:**

- **Giao diện người dùng:** Giao diện phần mềm cần trực quan, dễ sử dụng và dễ điều hướng.
- **Khả năng đào tạo:** Phần mềm cần đi kèm tài liệu hướng dẫn và chương trình đào tạo giúp nhóm sử dụng hiệu quả.
- **Hỗ trợ khách hàng:** Nhà cung cấp phần mềm phải cung cấp hỗ trợ khách hàng tốt trong trường hợp gặp sự cố.

- **Giá cả:**

- **Giá thành:** Phần mềm quản lý dự án có nhiều mức giá từ miễn phí đến các gói cao cấp. Hãy xác định ngân sách và tìm phần mềm phù hợp.
- **Mô hình cấp phép:** Một số phần mềm cấp phép theo người dùng, trong khi những phần mềm khác cấp phép theo tính năng hoặc dự án. Chọn mô hình cấp phép phù hợp với nhu cầu của dự án phần mềm.

- **Danh tiếng của nhà cung cấp:**

- **Kinh nghiệm:** Lựa chọn nhà cung cấp có uy tín và kinh nghiệm trong việc cung cấp giải pháp.
- **Hỗ trợ khách hàng:** Đảm bảo nhà cung cấp cung cấp hỗ trợ khách hàng chất lượng cao.

- **Cập nhật phần mềm:** Nhà cung cấp phần mềm nên thường xuyên cập nhật các tính năng mới và bản vá lỗi bảo mật.

Ngoài các tiêu chí trên, cũng nên cân nhắc khả năng tích hợp phần mềm với các công cụ khác, khả năng tùy chỉnh và khả năng mở rộng trong tương lai. Bằng cách xem xét kỹ lưỡng các yếu tố này, có thể lựa chọn phần mềm quản lý dự án phù hợp, giúp nâng cao hiệu quả và đạt được mục tiêu dự án đúng tiến độ và ngân sách.

Sử dụng phần mềm quản lý dự án giúp theo dõi tiến độ và tối ưu hóa hiệu quả quản lý, tiết kiệm thời gian và chi phí, đồng thời gia tăng tỷ lệ thành công cho dự án.

5.1.1. Trello

Trello là một công cụ quản lý dự án dựa trên mô hình Kanban. Với giao diện đơn giản và trực quan, Trello cho phép người dùng dễ dàng tạo bảng nhiệm vụ, phân công công việc và theo dõi tiến độ của từng nhiệm vụ trong dự án. Trello rất thích hợp cho các nhóm nhỏ và các dự án đơn giản, giúp tăng cường khả năng tổ chức và giao tiếp giữa các thành viên.

Trello là một ứng dụng có thể sử dụng miễn phí với những tính năng phục vụ nhu cầu cơ bản. Còn với các tính năng nâng cao, người dùng sẽ phải trả mức phí hàng tháng để sử dụng. Do đó, các doanh nghiệp vừa, nhỏ hoặc freelancer sẽ thích hợp với phần mềm này.

Nó có thể giúp sắp xếp các dự án trở nên dễ dàng bằng thao tác di chuyển từ trái sang phải các thẻ trong bảng làm việc cả trên máy tính lẫn điện thoại.

Tính năng nổi bật của phần mềm Trello

Phần mềm Trello cung cấp nhiều tính năng nổi bật hỗ trợ hiệu quả trong việc quản lý và tổ chức công việc. Dưới đây là các tính năng chính của Trello:

- **Sắp xếp và quản lý công việc:** Trello giúp tổ chức và phân loại công việc một cách rõ ràng, dựa trên các vị trí và tình trạng công việc. Các nhiệm vụ có thể được sắp xếp vào các danh sách riêng biệt để dễ dàng theo dõi và quản lý.
- **Tổng hợp và phân chia công việc:** Trello cho phép phân công nhiệm vụ cho từng thành viên trong nhóm. Mỗi nhiệm vụ có thể đi kèm với mô tả chi tiết, thời gian hoàn thành và trạng thái công việc, giúp mọi người trong nhóm nắm bắt tình hình tiến độ công việc.
- **Hỗ trợ làm việc nhóm:** Tính năng cộng tác của Trello cho phép các thành viên trong nhóm yêu cầu sự giúp đỡ từ nhau khi thực hiện các nhiệm vụ, thúc đẩy sự hợp tác và hỗ trợ trong quá trình làm việc.

- **Sử dụng đơn giản, linh hoạt và miễn phí:** Trello có giao diện thân thiện, dễ sử dụng và linh hoạt. Phần mềm này miễn phí sử dụng với các tính năng cơ bản, và chỉ yêu cầu trả phí nếu người dùng có nhu cầu sử dụng các tính năng nâng cao.

Nhờ những tính năng này, Trello trở thành một công cụ quản lý công việc hữu ích và hiệu quả, phù hợp với các nhóm làm việc từ nhỏ đến lớn.

Thành phần cơ bản trong Trello

Phần mềm Trello sử dụng một hệ thống thành phần đơn giản nhưng mạnh mẽ để tổ chức công việc. Dưới đây là các thành phần cơ bản trong Trello:

- **Card:** Card là các thẻ thông tin trong Trello, bao gồm các trường như *Title* (tiêu đề) và *Description* (mô tả). Card dùng để lưu trữ các công việc, có thể là nhiệm vụ (*task*), tính năng sản phẩm (*feature*), câu hỏi (*question*), hoặc các mục công việc khác. Mỗi Card còn có *checklist*, cho phép chia nhỏ công việc thành các bước cụ thể để theo dõi và hoàn thành.
- **List:** List là danh sách tổng hợp các Card có cùng tình trạng hoặc tính chất, ví dụ như một *List* có thể là *Doing* (đang thực hiện), *Done* (đã hoàn thành), hoặc các danh mục khác tùy theo nhu cầu của nhóm. List giúp phân loại và tổ chức công việc theo tiến độ.
- **Board:** Board là bảng chứa các List, có thể coi như là một dự án hoặc một công việc lớn. Mỗi Board có thể chứa nhiều List, giúp phân chia và theo dõi tình trạng tiến hành các đầu việc của dự án. Board giúp dễ dàng quản lý toàn bộ quá trình làm việc của nhóm.
- **Organization:** Organization là thành phần tổng hợp các Board và các thành viên trong nhóm. Đây là mức độ tổ chức cao hơn, giúp quản lý nhiều dự án và nhóm làm việc trong một không gian chung.

Các thành phần này giúp dễ dàng quản lý, phân chia công việc và theo dõi tiến độ trong một dự án hoặc công việc nhóm, từ đó nâng cao hiệu quả làm việc.

Đăng ký sử dụng Trello

Để đăng ký và sử dụng Trello, có thể thực hiện các bước sau:

1. Mở trình duyệt web và nhập địa chỉ: <http://trello.com>.
2. Tại trang chủ của Trello, tìm và click vào mục **Login** (Đăng nhập).
3. Có thể chọn một trong hai tùy chọn đăng nhập:
 - **Login with Google:** Đăng nhập bằng tài khoản Google cá nhân.
 - **Sign-up:** Đăng ký tài khoản mới nếu chưa có tài khoản Trello.

Sau khi hoàn tất các bước trên, sẽ có thể bắt đầu sử dụng Trello để quản lý công việc và dự án của mình.

5.1.2. Asana

Asana là công cụ quản lý dự án toàn diện, giúp lập kế hoạch, phân công nhiệm vụ, theo dõi tiến độ và báo cáo chi tiết về công việc. Asana đặc biệt hữu ích trong việc phân chia công việc chi tiết và theo dõi tiến độ công việc, giúp các nhóm duy trì năng suất làm việc và quản lý các nhiệm vụ dễ dàng.

Tính năng nổi bật của Asana

- **Thiết kế thanh dashboard:** Asana cung cấp một thanh dashboard trực quan, giúp người dùng dễ dàng hình dung và theo dõi công việc của mình. Người dùng có thể dễ dàng nắm bắt tiến độ và trạng thái các nhiệm vụ trong một cái nhìn tổng quan.
- **Chức năng Timeline:** Asana hỗ trợ tính năng *timeline* để tạo ra các dự án liên kết và đồng bộ với nhau. Chức năng này giúp tự động cập nhật tiến độ công việc và đảm bảo các nhiệm vụ và dự án khớp với nhau một cách liên tục, giúp việc quản lý trở nên dễ dàng hơn.
- **Lập kế hoạch và liên kết với các công cụ khác:** Asana hỗ trợ lên kế hoạch công việc và liên kết trực tiếp với email, tệp đính kèm, và nhiều công cụ khác trên cùng một nền tảng. Điều này giúp tối ưu hóa quy trình làm việc và giảm thiểu sự gián đoạn trong giao tiếp.
- **Tính năng xem hình ảnh lớn:** Asana cung cấp khả năng theo dõi lịch trình công việc rõ ràng, giúp người dùng dễ dàng phát hiện các lỗi hổng hoặc vấn đề trong tiến độ công việc và điều chỉnh kịp thời.
- **Báo cáo tiến độ:** Phần mềm cung cấp các báo cáo tiến độ chi tiết cho từng dự án và cho phép chia sẻ báo cáo này với tất cả thành viên trong nhóm. Điều này giúp mọi người trong nhóm đều nắm bắt được tình hình dự án và đóng góp hiệu quả hơn.
- **Ưu tiên công việc quan trọng:** Asana tự động ưu tiên những nhiệm vụ quan trọng nhất và hiển thị chúng trước, giúp người dùng tập trung vào những công việc cấp bách và cần thiết.
- **Các mẫu quy trình làm việc:** Phần mềm Asana được thiết kế với nhiều quy trình và mẫu làm việc có sẵn để người dùng lựa chọn và sử dụng. Điều này giúp tiết kiệm thời gian khi thiết lập các dự án hoặc công việc.

- **Không gian làm việc an toàn:** Asana cung cấp một không gian làm việc an toàn và bảo mật, giúp các thành viên trong nhóm chia sẻ thông tin một cách hiệu quả và đảm bảo chỉ những người có quyền truy cập mới thấy được thông tin liên quan.

Nhờ các tính năng này, Asana là công cụ hữu ích giúp doanh nghiệp tổ chức và quản lý công việc một cách khoa học, tối ưu hóa năng suất và đạt được mục tiêu một cách hiệu quả.

Đăng ký sử dụng Asana

1. Truy cập trang web <https://asana.com>.
2. Nhấn vào **Get Started** hoặc **Try Asana Free**.
3. Chọn phương thức đăng ký:
 - **Đăng ký với Google:** Chọn **Continue with Google** và đăng nhập với tài khoản Google cá nhân.
 - **Đăng ký bằng Email:** Chọn **Sign up with Email**, nhập địa chỉ email và tạo mật khẩu.
4. Xác minh email qua liên kết gửi đến hộp thư.
5. Hoàn tất thiết lập tài khoản và bắt đầu sử dụng Asana để tạo dự án và quản lý công việc.

5.1.3. Jira

Jira là một công cụ mạnh mẽ được thiết kế cho các dự án phần mềm, đặc biệt là trong việc quản lý quy trình Agile. Với Jira, các nhóm có thể tạo bảng nhiệm vụ, theo dõi lỗi, quản lý Sprint và theo dõi tiến độ chi tiết của từng phần công việc trong dự án. Jira cũng hỗ trợ phân tích và tối ưu hóa quy trình làm việc, đặc biệt trong các dự án phát triển phần mềm phức tạp.

Các thành phần và thuật ngữ cơ bản trong Jira

Các thành phần và thuật ngữ cơ bản mà người dùng cần nắm vững để sử dụng Jira hiệu quả:

1. **Roles (Vai trò):** Trong Jira, *roles* xác định vai trò của từng thành viên trong dự án. Các vai trò này giúp phân quyền truy cập và chỉ định trách nhiệm cụ thể, ví dụ như người chịu trách nhiệm phân bổ nguồn lực hoặc tham gia vào nhóm dự án. Vai trò giúp quản lý các quyền và nhiệm vụ trong dự án.
2. **Issue (Vấn đề):** *Issue* trong Jira đại diện cho các đơn vị công việc như nhiệm vụ (*task*), lỗi (*bug*), tính năng (*feature*), hoặc các công việc khác trong dự án. Đây là

yếu tố trung tâm để theo dõi tiến độ công việc, phân công nhiệm vụ và xử lý các vấn đề phát sinh trong suốt vòng đời dự án.

3. **Project (Dự án):** *Project* trong Jira là nơi chứa tất cả các *issue* liên quan đến một dự án cụ thể. Nó giúp quản trị viên thiết lập quyền truy cập và phân quyền cho các thành viên tham gia, đảm bảo chỉ những người có quyền mới có thể thực hiện các hành động nhất định trong dự án.
4. **Component (Thành phần):** *Component* là các phần nhỏ trong dự án, giúp phân chia công việc thành các nhóm đơn vị dễ quản lý. Các thành phần này giúp tổ chức sản phẩm dự án và phân loại công việc theo nhóm, từ đó dễ dàng theo dõi và điều phối tiến độ.
5. **Workflow (Quy trình công việc):** *Workflow* là quy trình làm việc từ khi một *issue* được tạo ra cho đến khi hoàn thành. Nó xác định các bước mà mỗi *issue* phải trải qua, bao gồm các trạng thái như "Mới tạo", "Đang thực hiện", và "Hoàn thành". *Workflow* giúp theo dõi tiến độ và kiểm soát quá trình làm việc của nhóm.
6. **Priority (Mức độ ưu tiên):** *Priority* xác định mức độ quan trọng của mỗi *issue*. Nó giúp nhóm làm việc quyết định vấn đề nào cần xử lý trước và vấn đề nào có thể trì hoãn, từ đó tối ưu hóa việc phân bổ nguồn lực và thời gian.
7. **Status (Trạng thái):** *Status* chỉ ra tình trạng hiện tại của một *issue* trong *workflow*. Các trạng thái phổ biến bao gồm "Mới tạo", "Đang tiến hành", "Đang chờ đánh giá", và "Hoàn thành". Trạng thái giúp nhóm hiểu rõ vị trí hiện tại của công việc và các bước tiếp theo cần thực hiện.

Những thành phần và thuật ngữ trên là cơ sở để người dùng hiểu và vận hành hiệu quả Jira trong việc quản lý công việc và dự án.

Đăng ký sử dụng Jira

1. Truy cập trang web <https://jira.atlassian.com>.
2. Nhấn vào **Get it free** (Dùng thử miễn phí).
3. Chọn phương thức đăng ký:
 - **Đăng ký với Google:** Chọn **Sign up with Google** và đăng nhập với tài khoản Google cá nhân.
 - **Đăng ký bằng Email:** Chọn **Sign up with Email**, nhập địa chỉ email và tạo mật khẩu.
4. Xác minh email của qua liên kết gửi đến hộp thư.
5. Hoàn tất thiết lập tài khoản và bắt đầu tạo dự án và quản lý công việc trên Jira.

5.1.4. Slack

Slack là công cụ giao tiếp nhóm mạnh mẽ, giúp các thành viên trong nhóm kết nối và chia sẻ thông tin một cách nhanh chóng. Slack hỗ trợ các kênh giao tiếp tức thời, giúp tăng cường sự phối hợp giữa các nhóm và dễ dàng chia sẻ tài liệu. Mặc dù không phải là công cụ quản lý dự án chính, nhưng Slack đóng vai trò quan trọng trong việc duy trì liên lạc và trao đổi thông tin trong suốt quá trình làm việc.

Các tính năng nổi bật của Slack

Ứng dụng Slack tích hợp nhiều tính năng hữu ích, hỗ trợ tối đa các hoạt động làm việc nhóm, bao gồm:

1. **Cài đặt nhắc nhở:** Slack cho phép thiết lập nhắc nhở để ghi nhớ các công việc cần làm. Mỗi khi đến thời gian thực hiện, ứng dụng sẽ tự động gửi thông báo nhắc nhở, giúp tránh tình trạng bỏ lỡ các công việc quan trọng.
2. **Ghim tin nhắn:** Tính năng này cho phép ghim những tin nhắn quan trọng, giúp các thành viên trong nhóm dễ dàng đọc và theo dõi thông tin mà không bị lạc mất trong các cuộc trò chuyện.
3. **Chủ đề:** Slack cho phép người dùng tùy chỉnh chủ đề của các cuộc trò chuyện, giúp dễ dàng phân biệt các kênh và nhóm theo sở thích cá nhân hoặc đặc thù công việc. Điều này giúp tạo sự linh hoạt và tổ chức trong quá trình giao tiếp.
4. **Chỉnh sửa nội dung tin nhắn:** Slack cung cấp tính năng chỉnh sửa tin nhắn đã gửi. Người dùng có thể dễ dàng sửa lại nội dung tin nhắn khi cần thiết, giúp cải thiện tính chính xác và rõ ràng của thông tin được chia sẻ.
5. **Liên kết với ứng dụng lưu trữ khác:** Slack tích hợp với nhiều ứng dụng lưu trữ đám mây như Github, Google Drive, Dropbox, v.v., giúp mở rộng khả năng lưu trữ và chia sẻ dữ liệu trong nhóm mà không cần rời khỏi ứng dụng.
6. **Emoji:** Người dùng có thể gửi các biểu tượng cảm xúc (emoji) trong tin nhắn, giúp giao tiếp thêm sinh động và dễ hiểu. Slack còn cho phép tạo thêm emoji tùy chỉnh để đáp ứng nhu cầu sử dụng của nhóm.
7. **Team directory:** Tính năng *Team directory* hiển thị thông tin chi tiết về từng thành viên trong nhóm, bao gồm chức vụ, trạng thái online hoặc offline. Điều này giúp dễ dàng tìm kiếm và tương tác với các thành viên trong nhóm.

Những tính năng này giúp Slack trở thành công cụ mạnh mẽ hỗ trợ làm việc nhóm hiệu quả, từ việc quản lý công việc đến cải thiện giao tiếp và tổ chức trong môi trường làm việc.

Đăng ký sử dụng Slack

1. Truy cập trang web <https://slack.com>.
2. Nhấn vào **Get Started** hoặc **Try for free**.
3. Chọn phương thức đăng ký:
 - **Đăng ký với Google:** Chọn **Sign in with Google** và đăng nhập bằng tài khoản Google.
 - **Đăng ký bằng Email:** Nhập địa chỉ email và tạo mật khẩu.
4. Xác minh email qua liên kết gửi đến hộp thư.
5. Hoàn tất thiết lập tài khoản và bắt đầu sử dụng Slack để giao tiếp và làm việc nhóm.

5.1.5. GitHub/GitLab

GitHub và GitLab là các nền tảng quản lý mã nguồn, cho phép các nhóm phát triển phần mềm theo dõi các thay đổi mã nguồn, hợp nhất các nhánh và quản lý phiên bản dự án. Cả hai công cụ này đều cung cấp các tính năng mạnh mẽ để hỗ trợ việc cộng tác trong phát triển phần mềm, như theo dõi lỗi và quản lý tài nguyên.

Các tính năng nổi bật của GitHub và GitLab

GitHub và GitLab là hai nền tảng quản lý mã nguồn dựa trên hệ thống kiểm soát phiên bản Git, hỗ trợ các nhà phát triển trong việc quản lý, lưu trữ và cộng tác trên các dự án phần mềm. Tuy GitHub và GitLab đều là những nền tảng phổ biến để quản lý phiên bản và hợp tác trong phát triển phần mềm, nhưng chúng có những điểm khác biệt đáng kể. Dưới đây là các tính năng nổi bật của mỗi nền tảng:

GitHub:

- **Quản lý mã nguồn với Git:** GitHub cung cấp môi trường để lưu trữ và quản lý mã nguồn sử dụng Git, cho phép theo dõi lịch sử thay đổi và quản lý các phiên bản của dự án.
- **Cộng tác và làm việc nhóm:** Hỗ trợ pull request, issue tracking và code review, giúp các thành viên trong nhóm dễ dàng đóng góp và xem xét mã nguồn của nhau.
- **GitHub Actions:** Cung cấp công cụ tích hợp liên tục (CI/CD) để tự động hóa quy trình xây dựng, kiểm thử và triển khai ứng dụng.
- **GitHub Pages:** Cho phép tạo và lưu trữ các trang web tĩnh trực tiếp từ kho mã nguồn.

- **Bảo mật và quyền truy cập:** Cung cấp các tính năng bảo mật như xác thực hai yếu tố, quyền truy cập linh hoạt và khả năng kiểm soát ai có thể xem hoặc đóng góp vào kho mã nguồn.
- **Tích hợp và API:** Hỗ trợ tích hợp với nhiều công cụ và dịch vụ khác, cùng với API mạnh mẽ để tùy chỉnh và mở rộng chức năng.

GitLab:

- **Quản lý mã nguồn và DevOps tích hợp:** GitLab không chỉ cung cấp quản lý mã nguồn mà còn tích hợp các công cụ DevOps từ lập kế hoạch, phát triển đến triển khai và giám sát.
- **CI/CD mạnh mẽ:** GitLab CI/CD cho phép tự động hóa quy trình kiểm thử và triển khai, hỗ trợ các pipeline tùy chỉnh.
- **Quản lý dự án:** Cung cấp các công cụ như issue tracking, milestones, và boards để quản lý công việc và tiến độ dự án.
- **Container Registry:** Tích hợp kho lưu trữ container, cho phép lưu trữ và quản lý các container Docker trực tiếp trong GitLab.
- **Bảo mật và tuân thủ:** Cung cấp các tính năng kiểm tra bảo mật mã nguồn, quản lý bí mật và tuân thủ quy định.
- **Self-hosted và Cloud-hosted:** GitLab có thể được triển khai trên máy chủ riêng hoặc sử dụng dịch vụ đám mây, linh hoạt cho nhu cầu của doanh nghiệp.

Đăng ký sử dụng GitHub và GitLab

Đăng ký tài khoản GitHub:

1. Truy cập trang web <https://github.com>.
2. Nhấn vào nút **Sign up**.
3. Nhập **email**, tạo **mật khẩu** và chọn **tên người dùng**.
4. Xác minh email qua liên kết gửi đến hộp thư.
5. Hoàn tất thiết lập tài khoản và bắt đầu sử dụng GitHub.

Đăng ký tài khoản GitLab:

1. Truy cập trang web <https://gitlab.com>.
2. Nhấn vào **Register**.

3. Nhập **tên người dùng**, **email** và tạo **mật khẩu**.
4. Xác minh email qua liên kết gửi đến hộp thư.
5. Hoàn tất thiết lập tài khoản và bắt đầu sử dụng GitLab.

5.2. Sử dụng các công cụ quản lý dự án

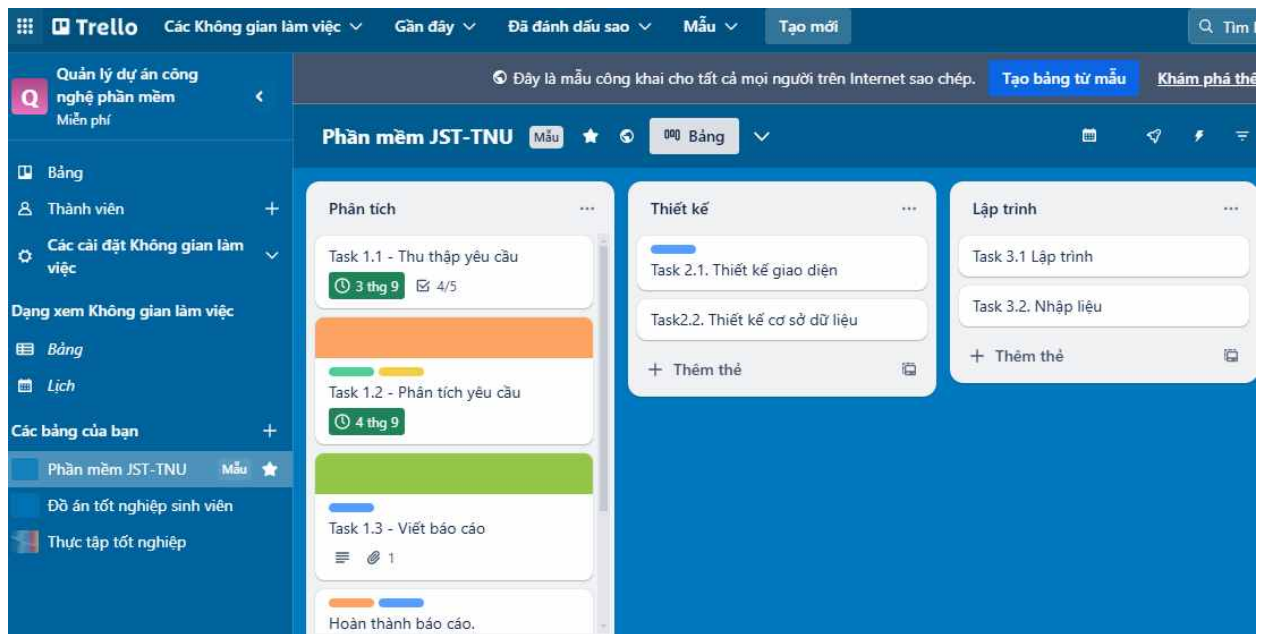
Trong phần này, chúng ta sẽ tìm hiểu cách sử dụng các công cụ quản lý dự án phổ biến như Trello, Asana, Jira, Slack và GitHub/GitLab để quản lý nhóm, theo dõi tiến độ, và lập báo cáo chi tiết về tiến độ, chi phí và kết quả dự án cho các bên liên quan.

5.2.1. Theo dõi tiến độ

Các công cụ như Asana, Trello và Jira giúp theo dõi tiến độ dự án thông qua các bảng nhiệm vụ và các chỉ số về tiến độ công việc. Người quản lý dự án có thể dễ dàng nắm bắt tình hình công việc, nhận diện các nhiệm vụ đang bị trì hoãn và có các biện pháp điều chỉnh kịp thời.

Sử dụng Trello

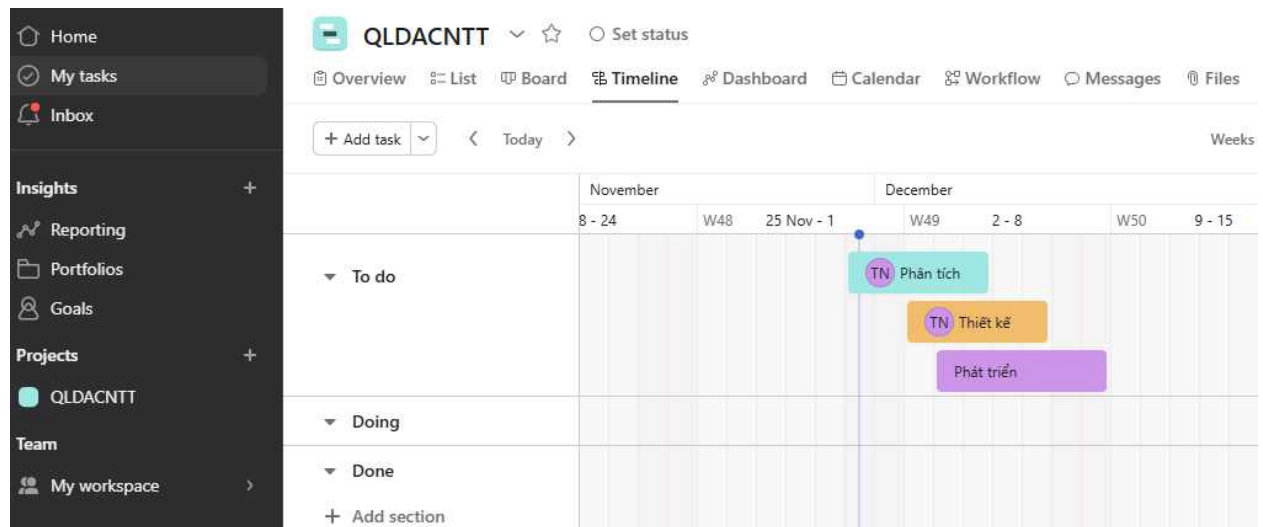
- **Tạo Board và List:**
 - **Board** đại diện cho dự án đang quản lý.
 - **List** chia công việc theo các giai đoạn như *To Do* (Cần làm), *Doing* (Đang làm), *Done* (Hoàn thành).
- **Sử dụng Card để quản lý nhiệm vụ:**
 - Tạo **Card** cho từng nhiệm vụ cụ thể.
 - Thêm mô tả, gán thành viên phụ trách, đặt ngày đến hạn, và thêm checklist để theo dõi các bước công việc.
 - Di chuyển **Card** giữa các **List** để cập nhật tiến độ.
- **Theo dõi và cập nhật tiến độ:**
 - Sử dụng nhãn (labels) để phân loại công việc.
 - Theo dõi hoạt động (activity) trên từng **Card** để nắm bắt thay đổi.



Hình 5.1 Minh họa sử dụng Trello trong quản lý dự án

Sử dụng Asana

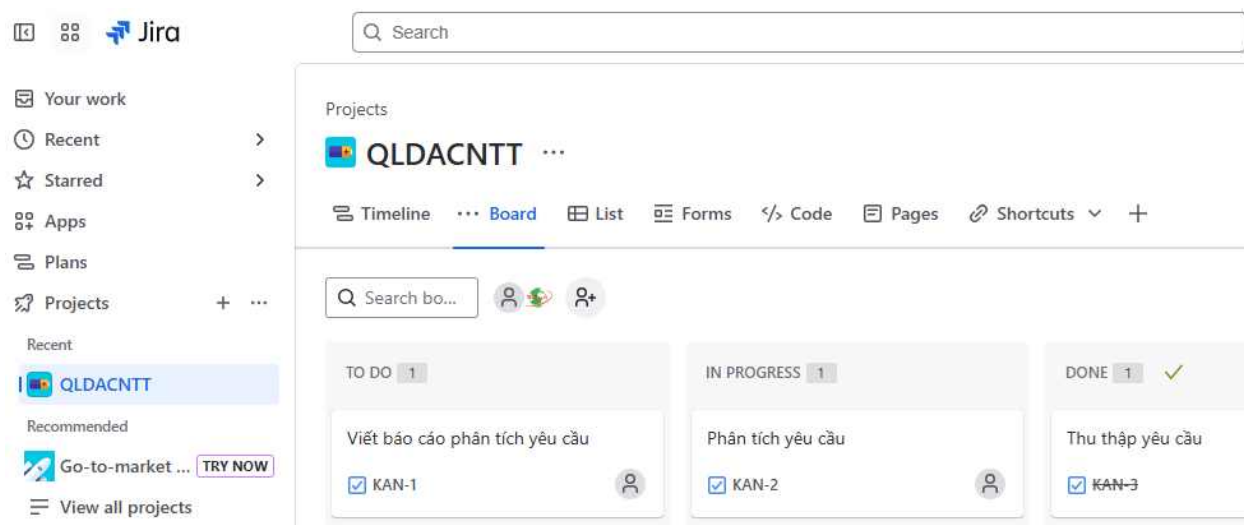
- **Tạo dự án và nhiệm vụ:**
 - Tạo **Project** mới cho dự án.
 - Thêm **Tasks** (nhiệm vụ) và **Subtasks** (nhiệm vụ con) cho các công việc cụ thể.
 - Gán nhiệm vụ cho thành viên, đặt ngày đến hạn và mức độ ưu tiên.
- **Sử dụng Timeline và Calendar:**
 - **Timeline** cho phép xem toàn bộ dự án dưới dạng biểu đồ Gantt.
 - **Calendar** giúp theo dõi nhiệm vụ theo ngày tháng.
- **Theo dõi tiến độ:**
 - Sử dụng **Progress** để cập nhật tình hình dự án.
 - Thêm **Milestones** để đánh dấu các cột mốc quan trọng.



Hình 5.2. Màn hình làm việc quản lý dự án với Asana

Sử dụng Jira

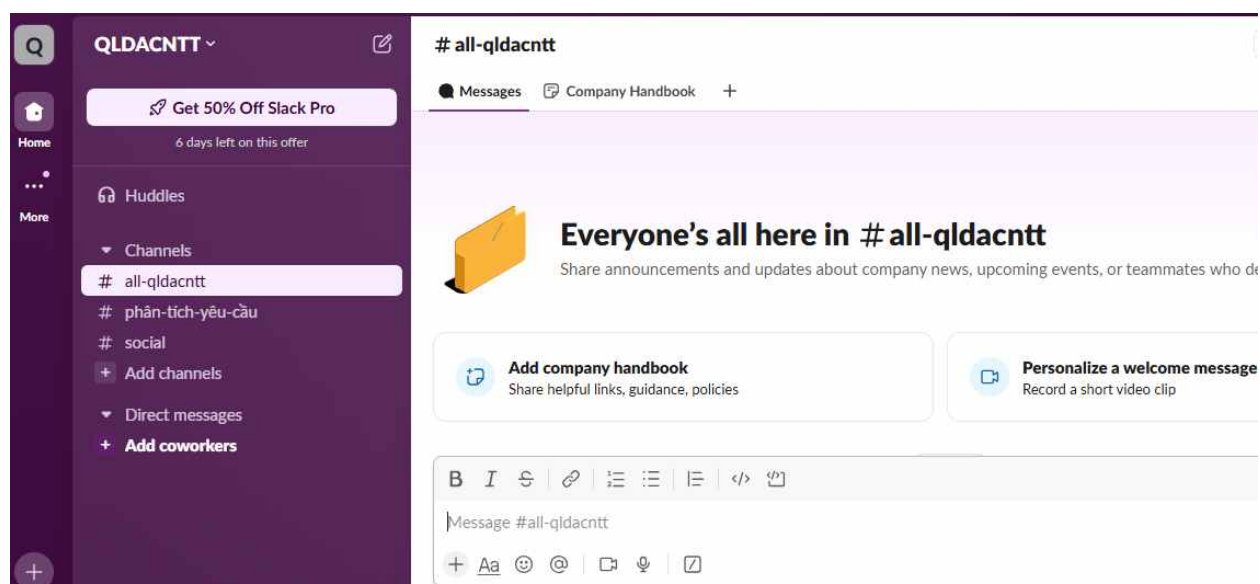
- **Tạo dự án và Issue:**
 - Tạo **Project** mới, chọn loại dự án phù hợp (Scrum, Kanban).
 - Tạo **Issue** cho từng nhiệm vụ, lỗi hoặc yêu cầu mới.
- **Quản lý Workflow:**
 - Sử dụng **Workflow** để định nghĩa các trạng thái của **Issue** như *To Do*, *In Progress*, *In Review*, *Done*.
 - Chuyển đổi trạng thái của **Issue** để phản ánh tiến độ thực tế.
- **Sử dụng Board:**
 - **Kanban Board** hoặc **Scrum Board** giúp trực quan hóa tiến độ công việc.
 - Kéo thả **Issue** giữa các cột tương ứng với trạng thái.



Hình 5.3. Màn hình làm việc quản lý dự án với Jira

Sử dụng Slack

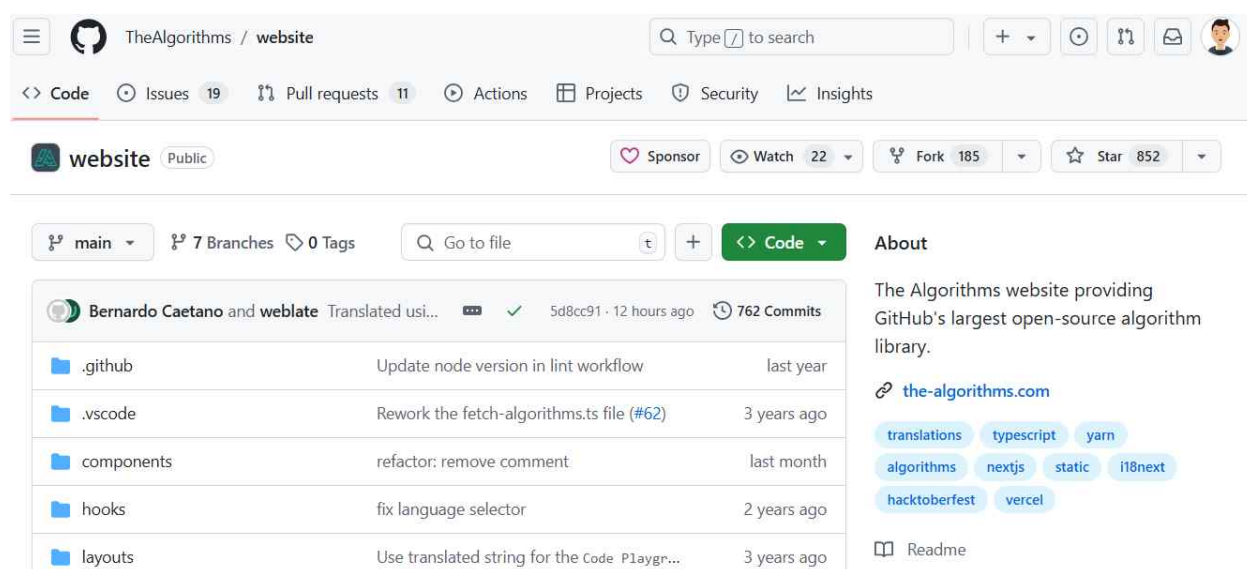
- **Tạo các kênh làm việc:**
 - Tạo **Channels** riêng cho từng dự án hoặc nhóm công việc.
- **Sử dụng tích hợp với công cụ khác:**
 - Kết nối Slack với Trello, Asana, Jira, GitHub/GitLab để nhận thông báo cập nhật tiến độ trực tiếp.
 - Thiết lập **Bot** để nhắc nhở và cập nhật công việc.
- **Giao tiếp và cập nhật:**
 - Sử dụng tin nhắn, cuộc gọi để thảo luận và cập nhật tình hình công việc.
 - Chia sẻ tài liệu và thông tin quan trọng trong kênh chung.



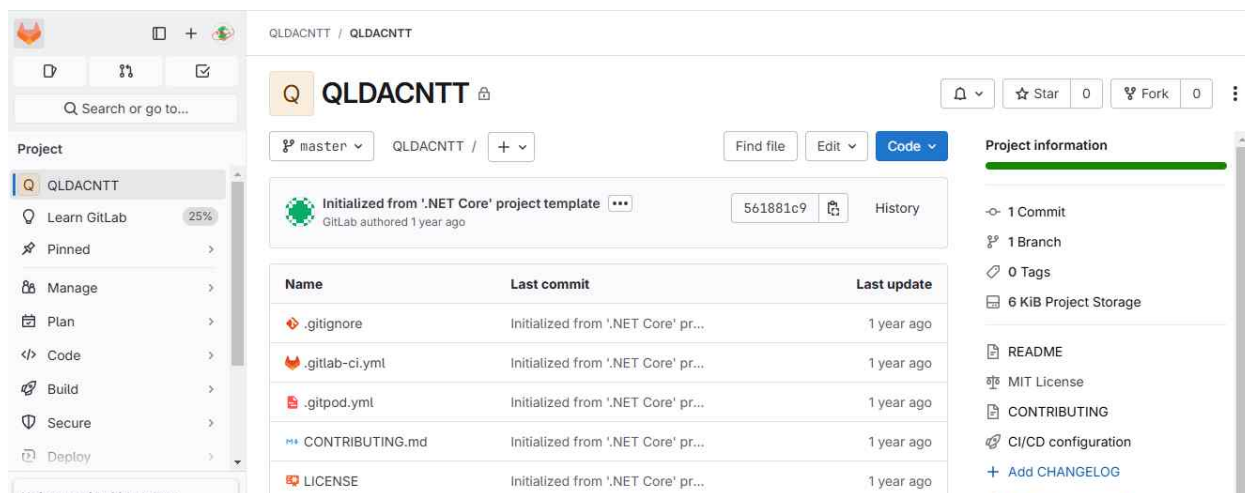
Hình 5.4. Màn hình làm việc quản lý dự án với Slack

Sử dụng GitHub/GitLab

- **Quản lý Issue và Pull Request:**
 - Tạo **Issue** để ghi nhận nhiệm vụ, lỗi cần xử lý.
 - Sử dụng **Pull Request** (GitHub) hoặc **Merge Request** (GitLab) để đề xuất thay đổi mã nguồn.
- **Sử dụng Projects/Boards:**
 - Tạo **Projects** với các **Kanban Board** để quản lý công việc.
 - Sắp xếp **Issue** vào các cột như *Backlog*, *In Progress*, *Review*, *Done*.
- **Theo dõi tiến độ phát triển mã nguồn:**
 - Kiểm tra lịch sử commit để nắm bắt các thay đổi.
 - Sử dụng **Code Review** để đảm bảo chất lượng mã nguồn.



Hình 5.5. Màn hình làm việc với Github



Hình 5.6. Màn hình làm việc với Github

5.2.2. Tạo báo cáo tiến độ

Báo cáo tiến độ là một phần quan trọng trong quá trình quản lý dự án, giúp các bên liên quan, bao gồm cả nhà quản lý và các thành viên trong nhóm, nắm bắt được tình hình hiện tại của dự án. Thông qua các báo cáo này, các chỉ số quan trọng như tiến độ công việc, mức độ hoàn thành các nhiệm vụ, và các vấn đề phát sinh trong quá trình thực hiện sẽ được trình bày một cách rõ ràng và chi tiết. Việc cập nhật tiến độ kịp thời giúp các bên liên quan có cái nhìn tổng quan về dự án, nhận diện được các vấn đề hoặc sự chậm trễ có thể ảnh hưởng đến kết quả cuối cùng, và từ đó đưa ra các quyết định kịp thời để điều chỉnh kế hoạch hoặc phân bổ nguồn lực phù hợp.

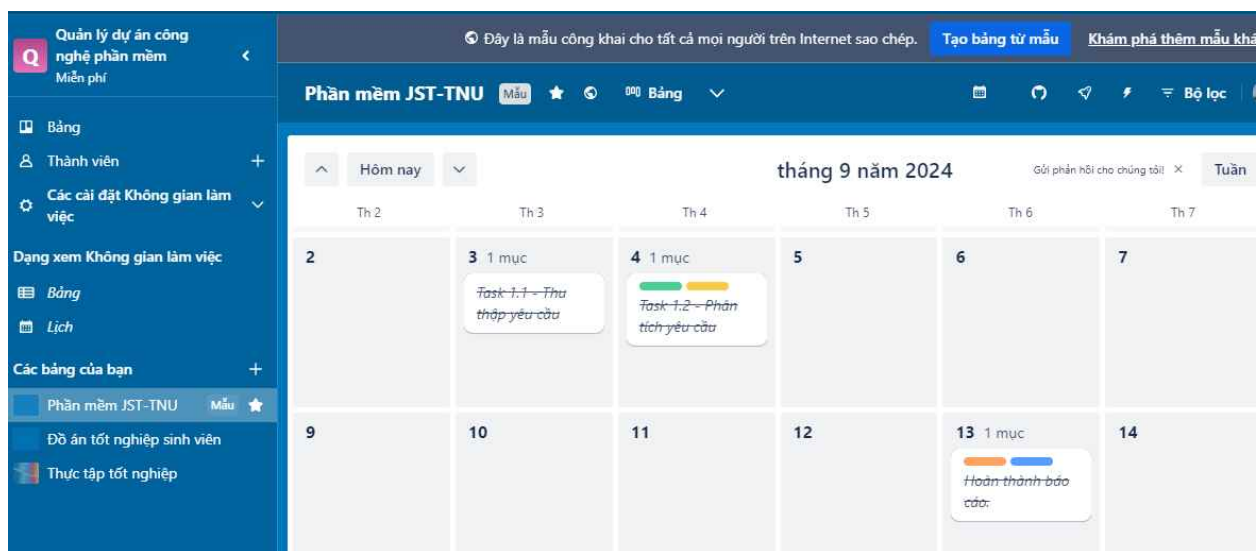
Các công cụ quản lý dự án hiện nay đều tích hợp tính năng tạo báo cáo tự động, giúp người quản lý dễ dàng thu thập và tổng hợp dữ liệu từ các nguồn khác nhau mà không cần phải làm việc thủ công quá nhiều. Những công cụ này có thể tự động cập nhật các chỉ số về tiến độ công việc, thời gian hoàn thành dự kiến, cũng như các thay đổi trong kế hoạch dự án. Điều này không chỉ giúp tiết kiệm thời gian mà còn giảm thiểu sai sót trong quá trình thu thập và báo cáo thông tin. Các báo cáo tự động thường được trình bày trực quan, dễ hiểu, với các biểu đồ và bảng biểu rõ ràng, giúp người nhận báo cáo dễ dàng nắm bắt được thông tin và hiểu được tình hình dự án một cách nhanh chóng.

Hơn nữa, các báo cáo này có thể được cá nhân hóa để phục vụ các nhu cầu khác nhau của các bên liên quan. Ví dụ, một báo cáo có thể được tùy chỉnh để tập trung vào các chỉ số về tiến độ công việc của nhóm phát triển, trong khi một báo cáo khác có thể chi tiết hơn về chi phí và ngân sách của dự án. Điều này giúp các nhà quản lý và các bên liên quan có cái nhìn chi tiết và toàn diện về dự án, đồng thời cải thiện khả năng ra quyết định và điều chỉnh chiến lược dự án khi cần thiết.

Nhờ vào các tính năng này, việc tạo báo cáo tiến độ trở nên nhanh chóng, dễ dàng và chính xác hơn, góp phần quan trọng vào việc duy trì sự minh bạch và hiệu quả trong quản lý dự án.

Sử dụng Trello

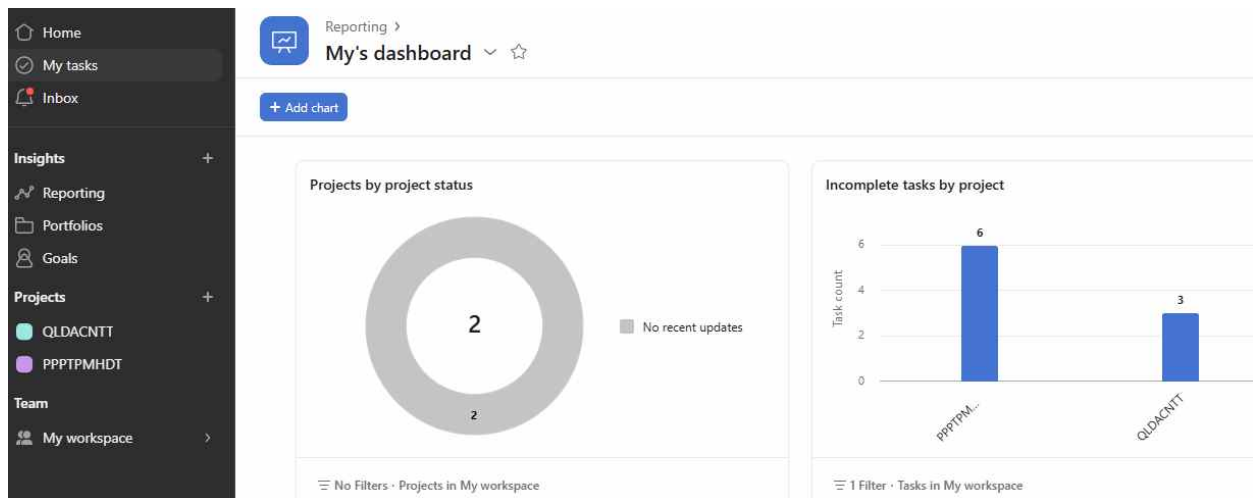
- **Sử dụng các Tiện ích bổ sung (Power-Ups):**
 - **Kích hoạt Tiện ích Lịch (Calendar Power-Up):** Cho phép xem và xuất lịch trình công việc một cách trực quan.
 - **Sử dụng Tiện ích Báo cáo (Reporting Power-Ups) như Blue Cat Reports:** Để tạo báo cáo tùy chỉnh theo nhu cầu.
- **Xuất dữ liệu:**
 - **Xuất bảng Trello sang định dạng CSV hoặc JSON:** Để phân tích dữ liệu sâu hơn hoặc sử dụng trong các công cụ khác.



Hình 5.7. Màn hình xem lịch trình công việc với Trello

Sử dụng Asana

- **Sử dụng Bảng điều khiển (Dashboard) và Báo cáo (Reports):**
 - **Truy cập Bảng điều khiển:** Để xem các biểu đồ tiến độ công việc.
 - **Tạo Báo cáo tùy chỉnh (Custom Reports):** Dựa trên các tiêu chí như người phụ trách, ngày đến hạn.
- **Cập nhật Trạng thái Dự án:**
 - **Sử dụng Cập nhật Trạng thái Dự án (Project Status Update):** Để thông báo tiến độ cho các bên liên quan.



Hình 5.8. Màn hình xem báo cáo công việc với asana

Sử dụng Jira

- **Sử dụng công cụ báo cáo tích hợp:**
 - **Truy cập Báo cáo (Reports):** Để xem các biểu đồ như Tiến độ Tăng dần (*Burnup*), Tiến độ Giảm dần (*Burndown*), Tốc độ (*Velocity*).
- **Tạo Bảng điều khiển tùy chỉnh (Dashboard):**
 - **Thêm các Tiện ích (Gadgets):** Để hiển thị thông tin quan trọng.
 - **Chia sẻ Bảng điều khiển:** Với nhóm và quản lý để mọi người cùng theo dõi.

Sử dụng Slack

- **Sử dụng ứng dụng tích hợp:**
 - **Kết nối với công cụ báo cáo:** Để nhận tóm tắt tiến độ trong kênh Slack.
- **Thiết lập thông báo định kỳ:**
 - **Sử dụng Nhắc nhở Lên lịch (Scheduled Reminders):** Để gửi báo cáo tiến độ hàng ngày, hàng tuần cho nhóm.

Sử dụng GitHub/GitLab

- **Sử dụng Thống kê (Insights/Analytics):**
 - **Truy cập Thống kê:** Để xem các số liệu về cam kết (commit), yêu cầu hợp nhất (pull request), vấn đề (issue).
- **Tạo báo cáo từ Vấn đề và Yêu cầu Hợp nhất:**

- **Lọc và xuất danh sách Vấn đề, Yêu cầu Hợp nhất:** Theo trạng thái, người phụ trách.

5.2.3. Lập báo cáo chi tiết về dự án

Các công cụ như Trello, Jira và Asana cho phép lập báo cáo chi tiết về dự án, từ tiến độ công việc đến các vấn đề phát sinh và các nguồn lực sử dụng trong dự án. Những báo cáo này giúp các bên liên quan đánh giá tình hình dự án và đưa ra quyết định kịp thời.

Sử dụng Trello

- **Tổng hợp thông tin dự án:**
 - Thu thập dữ liệu từ các **Card**, bao gồm thời gian thực hiện, người phụ trách.
- **Sử dụng công cụ hỗ trợ:**
 - Sử dụng **Third-party Apps** như **Bridge24** để tạo báo cáo chi tiết và biểu đồ.

Sử dụng Asana

- **Sử dụng Portfolio:**
 - Tạo **Portfolio** để tổng hợp nhiều dự án.
 - Theo dõi tiến độ, chi phí và kết quả tổng quan.
- **Xuất báo cáo chi tiết:**
 - Xuất dữ liệu dự án sang Excel hoặc Google Sheets để phân tích sâu hơn.

Sử dụng Jira

- **Sử dụng Advanced Roadmaps:**
 - Lập kế hoạch và báo cáo ở cấp độ cao hơn.
 - Theo dõi phụ thuộc giữa các nhiệm vụ, dự án.
- **Báo cáo thời gian và chi phí:**
 - Sử dụng **Time Tracking** để báo cáo thời gian làm việc.
 - Tích hợp với công cụ quản lý chi phí nếu cần.

Sử dụng Slack

- **Thu thập thông tin giao tiếp:**

- Tổng hợp các quyết định quan trọng, thảo luận trong các kênh.
- **Tích hợp với công cụ báo cáo:**
 - Sử dụng ứng dụng như **Polly** hoặc **Standuply** để thu thập thông tin và tạo báo cáo.

Sử dụng GitHub/GitLab

- **Báo cáo phát triển phần mềm:**
 - Tổng hợp thông tin từ **Commits**, **Pull Requests** để đánh giá tiến độ.
- **Sử dụng Wiki và README:**
 - Ghi chép chi tiết về dự án, hướng dẫn sử dụng, kết quả đạt được.

Câu hỏi ôn tập cuối chương

1. Công cụ nào dưới đây sử dụng mô hình Kanban để quản lý công việc?
2. Asana chủ yếu được sử dụng để làm gì trong quản lý dự án?
3. Công cụ nào hỗ trợ giao tiếp nhóm và chia sẻ thông tin tức thời?
4. Điểm mạnh của Jira trong quản lý dự án phần mềm là gì?
5. GitHub và GitLab chủ yếu được sử dụng để làm gì trong các dự án phần mềm?
6. Khi nào nên sử dụng Trello trong một dự án nhỏ?
7. Công cụ nào dưới đây giúp theo dõi tiến độ công việc và lập báo cáo chi tiết?
8. Slack phù hợp nhất với mục đích gì trong một nhóm dự án?
9. Jira hỗ trợ tốt nhất trong việc nào sau đây?
10. Tính năng báo cáo tiến độ trong Asana giúp gì cho người quản lý dự án?

Chương 6: Phân tích, báo cáo và kiểm tra dự án

Nội dung chính của chương

Chương này trình bày về phân tích, báo cáo và kiểm tra dự án trong quản lý phần mềm, tập trung vào việc sử dụng các chỉ số KPIs để đo lường hiệu quả và tiến độ. Nội dung bao gồm định nghĩa, vai trò và cách áp dụng các KPIs phổ biến như EV, PV, CPI, SPI để đánh giá hiệu suất và tình hình dự án. Ngoài ra, chương cũng hướng dẫn cách xây dựng báo cáo tiến độ và báo cáo kết thúc dự án, bao gồm các thành phần chính như tình trạng công việc, nguồn lực, rủi ro và các vấn đề tồn đọng. Bên cạnh đó, các kỹ thuật phân tích kết quả thực tế so với kế hoạch và cách sử dụng báo cáo để cải thiện quản lý dự án cũng được đề cập chi tiết, nhằm cung cấp cái nhìn toàn diện và hỗ trợ ra quyết định hiệu quả cho các bên liên quan.

Mục tiêu cần đạt của chương

Hiểu các chỉ số KPIs dùng để đo lường và báo cáo hiệu quả dự án.

- Sinh viên nắm vững khái niệm và vai trò của KPIs trong quản lý dự án.
- Sinh viên có thể áp dụng các chỉ số KPIs như **EV, PV, AC, CPI, SPI, EAC**, và **VAC** để đánh giá tiến độ và kết quả dự án.
- Sinh viên hiểu cách chọn KPIs phù hợp và sử dụng chúng để cải thiện quản lý dự án.

Hiểu cách xây dựng báo cáo tiến độ và báo cáo kết thúc dự án.

- Sinh viên biết cách xây dựng báo cáo tiến độ và báo cáo kết thúc dự án.
- Sinh viên hiểu cách trình bày kết quả thực tế và so sánh với kế hoạch dự án.
- Sinh viên có kỹ năng sử dụng báo cáo để truyền đạt tình trạng và kết quả của dự án.

6.1. KPIs và các chỉ số được sử dụng để đo lường và báo cáo hiệu quả dự án.

Mục tiêu: Giúp sinh viên hiểu cách thiết lập và sử dụng KPIs nhằm đo lường hiệu suất của dự án một cách hiệu quả, từ đó đưa ra các điều chỉnh phù hợp để đảm bảo đạt mục tiêu.

6.1.1. Định nghĩa KPIs

6.1.1.1. Khái niệm về KPIs (Key Performance Indicators)

Định nghĩa: KPIs là viết tắt của "Key Performance Indicators" - các chỉ số chính dùng để đo lường hiệu quả của một hoạt động hay một dự án cụ thể so với các mục tiêu đề ra.

Mục đích: Các KPIs giúp định lượng mức độ thành công trong việc đạt được các mục tiêu chiến lược, giúp các nhà quản lý và các bên liên quan hiểu rõ liệu dự án có tiến triển như mong đợi hay không.

Phân biệt với các chỉ số khác: Không giống như các chỉ số chung chung khác, KPIs tập trung vào các yếu tố quan trọng nhất, thường là những yếu tố quyết định tới sự thành công của dự án.

6.1.1.2. Vai trò của KPIs trong Quản lý Dự án

Đo lường hiệu quả: KPIs giúp đánh giá xem các hoạt động của dự án có đạt được mục tiêu ban đầu không, bằng cách đo lường các kết quả đầu ra cụ thể.

Giám sát tiến độ: KPIs cung cấp dữ liệu thực tế về tiến độ, giúp nhận diện các vấn đề ngay khi phát sinh, từ đó quản lý có thể kịp thời điều chỉnh kế hoạch và tối ưu hóa nguồn lực.

Động lực và tập trung cho đội ngũ: KPIs giúp đội ngũ dự án hiểu rõ các mục tiêu cần đạt, từ đó tăng động lực làm việc và tập trung vào những nhiệm vụ quan trọng nhất.

Đánh giá sau dự án: Cuối cùng, KPIs cũng là thước đo quan trọng để đánh giá tổng kết dự án, xác định những thành công và những điểm cần cải thiện trong tương lai.

6.1.1.3. Các Đặc điểm Của KPIs Hiệu Quả

Cụ thể và rõ ràng: KPIs phải đo lường những yếu tố có tác động trực tiếp đến thành công của dự án và được xác định rõ ràng để tránh gây hiểu nhầm.

Có thể đo lường được: Một KPI hiệu quả phải có thể đo lường bằng số liệu cụ thể, để dễ dàng kiểm tra và so sánh trong suốt quá trình thực hiện dự án.

Có thể đạt được: KPIs phải khả thi, phù hợp với nguồn lực và thời gian của dự án.

Có liên quan đến mục tiêu: KPIs cần phải có ý nghĩa đối với các mục tiêu chiến lược của dự án. Nếu không, KPIs sẽ không đóng góp nhiều cho việc đánh giá hiệu suất.

Có thời hạn cụ thể: Mỗi KPI phải có khung thời gian thực hiện rõ ràng để đảm bảo tiến độ và hiệu quả.

6.1.1.4. Các Cấp Độ KPIs Thường Gặp

KPIs cấp dự án: Đo lường hiệu suất của toàn bộ dự án, ví dụ như thời gian hoàn thành, chi phí thực tế so với ngân sách, và chất lượng đầu ra.

KPIs cấp nhóm: Đánh giá hiệu suất làm việc của từng nhóm trong dự án, chẳng hạn như hiệu quả của nhóm phát triển, nhóm kiểm thử, hoặc nhóm quản lý.

KPIs cá nhân: Đo lường hiệu suất của từng thành viên, chẳng hạn như số lượng công việc hoàn thành đúng hạn hoặc mức độ đóng góp vào thành công chung.

6.1.1.5. Ví dụ về KPIs trong Quản lý Dự án

Thời gian hoàn thành: Xác định liệu dự án có đi đúng tiến độ hay không.

Số lượng công việc hoàn thành: Đo lường phần trăm công việc đã hoàn tất so với kế hoạch.

Mức độ hài lòng của khách hàng: Đánh giá mức độ khách hàng hài lòng với kết quả dự án.

Chi phí thực tế so với ngân sách: So sánh chi phí thực tế của dự án với ngân sách ban đầu, từ đó đánh giá khả năng quản lý tài chính.

6.1.2. Các chỉ số KPIs phổ biến

Các chỉ số hiệu suất (KPIs) trong quản lý dự án giúp đánh giá sự tiến bộ, chất lượng, chi phí và hiệu suất của đội ngũ trong suốt quá trình thực hiện dự án. Dưới đây là một số KPIs phổ biến được sử dụng trong các lĩnh vực khác nhau của dự.

6.1.2.1. KPIs về Tiến Độ Dự Án

a. PV (Planned Value)

$$PV = \left(\frac{\text{Công việc dự kiến hoàn thành}}{\text{Tổng công việc}} \right) \times \text{BAC (Budget at Completion)}$$

- **% công việc theo kế hoạch:** tỷ lệ phần trăm công việc dự kiến hoàn thành trong một thời gian cụ thể.
- **BAC (Budget at Completion):** tổng ngân sách đã được phê duyệt cho toàn bộ dự án.

PV cho biết dự định sẽ hoàn thành bao nhiêu công việc tính đến thời điểm hiện tại.

b. EV (Earned Value)

$$EV = \left(\frac{\text{Công việc đã hoàn thành}}{\text{Tổng công việc}} \right) \times \text{BAC (Budget at Completion)}$$

BAC (Budget at Completion) là tổng ngân sách dự án được phê duyệt.

EV giúp xác định giá trị công việc thực tế mà dự án đã đạt được. Nếu EV lớn hơn kế hoạch (PV), dự án đang tiến triển tốt hơn dự kiến. Ngược lại, nếu EV thấp hơn PV, dự án có thể đang chậm tiến độ.

Ví dụ:

Giả sử một dự án có **BAC = 500.000 USD** và theo kế hoạch, dự án sẽ hoàn thành 40% công việc vào thời điểm hiện tại. Tuy nhiên, thực tế, dự án chỉ hoàn thành 30% công việc.

Tính EV theo kế hoạch:

$$EV = 30\% \times 500.000 = 150.000 \text{ USD}$$

Trong trường hợp này, giá trị công việc đã hoàn thành (EV) là **150.000 USD**.

So sánh EV với PV:

Nếu theo kế hoạch, dự án dự kiến hoàn thành 40% công việc vào thời điểm này, thì giá trị công việc dự kiến là:

$$PV = 40\% \times 500.000 = 200.000 \text{ USD}$$

EV = 150.000 USD (thực tế hoàn thành)

PV = 200.000 USD (dự kiến hoàn thành)

EV thấp hơn PV, có nghĩa là dự án đang chậm tiến độ so với kế hoạch.

EV là chỉ số quan trọng để theo dõi tiến độ của dự án và đánh giá khả năng hoàn thành các mục tiêu dự án trong thời gian và ngân sách đã xác định.

c. Schedule Performance Index (SPI)

Công thức:

$$SPI = \frac{EV}{PV}$$

- **EV (Earned Value):** Giá trị công việc đã hoàn thành.
- **PV (Planned Value):** Giá trị công việc dự kiến hoàn thành theo kế hoạch.

SPI đo lường tiến độ của dự án. Nếu $SPI > 1$ dự án đang đi nhanh hơn kế hoạch, nếu $SPI < 1$ dự án đang chậm hơn so với kế hoạch.

Ví dụ: Giả sử một dự án có PV = 50.000 USD (theo kế hoạch) và EV = 40.000 USD (giá trị công việc hoàn thành).

$$SPI = \frac{40.000}{50.000} = 0,8$$

SPI = 0,8 có nghĩa là dự án chỉ hoàn thành 80% so với kế hoạch tại thời điểm đánh giá. Điều này cho thấy dự án đang chậm và có thể cần điều chỉnh lại tiến độ hoặc tăng nguồn lực để hoàn thành đúng thời hạn.

d. Schedule Variance (SV)

Công thức:

$$SV = EV - PV$$

SV đo lường sự khác biệt giữa giá trị công việc thực tế hoàn thành và giá trị công việc dự kiến theo kế hoạch. SV dương có nghĩa là dự án đi trước kế hoạch, SV âm nghĩa là dự án chậm hơn kế hoạch.

Ví dụ: Với PV = 50.000 USD và EV = 40.000 USD như ví dụ trên:

$$SV = 40.000 - 50.000 = -10.000 \text{ USD}$$

SV = -10.000 USD cho thấy dự án đang chậm hơn so với tiến độ dự kiến với mức chênh lệch 10.000 USD, điều này có thể gây ra rủi ro nếu không có biện pháp xử lý kịp thời.

e. Tỷ lệ hoàn thành công việc (Task Completion Rate – TCR): Đo lường số lượng công việc đã hoàn thành so với tổng số công việc đã được lên kế hoạch. Chỉ số này giúp đánh giá tiến độ và xác định xem dự án có đang đi đúng hướng không.

Công thức:

$$TCR = \left(\frac{\text{Số nhiệm vụ hoàn thành}}{\text{Tổng số nhiệm vụ}} \right) \times 100$$

Ví dụ: Giả sử một dự án có kế hoạch thực hiện 50 nhiệm vụ. Tính đến thời điểm báo cáo, 30 nhiệm vụ đã được hoàn thành.

$$\text{Tỷ lệ hoàn thành công việc} = \frac{30}{50} \times 100 = 60\%$$

Tỷ lệ 60% cho thấy dự án đã hoàn thành hơn một nửa số nhiệm vụ theo kế hoạch. Tuy nhiên, nếu đây là điểm giữa của thời gian dự án, tỷ lệ này có thể cho thấy dự án đang

đi đúng tiến độ. Nếu tỷ lệ thấp hơn mong đợi, quản lý cần xem xét các nguyên nhân và thực hiện điều chỉnh để đảm bảo hoàn thành dự án đúng hạn.

f. Thời gian hoàn thành nhiệm vụ (Task Cycle Time – TCT): Đo thời gian trung bình cần thiết để hoàn thành một công việc hoặc một nhiệm vụ cụ thể. Đây là một chỉ số quan trọng để xác định hiệu suất của đội ngũ dự án và phát hiện những chỗ tắc nghẽn trong quy trình.

Công thức:

$$\text{Thời gian hoàn thành nhiệm vụ} = \frac{\sum \text{Thời gian hoàn thành từng nhiệm vụ}}{\text{Số lượng nhiệm vụ}}$$

Nếu một dự án có 5 nhiệm vụ với thời gian hoàn thành lần lượt là 3 ngày, 5 ngày, 2 ngày, 4 ngày, và 6 ngày, thì thời gian hoàn thành nhiệm vụ trung bình là:

$$\text{Thời gian hoàn thành nhiệm vụ} = \frac{3 + 5 + 2 + 4 + 6}{5} = 4 \text{ ngày}$$

Thời gian hoàn thành nhiệm vụ trung bình là 4 ngày, cho thấy tốc độ hoàn thành của nhóm. Nếu thời gian này quá dài hoặc có sự dao động lớn, quản lý có thể cần cải tiến quy trình hoặc phân bổ nguồn lực tốt hơn để cải thiện hiệu quả.

g. Tỷ lệ chậm trễ (On-Time Completion Percentage – OTCP): Đánh giá tỷ lệ các nhiệm vụ hoàn thành đúng thời hạn. Chỉ số này giúp quản lý theo dõi sự tuân thủ thời gian, xác định các vấn đề gây chậm trễ và kịp thời đưa ra các điều chỉnh.

Công thức:

$$\text{Tỷ lệ chậm trễ (\%)} = \frac{\text{Số nhiệm vụ hoàn thành đúng thời hạn}}{\text{Tổng số nhiệm vụ}} \times 100$$

Ví dụ: Nếu có 40 nhiệm vụ trong dự án, trong đó 30 nhiệm vụ hoàn thành đúng thời hạn, thì tỷ lệ hoàn thành đúng thời hạn là:

$$\text{Tỷ lệ chậm trễ} = \frac{30}{40} \times 100 = 75\%$$

Tỷ lệ 75% cho thấy đa số các nhiệm vụ đã hoàn thành đúng thời hạn, nhưng 25% nhiệm vụ bị trễ. Điều này có thể là tín hiệu để nhóm dự án tập trung hơn vào việc cải thiện tuân thủ thời gian cho các nhiệm vụ tiếp theo hoặc điều chỉnh nguồn lực cho các nhiệm vụ quan trọng sắp tới.

6.1.2.2. KPIs về Ngân Sách và Chi Phí

a. BAC (Budget at Completion) - Ngân sách hoàn thành

BAC là ngân sách tổng thể của dự án, được ước tính trước khi dự án bắt đầu. Đây là số tiền dự tính sẽ chi cho toàn bộ dự án. BAC giúp cung cấp bức tranh tổng thể về chi phí của dự án và sẽ được so sánh với các chỉ số chi phí thực tế trong suốt quá trình thực hiện dự án.

Giả sử ngân sách ban đầu của dự án là 200.000 USD. Điều này có nghĩa là khi bắt đầu dự án, tổng chi phí dự tính sẽ là 200.000 USD.

b. Cost Variance (CV)

Công thức:

$$CV = EV - AC$$

CV cho biết sự chênh lệch giữa chi phí thực tế và chi phí theo kế hoạch. CV dương cho thấy chi phí tiết kiệm, CV âm cho thấy chi phí vượt quá kế hoạch.

Ví dụ: Với $EV = 40.000$ USD và $AC = 45.000$ USD:

$$CV = 40.000 - 45.000 = -5.000 \text{ USD}$$

$CV = -5.000$ USD cho thấy dự án đã chi tiêu vượt quá dự toán 5.000 USD, có thể cần điều chỉnh lại kế hoạch tài chính hoặc xem xét các phần chi phí không hiệu quả.

c. Chi phí thực tế so với ngân sách (Budget Variance): So sánh chi phí thực tế của dự án với ngân sách ban đầu, giúp nhận diện xem dự án có đang vượt ngân sách hay không. Chỉ số này là một công cụ quan trọng để quản lý tài chính trong dự án.

Công thức:

$$BV = \text{Chi phí thực tế} - \text{Ngân sách}$$

Ví dụ:

- Ngân sách dự án ban đầu là \$100,000.
- Chi phí thực tế trong quá trình thực hiện dự án là \$120,000.

$$BV = 120,000 - 100,000 = 20,000$$

Chỉ số **BV = 20,000** cho thấy dự án đã vượt ngân sách \$20,000. Điều này có thể là dấu hiệu cần kiểm tra lại các khoản chi phí và tìm cách điều chỉnh để giữ cho dự án trong giới hạn ngân sách.

d. TCPI (To Complete Performance Index) - Chỉ số hiệu suất cần đạt được để hoàn thành dự án trong ngân sách

Công thức:

$$TCPI = \frac{BAC - EV}{BAC - AC}$$

- **BAC:** Ngân sách hoàn thành.
- **EV:** Giá trị công việc đã hoàn thành.
- **AC:** Chi phí thực tế đã chi cho công việc.

TCPI giúp đo lường hiệu suất cần thiết trong tương lai để hoàn thành dự án trong phạm vi ngân sách đã thiết lập. Nếu TCPI cao, dự án cần cải thiện hiệu suất để giữ chi phí trong phạm vi ngân sách.

Ví dụ: Nếu BAC = 200.000 USD, EV = 80.000 USD và AC = 90.000 USD:

$$TCPI = \frac{200.000 - 80.000}{200.000 - 90.000} = \frac{120.000}{110.000} \approx 1,09$$

TCPI = 1,09 cho thấy hiệu suất cần phải đạt được từ bây giờ đến khi hoàn thành dự án để đảm bảo chi phí không vượt quá ngân sách. Điều này có nghĩa là, để hoàn thành trong ngân sách, đội ngũ cần phải đạt hiệu suất vượt trội.

e. EAC (Estimate at Completion) - Ước tính chi phí khi hoàn thành

Công thức:

$$EAC = AC + (BAC - EV)$$

EAC dự báo chi phí cuối cùng của dự án dựa trên hiệu suất thực tế cho đến nay. Nó giúp dự báo tổng chi phí mà dự án sẽ tiêu tốn khi hoàn thành, cho phép các nhà quản lý điều chỉnh kế hoạch chi tiêu nếu cần thiết.

Ví dụ: Giả sử BAC = 200.000 USD, AC = 90.000 USD và EV = 80.000 USD:

$$EAC = 90.000 + (200.000 - 80.000) = 90.000 + 120.000 = 210.000$$

EAC = 210.000 USD cho thấy chi phí dự kiến của dự án khi hoàn thành sẽ cao hơn ngân sách ban đầu (200.000 USD). Điều này cho thấy dự án có thể cần biện pháp kiểm soát chi phí hoặc điều chỉnh ngân sách để tránh vượt quá.

f. Tỷ lệ hoàn vốn đầu tư (Return on Investment - ROI): Đo lường lợi nhuận từ dự án so với chi phí đầu tư, giúp đánh giá giá trị của dự án đối với công ty.

Công thức:

$$ROI = \frac{\text{Lợi nhuận thu được từ dự án}}{\text{Chi phí đầu tư}} \times 100$$

Ví dụ:

- Chi phí đầu tư ban đầu là \$200,000.
- Lợi nhuận thu được từ dự án là \$500,000.

$$ROI = \frac{500,000 - 200,000}{200,000} \times 100 = \frac{300,000}{200,000} \times 100 = 150\%$$

ROI = 150% có nghĩa là dự án đã mang lại 150% lợi nhuận so với chi phí đầu tư ban đầu, cho thấy đây là một dự án có lợi nhuận rất cao.

g. Tỷ lệ chi phí trên tổng số giờ làm việc (Cost Performance Index - CPI): Đây là chỉ số đo lường hiệu quả sử dụng nguồn lực bằng cách so sánh giữa chi phí thực tế và chi phí dự tính để thực hiện công việc đã hoàn thành.

Công thức:

$$CPI = \frac{\text{Giá trị công việc đã hoàn thành (EV)}}{\text{Chi phí thực tế (AC)}}$$

Ví dụ:

- Giá trị công việc đã hoàn thành (EV) là \$90,000.
- Chi phí thực tế (AC) là \$100,000.

$$CPI = \frac{90,000}{100,000} = 0.9$$

CPI = 0.9 có nghĩa là dự án đang chi tiêu nhiều hơn so với giá trị công việc đã hoàn thành. Một CPI dưới 1 cho thấy dự án đang có hiệu quả chi phí kém, và có thể cần phải tìm cách cải thiện quản lý tài chính để giảm chi phí hoặc tăng giá trị công việc hoàn thành.

6.1.2.3. KPIs về Chất Lượng Dự Án

a. Tỷ lệ lỗi (Error Rate)

Đánh giá số lượng lỗi hoặc vấn đề phát sinh trong quá trình triển khai dự án. Chỉ số này là thước đo chất lượng sản phẩm và hiệu quả của quy trình kiểm tra chất lượng.

$$\text{Error Rate} = \left(\frac{\text{Số lượng lỗi}}{\text{Tổng số công việc hoặc nhiệm vụ}} \right) \times 100$$

Ví dụ:

Trong một dự án phần mềm, có 10 lỗi phát sinh trong tổng số 500 dòng mã được triển khai.

$$\text{Tỷ lệ lỗi} = (10 / 500) * 100 = 2\%$$

Tỷ lệ lỗi giúp đánh giá mức độ chất lượng của sản phẩm dự án. Nếu tỷ lệ lỗi cao, cần cải thiện quy trình kiểm thử và kiểm tra chất lượng. Chỉ số này cho biết sự ổn định của sản phẩm và hiệu quả của việc kiểm soát chất lượng.

b. Tỷ lệ tái thực hiện nhiệm vụ (Rework Rate)

Đo lường phần trăm công việc phải làm lại do lỗi hoặc không đạt yêu cầu ban đầu. Chỉ số này phản ánh hiệu quả của quy trình sản xuất hoặc phát triển, cũng như chất lượng đầu ra của dự án.

Công thức:

$$\text{Rework Rate} = \left(\frac{\text{Số công việc tái làm}}{\text{Tổng số công việc}} \right) \times 100$$

Ví dụ:

Có 5 nhiệm vụ cần làm lại trong tổng số 50 nhiệm vụ đã hoàn thành.

$$\text{Tỷ lệ tái làm} = (5 / 50) * 100 = 10\%$$

Tỷ lệ tái thực hiện cao cho thấy có vấn đề trong quy trình thực hiện công việc, chẳng hạn như việc không đáp ứng yêu cầu ban đầu hoặc các lỗi không được phát hiện sớm. Chỉ số này giúp đo lường hiệu quả quy trình và chất lượng của công việc.

c. Mức độ hài lòng của khách hàng (Customer Satisfaction Score)

Đây là một chỉ số đo lường sự hài lòng của khách hàng với sản phẩm hoặc dịch vụ của dự án, thông qua khảo sát hoặc đánh giá phản hồi từ khách hàng.

Công thức:

$$\text{Customer Satisfaction Score} = \left(\frac{\text{Tổng điểm khảo sát}}{\text{Số lượng phản hồi}} \right) \times 100$$

Ví dụ:

Sau khi khảo sát, khách hàng đánh giá mức độ hài lòng với điểm trung bình là 8/10, với 50 phản hồi.

$$\text{Customer Satisfaction Score} = (8 * 50 / 10) = 40\%$$

Chỉ số này phản ánh mức độ hài lòng của khách hàng về sản phẩm hoặc dịch vụ của dự án. Mức độ hài lòng cao thường có thể dẫn đến khách hàng quay lại hoặc giới thiệu cho người khác, giúp gia tăng giá trị lâu dài cho doanh nghiệp.

6.1.2.4. KPIs về Đội Ngũ và Hiệu Suất Làm Việc

a. Tỷ lệ thay đổi nhân sự (Employee Turnover Rate)

Đo lường tỷ lệ nhân sự rời khỏi đội ngũ dự án trong một thời gian cụ thể, giúp đánh giá mức độ ổn định và sự hài lòng của nhân sự trong dự án.

Công thức:

$$\text{Employee Turnover Rate} = \left(\frac{\text{Số nhân viên rời đi}}{\text{Số nhân viên tổng}} \right) \times 100$$

Ví dụ:

Trong một dự án, có 2 nhân viên rời đi trong tổng số 20 nhân viên.

$$\text{Employee Turnover Rate} = (2 / 20) * 100 = 10\%$$

Chỉ số này đo lường sự ổn định của đội ngũ nhân sự. Tỷ lệ thay đổi nhân sự cao có thể cho thấy sự bất mãn trong đội ngũ, có thể ảnh hưởng đến hiệu quả công việc và sự liên tục của dự án.

b. Hiệu suất cá nhân (Individual Productivity)

Đánh giá số lượng công việc mà mỗi thành viên trong nhóm hoàn thành trong một khoảng thời gian nhất định. Chỉ số này giúp quản lý theo dõi hiệu suất cá nhân và tối ưu hóa việc phân bổ công việc.

$$P = \frac{\text{Số công việc hoàn thành}}{\text{Thời gian làm việc}}$$

Ví dụ:

Một nhân viên hoàn thành 10 nhiệm vụ trong 40 giờ làm việc.

$$\text{Individual Productivity} = 10 / 40 = 0.25 \text{ nhiệm vụ/giờ.}$$

Chỉ số này giúp đánh giá hiệu suất của mỗi thành viên trong nhóm. Nhân viên có năng suất cao thường hoàn thành nhiều nhiệm vụ trong thời gian ngắn, giúp nâng cao hiệu quả của cả đội ngũ.

c. Sự gắn kết của đội ngũ (Employee Engagement Score)

Đo lường mức độ gắn kết và cam kết của đội ngũ với dự án, thường thông qua các khảo sát hoặc đánh giá nội bộ.

Công thức:

$$\text{Employee Engagement Score} = \frac{\text{Điểm khảo sát về sự gắn kết}}{\text{Số lượng khảo sát}} \times 100$$

Ví dụ:

Sau khảo sát về sự gắn kết, 80% nhân viên đánh giá sự hài lòng với công việc và đội nhóm.

$$\text{Employee Engagement Score} = 80\%$$

Chỉ số này đo lường mức độ gắn kết của nhân viên đối với công ty và dự án. Mức độ gắn kết cao giúp tăng hiệu suất làm việc và giữ chân nhân sự tài năng.

6.1.2.5. KPIs về Thời Gian và Hiệu Quả Quy Trình

a. Tỷ lệ hoàn thành theo kế hoạch (Planned vs. Actual Hours)

Đo lường thời gian thực tế so với thời gian đã lên kế hoạch để hoàn thành nhiệm vụ, giúp nhận diện các sai lệch và đưa ra biện pháp điều chỉnh khi cần thiết.

Công thức:

$$\text{Planned vs. Actual Hours} = \left(\frac{\text{Thời gian thực tế}}{\text{Thời gian đã lên kế hoạch}} \right) \times 100$$

Ví dụ:

Thời gian thực tế là 120 giờ trong khi thời gian đã lên kế hoạch là 100 giờ.

$$\text{Planned vs. Actual Hours} = (120 / 100) * 100 = 120\%$$

Chỉ số này giúp đánh giá mức độ hoàn thành các nhiệm vụ dựa trên thời gian đã lên kế hoạch. Nếu tỷ lệ vượt quá 100%, có thể có sự chậm trễ hoặc sự cố phát sinh trong quá trình thực hiện công việc.

b. Tỷ lệ hiệu quả của quy trình (Process Efficiency Rate)

Đo tỷ lệ các quy trình hiệu quả bằng cách so sánh thời gian hoặc tài nguyên thực tế với tiêu chuẩn đã đặt ra, giúp tối ưu hóa quy trình làm việc.

Công thức:

$$\text{Process Efficiency Rate} = \left(\frac{\text{Thời gian chuẩn}}{\text{Thời gian thực tế}} \right) \times 100$$

Ví dụ:

Quy trình chuẩn yêu cầu 50 giờ, nhưng thời gian thực tế là 60 giờ.

$$\text{Process Efficiency Rate} = (50 / 60) * 100 = 83.33\%$$

Tỷ lệ này đo lường mức độ hiệu quả của quy trình làm việc. Nếu tỷ lệ này thấp, có thể cần cải thiện quy trình để sử dụng tài nguyên hiệu quả hơn.

c. Thời gian phản hồi (Response Time)

Đo lường thời gian trung bình để giải quyết các vấn đề hoặc yêu cầu phát sinh trong dự án, giúp đánh giá mức độ linh hoạt và khả năng phản ứng của đội ngũ dự án.

Công thức:

$$\text{Response Time} = \frac{\text{Tổng thời gian phản hồi}}{\text{Số lượng yêu cầu}}$$

Ví dụ:

Tổng thời gian phản hồi là 240 phút cho 60 yêu cầu.

$$\text{Response Time} = 240 / 60 = 4 \text{ phút/yêu cầu}$$

Thời gian phản hồi đo lường độ linh hoạt của đội ngũ trong việc xử lý các yêu cầu và vấn đề phát sinh. Chỉ số này giúp đánh giá mức độ phản ứng nhanh chóng của nhóm đối với tình huống.

6.1.2.6. KPIs về Khả Năng Phản Ứng và Thích Ứng

a. Thời gian khắc phục sự cố (Issue Resolution Time)

Đo lường thời gian cần thiết để giải quyết các sự cố phát sinh trong dự án. Chỉ số này phản ánh khả năng của nhóm trong việc xử lý vấn đề kịp thời.

Công thức:

$$\text{Issue Resolution Time} = \frac{\text{Tổng thời gian giải quyết sự cố}}{\text{Số sự cố}}$$

Ví dụ:

Tổng thời gian khắc phục sự cố là 120 phút cho 6 sự cố.

$$\text{Issue Resolution Time} = 120 / 6 = 20 \text{ phút/sự cố}$$

Thời gian khắc phục sự cố đo lường khả năng của nhóm trong việc giải quyết vấn đề phát sinh. Thời gian càng ngắn, khả năng phản ứng và xử lý sự cố của nhóm càng nhanh.

b. Tỷ lệ thích ứng với thay đổi (Adaptability Rate)

Đánh giá khả năng của đội ngũ dự án trong việc thích ứng với các thay đổi về yêu cầu, công nghệ hoặc môi trường làm việc.

Công thức:

$$\text{Adaptability Rate} = \left(\frac{\text{Số lượng thay đổi được xử lý}}{\text{Tổng số thay đổi}} \right) \times 100$$

Ví dụ:

Trong 10 thay đổi yêu cầu, nhóm đã xử lý được 9 thay đổi.

$$\text{Adaptability Rate} = (9 / 10) * 100 = 90\%$$

Tỷ lệ này đo lường khả năng của đội ngũ trong việc thích ứng với các thay đổi trong dự án. Tỷ lệ cao cho thấy đội ngũ linh hoạt và có khả năng thay đổi nhanh chóng khi cần thiết.

6.2. Xây dựng báo cáo tiến độ và báo cáo kết thúc dự án

6.2.1. Nội dung báo cáo tiến độ

6.2.1.1. Mục tiêu của Báo Cáo Tiến Độ

- Cung cấp thông tin chi tiết về tình hình hiện tại của dự án, giúp quản lý và các bên liên quan đánh giá tiến độ so với kế hoạch ban đầu.
- Xác định các rủi ro, khó khăn hoặc vấn đề tồn tại ảnh hưởng đến tiến độ dự án.
- Đưa ra các đề xuất và biện pháp cần thiết để cải thiện hoặc điều chỉnh kế hoạch nhằm đạt được mục tiêu.

6.2.1.2. Các Thành Phần Chính của Báo Cáo Tiến Độ

- **Tóm tắt dự án:** Một phần ngắn gọn, nhắc lại mục tiêu dự án, các mốc quan trọng và thời gian dự kiến. Phần này giúp người đọc có cái nhìn tổng quan trước khi đi vào chi tiết.
- **Tiến độ công việc:**
 - **Tình trạng hiện tại:** Báo cáo về các nhiệm vụ đã hoàn thành, đang thực hiện, và chưa bắt đầu.
 - **Tỷ lệ hoàn thành công việc:** Đưa ra tỷ lệ phần trăm công việc đã hoàn thành so với kế hoạch, giúp so sánh và đánh giá nhanh tiến độ.
 - **Thời gian thực hiện:** So sánh thời gian thực tế với thời gian đã lên kế hoạch cho từng nhiệm vụ, để xác định xem dự án có đi đúng tiến độ hay không.
- **Nguồn lực sử dụng:**
 - **Nhân sự:** Đánh giá tình hình sử dụng nhân sự, bao gồm số lượng nhân viên tham gia, vai trò của họ, và đánh giá năng suất của từng thành viên nếu có.
 - **Ngân sách:** Cung cấp thông tin về chi phí thực tế so với ngân sách ban đầu, đánh giá các khoản chi vượt ngân sách (nếu có) và các giải pháp tối ưu hóa chi phí.
- **Đánh giá rủi ro:**
 - **Rủi ro hiện tại:** Liệt kê các rủi ro đã xảy ra hoặc có nguy cơ ảnh hưởng đến tiến độ dự án, như thiếu nhân sự, vượt ngân sách, hay thay đổi yêu cầu.
 - **Giải pháp khắc phục:** Đưa ra các biện pháp đã thực hiện để khắc phục rủi ro, đồng thời đề xuất các phương án dự phòng để giảm thiểu tác động trong tương lai.
- **Các vấn đề tồn đọng:**
 - **Danh sách các vấn đề:** Bao gồm các vấn đề đã phát sinh trong quá trình thực hiện và chưa được giải quyết hoàn toàn.

- **Phân tích nguyên nhân:** Đánh giá nguyên nhân của các vấn đề tồn đọng để xác định liệu chúng có liên quan đến quản lý nguồn lực, kế hoạch, hay các yếu tố khác.
- **Kế hoạch giải quyết:** Đưa ra các hành động cụ thể để giải quyết vấn đề tồn đọng trong thời gian sớm nhất.
- **Các mốc quan trọng và hoạt động sắp tới:**
 - **Mốc quan trọng đã đạt được:** Nêu rõ các mốc hoặc cột mốc quan trọng đã hoàn thành và ảnh hưởng của chúng đến dự án.
 - **Mốc sắp tới:** Đưa ra danh sách các mốc hoặc hoạt động sắp tới trong giai đoạn tiếp theo, kèm theo thời gian dự kiến và các nguồn lực cần thiết.
 - **Yêu cầu từ các bên liên quan:** Nếu có bất kỳ yêu cầu hoặc thay đổi từ các bên liên quan, cần ghi rõ nội dung và tác động đến kế hoạch ban đầu.
- **Đề xuất và kiến nghị:**
 - Đưa ra các đề xuất cụ thể để cải thiện tiến độ hoặc tối ưu hóa nguồn lực nếu cần thiết.
 - Kiến nghị các điều chỉnh về lịch trình, nguồn lực, hoặc ngân sách nếu có các yếu tố mới phát sinh.

6.2.1.3. Hình thức và Tần suất Báo Cáo Tiến Độ

- **Hình thức báo cáo:** Báo cáo tiến độ có thể ở dạng tài liệu viết, báo cáo điện tử, hoặc trình bày bằng các công cụ đồ họa như biểu đồ Gantt, sơ đồ tiến độ, giúp thông tin dễ nắm bắt và trực quan.
- **Tần suất báo cáo:** Tần suất báo cáo có thể tùy thuộc vào quy mô và tính chất của dự án, thường là hàng tuần, hàng tháng hoặc theo từng giai đoạn quan trọng của dự án.

6.2.1.4. Ví dụ

Báo Cáo Tiến Độ Dự Án – Phát Triển Phần Mềm Quản Lý Công Việc Cá Nhân Trên Điện Thoại

Trong báo cáo tiến độ dự án, các chỉ số KPI được sử dụng để đo lường và đánh giá hiệu quả tiến độ tại nhiều nội dung chính, giúp quản lý và các bên liên quan có cơ sở định lượng để đưa ra quyết định. Dưới đây là cách các KPI có thể được áp dụng trong từng nội dung của báo cáo tiến độ:

a. Tiến Độ Công Việc

Tỷ lệ hoàn thành công việc: KPI này đo lường phần trăm công việc đã hoàn thành so với kế hoạch. Đây là một chỉ số quan trọng giúp đánh giá nhanh liệu dự án đang đi đúng tiến độ hay không.

Dữ liệu:

- Tổng số công việc hoàn thành: 80 công việc
- Tổng số công việc kế hoạch: 160 công việc

Tỷ lệ hoàn thành công việc = $80/160 \times 100 = 50\%$

Phân tích: Dự án đã hoàn thành 50% công việc, cho thấy tiến độ đạt được một nửa so với kế hoạch ban đầu. Điều này cho thấy dự án đang đi đúng hướng và có thể kịp thời gian hoàn thành.

Thời gian thực hiện so với kế hoạch: KPI này đo lường mức độ chênh lệch giữa thời gian thực tế và thời gian dự kiến, giúp phát hiện các công việc bị chậm trễ.

Dữ liệu:

Thời gian thực tế: 200 giờ

Thời gian dự kiến: 180 giờ

Chênh lệch thời gian = $200 - 180 = 20$

Phân tích: Thời gian thực tế đã vượt qua 11.11% so với kế hoạch ban đầu, cho thấy có một số công việc mất thêm thời gian. Điều này có thể cần được điều chỉnh để không ảnh hưởng đến các giai đoạn tiếp theo.

b. Nguồn Lực Sử Dụng

Tỷ lệ sử dụng ngân sách: KPI này đo lường phần trăm ngân sách đã được sử dụng so với ngân sách ban đầu. Đây là một chỉ số tài chính quan trọng trong việc quản lý chi phí dự án.

Dữ liệu:

Ngân sách đã sử dụng: 50,000 USD

Ngân sách ban đầu: 100,000 USD

Tỷ lệ sử dụng ngân sách = $(50,000/100,000) \times 100 = 50\%$

Phân tích: Dự án đã sử dụng 50% ngân sách trong vòng 2 tháng đầu. Cần phải theo dõi chặt chẽ để tránh vượt ngân sách trong các giai đoạn tiếp theo.

Hiệu suất làm việc của nhân sự: KPI này đánh giá năng suất làm việc của từng thành viên hoặc nhóm dựa trên công việc đã hoàn thành và thời gian làm việc.

Dữ liệu:

Tổng số công việc hoàn thành: 80 công việc

Tổng thời gian làm việc: 320 giờ

Hiệu suất làm việc = $80/320 = 0.25$ công việc/ giờ

Phân tích: Mỗi nhân viên hoàn thành 0.25 công việc mỗi giờ, một con số hợp lý cho một nhóm có kỹ năng chuyên môn cao. Tuy nhiên, cần kiểm tra xem liệu quy trình làm việc có thể tối ưu hóa thêm hay không.

c. Đánh Giá Rủi Ro

Số lượng rủi ro phát sinh: KPI này đo lường mức độ và tần suất các rủi ro phát sinh trong dự án, giúp xác định khả năng quản lý và giảm thiểu rủi ro.

Dữ liệu:

Số lượng rủi ro phát sinh: 3 rủi ro (chủ yếu liên quan đến việc không đáp ứng kịp thời yêu cầu của khách hàng).

Phân tích: Ba rủi ro phát sinh trong hai tháng đầu cho thấy mức độ kiểm soát rủi ro là hợp lý, tuy nhiên, cần chú ý hơn đến việc đáp ứng yêu cầu khách hàng trong các giai đoạn tiếp theo.

Thời gian phản hồi và giải quyết rủi ro: KPI này đo lường thời gian trung bình để giải quyết các rủi ro.

Dữ liệu:

- Thời gian phản hồi: 24 giờ
- Thời gian giải quyết rủi ro: 48 giờ

Phân tích: Với thời gian phản hồi và giải quyết rủi ro là 48 giờ, nhóm dự án cần nâng cao hiệu quả giải quyết vấn đề nhanh chóng hơn, đặc biệt là các vấn đề phát sinh từ phía khách hàng.

d. Các Mốc Quan Trọng và Hoạt Động Sắp Tới

Tỷ lệ hoàn thành mốc quan trọng: KPI này đo lường phần trăm các mốc quan trọng đã hoàn thành so với tổng số các mốc trong kế hoạch.

Dữ liệu:

Tổng số mốc quan trọng trong kế hoạch: 5 mốc

Mốc quan trọng đã hoàn thành: 3 mốc

Tỷ lệ hoàn thành mốc quan trọng = $(3/5) \times 100 = 60\%$

Phân tích: Dự án đã hoàn thành 60% các mốc quan trọng. Cần phải đẩy nhanh tiến độ để đảm bảo hoàn thành toàn bộ các mốc theo kế hoạch.

e. Các Vấn Đề Tồn Động

Số lượng vấn đề tồn đọng: KPI này đo lường số lượng vấn đề chưa được giải quyết.

Dữ liệu:

Vấn đề chưa giải quyết: 2 vấn đề liên quan đến tính năng giao diện người dùng.

Phân tích: Dự án hiện còn 2 vấn đề chưa được giải quyết, điều này cho thấy nhóm cần tập trung vào việc khắc phục các vấn đề tồn đọng để không làm trì hoãn tiến độ.

Thời gian tồn đọng trung bình: KPI này đo lường thời gian trung bình mà các vấn đề tồn tại trước khi được giải quyết.

Dữ liệu:

Thời gian tồn đọng trung bình: 36 giờ

Phân tích: Thời gian tồn đọng trung bình là 36 giờ, nhóm cần giảm thiểu thời gian chờ đợi trước khi giải quyết các vấn đề phát sinh để cải thiện hiệu suất làm việc.

f. Đề Xuất và Kiến Nghị

KPI cải thiện: Dựa trên các kết quả hiện tại, dự án cần tập trung vào các mục tiêu sau:

- Tối ưu hóa thời gian giải quyết vấn đề để giảm thiểu thời gian tồn đọng.

- Cải thiện chất lượng quy trình phát triển để giảm tỷ lệ lỗi và tỷ lệ tái làm.
- Theo dõi chặt chẽ ngân sách để tránh vượt ngân sách trong giai đoạn tiếp theo.

Phân tích: Các đề xuất trên sẽ giúp nhóm dự án cải thiện hiệu quả trong các giai đoạn còn lại của dự án.

g. Tổng Kết

Báo cáo tiến độ dự án cho thấy rằng phần mềm quản lý công việc cá nhân đang đi đúng hướng, mặc dù có một số vấn đề cần cải thiện, đặc biệt là về thời gian và tỷ lệ hoàn thành các mốc quan trọng. Việc theo dõi chặt chẽ các chỉ số KPI sẽ giúp nhóm quản lý điều chỉnh kịp thời và đảm bảo dự án hoàn thành đúng tiến độ.

6.2.2. Nội dung báo cáo kết thúc dự án

Báo cáo kết thúc dự án là tài liệu quan trọng được lập sau khi dự án đã hoàn thành. Nó cung cấp cái nhìn tổng quan về toàn bộ dự án, đánh giá hiệu quả thực hiện, các vấn đề gặp phải, cũng như bài học kinh nghiệm cho các dự án tiếp theo. Báo cáo kết thúc giúp các bên liên quan, bao gồm đội ngũ dự án, khách hàng và nhà quản lý, có cái nhìn rõ ràng về kết quả thực hiện dự án và các vấn đề cần cải tiến.

6.2.2.1. Tổng Quan Về Dự Án

Mục tiêu dự án: Tóm tắt các mục tiêu và yêu cầu chính của dự án phần mềm. Các mục tiêu có thể bao gồm các yêu cầu kỹ thuật, mục tiêu kinh doanh, và yêu cầu người dùng.

Phạm vi dự án: Mô tả các tính năng và chức năng chính của phần mềm đã được hoàn thành, xác định rõ các phần mềm đã triển khai và các tính năng chưa thực hiện được (nếu có).

Thời gian thực hiện: Báo cáo thời gian bắt đầu và kết thúc của dự án, so sánh với kế hoạch ban đầu. Phân tích lý do cho bất kỳ sự chậm trễ nào (nếu có).

6.2.2.2. Kết Quả Dự Án

Tỷ lệ hoàn thành mục tiêu: Đánh giá tỷ lệ phần trăm các mục tiêu dự án đã hoàn thành. So sánh giữa các mục tiêu đã đạt được và các mục tiêu chưa hoàn thành.

Chất lượng phần mềm: Đánh giá chất lượng của phần mềm dựa trên các chỉ số như tỷ lệ lỗi, mức độ hài lòng của người dùng cuối, kết quả kiểm thử (unit testing, system testing, v.v.).

Ngân sách dự án: Đưa ra báo cáo tài chính chi tiết của dự án, so sánh ngân sách dự kiến với ngân sách thực tế đã sử dụng. Phân tích nguyên nhân của sự sai lệch, nếu có.

6.2.2.3. Các Vấn Đề Gặp Phải Trong Quá Trình Thực Hiện

Các vấn đề và rủi ro phát sinh: Liệt kê các vấn đề đã gặp phải trong suốt quá trình thực hiện, chẳng hạn như khó khăn về kỹ thuật, thay đổi yêu cầu từ khách hàng, hay sự thiếu hụt nguồn lực.

Giải pháp đã thực hiện: Phân tích các biện pháp đã được đưa ra để khắc phục các vấn đề và rủi ro. Đánh giá mức độ hiệu quả của các biện pháp này.

6.2.2.4. Đánh Giá Đội Ngũ Và Nguồn Lực

Hiệu suất làm việc của đội ngũ: Đánh giá mức độ hoàn thành công việc của đội ngũ phát triển phần mềm, sự hợp tác giữa các thành viên và sự đóng góp của từng cá nhân.

Thay đổi nguồn lực: Trình bày về các thay đổi trong nguồn lực dự án, bao gồm việc thay thế nhân sự hoặc bổ sung các chuyên gia, và tác động của những thay đổi này đến tiến độ và chất lượng dự án.

6.2.2.5. Đánh Giá Quy Trình Và Công Cụ

Quy trình quản lý dự án: Phân tích các quy trình đã được sử dụng trong dự án, ví dụ như Agile, Waterfall, Scrum, hoặc Kanban. Đánh giá hiệu quả của các phương pháp này trong việc quản lý tiến độ và chất lượng của dự án.

Công cụ hỗ trợ dự án: Liệt kê các công cụ phần mềm đã được sử dụng trong quá trình quản lý dự án, theo dõi tiến độ, quản lý mã nguồn, kiểm thử, và giao tiếp nhóm. Đánh giá sự hiệu quả của các công cụ này.

6.2.2.6. Đánh Giá Sự Thỏa Mãn Của Các Bên Liên Quan

Khách hàng và người dùng cuối: Đánh giá mức độ hài lòng của khách hàng và người dùng cuối đối với sản phẩm phần mềm được triển khai, bao gồm sự phản hồi về tính năng, giao diện, hiệu suất và độ ổn định của phần mềm.

Các bên liên quan: Đánh giá sự hài lòng của các bên liên quan khác như nhà đầu tư, đội ngũ quản lý và đối tác kỹ thuật.

6.2.2.7. Các Bài Học Kinh Nghiệm

Bài học từ dự án: Liệt kê các bài học rút ra từ quá trình triển khai dự án. Chỉ ra những điểm mạnh, yếu trong quá trình thực hiện và đưa ra những bài học hữu ích cho các dự án tiếp theo.

Đề xuất cải tiến: Đưa ra các đề xuất về quy trình, công cụ, hoặc chiến lược có thể cải thiện trong các dự án phần mềm tiếp theo.

6.2.2.8. Kết Luận Và Đề Xuất Tiếp Theo

Kết luận: Tóm tắt các kết quả và thành công của dự án, các mục tiêu đã đạt được và các mục tiêu chưa hoàn thành.

Đề xuất tiếp theo: Cung cấp các gợi ý cho các bước tiếp theo, như bảo trì phần mềm, phát triển các tính năng mới, hoặc mở rộng phần mềm sang các thị trường khác.

6.2.2.9. Ví Dụ Minh Họa

Giả sử, dự án "Ứng dụng Quản Lý Công Việc Cá Nhân" đã được hoàn thành:

Mục tiêu dự án: Xây dựng một ứng dụng giúp người dùng quản lý công việc cá nhân trên thiết bị di động với các tính năng như lập kế hoạch, nhắc nhở và phân loại công việc.

Kết quả: 95% tính năng đã hoàn thành, bao gồm lập kế hoạch, nhắc nhở và phân loại công việc. 100% tính năng kiểm thử thành công. Tuy nhiên, tính năng chia sẻ công việc chưa hoàn thiện.

Ngân sách: Ngân sách dự án ban đầu là 500 triệu đồng, ngân sách thực tế là 480 triệu đồng, do giảm bớt chi phí phát triển không cần thiết.

Các vấn đề gặp phải: Đã gặp vấn đề trong việc tích hợp API nhắc nhở với các dịch vụ bên ngoài, nhưng đã giải quyết nhanh chóng bằng cách thay đổi phương thức tích hợp.

4. Kết Luận

Báo cáo kết thúc dự án là công cụ quan trọng giúp các bên liên quan có cái nhìn tổng quan về hiệu quả và các vấn đề đã gặp phải trong suốt quá trình thực hiện dự án phần mềm. Việc lập báo cáo kết thúc giúp xác định các thành công, thất bại và những bài học quan trọng có thể ứng dụng cho các dự án tương lai. Thông qua đó, các nhà quản lý có thể đưa ra các cải tiến trong quy trình làm việc và cải thiện hiệu quả của các dự án sau này.

5. Tài Liệu Tham Khảo

- **PMI - Guide to the Project Management Body of Knowledge (PMBOK® Guide)**
- **Harvard Business Review**

Câu hỏi ôn tập cuối chương

1. KPIs là gì và tại sao chúng lại quan trọng trong quản lý dự án phần mềm?
2. Các tiêu chí nào giúp xác định một KPI hiệu quả trong dự án?
3. Làm thế nào để đo lường tiến độ dự án bằng các chỉ số như EV, PV, và SPI?
4. Tỷ lệ hoàn thành công việc (Task Completion Rate) có ý nghĩa gì trong việc đánh giá tiến độ dự án?
5. Các bước cơ bản để xây dựng một báo cáo tiến độ dự án bao gồm những gì?
6. So sánh sự khác biệt giữa báo cáo tiến độ và báo cáo kết thúc dự án.
7. Thời gian khắc phục sự cố (Issue Resolution Time) được tính toán như thế nào và phản ánh điều gì về hiệu suất dự án?
8. Mức độ hài lòng của khách hàng (Customer Satisfaction Score) được đo lường bằng cách nào?
9. Làm thế nào để áp dụng các bài học kinh nghiệm từ một dự án vào các dự án trong tương lai?
10. Vai trò của báo cáo kết thúc dự án trong việc cải thiện chất lượng quản lý dự án là gì?

Chương 7: Vai trò và ứng dụng kỹ năng mềm trong quản lý dự án phần mềm

Nội dung chính của chương

Chương này tập trung trình bày về các kỹ năng mềm cần thiết trong quản lý dự án phần mềm, bao gồm kỹ năng giao tiếp, lãnh đạo, đàm phán, và giải quyết xung đột. Chương nhấn mạnh vai trò quan trọng của giao tiếp hiệu quả trong việc truyền đạt thông tin, quản lý thay đổi, và xây dựng sự gắn kết nhóm. Kỹ năng lãnh đạo được phân tích qua việc định hướng mục tiêu, tạo động lực, và quản lý đội nhóm, giúp đảm bảo tiến độ và chất lượng dự án. Đàm phán và xử lý xung đột được đề cập như những công cụ quan trọng để duy trì môi trường làm việc tích cực, giải quyết mâu thuẫn, và đạt được sự đồng thuận giữa các bên liên quan. Bên cạnh đó, chương cũng đề xuất các phương pháp quản lý thời gian và phân công công việc hiệu quả, đồng thời nhấn mạnh việc xây dựng tinh thần đồng đội và không gian làm việc hợp tác để tối ưu hóa hiệu suất nhóm.

Mục tiêu cần đạt của chương

- Hiểu vai trò của giao tiếp, lãnh đạo và giải quyết xung đột.
- Vận dụng kỹ năng mềm để thúc đẩy hiệu quả làm việc nhóm.

7.1. Vai trò của kỹ năng giao tiếp, lãnh đạo, đàm phán và giải quyết xung đột trong quản lý dự án phần mềm.



Hình 7.1. Các kỹ năng mềm trong quản lý dự án phần mềm

7.1.1. Kỹ năng giao tiếp trong quản lý dự án phần mềm

Kỹ năng giao tiếp là yếu tố thiết yếu trong quản lý dự án, đặc biệt là trong các dự án phần mềm có sự tham gia của nhiều bộ phận. Giao tiếp hiệu quả giúp truyền đạt thông tin chính xác, giảm thiểu hiểu lầm, và thúc đẩy sự hợp tác giữa các thành viên.

7.1.1.1. Vai trò của kỹ năng giao tiếp trong quản lý dự án phần mềm

Đảm bảo hiểu biết chung: Giao tiếp rõ ràng giúp đảm bảo tất cả các thành viên hiểu được mục tiêu, nhiệm vụ, và các yêu cầu kỹ thuật của dự án.

Xây dựng niềm tin và động viên đội nhóm: Khi các thành viên được thông báo thường xuyên và rõ ràng, họ cảm thấy mình là một phần quan trọng của dự án, từ đó tăng cường động lực và sự gắn kết.

Giải quyết vấn đề nhanh chóng: Giao tiếp hiệu quả giúp các vấn đề được phát hiện sớm và đưa ra giải pháp kịp thời, tránh ảnh hưởng đến tiến độ và chất lượng dự án.

7.1.1.2. Các phương pháp giao tiếp hiệu quả trong quản lý dự án phần mềm

a. Giao tiếp bằng văn bản (Written Communication)

Phương pháp: Sử dụng các tài liệu như email, báo cáo, và bản ghi chú để ghi lại thông tin quan trọng, các thay đổi và yêu cầu của dự án.

Ưu điểm: Ghi lại lịch sử của các quyết định và tiến trình; thuận tiện cho các thành viên xem lại thông tin khi cần thiết.

Ứng dụng:

- Viết email tóm tắt buổi họp và các quyết định quan trọng.
- Tạo báo cáo tiến độ, bản mô tả yêu cầu (SRS), và tài liệu hướng dẫn kỹ thuật cho các thành viên trong nhóm.

b. Giao tiếp bằng lời nói (Verbal Communication)

Phương pháp: Sử dụng cuộc họp nhóm, gọi điện, hoặc thảo luận trực tiếp để trao đổi thông tin nhanh chóng, giải đáp thắc mắc, và xử lý vấn đề tức thời.

Ưu điểm: Tạo điều kiện cho sự phản hồi ngay lập tức; giúp làm rõ các điểm phức tạp thông qua tương tác hai chiều.

Ứng dụng:

- Tổ chức các cuộc họp dự án định kỳ để cập nhật tiến độ và thảo luận về các vấn đề mới phát sinh.

- Đối thoại trực tiếp với các thành viên khi cần làm rõ các yêu cầu hoặc thay đổi trong dự án.

c. Lắng nghe tích cực (Active Listening)

Phương pháp: Lắng nghe cẩn thận, không ngắt lời, ghi chú lại ý kiến của người nói, và đưa ra phản hồi khi cần thiết.

Ưu điểm: Giúp thấu hiểu sâu sắc quan điểm của người nói, xây dựng mối quan hệ tốt đẹp, và hạn chế hiểu lầm.

Ứng dụng:

- Lắng nghe kỹ phản hồi từ các thành viên về các vấn đề kỹ thuật hoặc khó khăn trong công việc.
- Áp dụng trong các buổi đánh giá tiến độ, nơi các thành viên có thể chia sẻ các thách thức hoặc sáng kiến.

d. Phản hồi xây dựng (Constructive Feedback)

Phương pháp: Đưa ra các phản hồi cụ thể, tập trung vào hành vi hoặc kết quả công việc, thay vì nhắm vào cá nhân.

Ưu điểm: Giúp cải thiện hiệu suất của đội nhóm, đồng thời xây dựng mối quan hệ tích cực và tôn trọng.

Ứng dụng:

- Trong buổi họp cá nhân, đưa ra lời khen ngợi về các điểm mạnh và gợi ý cải thiện các điểm yếu của từng thành viên.
- Đánh giá và phản hồi công việc của nhóm theo cách hỗ trợ, thay vì chỉ trích.

e. Giao tiếp phi ngôn ngữ (Non-verbal Communication)

Phương pháp: Sử dụng ngôn ngữ cơ thể, ánh mắt, nét mặt, và cách đứng/ngồi để tạo ra sự tương tác thân thiện và cởi mở.

Ưu điểm: Truyền đạt được sự tự tin, tạo ra sự kết nối thân thiện và thoải mái trong giao tiếp.

Ứng dụng:

- Trong các buổi họp trực tiếp, sử dụng ngôn ngữ cơ thể để thể hiện sự quan tâm và đồng cảm với người đối diện.

- Dùng ánh mắt và cử chỉ để tạo cảm giác thoải mái và thúc đẩy sự tin tưởng.

7.1.1.3. Ứng dụng cụ thể của kỹ năng giao tiếp trong quản lý dự án phần mềm

Truyền đạt yêu cầu dự án: Đảm bảo tất cả các bên liên quan (khách hàng, đội phát triển, quản lý dự án) đều hiểu rõ các yêu cầu và mục tiêu dự án. Tạo các tài liệu mô tả yêu cầu và gửi tới các thành viên để họ nắm rõ nhiệm vụ.

Quản lý thay đổi: Khi có yêu cầu thay đổi trong dự án, kỹ năng giao tiếp giúp thông báo kịp thời và thảo luận để đạt được sự đồng thuận. Việc này giúp tránh nhầm lẫn và giảm thiểu rủi ro cho dự án.

Giải quyết vấn đề: Kỹ năng giao tiếp giúp quản lý dự án dễ dàng xác định nguồn gốc vấn đề và đưa ra giải pháp. Thông qua các cuộc thảo luận trực tiếp, quản lý có thể hiểu rõ các khó khăn của thành viên và phối hợp tìm giải pháp.

Đánh giá tiến độ và hiệu suất: Các cuộc họp cập nhật tiến độ giúp đánh giá công việc, đưa ra phản hồi và điều chỉnh kế hoạch nếu cần. Quản lý dự án cần giao tiếp rõ ràng để giữ đội nhóm luôn đi đúng hướng.

Xây dựng văn hóa nhóm: Giao tiếp cởi mở và khuyến khích lắng nghe giúp tạo ra một môi trường làm việc tích cực, nơi mọi người cảm thấy thoải mái để chia sẻ ý tưởng và góp ý. Điều này giúp tăng cường sự gắn bó và hiệu quả làm việc của nhóm.

7.1.2. Kỹ năng lãnh đạo

7.1.2.1. Giới thiệu về kỹ năng lãnh đạo trong quản lý dự án phần mềm

Kỹ năng lãnh đạo đóng vai trò quan trọng trong việc định hướng, tổ chức, và quản lý đội ngũ thực hiện dự án phần mềm. Một người lãnh đạo giỏi không chỉ tạo động lực cho đội nhóm mà còn đảm bảo rằng dự án được tiến hành theo đúng mục tiêu, tiến độ, và tiêu chuẩn chất lượng. Lãnh đạo hiệu quả là chìa khóa giúp đội nhóm vượt qua các khó khăn, thích ứng với thay đổi và luôn duy trì sự phối hợp nhịp nhàng giữa các thành viên.

7.1.2.2. Các yếu tố cốt lõi của kỹ năng lãnh đạo

a. Định hướng và mục tiêu rõ ràng

Vai trò: Người lãnh đạo cần xác định rõ ràng mục tiêu và định hướng của dự án, đồng thời truyền đạt chúng một cách cụ thể tới các thành viên.

Phương pháp: Thiết lập mục tiêu theo mô hình SMART (Specific, Measurable, Achievable, Relevant, Time-bound), đảm bảo mỗi thành viên đều hiểu rõ vai trò của mình trong việc đạt được mục tiêu này.

Phương pháp Thiết lập mục tiêu theo mô hình SMART là một công cụ quản lý hiệu quả để giúp bạn xác định các mục tiêu rõ ràng, dễ đo lường và có thể đạt được trong một thời gian cụ thể. SMART là một từ viết tắt đại diện cho các tiêu chí sau:

Specific (Cụ thể): Để mục tiêu hiệu quả, bạn cần phải xác định rõ ràng mục tiêu của mình là gì. Điều này có nghĩa là mục tiêu phải rõ ràng và không được mơ hồ. Mục tiêu cụ thể giúp bạn biết chính xác những gì cần làm để đạt được nó.

Câu hỏi cần tự hỏi: Mục tiêu này cụ thể là gì? Ai sẽ tham gia vào mục tiêu này? Ở đâu? Khi nào? Tại sao mục tiêu này lại quan trọng?

Ví dụ: Thay vì nói "Tăng trưởng doanh thu", bạn có thể nói "Tăng trưởng doanh thu từ khách hàng hiện tại trong khu vực TP.HCM".

Measurable (Có thể đo lường): Một mục tiêu SMART phải có khả năng đo lường. Điều này có nghĩa là bạn phải có chỉ số hoặc tiêu chí để theo dõi tiến độ và đánh giá khi nào bạn đã đạt được mục tiêu.

Câu hỏi cần tự hỏi: Làm sao tôi biết mình đã đạt được mục tiêu này? Những chỉ số nào giúp tôi theo dõi sự tiến bộ?

Ví dụ: "Tăng doanh thu lên 10% trong quý này" là một mục tiêu có thể đo lường được, vì bạn có thể dễ dàng theo dõi sự thay đổi về doanh thu.

Achievable (Có thể đạt được): Mục tiêu của bạn phải là khả thi, tức là nó phải thực tế và có thể đạt được với các nguồn lực hiện tại của bạn (thời gian, tiền bạc, nhân lực, kiến thức, v.v.). Mặc dù mục tiêu cần phải thách thức, nhưng nó cũng cần phải thực tế để bạn không cảm thấy quá khó khăn hoặc thất vọng.

Câu hỏi cần tự hỏi: Mục tiêu này có thể đạt được với các nguồn lực hiện tại không? Cần phải làm gì để đạt được mục tiêu này?

Ví dụ: Một mục tiêu như "Tăng trưởng doanh thu 50% trong 1 tháng" có thể không khả thi nếu bạn chưa có đủ nhân lực hoặc tài nguyên để đạt được nó. Thay vào đó, mục tiêu "Tăng trưởng doanh thu 10% trong 3 tháng" sẽ thực tế hơn.

Relevant (Phù hợp): Mục tiêu phải **phù hợp** với các mục tiêu lớn hơn của bạn. Nó cần phải có ý nghĩa và liên quan đến các ưu tiên và chiến lược dài hạn của bạn. Mục

tiêu này sẽ giúp bạn giữ đúng hướng và không bị phân tâm vào những thứ không quan trọng.

Câu hỏi cần tự hỏi: Mục tiêu này có phù hợp với các mục tiêu lớn hơn của tôi không? Nó có liên quan đến chiến lược hoặc giá trị của tổ chức hay không?

Ví dụ: Nếu công ty của bạn đang tập trung vào việc phát triển sản phẩm mới, mục tiêu "Nâng cao kỹ năng thiết kế sản phẩm cho đội ngũ nhân viên trong 6 tháng" sẽ phù hợp hơn là mục tiêu "Tăng số lượng người theo dõi trên mạng xã hội".

Time-bound (Có giới hạn thời gian): Mỗi mục tiêu cần phải có một **thời gian hoàn thành** rõ ràng. Mục tiêu có giới hạn thời gian giúp bạn duy trì động lực và kỷ luật trong việc thực hiện. Khi có thời gian rõ ràng, bạn có thể dễ dàng xác định khi nào mục tiêu đã được hoàn thành.

Câu hỏi cần tự hỏi: Khi nào tôi muốn hoàn thành mục tiêu này? Có thể chia nhỏ mục tiêu thành các mốc thời gian cụ thể không?

Ví dụ: "Tăng trưởng doanh thu lên 10% trong quý này" là một mục tiêu có thời hạn rõ ràng (quý này), giúp bạn dễ dàng lập kế hoạch hành động và theo dõi tiến độ.

Ví dụ về một mục tiêu SMART:

Mục tiêu: "Tăng trưởng doanh thu của cửa hàng online thêm 15% trong 3 tháng tới bằng cách mở rộng chiến dịch quảng cáo trên Facebook và Instagram, đạt 1000 đơn hàng mỗi tháng."

- **Specific:** Tăng trưởng doanh thu 15%.
- **Measurable:** Đo lường bằng doanh thu và số lượng đơn hàng.
- **Achievable:** 15% là mục tiêu có thể đạt được dựa trên các nguồn lực hiện có.
- **Relevant:** Liên quan đến mục tiêu mở rộng kinh doanh online.
- **Time-bound:** Trong vòng 3 tháng tới

Ứng dụng: Lập kế hoạch chi tiết và giao nhiệm vụ cụ thể cho từng thành viên dựa trên năng lực và thế mạnh cá nhân.

b. Khả năng ra quyết định

Vai trò: Lãnh đạo dự án phần mềm cần có khả năng ra quyết định nhanh chóng và chính xác, đặc biệt trong các tình huống khó khăn hay khi có sự thay đổi trong yêu cầu dự án.

Phương pháp: Phân tích dữ liệu, xem xét ý kiến từ các thành viên, đánh giá các kịch bản khác nhau và chọn ra phương án tốt nhất cho dự án.

Ứng dụng: Quyết định về các phương án phát triển, lựa chọn công nghệ phù hợp, và phân bổ nguồn lực hợp lý.

c. Động viên và truyền cảm hứng

Vai trò: Lãnh đạo hiệu quả biết cách tạo động lực cho đội nhóm, khuyến khích sự sáng tạo và giúp thành viên phát huy hết khả năng của mình.

Phương pháp: Đưa ra lời khen ngợi và công nhận thành tích cá nhân, khuyến khích phát triển kỹ năng và cung cấp cơ hội học hỏi.

Ứng dụng: Tạo môi trường làm việc tích cực, tổ chức các buổi trao đổi ý tưởng hoặc hỗ trợ nhân viên tham gia vào các khóa đào tạo nâng cao.

d. Trao quyền và phát triển đội nhóm

Vai trò: Lãnh đạo không chỉ giám sát mà còn biết cách trao quyền, tạo điều kiện cho các thành viên tự chủ trong công việc, từ đó phát triển năng lực lãnh đạo ở cấp độ nhóm.

Phương pháp: Phân công nhiệm vụ phù hợp với khả năng của mỗi người, khuyến khích các thành viên tự chịu trách nhiệm và đưa ra quyết định trong phạm vi công việc của mình.

Ứng dụng: Đưa ra các nhiệm vụ thách thức, tổ chức các nhóm nhỏ để các thành viên có thể luân phiên dẫn dắt.

e. Kỹ năng lắng nghe và thấu hiểu

Vai trò: Lãnh đạo cần lắng nghe và hiểu rõ nhu cầu, khó khăn, cũng như ý kiến của các thành viên, từ đó đưa ra hỗ trợ hoặc điều chỉnh phù hợp.

Phương pháp: Thường xuyên trao đổi với các thành viên, thể hiện sự quan tâm chân thành, lắng nghe những phản hồi và ý kiến đóng góp.

Ứng dụng: Tổ chức các buổi họp nhỏ định kỳ, tạo điều kiện cho các thành viên chia sẻ khó khăn và sáng kiến.

3. Các phong cách lãnh đạo hiệu quả trong quản lý dự án phần mềm

- **Lãnh đạo kiểu dân chủ (Democratic Leadership):**

Khuyến khích mọi người cùng tham gia vào quá trình ra quyết định. Phong cách này phù hợp khi cần sự đóng góp từ các thành viên, đặc biệt là khi có nhiều giải pháp khả thi cho một vấn đề kỹ thuật.

- **Lãnh đạo chỉ huy (Directive Leadership):**

Trong các dự án có yêu cầu cao về thời gian và độ chính xác, người lãnh đạo có thể cần ra quyết định nhanh và hướng dẫn cụ thể. Phong cách này phù hợp cho các tình huống khẩn cấp hoặc khi các thành viên thiếu kinh nghiệm.

- **Lãnh đạo hỗ trợ (Supportive Leadership):**

Tập trung vào việc cung cấp các hỗ trợ cần thiết cho thành viên, xây dựng lòng tin và động viên, phù hợp trong môi trường làm việc sáng tạo và khi nhóm cần sự gắn kết.

7.1.2.4. Ứng dụng của kỹ năng lãnh đạo trong quản lý dự án phần mềm

Định hướng phát triển và tạo ra giá trị: Người lãnh đạo định hướng và duy trì sự nhất quán trong suốt quá trình phát triển phần mềm, đồng thời đảm bảo rằng sản phẩm cuối cùng mang lại giá trị cho khách hàng.

Quản lý rủi ro và xử lý sự cố: Lãnh đạo hiệu quả giúp nhận diện sớm các rủi ro, chuẩn bị các phương án phòng tránh và xử lý các sự cố kỹ thuật hoặc xung đột phát sinh trong nhóm.

Xây dựng và duy trì văn hóa nhóm: Lãnh đạo giỏi tạo ra một môi trường làm việc tích cực, nơi các thành viên có động lực làm việc và sẵn sàng chia sẻ kiến thức, kinh nghiệm để cùng nhau phát triển.

Thúc đẩy hiệu quả và năng suất của nhóm: Bằng cách cung cấp tầm nhìn rõ ràng và hỗ trợ thích hợp, người lãnh đạo có thể tối ưu hóa năng lực của từng thành viên, tạo ra hiệu quả và sự gắn kết cao trong quá trình làm việc.

7.1.3. Kỹ năng đàm phán

7.1.3.1. Giới thiệu về kỹ năng đàm phán trong quản lý dự án phần mềm

Trong môi trường phát triển phần mềm, đàm phán đóng vai trò quan trọng giúp người quản lý dự án đạt được sự đồng thuận về ngân sách, phạm vi công việc, thời gian hoàn thành, và các yêu cầu kỹ thuật với các bên liên quan. Đàm phán hiệu quả giúp tối ưu hóa tài nguyên, duy trì mối quan hệ tích cực và tạo ra các thỏa thuận công bằng cho cả đội nhóm và khách hàng.

7.1.3.2. Các yếu tố cốt lõi của kỹ năng đàm phán

a. Chuẩn bị kỹ lưỡng

Vai trò: Hiểu rõ nhu cầu, lợi ích của tất cả các bên liên quan, và dự đoán các vấn đề có thể phát sinh trước khi bước vào cuộc đàm phán.

Phương pháp: Tìm hiểu thông tin liên quan, xác định rõ mục tiêu của bản thân và của bên đối tác, chuẩn bị phương án thay thế nếu cần.

Ứng dụng: Lập danh sách các yếu tố quan trọng như thời gian, chi phí, chất lượng và đưa ra các phương án linh hoạt để đạt được đồng thuận.

b. Lắng nghe và thấu hiểu đối tác

Vai trò: Lắng nghe giúp hiểu được các yêu cầu và quan điểm của đối tác, từ đó dễ dàng tìm ra phương án có lợi cho cả hai bên.

Phương pháp: Sử dụng kỹ thuật lắng nghe chủ động, tóm tắt lại ý kiến của đối tác để đảm bảo hiểu đúng.

Ứng dụng: Lắng nghe các ý kiến của khách hàng hoặc các phòng ban khác để tìm ra giải pháp đáp ứng yêu cầu kỹ thuật mà vẫn đảm bảo lợi ích của dự án.

c. Linh hoạt và sáng tạo trong giải pháp

Vai trò: Đàm phán không chỉ là thương lượng, mà còn là quá trình tìm ra các giải pháp sáng tạo để đáp ứng nhu cầu của tất cả các bên.

Phương pháp: Xem xét các tùy chọn thay thế, và mở rộng phạm vi của các phương án thỏa thuận nếu cần thiết.

Ứng dụng: Tìm ra các phương án thay thế khi có sự chênh lệch về ngân sách hoặc yêu cầu thời gian mà không ảnh hưởng đến chất lượng.

d. Xác định các điểm có thể thỏa hiệp

Vai trò: Để đạt được thỏa thuận, đôi bên cần hiểu rõ đâu là những yếu tố có thể thay đổi và đâu là những giới hạn không thể vượt qua.

Phương pháp: Xác định các yếu tố ưu tiên và sẵn sàng điều chỉnh những yếu tố ít quan trọng.

Ứng dụng: Sẵn sàng thỏa hiệp về một số tính năng không quan trọng hoặc thời gian triển khai các phần phụ của dự án.

e. Kết thúc đàm phán với cam kết rõ ràng

Vai trò: Kết thúc đàm phán là giai đoạn quan trọng, đòi hỏi phải có sự thống nhất rõ ràng về kết quả cuối cùng.

Phương pháp: Đảm bảo rằng tất cả các thỏa thuận đều được xác nhận rõ ràng, có văn bản nếu cần.

Ứng dụng: Xác nhận thỏa thuận cuối cùng bằng văn bản để tránh sự mơ hồ, đảm bảo cả hai bên đều hiểu rõ các cam kết đã thống nhất.

7.1.3.3. Các phương pháp đàm phán trong quản lý dự án phần mềm

a. Đàm phán dựa trên lợi ích (Interest-based negotiation)

Mô tả: Phương pháp này tập trung vào lợi ích và nhu cầu cơ bản của các bên thay vì vị trí hay lập trường cố định. Điều này cho phép các bên hiểu sâu hơn về lý do đằng sau yêu cầu của nhau, từ đó dễ dàng tìm được giải pháp hợp tác.

Cách thực hiện: Tạo ra môi trường giao tiếp cởi mở, khuyến khích các bên chia sẻ mục tiêu dài hạn và những yếu tố quan trọng mà họ mong muốn đạt được.

Ứng dụng: Trong đàm phán với khách hàng về yêu cầu chức năng của phần mềm, thay vì chỉ tranh luận về ngân sách, có thể tìm hiểu những yếu tố quan trọng nhất mà khách hàng ưu tiên để điều chỉnh chi phí theo cách đôi bên cùng có lợi.

b. Đàm phán hợp tác (Collaborative negotiation)

Mô tả: Đây là phương pháp đàm phán mà các bên cùng làm việc để tìm ra giải pháp tối ưu, với mục tiêu là mang lại lợi ích tốt nhất cho tất cả mọi người tham gia.

Cách thực hiện: Tạo không gian để cả hai bên có thể chia sẻ thông tin một cách minh bạch và đưa ra các giải pháp sáng tạo nhằm đáp ứng mục tiêu của nhau.

Ứng dụng: Khi đàm phán giữa các nhóm phát triển và đội ngũ kiểm thử, có thể cùng bàn bạc để điều chỉnh quy trình làm việc, giúp kiểm thử có thể phát hiện lỗi sớm và giảm áp lực lên nhóm phát triển.

c. Đàm phán cạnh tranh (Competitive negotiation)

Mô tả: Phương pháp này thường được sử dụng khi một bên cần đạt được kết quả có lợi cho mình mà không quá quan tâm đến lợi ích của bên còn lại.

Cách thực hiện: Đặt ra các yêu cầu rõ ràng và kiên quyết bảo vệ quan điểm của mình để đạt được lợi ích tối đa.

Ứng dụng: Khi đàm phán với nhà cung cấp dịch vụ để giảm giá hoặc đàm phán với nhân viên về việc phân bổ thời gian làm việc, có thể sử dụng phương pháp này để bảo vệ lợi ích của dự án.

d. Đàm phán thỏa hiệp (Compromising negotiation)

Mô tả: Phương pháp này yêu cầu cả hai bên đều từ bỏ một phần yêu cầu của mình để đi đến một thỏa thuận chung.

Cách thực hiện: Xác định rõ các điểm có thể thỏa hiệp, đặt ra giới hạn và cố gắng đạt được sự đồng thuận.

Ứng dụng: Trong đàm phán về thời gian giao hàng của sản phẩm, nếu cả hai bên không đạt được thống nhất về thời hạn hoàn thành, có thể giảm khối lượng công việc để đảm bảo thời hạn.

7.1.4. Kỹ năng giải quyết xung đột

7.1.4.1. Giới thiệu về kỹ năng giải quyết xung đột trong quản lý dự án phần mềm

Xung đột là điều khó tránh khỏi trong các dự án phần mềm do sự khác biệt về quan điểm, lợi ích, hay áp lực về thời gian. Khả năng giải quyết xung đột giúp người quản lý dự án duy trì môi trường làm việc tích cực và tăng cường sự hợp tác giữa các thành viên trong nhóm. Giải quyết xung đột hiệu quả giúp các thành viên tập trung vào mục tiêu chung và tăng năng suất làm việc.

7.1.4.2. Các phương pháp giải quyết xung đột

a. Nhận diện nguyên nhân xung đột

Vai trò: Hiểu rõ nguồn gốc của xung đột là bước đầu để giải quyết vấn đề một cách hiệu quả.

Phương pháp: Lắng nghe các bên liên quan, phân tích nguyên nhân cốt lõi của xung đột thay vì chỉ tập trung vào biểu hiện bên ngoài.

Ứng dụng: Xác định nguyên nhân chính của xung đột, như khác biệt về phong cách làm việc, mục tiêu không rõ ràng hoặc hạn chế tài nguyên.

b. Lựa chọn phương pháp giải quyết phù hợp

Vai trò: Không phải mọi xung đột đều có cùng một cách giải quyết; cần có phương pháp phù hợp với từng tình huống.

Phương pháp: Dựa vào mô hình Thomas-Kilmann, sử dụng các phương pháp như hợp tác, thỏa hiệp, tránh né, cạnh tranh hoặc nhượng bộ, tùy vào hoàn cảnh.

Ứng dụng: Trong xung đột giữa các thành viên về phân công công việc, có thể chọn phương pháp thỏa hiệp hoặc hợp tác để đạt được mục tiêu chung.

c. Thúc đẩy trao đổi và giao tiếp cởi mở

Vai trò: Giải quyết xung đột hiệu quả đòi hỏi sự giao tiếp cởi mở để các bên có thể hiểu nhau hơn.

Phương pháp: Tổ chức buổi họp để các bên chia sẻ suy nghĩ và quan điểm của mình, khuyến khích lắng nghe chủ động.

Ứng dụng: Tạo môi trường thảo luận cởi mở, cho phép mọi người bày tỏ quan điểm và giải quyết xung đột một cách tích cực.

d. Đưa ra giải pháp và thống nhất kết quả

Vai trò: Một khi đã có các phương án giải quyết, người quản lý cần giúp các bên đạt được sự đồng thuận.

Phương pháp: Đưa ra các giải pháp công bằng, đảm bảo mọi bên đều đồng ý với kết quả cuối cùng, có thể ghi lại các cam kết nếu cần.

Ứng dụng: Sau khi xác định được giải pháp, ghi lại cam kết của các thành viên trong nhóm để đảm bảo tất cả đều tuân theo thỏa thuận.

e. Theo dõi và đánh giá kết quả

Vai trò: Xung đột có thể tái diễn nếu không có sự theo dõi. Việc này giúp đảm bảo rằng các giải pháp được thực hiện và có hiệu quả.

Phương pháp: Theo dõi kết quả và đánh giá phản hồi từ các bên sau khi xung đột được giải quyết.

Ứng dụng: Gặp gỡ các thành viên liên quan sau một thời gian để đánh giá lại tình hình và đảm bảo rằng không có xung đột nào mới phát sinh.

7.1.4.2. Các phương pháp giải quyết xung đột trong quản lý dự án phần mềm

a. Phương pháp đối đầu trực tiếp (Direct confrontation)

Mô tả: Phương pháp này đòi hỏi giải quyết xung đột ngay tại thời điểm phát sinh, giúp ngăn chặn vấn đề trở nên phức tạp hơn.

Cách thực hiện: Tạo điều kiện để các bên liên quan thảo luận trực tiếp, xác định rõ vấn đề, và giải quyết mâu thuẫn ngay lập tức.

Ứng dụng: Nếu có xung đột về trách nhiệm giữa các thành viên nhóm, tổ chức một buổi họp ngay lập tức để làm rõ nhiệm vụ của từng người.

b. Phương pháp hợp tác (Collaborative approach)

Mô tả: Khuyến khích các bên hợp tác để tìm ra giải pháp chung, trong đó cả hai bên đều có lợi.

Cách thực hiện: Thảo luận cùng các bên liên quan để tìm ra mục tiêu chung, sau đó đề xuất các giải pháp sao cho mọi người đều cảm thấy hài lòng.

Ứng dụng: Trong trường hợp có xung đột về yêu cầu kỹ thuật, có thể để đội kỹ sư và khách hàng cùng hợp tác để tìm ra phương án phù hợp nhất với ngân sách và thời gian.

c. Phương pháp tránh né (Avoidance)

Mô tả: Phương pháp này được sử dụng khi xung đột không quan trọng hoặc khi việc giải quyết xung đột ngay lập tức có thể gây ra hậu quả nghiêm trọng.

Cách thực hiện: Trì hoãn giải quyết vấn đề hoặc tránh tiếp xúc với vấn đề cho đến khi có phương án rõ ràng.

Ứng dụng: Trong xung đột về các yêu cầu chưa rõ ràng từ khách hàng, người quản lý có thể chờ đến khi khách hàng cung cấp thêm thông tin để tránh hiểu nhầm và mất thời gian giải quyết những chi tiết nhỏ nhặt.

d. Phương pháp thỏa hiệp (Compromising)

Mô tả: Tìm kiếm một giải pháp trung hòa, chấp nhận cả hai bên cùng nhượng bộ để đạt được thỏa thuận.

Cách thực hiện: Cân nhắc và đưa ra các giải pháp đôi bên cùng chấp nhận, mặc dù không đạt được tất cả các mục tiêu ban đầu.

Ứng dụng: Trong các dự án phần mềm với hạn chế về thời gian và nguồn lực, cả nhóm phát triển và khách hàng có thể thỏa hiệp về yêu cầu của phần mềm, tập trung vào các tính năng chính và cắt giảm các tính năng phụ để đạt mục tiêu.

e. Phương pháp hòa giải (Mediation)

Mô tả: Một bên thứ ba trung lập được mời vào để giúp các bên tranh chấp tìm ra giải pháp hợp lý.

Cách thực hiện: Bên thứ ba trung gian lắng nghe ý kiến của các bên, sau đó đưa ra những đề xuất để các bên cùng thảo luận và đi đến thỏa thuận.

Ứng dụng: Khi có mâu thuẫn sâu sắc giữa các thành viên về các quyết định quan trọng của dự án, mời một quản lý cấp cao hoặc chuyên gia vào làm trung gian hòa giải.

f. Phương pháp giải quyết vấn đề (Problem-solving approach)

Mô tả: Xung đột được nhìn nhận như một vấn đề cần giải quyết, và các bên làm việc cùng nhau để tìm ra nguyên nhân gốc rễ.

Cách thực hiện: Phân tích kỹ lưỡng vấn đề, tìm hiểu nguyên nhân và đề xuất giải pháp dựa trên dữ liệu và phân tích thay vì cảm xúc.

Ứng dụng: Khi xảy ra xung đột về sự cố kỹ thuật hoặc phân bổ tài nguyên, nhóm kỹ thuật có thể họp lại để phân tích chi tiết vấn đề và tìm ra giải pháp khắc phục dựa trên dữ liệu.

7.2. Sử dụng kỹ năng mềm để duy trì môi trường làm việc tích cực và hợp tác trong nhóm, thúc đẩy hiệu quả làm việc và xử lý xung đột, đảm bảo tiến độ và chất lượng dự án.

7.2.1. Thúc đẩy hiệu quả làm việc

Thúc đẩy hiệu quả làm việc trong một nhóm dự án đòi hỏi sự kết hợp của nhiều kỹ năng mềm nhằm tạo động lực, tăng cường sự tập trung và tinh thần hợp tác giữa các thành viên. Dưới đây là các phương pháp và cách ứng dụng cụ thể để tối ưu hiệu quả làm việc trong nhóm:

7.2.1.1. Giao tiếp tích cực và rõ ràng

Phương pháp: Giao tiếp hiệu quả trong nhóm đòi hỏi sự minh bạch và tính thường xuyên. Các thành viên cần được cập nhật đầy đủ về tiến độ và kỳ vọng, và được khuyến khích thảo luận khi gặp khó khăn. Các công cụ hỗ trợ như Slack, Microsoft Teams, và Asana giúp đảm bảo thông tin không bị bỏ sót.

Ứng dụng: Đặt các cuộc họp ngắn (stand-up meeting) hàng ngày để tất cả các thành viên cập nhật tiến độ và trao đổi các vấn đề gặp phải. Khuyến khích chia sẻ ý kiến và lắng nghe phản hồi từ các thành viên khác để tạo ra sự thấu hiểu và đồng thuận.

Ví dụ: Trong một dự án phát triển phần mềm, trưởng nhóm tổ chức buổi họp sáng ngắn (stand-up meeting) vào mỗi ngày. Tại đây, từng thành viên cập nhật về công việc họ đã làm hôm qua, những gì sẽ làm hôm nay và nêu rõ bất kỳ vấn đề nào đang gặp phải.

Điều này giúp cả nhóm dễ dàng hiểu rõ tình hình, hỗ trợ nhau kịp thời và đảm bảo không ai bị gián đoạn.

7.2.1.2. Khuyến khích và tạo động lực

Phương pháp: Động viên kịp thời và khen ngợi thành viên khi họ hoàn thành tốt nhiệm vụ, đặc biệt trong những công việc khó khăn hoặc quan trọng. Xây dựng một hệ thống thưởng để ghi nhận thành tích, chẳng hạn như khen thưởng dưới dạng lời khen công khai, vinh danh, hoặc thậm chí là các phần thưởng vật chất.

Ứng dụng: Tổ chức buổi trao đổi, vinh danh định kỳ hoặc lễ kỷ niệm nhỏ khi hoàn thành một cột mốc trong dự án. Nhấn mạnh tầm quan trọng của đóng góp cá nhân và tinh thần đồng đội trong nhóm để mỗi thành viên đều cảm thấy có giá trị và động lực làm việc cao hơn.

Ví dụ: Khi một thành viên hoàn thành phần việc khó trước thời hạn, trưởng nhóm công khai khen ngợi nỗ lực này trong buổi họp tuần, thậm chí tặng thêm phần thưởng nhỏ (như phiếu mua hàng). Sự ghi nhận này tạo cảm giác được đánh giá cao và thúc đẩy các thành viên khác cũng phấn đấu hoàn thành công việc với chất lượng cao.

7.2.1.3. Quản lý thời gian và phân công công việc hợp lý

Phương pháp: Phân chia công việc theo năng lực và sở trường của từng thành viên nhằm tăng năng suất làm việc và tránh tình trạng quá tải. Sử dụng các công cụ lập kế hoạch như bảng Kanban, sơ đồ Gantt hoặc phần mềm quản lý dự án để giúp mọi người theo dõi tiến độ và ưu tiên công việc một cách hiệu quả.

Ứng dụng: Trước khi bắt đầu một giai đoạn dự án, tổ chức các cuộc họp phân tích kỹ năng từng người để giao nhiệm vụ phù hợp. Điều này không chỉ đảm bảo mỗi thành viên làm đúng chuyên môn mà còn giúp tăng tính trách nhiệm và hiệu quả công việc.

Ví dụ: Trong một dự án thiết kế ứng dụng, trưởng nhóm tạo một bảng Kanban trên Trello, chia thành các cột "Việc cần làm", "Đang làm" và "Hoàn thành". Các nhiệm vụ cụ thể như "Thiết kế giao diện đăng nhập", "Lập trình tính năng đăng ký" được phân công cho các thành viên có kỹ năng tương ứng. Khi ai đó hoàn thành công việc, họ di chuyển thẻ sang cột "Hoàn thành" để mọi người cùng biết và tiếp tục chuyển sang nhiệm vụ khác.

7.2.1.4. Xây dựng môi trường học hỏi

Phương pháp: Khuyến khích chia sẻ kiến thức và kỹ năng thông qua các buổi đào tạo, chia sẻ kinh nghiệm, hoặc tổ chức hội thảo nội bộ. Tạo điều kiện cho các thành viên học hỏi lẫn nhau để gia tăng năng lực chung của nhóm và làm việc hiệu quả hơn.

Ứng dụng: Thiết lập các buổi thảo luận hàng tuần hoặc các buổi đào tạo kỹ năng từ các thành viên có kinh nghiệm. Ví dụ, nếu một thành viên có thể mạnh về lập trình, họ có thể chia sẻ cách sử dụng một công cụ hoặc framework mới, giúp cả nhóm làm việc hiệu quả hơn với các công cụ đó.

Ví dụ: Nếu dự án sử dụng một công nghệ mới (như một framework lập trình), trưởng nhóm tổ chức một buổi hướng dẫn mà các thành viên có kinh nghiệm sẽ chia sẻ cách cài đặt, sử dụng framework và giải đáp thắc mắc. Sau buổi này, những thành viên khác sẽ làm việc hiệu quả hơn và giảm thiểu lỗi phát sinh.

7.2.1.5. Xây dựng tinh thần làm việc nhóm

Phương pháp: Tăng cường sự gắn kết trong nhóm thông qua các hoạt động xây dựng đội ngũ, các trò chơi tập thể, hoặc buổi giao lưu ngoài công việc. Những hoạt động này giúp các thành viên hiểu rõ nhau hơn, xây dựng lòng tin và hỗ trợ nhau khi làm việc.

Ứng dụng: Tổ chức các buổi gặp gỡ ngoài giờ làm việc, như bữa trưa hoặc hoạt động thể thao, để các thành viên có cơ hội gắn bó và xây dựng mối quan hệ cá nhân tốt đẹp. Điều này sẽ giúp tạo môi trường làm việc hòa đồng và hỗ trợ lẫn nhau.

Ví dụ: Định kỳ hàng tháng, nhóm tổ chức một buổi ăn trưa hoặc hoạt động thể thao ngắn sau giờ làm việc. Trong môi trường không chính thức này, các thành viên dễ dàng trò chuyện, chia sẻ về cả công việc lẫn cuộc sống cá nhân. Khi trở lại công việc, sự gắn kết giữa họ sẽ giúp dễ dàng hợp tác và xử lý các vấn đề hiệu quả hơn.

7.2.2. Xử lý xung đột

Trong môi trường làm việc nhóm, xung đột là điều không thể tránh khỏi. Đặc biệt trong các dự án phần mềm có tính chất phức tạp và nhiều bộ phận liên quan, các xung đột có thể nảy sinh từ sự khác biệt trong phong cách làm việc, hiểu biết về công nghệ, hay sự kỳ vọng không đồng nhất về tiến độ. Xử lý xung đột hiệu quả giúp duy trì sự hài hòa trong nhóm và đảm bảo tiến độ dự án. Dưới đây là các phương pháp chi tiết để giải quyết xung đột cùng với ví dụ cụ thể:

7.2.2.1. Lắng nghe và thấu hiểu

Phương pháp: Khi có xung đột, trước tiên cần để các bên liên quan trình bày quan điểm của mình. Lắng nghe chủ động không chỉ giúp bạn hiểu rõ nguồn gốc vấn đề mà còn thể hiện sự tôn trọng và tạo điều kiện để các bên giải tỏa căng thẳng.

Ví dụ: Trong một dự án, hai lập trình viên có ý kiến trái ngược về cách xử lý dữ liệu. Trưởng nhóm tổ chức một buổi họp riêng với cả hai, lắng nghe ý kiến từng người mà không phán xét, giúp họ hiểu rõ quan điểm của nhau và tìm ra điểm chung.

7.2.2.2. Phân tích nguyên nhân gốc rễ của xung đột

Phương pháp: Để giải quyết xung đột triệt để, cần tìm hiểu nguyên nhân sâu xa. Nguyên nhân có thể bắt nguồn từ sự khác biệt trong cách tiếp cận, hiểu biết về công nghệ, hoặc mâu thuẫn về vai trò, trách nhiệm.

Ví dụ: Trong một dự án phát triển ứng dụng, một thành viên cảm thấy bị phân công quá nhiều công việc, dẫn đến mâu thuẫn với trưởng nhóm. Sau khi phân tích, trưởng nhóm nhận ra rằng sự phân công không rõ ràng dẫn đến hiểu lầm, và sau đó đã điều chỉnh lại trách nhiệm để cân bằng khối lượng công việc.

7.2.2.3. Tìm kiếm giải pháp đôi bên cùng có lợi

Phương pháp: Khi xử lý xung đột, nên cố gắng tìm ra giải pháp mà cả hai bên đều hài lòng. Các thành viên sẽ có động lực và cảm giác công bằng khi họ thấy quyền lợi và quan điểm của mình được xem xét.

Ví dụ: Một thành viên muốn thay đổi giao diện ứng dụng để tối ưu hiệu suất, trong khi người khác muốn giữ thiết kế cũ để đảm bảo tính ổn định. Trưởng nhóm tổ chức một cuộc thảo luận và đề xuất thử nghiệm thiết kế mới trong một tính năng nhỏ, từ đó đánh giá ảnh hưởng trước khi áp dụng rộng rãi.

7.2.2.4. Áp dụng kỹ năng thương lượng

Phương pháp: Trong một số trường hợp, thương lượng là cách tốt để các bên nhượng bộ một phần và đạt được sự đồng thuận. Thương lượng cần đi kèm với thái độ tôn trọng, nhường nhịn và sẵn sàng hợp tác.

Ví dụ: Trong một cuộc họp về lịch trình, hai nhóm có yêu cầu mốc thời gian khác nhau. Sau khi thương lượng, họ đồng ý chọn một mốc thời gian trung gian mà cả hai bên đều có thể chấp nhận, đồng thời bổ sung thêm tài nguyên để hoàn thành công việc sớm hơn dự kiến.

7.2.2.5. Sử dụng trung gian hòa giải

Phương pháp: Khi xung đột trở nên khó kiểm soát, việc nhờ một người trung gian có uy tín để hòa giải có thể giúp giải quyết vấn đề. Người này sẽ đóng vai trò dẫn dắt, đảm bảo cuộc thảo luận diễn ra trong không khí bình tĩnh và đưa ra góc nhìn khách quan.

Ví dụ: Hai thành viên của nhóm phát triển bất đồng về cách viết mã nguồn. Trưởng nhóm, với kinh nghiệm và kiến thức của mình, tham gia làm trung gian, đưa ra các ví dụ thực tiễn và cùng cả hai thành viên thảo luận để thống nhất một quy trình mã hóa phù hợp cho nhóm.

7.2.2.6. Đưa ra quyết định cuối cùng nếu cần thiết

Phương pháp: Trong một số trường hợp, nếu không thể đạt được sự đồng thuận, người lãnh đạo phải đưa ra quyết định cuối cùng để đảm bảo tiến độ và tránh kéo dài xung đột. Quyết định này cần dựa trên lợi ích chung của dự án và các mục tiêu đã đề ra.

Ví dụ: Nếu một thành viên phản đối một phương pháp kỹ thuật mà nhóm đã nhất trí sử dụng, và không có đủ thời gian để thử phương pháp khác, trưởng nhóm quyết định tiếp tục với phương pháp đã chọn, đồng thời giải thích rõ ràng lý do để tránh tạo ra bất mãn.

7.2.3. Đảm bảo tiến độ và chất lượng dự án

Kỹ năng mềm không chỉ giúp duy trì một môi trường làm việc tích cực, mà còn đóng vai trò quan trọng trong việc đảm bảo tiến độ và chất lượng của dự án phần mềm. Việc áp dụng các kỹ năng mềm phù hợp sẽ giúp giải quyết các vấn đề phát sinh nhanh chóng, tạo sự gắn kết trong nhóm và hỗ trợ quá trình đạt được các mục tiêu dự án.

7.2.3.1. Xây dựng tinh thần làm việc nhóm và sự gắn kết

Phương pháp: Một nhóm có sự đoàn kết sẽ có khả năng hoàn thành công việc hiệu quả hơn và giúp duy trì tiến độ dự án. Các kỹ năng mềm như giao tiếp hiệu quả, lắng nghe, và khả năng hợp tác là chìa khóa để xây dựng sự gắn kết trong nhóm.

Ví dụ: Để đảm bảo tiến độ và chất lượng, nhóm có thể áp dụng phương pháp Agile, nơi các thành viên thường xuyên gặp gỡ trong các cuộc họp ngắn (daily stand-ups) để báo cáo tiến độ và chia sẻ khó khăn. Điều này giúp sớm nhận diện các vấn đề và tạo ra cơ hội giải quyết kịp thời.

Lợi ích: Các cuộc họp và tương tác thường xuyên giúp giữ cho tất cả thành viên đồng bộ về tiến độ và mục tiêu. Đồng thời, tạo ra một không gian mở để mọi người cảm thấy thoải mái khi chia sẻ ý tưởng và phản hồi.

7.2.3.2. Quản lý thời gian và phân chia công việc hợp lý

Phương pháp: Kỹ năng quản lý thời gian là một trong những yếu tố quan trọng nhất để đảm bảo tiến độ dự án. Người lãnh đạo cần có khả năng phân chia công việc hợp lý, ưu tiên các nhiệm vụ quan trọng và kiểm soát được tiến độ công việc của từng thành viên.

Ví dụ: Sử dụng phương pháp như **Eisenhower Matrix** (phân loại công việc theo độ quan trọng và tính cấp bách) hoặc các công cụ quản lý dự án như **Trello** hoặc **Asana** để phân công công việc, giám sát tiến độ và tránh chậm trễ.

Lợi ích: Kỹ năng này giúp nhóm nhận diện được công việc cần làm ngay và tránh việc trì hoãn các công việc quan trọng. Hơn nữa, nó giúp duy trì sự linh hoạt trong quản lý các thay đổi, nếu có.

7.2.3.3. Kỹ năng giao tiếp và truyền đạt thông tin

Phương pháp: Giao tiếp hiệu quả là nền tảng của một dự án thành công. Kỹ năng giao tiếp không chỉ giúp các thành viên trong nhóm hiểu rõ yêu cầu công việc mà còn giúp quản lý truyền đạt đúng đắn các thông tin quan trọng, đảm bảo các quyết định kịp thời.

Ví dụ: Thường xuyên tổ chức các cuộc họp báo cáo tiến độ, nơi mỗi thành viên cập nhật công việc của mình và nhận phản hồi từ các đồng nghiệp. Đồng thời, đảm bảo rằng tất cả các yêu cầu và thay đổi từ khách hàng được truyền đạt rõ ràng đến tất cả các bộ phận liên quan.

Lợi ích: Giúp đảm bảo rằng không có thông tin nào bị bỏ sót, các yêu cầu và thay đổi được cập nhật kịp thời, từ đó giúp nhóm duy trì đúng tiến độ và đạt chất lượng như mong đợi.

7.2.3.4. Lãnh đạo bằng sự đồng cảm và khích lệ

Phương pháp: Kỹ năng lãnh đạo là yếu tố giúp duy trì sự động viên và khích lệ trong nhóm, đặc biệt khi đối mặt với các thử thách khó khăn. Người lãnh đạo có khả năng tạo ra một môi trường nơi mỗi thành viên cảm thấy động lực và niềm tin vào dự án.

Ví dụ: Khi nhóm gặp phải sự cố kỹ thuật hoặc tiến độ bị chậm trễ, người lãnh đạo cần thể hiện sự thấu hiểu và hỗ trợ, giúp đội ngũ vượt qua khó khăn. Khích lệ các thành viên có thành tích tốt và đưa ra các giải pháp để hỗ trợ người gặp khó khăn.

Lợi ích: Sự động viên sẽ giúp giảm căng thẳng trong nhóm, thúc đẩy sự sáng tạo và cải thiện tinh thần làm việc, từ đó giúp nhóm đạt được tiến độ và chất lượng công việc.

7.2.3.5. Giải quyết xung đột để duy trì tiến độ và chất lượng

Phương pháp: Trong mỗi nhóm, xung đột là điều không thể tránh khỏi. Kỹ năng giải quyết xung đột giúp người lãnh đạo và các thành viên duy trì sự hòa hợp, từ đó giảm thiểu ảnh hưởng tiêu cực lên tiến độ và chất lượng của dự án.

Ví dụ: Khi có sự bất đồng về phương pháp hoặc kế hoạch làm việc, người lãnh đạo có thể tổ chức một cuộc họp để giải quyết vấn đề, lắng nghe quan điểm của các bên và đưa ra quyết định hợp lý. Việc xử lý xung đột kịp thời sẽ tránh được sự bế tắc và giúp nhóm tiếp tục làm việc hiệu quả.

Lợi ích: Giải quyết xung đột giúp duy trì bầu không khí làm việc tích cực, tránh kéo dài các vấn đề không đáng có và giúp nhóm tiếp tục tập trung vào công việc chính.

7.2.3.6. Đảm bảo sự linh hoạt và thích nghi trong công việc

Phương pháp: Kỹ năng thích nghi và linh hoạt giúp đội ngũ dự án đối phó với những thay đổi bất ngờ trong yêu cầu khách hàng, công nghệ hoặc môi trường làm việc.

Ví dụ: Nếu khách hàng yêu cầu thay đổi vào giai đoạn gần cuối dự án, người lãnh đạo và nhóm cần có khả năng điều chỉnh kế hoạch và tài nguyên mà không làm gián đoạn quá trình thực hiện.

Lợi ích: Khả năng thích nghi sẽ giúp dự án không bị tụt lại phía sau, từ đó tiến độ và chất lượng dự án được đảm bảo ngay cả khi có thay đổi hoặc sự cố xảy ra.

7.2.3.7. Eisenhower Matrix

Eisenhower Matrix (còn gọi là **Eisenhower Box** hoặc **Urgent-Important Matrix**) là một công cụ quản lý thời gian nổi tiếng, được đặt tên theo Tổng thống Mỹ Dwight D. Eisenhower, người nổi tiếng với khả năng đưa ra quyết định hiệu quả. Phương pháp này giúp phân loại các công việc dựa trên hai tiêu chí chính: **Mức độ quan trọng** và **Mức độ cấp bách**. Nó giúp xác định công việc nào cần ưu tiên, công việc nào có thể hoãn lại và công việc nào có thể ủy thác.

a. Cấu trúc của Eisenhower Matrix

Eisenhower Matrix chia công việc thành 4 nhóm, dựa trên hai tiêu chí:

- **Quan trọng và Cấp bách (Do first)**

Công việc nằm trong nhóm này là những nhiệm vụ cần được thực hiện ngay lập tức vì chúng vừa quan trọng lại vừa cấp bách. Đây là những công việc có thể có tác động lớn đến dự án hoặc công việc của bạn, và bạn cần phải làm chúng ngay lập tức.

Ví dụ: Xử lý một lỗi nghiêm trọng trong sản phẩm phần mềm, phản hồi email khẩn cấp từ khách hàng.

- **Quan trọng nhưng Không Cấp bách (Schedule)**

Các công việc này có thể không cần thực hiện ngay lập tức nhưng chúng vẫn rất quan trọng và cần được lên kế hoạch thực hiện. Những công việc này thường là các nhiệm vụ chiến lược dài hạn hoặc công việc quan trọng nhưng không gây ảnh hưởng ngay lập tức nếu không được giải quyết.

Ví dụ: Lên kế hoạch phát triển phần mềm mới, cải thiện kỹ năng lập trình, hoặc xây dựng mối quan hệ với khách hàng lâu dài.

- **Không quan trọng nhưng Cấp bách (Delegate)**

Công việc trong nhóm này là những công việc không quá quan trọng đối với mục tiêu dài hạn của bạn nhưng lại có tính cấp bách. Bạn có thể ủy thác công việc này cho người khác để tiết kiệm thời gian và tập trung vào các nhiệm vụ quan trọng hơn.

Ví dụ: Giải quyết các email không quan trọng, tổ chức các cuộc họp nội bộ không liên quan trực tiếp đến mục tiêu chính của dự án.

- **Không quan trọng và Không Cấp bách (Eliminate)**

Những công việc này không quan trọng và cũng không cấp bách. Đây là những nhiệm vụ hoặc hoạt động tiêu tốn thời gian nhưng không mang lại giá trị đáng kể. Bạn có thể loại bỏ hoặc giảm thiểu việc làm những công việc này.

Ví dụ: Duyệt mạng xã hội trong giờ làm việc, tham gia vào các cuộc trò chuyện không cần thiết trong công ty.

b. Cách sử dụng Eisenhower Matrix

Liệt kê tất cả các công việc và nhiệm vụ cần làm.

Bắt đầu bằng cách viết ra tất cả các nhiệm vụ bạn phải thực hiện trong một ngày hoặc tuần.

Phân loại các công việc vào các nhóm trong ma trận.

Xác định mỗi công việc thuộc nhóm nào trong Eisenhower Matrix dựa trên tính quan trọng và cấp bách.

Đưa ra quyết định về ưu tiên và hành động.

- **Do first:** Thực hiện ngay lập tức các công việc quan trọng và cấp bách.
- **Schedule:** Lên lịch và lên kế hoạch thực hiện các công việc quan trọng nhưng không cấp bách.
- **Delegate:** Giao cho người khác các công việc không quan trọng nhưng cấp bách.
- **Eliminate:** Loại bỏ hoặc giảm thiểu các công việc không quan trọng và không cấp bách.

c. Lợi ích của Eisenhower Matrix

- **Giúp tập trung vào điều quan trọng:** Phương pháp này giúp nhận diện được đâu là những công việc thực sự cần sự chú ý và đâu là những công việc có thể giao cho người khác hoặc loại bỏ.
- **Tăng hiệu quả làm việc:** Bằng cách ưu tiên đúng công việc, có thể giảm thiểu thời gian dành cho những nhiệm vụ không cần thiết, từ đó tăng hiệu quả công việc.
- **Giúp giảm stress:** Khi bạn có một kế hoạch rõ ràng, sẽ dễ dàng hơn để xử lý các nhiệm vụ mà không cảm thấy bị choáng ngợp.

d. Ví dụ về Eisenhower Matrix

Giả sử là quản lý dự án phần mềm và có một số công việc cần làm trong một ngày làm việc:

Công việc	Quan trọng	Cấp bách	Nhóm trong ma trận
Xử lý lỗi nghiêm trọng của phần mềm	Có	Có	Do First
Lên kế hoạch phát triển tính năng mới	Có	Không	Schedule
Phản hồi email của	Có	Có	Do First

khách hàng			
Kiểm tra các tài liệu không liên quan đến dự án	Không	Không	Eliminate
Giao công việc cho các thành viên trong nhóm	Có	Không	Schedule
Tổ chức cuộc họp không cần thiết	Không	Có	Delegate

Qua bảng trên, ta sẽ ưu tiên giải quyết các vấn đề quan trọng và cấp bách trước, đồng thời lên kế hoạch cho các công việc quan trọng nhưng không cấp bách, giao các công việc không quan trọng cho người khác và loại bỏ các công việc không cần thiết.

Phương pháp **Eisenhower Matrix** là một công cụ đơn giản nhưng rất hiệu quả giúp tăng cường khả năng quản lý thời gian và quyết định ưu tiên công việc trong dự án.

Câu hỏi ôn tập cuối chương

1. Kỹ năng giao tiếp có vai trò gì trong việc đảm bảo tiến độ và chất lượng của dự án phần mềm?
2. Các phương pháp giao tiếp hiệu quả trong quản lý dự án phần mềm bao gồm những gì? Nêu ví dụ cụ thể.
3. Vai trò của kỹ năng lãnh đạo trong việc xây dựng động lực cho đội nhóm là gì?
4. Mô hình SMART được áp dụng như thế nào để thiết lập mục tiêu hiệu quả trong dự án phần mềm?
5. Kỹ năng đàm phán giúp giải quyết những vấn đề gì trong quản lý dự án phần mềm?
6. Các phương pháp xử lý xung đột trong quản lý dự án phần mềm là gì? Nêu ví dụ minh họa.
7. Lắng nghe tích cực (Active Listening) đóng vai trò như thế nào trong việc xây dựng mối quan hệ trong nhóm dự án?
8. Tại sao việc phân tích nguyên nhân gốc rễ của xung đột lại quan trọng trong quản lý dự án phần mềm?
9. Phương pháp Eisenhower Matrix hỗ trợ quản lý thời gian như thế nào? Nêu một ví dụ ứng dụng cụ thể.
10. Làm thế nào để xây dựng môi trường làm việc tích cực và hợp tác trong nhóm dự án phần mềm?

Chương 8: Quản lý và phân phối kết quả dự án

Nội dung chính của chương

Chương 8 trình bày về quy trình quản lý và phân phối kết quả dự án, tập trung vào các bước kiểm thử trước khi triển khai nhằm đảm bảo phần mềm đáp ứng đầy đủ yêu cầu kỹ thuật và người dùng. Nội dung bao gồm việc kiểm thử sản phẩm qua các giai đoạn như kiểm thử đơn vị, tích hợp, hệ thống, hiệu suất, bảo mật và chấp nhận người dùng, sử dụng các công cụ hỗ trợ hiện đại như JUnit, Selenium, Jira, và Git. Ngoài ra, chương cũng đề cập đến quy trình bảo trì phần mềm sau triển khai, bao gồm sửa lỗi, cải tiến và bảo trì phòng ngừa, cùng với việc thu thập phản hồi người dùng để nâng cao chất lượng sản phẩm. Các phương pháp phân tích như Pareto, A/B Testing và phân tích điểm đau được sử dụng để tối ưu hóa cải tiến và đáp ứng nhu cầu thay đổi của người dùng. Chương kết thúc bằng việc nhấn mạnh tầm quan trọng của lập kế hoạch bảo trì dài hạn để duy trì hiệu quả và sự hài lòng của khách hàng trong suốt vòng đời phần mềm.

Mục tiêu cần đạt của chương

- Hiểu quy trình phân phối sản phẩm và bảo trì sau triển khai.
- Biết nguyên tắc đảm bảo sản phẩm hoạt động đúng yêu cầu và đáp ứng nhu cầu người dùng.

8.1. Quy trình phân phối sản phẩm và kiểm thử trước khi triển khai nhằm đảm bảo phần mềm hoạt động đúng như yêu cầu.

Quy trình phân phối sản phẩm và kiểm thử trước khi triển khai là bước quan trọng trong quá trình phát triển phần mềm, giúp đảm bảo rằng phần mềm không chỉ hoàn thiện về tính năng mà còn hoạt động đúng như yêu cầu ban đầu.

8.1.1. Quy trình phân phối sản phẩm

8.1.1.1. Đánh giá và kiểm tra kết quả dự án

Mục tiêu: Đảm bảo sản phẩm đáp ứng yêu cầu và tiêu chuẩn đã được thiết lập.

Công cụ sử dụng:

- Kiểm thử tự động: Các công cụ như JUnit (cho Java), Appium (cho ứng dụng di động) giúp kiểm tra tính năng tự động của phần mềm, giảm thiểu lỗi do con người và tăng tốc quá trình kiểm thử.
- Jira hoặc TestRail: Sử dụng để theo dõi và quản lý các ca kiểm thử, quản lý các báo cáo lỗi và theo dõi sự tiến triển của việc khắc phục sự cố.

- SonarQube: Công cụ kiểm tra chất lượng mã nguồn, giúp phát hiện các vấn đề như lỗi bảo mật, mã không hiệu quả hoặc vi phạm quy chuẩn lập trình.

8.1.1.2. Xác định và chuyển giao kết quả

Mục tiêu: Đảm bảo rằng kết quả của dự án được chuyển giao đúng và đầy đủ cho khách hàng hoặc các bên liên quan.

Công cụ sử dụng:

- Git hoặc SVN (Subversion): Các hệ thống kiểm soát phiên bản giúp theo dõi sự thay đổi của mã nguồn và hỗ trợ việc bàn giao mã nguồn cho các nhóm khác nhau trong dự án.
- Confluence: Công cụ này giúp lưu trữ và chia sẻ tài liệu dự án, bao gồm tài liệu hướng dẫn sử dụng, tài liệu triển khai và các báo cáo sau khi hoàn thành.
- Slack hoặc Microsoft Teams: Dùng để giao tiếp và chia sẻ thông tin trong nhóm dự án, từ đó cải thiện quá trình bàn giao thông tin và tài liệu.

8.1.1.3. Theo dõi và đánh giá kết quả

Mục tiêu: Đảm bảo phần mềm hoạt động tốt và nhận phản hồi để cải tiến sản phẩm.

Công cụ sử dụng:

- Google Analytics hoặc Firebase Analytics: Dùng để theo dõi hành vi người dùng trong ứng dụng, đo lường hiệu suất của ứng dụng sau khi triển khai.
- Jira hoặc Trello: Công cụ quản lý dự án giúp theo dõi các vấn đề phát sinh và công việc đang thực hiện. Dự án có thể dễ dàng theo dõi tiến độ qua các task hoặc bug.
- SurveyMonkey hoặc Typeform: Các công cụ khảo sát giúp thu thập phản hồi từ người dùng cuối về các vấn đề gặp phải khi sử dụng sản phẩm.

8.1.1.4. Đóng dự án và phân phối kết quả

Mục tiêu: Kết thúc dự án và phân phối sản phẩm cho khách hàng hoặc các bên liên quan.

Công cụ sử dụng:

- **Post-mortem meetings (Họp tổng kết):** Dùng để phân tích các bài học kinh nghiệm từ dự án. Các công cụ hỗ trợ họp online như Zoom, Microsoft Teams giúp tổ chức các cuộc họp trực tuyến với các bên liên quan.
- **Báo cáo kết thúc dự án (Project Closure Report):** Tạo báo cáo tổng kết về tiến độ, kết quả dự án, chi phí và bài học kinh nghiệm. Các công cụ như Google Docs, Confluence có thể giúp trong việc tạo và chia sẻ các báo cáo này.
- **Asana hoặc Basecamp:** Là công cụ quản lý dự án giúp theo dõi các nhiệm vụ cuối cùng của dự án, đảm bảo tất cả các công việc được hoàn thành và bàn giao đúng hạn.

8.1.2. Kiểm thử trước khi triển khai

Kiểm thử trước khi triển khai là một trong những bước quan trọng trong quy trình phát triển phần mềm, nhằm đảm bảo rằng sản phẩm phần mềm sẽ hoạt động ổn định và chính xác khi được đưa vào môi trường thực tế. Mục tiêu của kiểm thử là phát hiện lỗi và sự cố trước khi phần mềm được triển khai chính thức, từ đó nâng cao chất lượng và giảm thiểu rủi ro cho người sử dụng cuối.

Mục đích của Kiểm thử trước khi triển khai

- **Đảm bảo chất lượng phần mềm:** Kiểm thử giúp phát hiện các lỗi phần mềm sớm, trước khi sản phẩm được đưa vào môi trường thực tế. Điều này giúp giảm thiểu các sự cố trong quá trình vận hành và duy trì phần mềm.
- **Đảm bảo tính khả dụng và ổn định:** Phần mềm cần phải ổn định, dễ sử dụng, và hoạt động liên tục mà không gặp phải các vấn đề nghiêm trọng sau khi triển khai.
- **Đáp ứng yêu cầu của khách hàng:** Kiểm thử giúp xác minh rằng phần mềm đáp ứng đúng các yêu cầu chức năng và phi chức năng mà khách hàng hoặc người dùng yêu cầu.

8.1.2.1. Kiểm thử đơn vị (Unit Testing)

Mục đích: Kiểm thử các đơn vị mã nguồn (như hàm, phương thức) một cách độc lập để đảm bảo chúng thực hiện đúng chức năng được yêu cầu.

Mô tả: Kiểm thử đơn vị tập trung vào việc kiểm tra các thành phần nhỏ nhất của ứng dụng, chẳng hạn như một hàm hoặc một phương thức trong lớp. Phương pháp này

nhằm phát hiện lỗi trong quá trình phát triển khi phần mềm vẫn còn ở mức cấu trúc cơ bản.

Quy trình kiểm thử:

- Viết các ca kiểm thử cho từng đơn vị mã nguồn, bao gồm các trường hợp bình thường và các trường hợp lỗi.
- Kiểm thử từng đơn vị riêng biệt và đảm bảo rằng các kết quả trả về của hàm/phương thức là chính xác.

Công cụ hỗ trợ:

- JUnit (Java) hoặc Kotest (Kotlin): Các framework kiểm thử phổ biến để viết và thực thi kiểm thử đơn vị tự động.
- Mockito: Một thư viện giúp kiểm thử đơn vị thông qua việc giả lập các đối tượng.

8.1.2.2. Kiểm thử tích hợp (Integration Testing)

Mục đích: Đảm bảo rằng các thành phần hoặc module trong hệ thống tương tác chính xác với nhau khi được tích hợp.

Mô tả: Kiểm thử tích hợp kiểm tra xem các phần của phần mềm, vốn được phát triển riêng lẻ, có làm việc đúng khi kết hợp lại với nhau hay không. Phương pháp này giúp phát hiện các vấn đề phát sinh từ sự tương tác giữa các thành phần.

Quy trình kiểm thử:

- Chọn các module hoặc thành phần có liên quan và kiểm thử khả năng tương tác của chúng.
- Thực hiện kiểm thử với các kết hợp của các module khác nhau để đảm bảo chúng phối hợp một cách chính xác.

Công cụ hỗ trợ:

- **Postman** hoặc **Insomnia**: Dùng để kiểm thử API trong kiểm thử tích hợp.
- **Spring Test (cho Java)** hoặc **Ktor Test (cho Kotlin)**: Công cụ kiểm thử cho các dịch vụ backend.
- ...

8.1.2.3. Kiểm thử hệ thống (System Testing)

Mục đích: Đảm bảo phần mềm hoạt động đúng như mong đợi trong một môi trường giả lập tương tự với môi trường sản xuất.

Mô tả: Kiểm thử hệ thống kiểm tra toàn bộ phần mềm để đảm bảo rằng nó đáp ứng đầy đủ các yêu cầu chức năng và phi chức năng. Đây là bước kiểm thử toàn diện trước khi đưa phần mềm vào môi trường thực tế.

Quy trình kiểm thử:

- Xác định tất cả các yêu cầu chức năng và phi chức năng của hệ thống.
- Kiểm tra các tính năng tổng thể của phần mềm, bao gồm kiểm tra giao diện người dùng, kiểm thử bảo mật, hiệu suất và khả năng tương thích.

Các loại kiểm thử hệ thống:

- **Kiểm thử chức năng:** Đảm bảo rằng phần mềm thực hiện đúng các tính năng theo yêu cầu.
- **Kiểm thử phi chức năng:** Bao gồm kiểm thử hiệu suất (speed, scalability), bảo mật (vulnerabilities), và độ tin cậy (reliability).

Công cụ hỗ trợ:

- **Selenium:** Kiểm thử tự động giao diện người dùng (UI).
- **LoadRunner:** Kiểm thử hiệu suất và tải.
- **OWASP ZAP:** Công cụ kiểm thử bảo mật cho các ứng dụng web.

8.1.2.4. Kiểm thử chấp nhận người dùng (User Acceptance Testing - UAT)

Mục đích: Đảm bảo phần mềm đáp ứng nhu cầu thực tế của người dùng cuối và có thể hoạt động hiệu quả trong môi trường thực tế.

Mô tả: Kiểm thử chấp nhận người dùng là giai đoạn cuối cùng trong kiểm thử phần mềm, nơi người dùng thực tế (hoặc đại diện của họ) kiểm tra phần mềm để đảm bảo rằng phần mềm đáp ứng các yêu cầu của họ và sẵn sàng cho triển khai. UAT thường được thực hiện sau khi tất cả các kiểm thử kỹ thuật khác đã hoàn thành.

Quy trình kiểm thử:

- Người dùng hoặc nhóm người dùng đại diện sẽ kiểm tra các tính năng chính của phần mềm, sử dụng nó trong môi trường gần giống với thực tế.

- Người dùng sẽ đánh giá phần mềm theo các tiêu chí như dễ sử dụng, tính đầy đủ của tính năng và khả năng đáp ứng nhu cầu công việc.
- Các lỗi hoặc yêu cầu thay đổi từ người dùng sẽ được ghi nhận và xử lý.

Công cụ hỗ trợ:

- **Trello hoặc Jira:** Quản lý các yêu cầu UAT và theo dõi các vấn đề phát sinh.
- **TestRail:** Công cụ quản lý kiểm thử giúp theo dõi các kịch bản kiểm thử và kết quả của UAT.

8.1.2.5. Kiểm thử hiệu suất (Performance Testing)

Mục đích: Đảm bảo rằng phần mềm có thể xử lý được khối lượng công việc yêu cầu mà không gặp phải sự cố về hiệu suất.

Mô tả: Kiểm thử hiệu suất tập trung vào việc xác định mức độ chịu tải của phần mềm dưới các điều kiện sử dụng khác nhau. Đây là kiểm thử quan trọng để đảm bảo phần mềm có thể chịu đựng được lưu lượng cao và hoạt động ổn định khi có nhiều người dùng hoặc tải lớn.

Các loại kiểm thử hiệu suất:

- **Kiểm thử tải (Load Testing):** Kiểm tra khả năng của hệ thống khi chịu tải bình thường và tải cực đại.
- **Kiểm thử căng thẳng (Stress Testing):** Đưa phần mềm vào các tình huống căng thẳng để xem hệ thống có thể hoạt động đúng khi vượt quá giới hạn chịu đựng hay không.
- **Kiểm thử khả năng mở rộng (Scalability Testing):** Đo lường khả năng mở rộng của hệ thống khi tăng thêm người dùng hoặc dữ liệu.

Công cụ hỗ trợ:

- **Apache JMeter:** Công cụ kiểm thử hiệu suất phổ biến.
- **Gatling:** Công cụ kiểm thử tải và hiệu suất với khả năng lập kịch bản mạnh mẽ.

8.1.2.6. Kiểm thử bảo mật (Security Testing)

Mục đích: Đảm bảo phần mềm không có lỗ hổng bảo mật và có thể bảo vệ thông tin người dùng cũng như dữ liệu quan trọng.

Mô tả: Kiểm thử bảo mật giúp phát hiện các lỗi bảo mật trong ứng dụng, chẳng hạn như các lỗ hổng mà tin tặc có thể khai thác. Kiểm thử này giúp ngăn ngừa các cuộc tấn công và bảo vệ dữ liệu người dùng.

Quy trình kiểm thử:

- Thực hiện các kiểm thử như kiểm thử lỗ hổng, kiểm thử xác thực và phân quyền, kiểm thử bảo mật dữ liệu.
- Kiểm tra các API và giao diện kết nối với hệ thống để phát hiện các lỗi bảo mật.

Công cụ hỗ trợ:

- **OWASP ZAP:** Công cụ kiểm thử bảo mật tự động giúp phát hiện các lỗ hổng bảo mật.
- **Burp Suite:** Công cụ phổ biến dùng để kiểm thử bảo mật các ứng dụng web.

8.1.3. Đảm bảo phần mềm hoạt động đúng như yêu cầu

Đảm bảo phần mềm hoạt động đúng như yêu cầu là một bước quan trọng trong quá trình kiểm thử trước khi triển khai. Điều này không chỉ giúp phần mềm hoạt động đúng chức năng mà còn đảm bảo sự hài lòng của người dùng cuối và các bên liên quan. Để thực hiện điều này, các nhóm phát triển và kiểm thử cần sử dụng các phương pháp và kỹ thuật kiểm thử thích hợp nhằm kiểm tra cả về yêu cầu chức năng lẫn phi chức năng của phần mềm.

8.1.3.1. Xác định yêu cầu phần mềm

Trước khi thực hiện kiểm thử để đảm bảo phần mềm hoạt động đúng như yêu cầu, việc đầu tiên là phải xác định rõ các yêu cầu phần mềm. Các yêu cầu này có thể được phân thành hai loại chính:

Yêu cầu chức năng (Functional Requirements): Đặc tả những tính năng và chức năng mà phần mềm cần phải thực hiện. Ví dụ: một ứng dụng thanh toán cần phải thực hiện giao dịch thanh toán, xuất hóa đơn, kiểm tra số dư tài khoản.

Yêu cầu phi chức năng (Non-functional Requirements): Đặc tả các yêu cầu không liên quan trực tiếp đến chức năng của phần mềm nhưng lại ảnh hưởng đến hiệu suất, bảo mật, khả năng mở rộng, và khả năng sử dụng. Ví dụ: thời gian phản hồi của hệ thống, tính ổn định trong trường hợp có nhiều người dùng đồng thời.

8.1.3.2. Lập kế hoạch kiểm thử để đảm bảo yêu cầu

Khi các yêu cầu phần mềm đã được xác định rõ ràng, việc tiếp theo là lập kế hoạch kiểm thử. Kế hoạch này sẽ giúp xác định các ca kiểm thử cụ thể để đảm bảo rằng phần mềm thực hiện đúng các yêu cầu đã đặt ra. Các bước kiểm thử bao gồm:

- **Xây dựng các kịch bản kiểm thử (Test Scenarios):** Mỗi kịch bản kiểm thử mô phỏng các tình huống cụ thể mà người dùng có thể gặp phải khi sử dụng phần mềm. Ví dụ, với một ứng dụng đặt vé máy bay, một kịch bản kiểm thử có thể là "người dùng chọn chuyến bay và thanh toán thành công".
- **Định nghĩa các ca kiểm thử (Test Cases):** Các ca kiểm thử là các tình huống kiểm tra cụ thể để kiểm tra xem phần mềm có thực hiện đúng chức năng yêu cầu hay không. Mỗi ca kiểm thử bao gồm các bước kiểm thử, đầu vào (input), và kết quả mong đợi (expected output).

8.1.3.3. Kiểm thử chức năng

Đảm bảo phần mềm hoạt động đúng chức năng là điều quan trọng nhất trong kiểm thử phần mềm. Các phương pháp kiểm thử chức năng sẽ kiểm tra xem phần mềm có thực hiện đúng các tính năng như đã được mô tả trong yêu cầu chức năng hay không. Các kỹ thuật kiểm thử chức năng bao gồm:

Kiểm thử theo yêu cầu (Requirement-based Testing): Kiểm thử được thiết kế dựa trên các yêu cầu phần mềm đã được xác định từ trước. Các kịch bản kiểm thử sẽ được xây dựng để kiểm tra mọi yêu cầu mà phần mềm cần thực hiện.

Kiểm thử chuỗi tác vụ (End-to-End Testing): Đảm bảo phần mềm thực hiện các chuỗi tác vụ từ đầu đến cuối một cách chính xác. Ví dụ, trong một hệ thống đặt vé máy bay, kiểm thử chuỗi tác vụ có thể kiểm tra quá trình từ khi người dùng chọn chuyến bay đến khi thanh toán hoàn tất.

8.1.3.4. Kiểm thử phi chức năng

Bên cạnh các yêu cầu chức năng, phần mềm cũng cần phải đáp ứng các yêu cầu phi chức năng để đảm bảo hiệu suất và tính ổn định khi triển khai. Việc kiểm thử phi chức năng sẽ đảm bảo phần mềm không chỉ thực hiện đúng chức năng mà còn có thể vận hành trong các điều kiện thực tế.

Kiểm thử hiệu suất (Performance Testing): Kiểm tra khả năng của phần mềm trong việc xử lý lượng người dùng hoặc dữ liệu lớn. Điều này giúp xác định xem phần mềm có thể duy trì hiệu suất ổn định trong điều kiện tải cao hay không.

Kiểm thử tải (Load Testing): Đo lường khả năng của phần mềm khi chịu một số lượng người dùng nhất định.

Kiểm thử căng thẳng (Stress Testing): Xác định giới hạn của phần mềm khi gặp phải tải cao bất ngờ.

Kiểm thử bảo mật (Security Testing): Đảm bảo phần mềm có các biện pháp bảo vệ dữ liệu và chống lại các lỗ hổng bảo mật. Việc kiểm thử bảo mật sẽ kiểm tra các lỗ hổng có thể bị khai thác bởi tin tặc để đảm bảo phần mềm không bị tấn công.

Kiểm thử khả năng sử dụng (Usability Testing): Đảm bảo phần mềm dễ sử dụng, giao diện người dùng thân thiện và dễ hiểu. Các bài kiểm thử này thường được thực hiện bởi người dùng cuối để kiểm tra trải nghiệm người dùng (UX).

8.1.3.5. Kiểm thử chấp nhận người dùng (User Acceptance Testing - UAT)

Mục đích: Kiểm thử chấp nhận người dùng là bước cuối cùng để đảm bảo phần mềm đáp ứng đầy đủ các yêu cầu của người dùng cuối và sẵn sàng triển khai vào môi trường thực tế.

Quy trình kiểm thử: Trong giai đoạn này, người dùng hoặc nhóm người dùng đại diện sẽ kiểm tra phần mềm dựa trên các kịch bản thực tế. Họ sẽ xác nhận rằng phần mềm không chỉ hoạt động đúng theo yêu cầu mà còn mang lại giá trị thực tiễn cho công việc của họ.

Công cụ hỗ trợ: Các công cụ như **Jira**, **Trello**, hoặc **TestRail** có thể giúp theo dõi các yêu cầu UAT và quản lý các vấn đề phát sinh trong quá trình kiểm thử.

8.1.3.6. Phản hồi và cải tiến

Sau khi thực hiện các ca kiểm thử, nhóm kiểm thử sẽ thu thập dữ liệu từ kết quả kiểm thử và gửi báo cáo lỗi (bug reports) nếu phát hiện vấn đề. Các phản hồi này sẽ giúp nhóm phát triển sửa lỗi, điều chỉnh hoặc cải tiến phần mềm để đảm bảo phần mềm cuối cùng đáp ứng hoàn hảo các yêu cầu của người dùng.

8.1.3.7. Kiểm thử lặp lại (Regression Testing)

Mục đích: Đảm bảo rằng việc sửa lỗi hoặc thay đổi mã nguồn không ảnh hưởng đến các chức năng hiện có của phần mềm.

Quy trình kiểm thử: Sau khi các lỗi được sửa, các ca kiểm thử cũ sẽ được chạy lại để kiểm tra xem các tính năng trước đó có còn hoạt động chính xác hay không. Điều

này giúp tránh tình trạng phần mềm hoạt động đúng ở một phần nhưng bị lỗi ở các phần khác do thay đổi mã nguồn.

Công cụ hỗ trợ: Các công cụ tự động như **Selenium**, **JUnit** có thể được sử dụng để thực hiện kiểm thử lại tự động.

Đảm bảo phần mềm hoạt động đúng như yêu cầu không chỉ là việc kiểm tra tính năng mà còn bao gồm việc kiểm tra tính ổn định, hiệu suất, bảo mật, và khả năng đáp ứng nhu cầu người dùng. Một quy trình kiểm thử đầy đủ, kết hợp với việc sử dụng các phương pháp kiểm thử khác nhau, sẽ giúp phát hiện và khắc phục các vấn đề tiềm ẩn trước khi phần mềm được triển khai chính thức. Các phương pháp kiểm thử cần được thực hiện nghiêm ngặt và liên tục trong suốt vòng đời phát triển phần mềm để đảm bảo phần mềm luôn đạt chất lượng cao nhất.

8.2. Quy trình bảo trì

8.2.1. Lập kế hoạch bảo trì

Bảo trì phần mềm sau khi triển khai là một giai đoạn quan trọng trong chu kỳ sống của phần mềm, nhằm duy trì hiệu quả hoạt động, khắc phục sự cố và đảm bảo rằng phần mềm luôn đáp ứng được yêu cầu của người dùng trong suốt thời gian sử dụng. Lập kế hoạch bảo trì là bước đầu tiên và thiết yếu trong việc đảm bảo phần mềm được bảo trì một cách có hệ thống và hiệu quả. Một kế hoạch bảo trì phần mềm tốt sẽ giúp giảm thiểu chi phí, thời gian và rủi ro liên quan đến các vấn đề phát sinh sau khi triển khai.

8.2.1.1. Xác định các loại bảo trì phần mềm

Trong lập kế hoạch bảo trì, việc phân loại các loại bảo trì là rất quan trọng. Mỗi loại bảo trì có các đặc điểm và mục tiêu khác nhau, giúp xác định các hành động và ưu tiên cụ thể. Các loại bảo trì phần mềm chính bao gồm:

Bảo trì sửa lỗi (Corrective Maintenance): Đây là quá trình khắc phục các lỗi và sự cố phát sinh trong phần mềm sau khi triển khai. Các lỗi này có thể do sai sót trong mã nguồn, thiếu sót trong yêu cầu, hoặc do thay đổi môi trường làm việc. Bảo trì sửa lỗi giúp khôi phục phần mềm về trạng thái hoạt động bình thường.

Bảo trì cải tiến (Adaptive Maintenance): Là quá trình điều chỉnh phần mềm để phù hợp với các thay đổi của môi trường bên ngoài, chẳng hạn như hệ điều hành, phần cứng mới hoặc các yêu cầu về bảo mật. Bảo trì cải tiến giúp phần mềm tiếp tục hoạt động hiệu quả trong bối cảnh thay đổi của công nghệ.

Bảo trì hoàn thiện (Perfective Maintenance): Là quá trình cải tiến tính năng và chức năng của phần mềm dựa trên phản hồi của người dùng, giúp phần mềm trở nên dễ sử dụng hơn, tối ưu hóa hiệu suất và tăng cường tính năng mới.

Bảo trì phòng ngừa (Preventive Maintenance): Mục đích của loại bảo trì này là dự đoán và ngăn ngừa các sự cố hoặc vấn đề có thể xảy ra trong tương lai, từ đó giảm thiểu các rủi ro và bảo vệ phần mềm khỏi các vấn đề tiềm ẩn.

8.2.1.2. Xây dựng chiến lược bảo trì



Hình 8.1: Quy trình bảo trì

Sau khi xác định các loại bảo trì, bước tiếp theo trong lập kế hoạch bảo trì là xây dựng chiến lược bảo trì chi tiết. Chiến lược này sẽ hướng dẫn các hoạt động bảo trì trong suốt vòng đời phần mềm và đảm bảo rằng các vấn đề được giải quyết một cách hiệu quả và đúng hạn. Các yếu tố cần xem xét khi xây dựng chiến lược bảo trì bao gồm:

Xác định phạm vi bảo trì: Cần xác định rõ ràng các phạm vi và mức độ bảo trì cho từng loại phần mềm, bao gồm các chức năng chính cần được bảo trì, sửa lỗi, và cải tiến.

Phân bổ nguồn lực: Cần xác định các nguồn lực cần thiết cho công tác bảo trì, bao gồm nhân lực, phần mềm hỗ trợ, và ngân sách. Việc phân bổ nguồn lực giúp bảo đảm rằng công tác bảo trì có thể được thực hiện kịp thời và hiệu quả.

Thời gian và tần suất bảo trì: Một phần quan trọng của chiến lược bảo trì là xác định thời gian và tần suất bảo trì. Việc bảo trì có thể được thực hiện theo kế hoạch định kỳ (hàng tháng, hàng quý) hoặc theo yêu cầu phát sinh (khi có lỗi hoặc vấn đề).

Ưu tiên bảo trì: Cần có tiêu chí để ưu tiên các tác vụ bảo trì. Ví dụ, các lỗi nghiêm trọng ảnh hưởng đến chức năng của phần mềm cần được xử lý trước, trong khi các cải tiến tính năng có thể được thực hiện sau.

8.2.1.3. Quy trình bảo trì phần mềm

Quy trình bảo trì phần mềm cần được thiết lập rõ ràng để đảm bảo rằng công tác bảo trì được thực hiện hiệu quả và có hệ thống. Quy trình bảo trì thường bao gồm các bước sau:

Nhận diện vấn đề: Khi phần mềm gặp sự cố hoặc cần cải tiến, bước đầu tiên là xác định rõ vấn đề. Điều này có thể được thực hiện thông qua phản hồi của người dùng, kiểm tra báo cáo lỗi, hoặc theo dõi hiệu suất của phần mềm.

Đánh giá và phân loại vấn đề: Sau khi nhận diện vấn đề, cần đánh giá mức độ nghiêm trọng và phân loại vấn đề. Ví dụ, lỗi gây ra sự cố cho người dùng có thể cần phải được xử lý ngay lập tức, trong khi các cải tiến tính năng có thể được lên kế hoạch trong một bản cập nhật sắp tới.

Thiết kế giải pháp: Sau khi đánh giá vấn đề, nhóm phát triển sẽ thiết kế giải pháp để khắc phục sự cố hoặc cải thiện phần mềm. Giải pháp có thể là một bản vá lỗi, cải tiến tính năng, hoặc nâng cấp phần mềm để tương thích với môi trường mới.

Kiểm thử giải pháp: Trước khi triển khai giải pháp, nó cần được kiểm thử để đảm bảo rằng vấn đề đã được giải quyết và không làm phát sinh lỗi mới.

Triển khai và giám sát: Sau khi giải pháp được kiểm thử và xác nhận, nó sẽ được triển khai vào hệ thống sản phẩm. Sau khi triển khai, phần mềm cần được giám sát để đảm bảo rằng không có vấn đề mới xuất hiện.

8.2.1.4. Công cụ hỗ trợ bảo trì

Việc sử dụng công cụ hỗ trợ bảo trì có thể giúp các nhóm phát triển và bảo trì phần mềm thực hiện công việc nhanh chóng và hiệu quả hơn. Một số công cụ hỗ trợ phổ biến trong công tác bảo trì bao gồm:

- **Công cụ theo dõi lỗi (Bug Tracking Systems):** Các công cụ như **Jira**, **Bugzilla**, và **Trello** giúp theo dõi các lỗi, yêu cầu tính năng mới và phản hồi từ người dùng.
- **Công cụ kiểm thử tự động (Automated Testing Tools):** Các công cụ kiểm thử tự động như **Selenium**, **JUnit**, và **TestComplete** giúp kiểm tra phần mềm sau mỗi lần thay đổi, đảm bảo rằng các lỗi không tái phát.
- **Công cụ quản lý phiên bản (Version Control Systems):** Công cụ như **Git** giúp theo dõi sự thay đổi mã nguồn và quản lý các bản sửa lỗi và cập nhật phần mềm.

8.2.1.5. Lập kế hoạch bảo trì dài hạn

Một phần quan trọng trong việc lập kế hoạch bảo trì là xác định các yêu cầu bảo trì dài hạn, nhằm đảm bảo rằng phần mềm luôn đáp ứng các yêu cầu của người dùng và duy trì tính ổn định qua thời gian. Các yếu tố cần xem xét trong kế hoạch bảo trì dài hạn bao gồm:

- **Duy trì tính tương thích:** Đảm bảo rằng phần mềm có thể hoạt động trên các nền tảng và công nghệ mới nhất trong suốt vòng đời của nó.
- **Cải tiến tính năng và tối ưu hóa:** Phần mềm cần được liên tục cải tiến để đáp ứng nhu cầu của người dùng và thích nghi với các thay đổi trong môi trường công nghệ.
- **Giảm thiểu chi phí bảo trì:** Lập kế hoạch bảo trì dài hạn giúp tối ưu hóa chi phí bảo trì bằng cách tránh các thay đổi không cần thiết và lên kế hoạch bảo trì định kỳ.

Lập kế hoạch bảo trì là một bước không thể thiếu trong việc duy trì chất lượng phần mềm sau khi triển khai. Một kế hoạch bảo trì chi tiết sẽ giúp các tổ chức phát triển phần mềm đáp ứng được nhu cầu của người dùng, giữ cho phần mềm luôn hoạt động ổn định, và duy trì được hiệu quả lâu dài. Việc phân loại các loại bảo trì, xây dựng chiến lược và quy trình bảo trì rõ ràng sẽ đảm bảo rằng phần mềm luôn đáp ứng được yêu cầu và sẵn sàng giải quyết các sự cố phát sinh trong suốt vòng đời sử dụng.

8.2.2. Đáp ứng nhu cầu người dùng

8.2.2.1. Khái niệm về đáp ứng nhu cầu người dùng

Đáp ứng nhu cầu người dùng là quá trình duy trì sự liên kết giữa nhà phát triển phần mềm và người dùng cuối sau khi phần mềm được triển khai. Mục tiêu là đảm bảo phần mềm không chỉ hoạt động đúng mà còn liên tục cải thiện để đáp ứng những thay đổi trong yêu cầu của người dùng và môi trường sử dụng.

8.2.2.2. Các bước thực hiện để đáp ứng nhu cầu người dùng

a. Thu thập và phân tích phản hồi người dùng

Công cụ thu thập phản hồi:

- **Khảo sát người dùng:** Sử dụng bảng hỏi, khảo sát trực tuyến hoặc email để thu thập ý kiến từ người dùng cuối về những vấn đề họ gặp phải hoặc tính năng họ mong muốn.
- **Chương trình hỗ trợ khách hàng:** Cung cấp kênh hỗ trợ trực tuyến, qua điện thoại, hoặc qua email để người dùng phản ánh trực tiếp các vấn đề.
- **Phân tích dữ liệu sử dụng phần mềm:** Dựa vào dữ liệu từ hệ thống (log, hành vi người dùng, thống kê lỗi) để phân tích các vấn đề hoặc yêu cầu chưa được đáp ứng.

Phân tích phản hồi:

- **Xác định các vấn đề phổ biến:** Phân loại phản hồi thành các nhóm chủ đề như lỗi phần mềm, yêu cầu tính năng mới, hoặc các vấn đề về hiệu suất.
- **Ưu tiên yêu cầu:** Đánh giá mức độ nghiêm trọng của các vấn đề và yêu cầu cải tiến để xác định những thay đổi cần thực hiện trước.

b. Xác định các nhu cầu mới và cải tiến phần mềm

Theo dõi xu hướng và thay đổi trong yêu cầu người dùng: Công nghệ, thị trường và nhu cầu người dùng luôn thay đổi, vì vậy cần thường xuyên theo dõi những thay đổi này để cập nhật phần mềm.

Đánh giá và phân tích các yêu cầu mới: Dựa trên phản hồi, phân tích các yêu cầu mới hoặc điều chỉnh các tính năng hiện tại của phần mềm để cải thiện trải nghiệm người dùng.

Cập nhật tài liệu yêu cầu: Cập nhật các tài liệu yêu cầu phần mềm để phản ánh các thay đổi và bổ sung tính năng mới.

c. Lập kế hoạch cập nhật và nâng cấp phần mềm

Lập kế hoạch cập nhật phần mềm định kỳ: Dựa trên phân tích phản hồi và nhu cầu người dùng, lập kế hoạch cho các bản cập nhật phần mềm định kỳ (ví dụ: mỗi quý, mỗi tháng) để cải tiến và sửa lỗi.

Đảm bảo tính tương thích và ổn định: Mỗi bản cập nhật phải được kiểm tra để đảm bảo tính tương thích với các hệ thống và thiết bị đang sử dụng phần mềm, đồng thời không làm gián đoạn hoạt động của phần mềm.

Thông báo và hướng dẫn người dùng: Cung cấp thông báo đầy đủ cho người dùng về các bản cập nhật, các tính năng mới, hoặc các sửa lỗi trong phần mềm. Cung cấp tài liệu hướng dẫn hoặc hướng dẫn trực tiếp cho người dùng nếu cần thiết.

d. Kiểm tra và đánh giá sau cập nhật

Theo dõi hiệu quả cập nhật: Sau khi phát hành bản cập nhật, theo dõi tác động của bản cập nhật đối với phần mềm và phản hồi của người dùng. Điều này có thể được thực hiện qua phân tích dữ liệu sử dụng hoặc tiếp tục thu thập phản hồi.

Phản hồi nhanh chóng và sửa lỗi: Nếu có vấn đề phát sinh sau bản cập nhật, cần có một quy trình sửa lỗi nhanh chóng để bảo vệ uy tín của phần mềm và sự hài lòng của người dùng.

8.2.2.3. Các phương pháp hiệu quả trong việc đáp ứng nhu cầu người dùng

Cải tiến liên tục: Duy trì một chu kỳ cải tiến liên tục bằng cách lắng nghe phản hồi từ người dùng và cập nhật phần mềm để cải thiện chất lượng và hiệu suất. Việc này cần được thực hiện đều đặn và có hệ thống. **Phân tích và ưu tiên cải tiến** có thể được thực hiện thông qua một số kỹ thuật sau để đảm bảo những thay đổi có giá trị nhất được thực hiện kịp thời và hiệu quả:

a. Phân tích SWOT (Strengths, Weaknesses, Opportunities, Threats)



Hình 8.2: SWOT

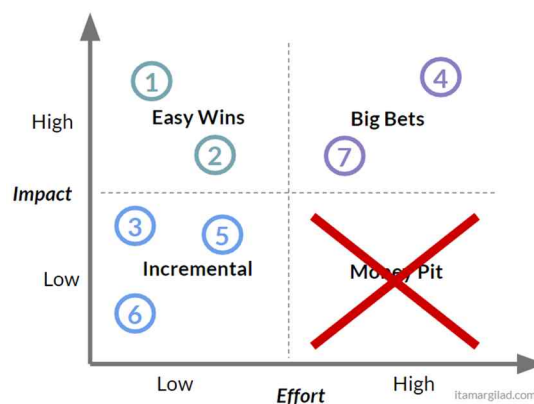
Mục đích: Xác định điểm mạnh, điểm yếu, cơ hội, và thách thức của ứng dụng để ưu tiên các cải tiến phù hợp.

Ví dụ: Một ứng dụng thương mại điện tử có thể phân tích SWOT như sau:

- Điểm mạnh: Giao diện người dùng thân thiện, tính năng lọc sản phẩm nhanh.
- Điểm yếu: Tốc độ tải trang chậm, khó khăn trong việc tìm kiếm sản phẩm.
- Cơ hội: Nhu cầu tăng cao về thương mại điện tử, mở rộng sang thị trường quốc tế.
- Thách thức: Cạnh tranh từ các ứng dụng khác có tốc độ nhanh hơn và các chương trình khuyến mãi hấp dẫn.

Hành động cải tiến: Ưu tiên cải thiện tốc độ tải trang để giữ chân người dùng và tăng khả năng cạnh tranh.

b. Ma trận Tác động - Độ phức tạp (Impact-Effort Matrix)



Hình 8.3: Ma trận tác động – độ phức tạp

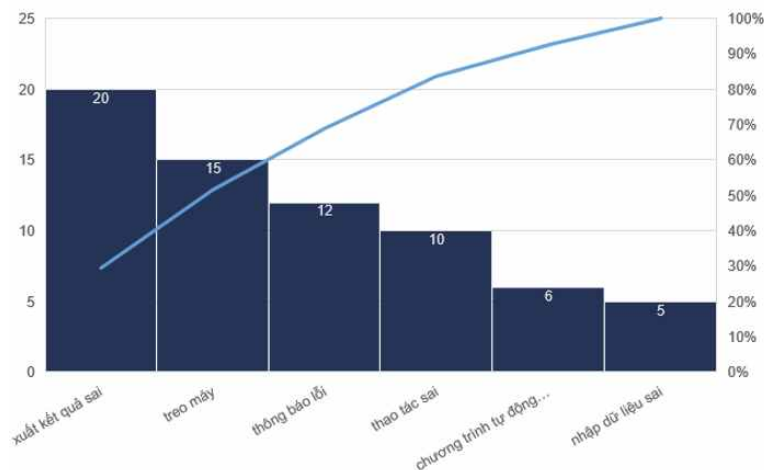
Mục đích: Sắp xếp các cải tiến dựa trên tác động và độ phức tạp của chúng, từ đó dễ dàng nhận diện các cải tiến có giá trị cao và ít tốn kém tài nguyên.

Ví dụ: Trong một ứng dụng nhắn tin, các cải tiến có thể được phân loại như sau:

- Tác động cao - Nỗ lực thấp: Thêm tùy chọn thay đổi màu nền ứng dụng.
- Tác động cao - Nỗ lực cao: Thêm tính năng gọi video.
- Tác động thấp - Nỗ lực thấp: Điều chỉnh màu sắc của các biểu tượng.
- Tác động thấp - Nỗ lực cao: Thay đổi hoàn toàn giao diện người dùng.

Hành động cải tiến: Bắt đầu với các cải tiến “tác động cao - nỗ lực thấp” để tạo ra giá trị cho người dùng một cách nhanh chóng.

c. Phân tích Pareto (Quy tắc 80/20)



Hình 8.4: Biểu đồ Pareto

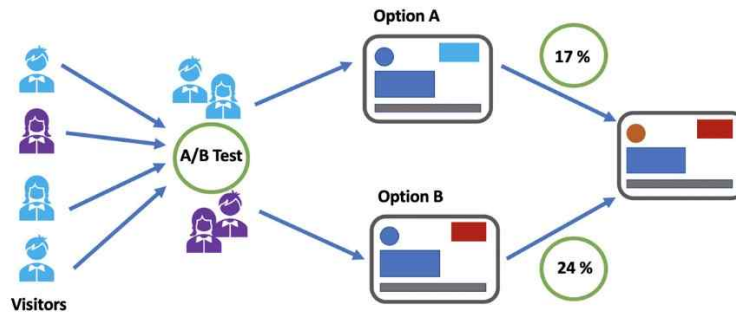
Mục đích: Tập trung giải quyết 20% vấn đề chính gây ra 80% khó chịu cho người dùng.

Ví dụ: Trong một ứng dụng quản lý chi tiêu cá nhân, dữ liệu cho thấy 80% khiếu nại đến từ hai vấn đề chính:

- Lỗi đồng bộ dữ liệu giữa các thiết bị.
- Giao diện phân loại chi tiêu khó sử dụng.

Hành động cải tiến: Đầu tiên, tập trung khắc phục lỗi đồng bộ dữ liệu và cải thiện giao diện phân loại chi tiêu, từ đó giảm đáng kể khiếu nại và tăng sự hài lòng.

c. Phân tích A/B Testing



Hình 8.5: Phân tích A/B Testting

Mục đích: Thử nghiệm hai phiên bản của cùng một tính năng để xác định phiên bản nào mang lại trải nghiệm tốt hơn.

Ví dụ: Để cải thiện trải nghiệm đặt hàng, một ứng dụng giao đồ ăn thực hiện A/B Testing:

- Phiên bản A: Người dùng thấy giao diện đặt hàng hiện tại.
- Phiên bản B: Người dùng thấy giao diện mới với thanh tiến trình đặt hàng.

Hành động cải tiến: Nếu phiên bản B giúp người dùng hoàn tất đơn hàng nhanh hơn và giảm tỷ lệ bỏ giỏ hàng, giao diện mới sẽ được triển khai rộng rãi.

e. Phân tích điểm đau (Pain Point Analysis)

Mục đích: Xác định các điểm gây khó chịu hoặc rào cản trong trải nghiệm người dùng và ưu tiên cải tiến.



Hình 8.6: Phân tích điểm đau

Ví dụ: Trong ứng dụng học trực tuyến, phản hồi từ người dùng cho thấy khó khăn trong việc theo dõi tiến độ học tập. Người dùng phải truy cập nhiều menu để xem tiến trình.

Hành động cải tiến: Thêm tính năng “tiến độ học tập” ngay tại màn hình chính để người dùng dễ dàng theo dõi tiến độ mà không cần truy cập nhiều menu.

f. Customer Journey Mapping (Lập bản đồ hành trình khách hàng)

Mục đích: Phân tích từng bước trong hành trình của người dùng để xác định các điểm yếu và cải tiến.

Ví dụ: Đối với một ứng dụng ngân hàng số, bản đồ hành trình người dùng từ lúc đăng ký tài khoản đến thực hiện giao dịch bao gồm các bước:

- Đăng ký tài khoản → Đăng nhập → Xác thực → Giao dịch tài chính.
- Phát hiện ra rằng người dùng gặp khó khăn nhiều ở bước “Xác thực”.

Hành động cải tiến: Tối ưu hóa quy trình xác thực, ví dụ rút ngắn thời gian và đơn giản hóa yêu cầu xác thực, để giúp người dùng hoàn tất đăng ký và sử dụng dịch vụ dễ dàng hơn.

g. Phân tích cạnh tranh

Mục đích: So sánh ứng dụng với các đối thủ để xác định các tính năng hoặc cải tiến cần thiết.

Ví dụ: Một ứng dụng chia sẻ ảnh nhận thấy đối thủ có tính năng “chỉnh sửa ảnh nâng cao” và “bộ lọc thời gian thực” mà ứng dụng của họ thiếu.

Hành động cải tiến: Nghiên cứu và phát triển các tính năng tương tự, ví dụ, bổ sung các bộ lọc và tùy chọn chỉnh sửa nâng cao, để giữ chân người dùng và tăng sức cạnh tranh.

h. User Persona và Phân khúc khách hàng

Mục đích: Xác định nhu cầu khác nhau của từng nhóm người dùng để cải tiến phù hợp.

Ví dụ: Một ứng dụng giáo dục phân khúc người dùng thành hai nhóm chính:

Học sinh: Cần tính năng đơn giản, bài học ngắn gọn.

Giáo viên: Cần công cụ quản lý lớp học và theo dõi tiến độ học tập.

Hành động cải tiến: Tối ưu hóa giao diện học sinh với trải nghiệm dễ sử dụng hơn, và thêm các công cụ quản lý lớp cho giáo viên. Phân khúc và đáp ứng nhu cầu cụ thể giúp gia tăng mức độ hài lòng của cả hai nhóm người dùng.

Tính minh bạch và giao tiếp rõ ràng: Khi triển khai cập nhật, cung cấp cho người dùng thông tin rõ ràng về lý do, tính năng mới, và cách sử dụng các cải tiến. Việc này giúp tạo sự tin tưởng và duy trì mối quan hệ tốt đẹp với người dùng.

Phản hồi nhanh chóng: Đảm bảo rằng tất cả các yêu cầu và vấn đề của người dùng đều được giải quyết nhanh chóng. Điều này có thể thông qua hệ thống hỗ trợ khách hàng hiệu quả hoặc thông qua các kênh trực tuyến như diễn đàn hoặc nhóm cộng đồng.

Đáp ứng nhu cầu người dùng là một phần quan trọng trong quá trình quản lý phần mềm sau khi triển khai. Việc thu thập phản hồi và cải tiến phần mềm dựa trên những phản hồi đó không chỉ giúp duy trì sự hài lòng của người dùng mà còn góp phần vào sự phát triển bền vững của phần mềm. Việc lập kế hoạch, thực hiện và theo dõi quá trình cập nhật và bảo trì phần mềm một cách cẩn thận sẽ giúp phần mềm tiếp tục hoạt động hiệu quả và phù hợp với nhu cầu thay đổi của người dùng.

Câu hỏi ôn tập cuối chương

1. Quy trình phân phối sản phẩm và kiểm thử trước khi triển khai có những mục tiêu chính nào?
2. Các công cụ thường được sử dụng trong quá trình kiểm thử tự động là gì? Nêu một vài ví dụ.
3. Mục tiêu của việc đánh giá và kiểm tra kết quả dự án là gì?
4. Kiểm thử đơn vị (Unit Testing) tập trung vào việc gì? Quy trình thực hiện kiểm thử đơn vị gồm những bước nào?
5. Kiểm thử tích hợp (Integration Testing) khác gì với kiểm thử hệ thống (System Testing)?
6. Kiểm thử chấp nhận người dùng (User Acceptance Testing - UAT) thường được thực hiện vào giai đoạn nào của dự án? Mục đích của kiểm thử này là gì?
7. Các loại kiểm thử phi chức năng phổ biến bao gồm những gì? Đưa ra ví dụ cho mỗi loại.
8. Trong quy trình bảo trì phần mềm, việc lập kế hoạch bảo trì dài hạn bao gồm các yếu tố nào?
9. Các công cụ hỗ trợ phổ biến trong bảo trì phần mềm là gì? Chúng giúp ích ra sao trong việc quản lý và bảo trì dự án?
10. Phân tích Pareto (Quy tắc 80/20) được áp dụng như thế nào trong việc cải thiện phần mềm? Đưa ra ví dụ minh họa.

Chương 9: Áp dụng kiến thức qua dự án mẫu

Nội dung chính của chương

Chương 9 tập trung vào việc áp dụng toàn diện kiến thức đã học qua việc thực hành trên một dự án mẫu. Nội dung được chia làm ba phần chính. Phần đầu, sinh viên sẽ học cách lập kế hoạch và triển khai một dự án mẫu, bao gồm các bước từ xác định mục tiêu, phân tích yêu cầu, lập lịch trình, dự toán ngân sách, và sử dụng công cụ quản lý. Phần thứ hai đi sâu vào quản lý các giai đoạn của dự án, giúp sinh viên hiểu cách theo dõi tiến độ, tối ưu hóa quy trình, xử lý các thách thức phát sinh và áp dụng các kỹ thuật hiện đại như kiểm thử tự động để nâng cao hiệu quả. Cuối cùng, chương trình hướng dẫn sinh viên đánh giá và báo cáo kết quả dự án mẫu. Qua đó, họ sẽ học cách xây dựng báo cáo, rút ra bài học kinh nghiệm và đề xuất cải tiến quy trình để áp dụng cho các dự án thực tế trong tương lai.

Mục tiêu cần đạt của chương

- Sinh viên biết cách lập kế hoạch, quản lý và triển khai một dự án phần mềm mẫu.
- Hiểu quy trình quản lý từng giai đoạn trong vòng đời phát triển phần mềm.
- Vận dụng kiến thức để đánh giá kết quả và rút kinh nghiệm từ các dự án mẫu.

9.1. Lập kế hoạch, triển khai một dự án mẫu

Dự án mẫu: xây dựng website đặt hàng trực tuyến cho cửa hàng bán lẻ

Đến nay, chúng ta đã tìm hiểu các kiến thức về quản lý dự án phần mềm. Phần này sẽ áp dụng các kiến thức đó trong một dự án mẫu, *xây dựng một website đặt hàng trực tuyến cho một cửa hàng bán lẻ*, nhằm mục tiêu giúp khách hàng dễ dàng xem sản phẩm, đặt hàng, và thanh toán trực tuyến. Dự án này được chọn vì nó mang tính thực tế cao, dễ hiểu, và có phạm vi đủ rõ ràng để thực hiện trong khuôn khổ học tập, thực hành các bước quản lý dự án phần mềm.

Lập kế hoạch là bước khởi đầu quan trọng trong bất kỳ dự án nào, đặc biệt đối với dự án xây dựng một website đặt hàng trực tuyến cho một cửa hàng bán lẻ. Một kế hoạch chi tiết giúp xác định rõ phạm vi công việc, phân tích yêu cầu và chuẩn bị các nguồn lực cần thiết để đảm bảo dự án thành công. Trong giai đoạn lập kế hoạch, nhóm dự án cần xác định các chức năng chính như danh mục sản phẩm, giỏ hàng và thanh toán, đồng thời giới hạn các chức năng phức tạp hơn như tích hợp trí tuệ nhân tạo hoặc kết nối với các hệ thống thanh toán quốc tế. Yêu cầu chức năng và phi chức năng cần được phân tích kỹ lưỡng và tài liệu hóa để tránh nhầm lẫn trong quá trình thực hiện. Ví dụ, yêu cầu phi chức năng có thể bao gồm tốc độ tải trang dưới 3 giây hoặc tính tương thích với nhiều trình duyệt. Việc sử dụng các công cụ như sơ đồ trường hợp sử dụng (use-case diagram) giúp

minh họa mối quan hệ giữa người dùng và hệ thống, đảm bảo rằng mọi yêu cầu đều được phản ánh đầy đủ và dễ hiểu.

Lịch trình dự án cũng là một phần không thể thiếu trong kế hoạch. Nhóm có thể chia dự án thành bốn giai đoạn chính: phân tích và thiết kế, phát triển, kiểm thử, và triển khai. Ví dụ, trong giai đoạn phân tích và thiết kế, nhóm sẽ tập trung vào thu thập yêu cầu từ khách hàng và thiết kế giao diện người dùng (UI) cùng kiến trúc hệ thống backend. Gantt Chart là công cụ hữu ích giúp theo dõi tiến độ của từng giai đoạn và phát hiện sớm các điểm nghẽn. Đồng thời, ngân sách dự án cần được dự toán cụ thể, bao gồm chi phí hosting, domain, và công cụ hỗ trợ. Sự chuẩn bị kỹ lưỡng này không chỉ giúp nhóm dự án tiết kiệm thời gian mà còn giảm thiểu rủi ro phát sinh.

Ngoài ra, việc lập kế hoạch không thể thiếu công tác quản lý rủi ro. Thay đổi yêu cầu từ khách hàng hoặc lỗi kỹ thuật là những rủi ro thường gặp trong các dự án công nghệ. Nhóm cần xây dựng một kế hoạch ứng phó rõ ràng, bao gồm việc dự phòng thời gian và tài nguyên cho các tình huống bất ngờ. Ví dụ, nếu khách hàng yêu cầu bổ sung tính năng mới vào giữa giai đoạn phát triển, nhóm cần đánh giá tác động và đề xuất giải pháp hợp lý như điều chỉnh lại lịch trình hoặc thêm các giai đoạn kiểm thử bổ sung. Điều này giúp dự án duy trì tiến độ mà vẫn đáp ứng yêu cầu của khách hàng. Lập kế hoạch kỹ lưỡng không chỉ là nền tảng cho sự thành công của dự án mà còn giúp nhóm dự án vận hành trơn tru và đạt được mục tiêu cuối cùng.

Phần sau đây, trình bày cụ thể các công việc cần thực hiện nhằm xác định mục tiêu từ đó lập kế hoạch triển khai trong quản lý dự án *xây dựng website đặt hàng trực tuyến cho một cửa hàng bán lẻ*.

9.1.1. Mục tiêu dự án

- Xây dựng một trang web cho phép người dùng xem danh mục sản phẩm, thêm sản phẩm vào giỏ hàng, và thanh toán trực tuyến.
- Tích hợp hệ thống quản lý đơn hàng để hỗ trợ chủ cửa hàng theo dõi và xử lý đơn hàng.
- Tích hợp cổng thanh toán đơn giản (ví dụ: thẻ nội địa hoặc ví điện tử).
- Đảm bảo giao diện thân thiện với người dùng và hoạt động ổn định trên các thiết bị khác nhau (máy tính và di động).
- Website phải tải nhanh (dưới 3 giây).
- Hệ thống có khả năng mở rộng trong tương lai.

9.1.2. Các bước lập kế hoạch dự án

Xác định phạm vi dự án

- **Phạm vi:** Mô tả rõ ràng các chức năng chính của website, và đồng thời xác định giới hạn các chức năng ngoài phạm vi dự án.
 - Phát triển website với các tính năng cơ bản như: trang chủ, danh mục sản phẩm, giỏ hàng, trang thanh toán.
 - Không bao gồm các tính năng phức tạp như AI hoặc tích hợp với nhiều hệ thống thanh toán quốc tế.
- **Giới hạn:**
 - Ngân sách và thời gian thực hiện giới hạn trong vòng 2 tháng.
 - Nhóm làm việc gồm 5 thành viên.

Xác định yêu cầu dự án:

- **Yêu cầu chức năng:**
 - Khách hàng có thể xem, tìm kiếm, và đặt hàng sản phẩm.
 - Chủ cửa hàng có thể xem và quản lý đơn hàng.
- **Yêu cầu phi chức năng:**
 - Website phải tải trong vòng dưới 3 giây.
 - Giao diện đơn giản, dễ sử dụng.

Lập lịch trình dự án

- Sử dụng công cụ như Trello hoặc Microsoft Project để theo dõi tiến độ.
- Chia dự án thành 4 giai đoạn chính:
 1. **Phân tích và thiết kế (2 tuần):**
 - Thu thập yêu cầu từ khách hàng (cửa hàng bán lẻ).
 - Thiết kế giao diện (UI) và kiến trúc hệ thống (backend).
 2. **Phát triển (4 tuần):**
 - Xây dựng các tính năng cơ bản (giỏ hàng, danh mục sản phẩm, quản lý đơn hàng).
 - Tích hợp cổng thanh toán cơ bản.

3. Kiểm thử (1 tuần):

- Kiểm tra hiệu suất, bảo mật, và trải nghiệm người dùng.
- Sửa lỗi nếu có.

4. Triển khai và bàn giao (1 tuần):

- Đưa website lên hosting.
- Hướng dẫn sử dụng cho chủ cửa hàng.

Dự toán ngân sách

- Phân bổ chi phí:
 - Hosting và domain: 1.000.000 đ.
 - Công cụ hỗ trợ (nếu cần): 600.000 đ.
 - Phần còn lại cho nhân lực và phát triển: chi phí thời gian được quy đổi theo giờ công.

Quản lý rủi ro

- **Rủi ro:**
 - Thay đổi yêu cầu từ khách hàng
 - Lỗi kỹ thuật phát sinh trong giai đoạn kiểm thử, triển khai.
- **Giải pháp:**
 - Theo dõi sát tiến độ, cập nhật khách hàng thường xuyên.
 - Sử dụng công cụ theo dõi lỗi như GitLab Issue Tracker.

Triển khai dự án

- **Phân công công việc:**
 - Mỗi thành viên chịu trách nhiệm một phần: giao diện, backend, kiểm thử, triển khai.
- **Sử dụng công cụ hỗ trợ:**
 - Sử dụng Trello hoặc Microsoft Project để phân bổ nhiệm vụ và theo dõi công việc.
 - Visual Studio Code để phát triển.
 - Postman để kiểm tra API.

- **Tích hợp và kiểm thử:**
 - Thực hiện kiểm thử từng module trước khi tích hợp.
 - Kiểm tra toàn bộ hệ thống khi hoàn thành.

Báo cáo tiến độ dự án

- **Mục tiêu của báo cáo tiến độ:**
 - Đảm bảo tất cả các thành viên và khách hàng nắm rõ trạng thái hiện tại.
 - Phát hiện sớm các vấn đề để điều chỉnh kịp thời.
- **Khi nào cần lập báo cáo tiến độ:**
 - Định kỳ (hàng tuần, theo kế hoạch dự án).
 - Khi cần đánh giá các nhiệm vụ đã hoàn thành.

Kết quả mong đợi

- Website hoạt động ổn định với các tính năng cơ bản.
- Chủ cửa hàng có thể quản lý đơn hàng dễ dàng.
- Khách hàng sử dụng website thuận tiện để mua sắm trực tuyến.

9.2. Quản lý các giai đoạn của dự án mẫu

Quản lý các giai đoạn của dự án mẫu bao gồm việc tối ưu hóa quy trình làm việc và xử lý hiệu quả các thách thức phát sinh. Các kỹ thuật và công cụ như Trello, GitLab, kiểm thử tự động, và ma trận rủi ro đóng vai trò quan trọng trong việc đảm bảo tiến độ và chất lượng dự án. Nhờ vào việc áp dụng những phương pháp này, có thể giải quyết vấn đề linh hoạt và hoàn thành dự án một cách chuyên nghiệp.

Trong phần tiếp theo trình bày việc áp dụng các phương pháp quản lý các giai đoạn vào dự án mẫu *xây dựng website đặt hàng trực tuyến cho cửa hàng bán lẻ*. Nội dung bao gồm việc tối ưu hóa tiến trình để đảm bảo hiệu quả công việc và xử lý các thách thức phát sinh trong quá trình thực hiện.

9.2.1. Tối ưu hóa tiến trình

Tối ưu hóa tiến trình là một yếu tố quan trọng trong quản lý dự án nhằm đảm bảo rằng mỗi giai đoạn được thực hiện một cách hiệu quả, đúng tiến độ, và tối ưu hóa nguồn lực. Trong dự án *xây dựng website đặt hàng trực tuyến cho một cửa hàng bán lẻ*, việc tối ưu hóa được thực hiện bằng cách phân công công việc rõ ràng, sử dụng các công cụ quản lý hiện đại, và áp dụng phương pháp theo dõi tiến độ thường xuyên. Các thành viên trong nhóm sẽ được giao nhiệm vụ cụ thể, ví dụ: một thành viên phụ trách thiết kế giao diện,

một thành viên phát triển backend, một thành viên kiểm thử và sửa lỗi. Việc sử dụng Trello giúp theo dõi trạng thái công việc theo thời gian thực, trong khi các công cụ như Gantt Chart hỗ trợ việc đánh giá tổng thể lịch trình dự án. Điều này đảm bảo các giai đoạn được tổ chức logic và dễ dàng điều chỉnh khi cần thiết.

Một yếu tố quan trọng khác trong tối ưu hóa tiến trình là ứng dụng các công cụ hỗ trợ kiểm thử và phát triển hiện đại. Đối với dự án này, kiểm thử tự động bằng Selenium được sử dụng để giảm thiểu thời gian kiểm thử thủ công và phát hiện lỗi nhanh chóng. Công cụ Postman được dùng để kiểm tra API tích hợp, đảm bảo tính nhất quán trong giao tiếp giữa các thành phần của hệ thống. Bên cạnh đó, việc lập ma trận rủi ro giúp nhóm dự án nhận diện và chuẩn bị trước các vấn đề tiềm ẩn, ví dụ như mất dữ liệu mã nguồn hoặc lỗi trong quá trình tích hợp thanh toán. Điều này không chỉ giúp nhóm sẵn sàng đối phó với các rủi ro mà còn duy trì tiến độ ổn định.

Kết quả của việc tối ưu hóa tiến trình không chỉ đảm bảo dự án được hoàn thành đúng hạn mà còn giúp giảm thiểu lỗi kỹ thuật và tăng cường hiệu suất làm việc nhóm. Các quy trình được tổ chức rõ ràng và sử dụng công cụ hiệu quả giúp nhóm dự án phát hiện sớm các vấn đề, từ đó có giải pháp điều chỉnh kịp thời. Điều này đặc biệt quan trọng trong các dự án mẫu như xây dựng website, nơi mà yêu cầu về thời gian và chất lượng luôn được đặt lên hàng đầu.

Phần sau đây, trình bày cụ thể các công việc cần thực hiện nhằm tối ưu hóa tiến trình trong quản lý dự án *xây dựng website đặt hàng trực tuyến cho một cửa hàng bán lẻ*.

Xác định mục tiêu tối ưu hóa

Tối ưu hóa tiến trình là bước quan trọng trong quản lý dự án nhằm đảm bảo rằng các giai đoạn thực hiện không chỉ hoàn thành đúng hạn mà còn đạt được hiệu quả cao nhất với nguồn lực hiện có. Việc xác định rõ ràng các mục tiêu tối ưu hóa là nền tảng để hướng dẫn nhóm dự án trong việc tổ chức và thực hiện các nhiệm vụ.

- **Đảm bảo dự án hoàn thành đúng tiến độ:** Mục tiêu hàng đầu của tối ưu hóa là giữ cho tiến trình dự án luôn bám sát lịch trình đã đề ra. Điều này yêu cầu nhóm dự án phải có công cụ và phương pháp quản lý phù hợp, chẳng hạn như theo dõi tiến độ hằng ngày qua các công cụ như Gantt Chart hoặc Trello. Ngoài ra, việc tổ chức các buổi họp ngắn định kỳ (daily stand-up) để cập nhật trạng thái công việc cũng giúp phát hiện kịp thời các rủi ro và có biện pháp điều chỉnh khi cần thiết.
- **Giảm thiểu chi phí không cần thiết:** Một trong những thách thức lớn của các dự án là kiểm soát chi phí phát sinh ngoài kế hoạch. Để tối ưu hóa chi phí, nhóm cần phân tích kỹ lưỡng các hạng mục chi tiêu ngay từ giai đoạn lập kế hoạch, đồng thời tránh các công việc lặp lại hoặc sử dụng tài nguyên không hiệu quả. Ví dụ, áp dụng kiểm thử tự động bằng Selenium thay vì kiểm thử thủ công sẽ giúp tiết kiệm thời gian và giảm chi phí nhân lực mà vẫn đảm bảo chất lượng sản phẩm. Bên

cạnh đó, việc sử dụng công cụ miễn phí hoặc mã nguồn mở trong quá trình phát triển (như Visual Studio Code hoặc GitHub) cũng là một cách hiệu quả để tiết kiệm ngân sách.

- **Nâng cao chất lượng sản phẩm và sự phối hợp giữa các thành viên nhóm:** Tối ưu hóa không chỉ dừng lại ở việc hoàn thành công việc nhanh chóng mà còn cần đảm bảo sản phẩm đạt tiêu chuẩn cao về hiệu suất và trải nghiệm người dùng. Việc sử dụng các công cụ như Postman để kiểm tra tính chính xác của API hoặc kiểm thử hồi quy (regression testing) để đảm bảo các thay đổi không ảnh hưởng đến hệ thống giúp tăng chất lượng sản phẩm. Đồng thời, sự phối hợp giữa các thành viên trong nhóm là yếu tố then chốt. Phân công công việc rõ ràng, giao tiếp thường xuyên, và hỗ trợ kỹ thuật kịp thời giúp các thành viên hoạt động hiệu quả hơn và giảm thiểu xung đột trong nhóm.

Phương pháp tối ưu hóa tiến trình

1. Phân chia công việc rõ ràng

- Mỗi thành viên được giao nhiệm vụ cụ thể, trong dự án 5 thành viên, gọi các thành viên là A, B, C, D và E là người quản lý dự án (Project Manager):
 - Thành viên A: Thiết kế giao diện (UI).
 - Thành viên B: Xây dựng chức năng backend (giỏ hàng, quản lý đơn hàng).
 - Thành viên C: Tích hợp thanh toán.
 - Thành viên D: Kiểm thử và sửa lỗi.
- Sử dụng Trello để theo dõi trạng thái công việc (To Do, In Progress, Done).

2. Theo dõi tiến độ thường xuyên

- Sử dụng báo cáo tiến độ hàng tuần để đánh giá mức độ hoàn thành.
- Tận dụng công cụ như Gantt Chart để hình dung lịch trình dự án.

3. Ứng dụng công cụ hỗ trợ

- Sử dụng GitLab để quản lý mã nguồn và theo dõi các thay đổi.
- Kiểm thử tự động qua Selenium để giảm thời gian kiểm tra thủ công.

4. Đánh giá rủi ro và điều chỉnh kịp thời

- Lập ma trận rủi ro:
 - **Ví dụ:** Rủi ro mất dữ liệu mã nguồn → Giải pháp: Thiết lập backup định kỳ trên GitHub.

- Cập nhật kế hoạch khi phát hiện các thay đổi từ khách hàng hoặc các vấn đề không mong muốn.

9.2.2. Xử lý các thách thức phát sinh

Trong quá trình thực hiện dự án, các thách thức phát sinh là điều không thể tránh khỏi, và khả năng xử lý hiệu quả sẽ quyết định sự thành công của dự án. Một thách thức phổ biến là thay đổi yêu cầu từ khách hàng, ví dụ như khách hàng yêu cầu bổ sung tính năng gợi ý sản phẩm trong giai đoạn kiểm thử. Để giải quyết, nhóm cần nhanh chóng phân tích tác động của yêu cầu mới đến ngân sách và tiến độ, đồng thời trao đổi với khách hàng để ưu tiên hóa hoặc đưa tính năng này vào giai đoạn bảo trì. Quá trình này đảm bảo rằng các thay đổi được quản lý hợp lý mà không làm gián đoạn nghiêm trọng đến tiến độ dự án.

Lỗi kỹ thuật là một thách thức khác thường gặp trong các dự án công nghệ, ví dụ như API thanh toán không tương thích với hệ thống backend. Nhóm có thể sử dụng Postman để kiểm tra và xác định vấn đề, sau đó thực hiện kiểm thử hồi quy (regression testing) nhằm đảm bảo các lỗi sửa không ảnh hưởng đến các chức năng khác. Đối với các vấn đề chậm tiến độ, việc phân tích nguyên nhân và điều chỉnh nguồn lực là cần thiết. Ví dụ, nhóm có thể tăng cường hỗ trợ kỹ thuật cho thành viên gặp khó khăn hoặc thay đổi thứ tự ưu tiên để hoàn thành các nhiệm vụ quan trọng trước.

Việc xử lý thách thức phát sinh không chỉ phụ thuộc vào kỹ thuật mà còn cần sự giao tiếp hiệu quả trong nhóm. Tăng cường các buổi họp ngắn định kỳ (daily stand-up) và sử dụng công cụ giao tiếp như Slack giúp đảm bảo thông tin được truyền đạt kịp thời và mọi vấn đề được giải quyết nhanh chóng. Tinh thần làm việc nhóm cũng cần được khuyến khích thông qua việc ghi nhận và hỗ trợ lẫn nhau, đảm bảo rằng các thành viên luôn đồng lòng hướng đến mục tiêu chung của dự án.

Phần sau đây, trình bày cụ thể các công việc cần thực hiện nhằm xử lý các thách thức phát sinh trong quản lý dự án *xây dựng website đặt hàng trực tuyến cho một cửa hàng bán lẻ*.

Nhận diện các thách thức

Trong quá trình triển khai dự án, việc đối mặt với các thách thức là điều không thể tránh khỏi. Những vấn đề này không chỉ ảnh hưởng đến tiến độ mà còn có thể gây tác động tiêu cực đến chất lượng sản phẩm và sự hài lòng của khách hàng. Dưới đây là những thách thức phổ biến có thể gặp trong dự án *xây dựng website đặt hàng trực tuyến cho một cửa hàng bán lẻ*.

- **Thay đổi yêu cầu từ khách hàng:** Đây là một trong những thách thức phổ biến nhất mà các nhóm dự án thường xuyên đối mặt. Ví dụ, khách hàng yêu cầu bổ sung tính năng gợi ý sản phẩm ngay trong giai đoạn kiểm thử. Yêu cầu này có thể làm thay đổi cấu trúc hệ thống hoặc đòi hỏi thêm thời gian và nguồn lực để thiết

kế, phát triển, và kiểm thử tính năng mới. Nếu không được quản lý tốt, những thay đổi này có thể gây gián đoạn tiến độ và vượt ngân sách đã dự toán. Vì vậy, nhóm dự án cần linh hoạt trong việc phân tích tác động của yêu cầu mới, đồng thời thảo luận với khách hàng để đưa ra giải pháp khả thi như thêm tính năng này vào giai đoạn bảo trì hoặc triển khai sau.

- **Lỗi kỹ thuật:** Các vấn đề kỹ thuật là một phần không thể thiếu trong quá trình phát triển phần mềm. Ví dụ, trong dự án mẫu, API thanh toán có thể không tương thích với hệ thống backend, hoặc giao diện website không hiển thị đúng trên một số thiết bị di động. Những lỗi này không chỉ ảnh hưởng đến trải nghiệm người dùng mà còn làm tăng thời gian sửa lỗi. Để giải quyết, nhóm cần sử dụng các công cụ chuyên dụng như Postman để kiểm tra API và thực hiện kiểm thử hồi quy (regression testing) để đảm bảo rằng các thay đổi không làm ảnh hưởng đến các chức năng khác trong hệ thống. Ngoài ra, việc lập kế hoạch kiểm thử kỹ lưỡng ngay từ đầu có thể giúp phát hiện lỗi sớm và giảm thiểu rủi ro.
- **Chậm tiến độ:** Một số thành viên không hoàn thành nhiệm vụ đúng hạn là nguyên nhân phổ biến dẫn đến chậm tiến độ dự án. Nguyên nhân có thể xuất phát từ sự thiếu kinh nghiệm, phân bổ nhiệm vụ không hợp lý, hoặc thiếu sự hỗ trợ kịp thời từ nhóm. Ví dụ, nếu một thành viên chịu trách nhiệm tích hợp công thanh toán gặp khó khăn kỹ thuật và không báo cáo kịp thời, công việc của các thành viên khác cũng bị đình trệ. Để giảm thiểu tác động, nhóm cần thực hiện các buổi họp ngắn định kỳ (daily stand-up) để cập nhật tiến độ và xử lý vấn đề ngay lập tức. Đồng thời, điều chỉnh lại lịch trình và ưu tiên xử lý các nhiệm vụ quan trọng là cách hiệu quả để giữ cho dự án đi đúng hướng.

Phương pháp xử lý thách thức

1. Giải quyết thay đổi yêu cầu

- Phân tích yêu cầu mới:
 - Xác định phạm vi thay đổi và ảnh hưởng đến tiến độ, chi phí.
 - Trao đổi với khách hàng để ưu tiên các yêu cầu quan trọng nhất.
- Cập nhật lại kế hoạch dự án với sự đồng thuận của nhóm và khách hàng.

2. Xử lý lỗi kỹ thuật

- Sử dụng Postman để kiểm tra và xác minh API tích hợp thanh toán.
- Thực hiện kiểm thử hồi quy (regression testing) khi sửa lỗi để đảm bảo không gây ảnh hưởng đến các chức năng khác.
- Hỗ trợ từ chuyên gia hoặc cộng đồng lập trình qua diễn đàn (Stack Overflow).

3. Khắc phục chậm tiến độ

- Xác định nguyên nhân gây chậm trễ: Ví dụ, thành viên gặp khó khăn về kỹ thuật hoặc phân bổ nhiệm vụ không hợp lý.
- Điều chỉnh nguồn lực:
 - Tăng hỗ trợ kỹ thuật từ các thành viên có kinh nghiệm hơn.
 - Thay đổi thứ tự ưu tiên để tập trung vào các nhiệm vụ quan trọng.

4. Tăng cường giao tiếp

- Tổ chức họp ngắn hằng ngày (daily stand-up) để cập nhật tiến độ và giải quyết vấn đề ngay lập tức.
- Sử dụng các công cụ giao tiếp như Slack hoặc Microsoft Teams để đảm bảo thông tin được truyền đạt đầy đủ.

Ví dụ về xử lý thách thức phát sinh cụ thể trong dự án mẫu này, nếu khách hàng yêu cầu bổ sung chức năng "gợi ý sản phẩm" khi đang ở giai đoạn kiểm thử, nhà quản lý dự án và team cần họp để phân tích tác động của yêu cầu phát sinh và đưa ra giải pháp, có thể như sau đây:

1. Phân tích tác động:

- Tính năng này yêu cầu thêm bao nhiêu thời gian và tài nguyên bổ sung.
- Dự kiến ảnh hưởng đến lịch trình kiểm thử và triển khai.

2. Giải pháp:

- Thảo luận với khách hàng để bổ sung vào giai đoạn bảo trì sau khi dự án chính hoàn thành.
- Nếu tính năng cần thiết ngay lập tức, cần thực hiện tái phân bổ nhiệm vụ và thời gian từ các giai đoạn khác.

9.3. Đánh giá và báo cáo kết quả thông qua dự án mẫu

Đánh giá và báo cáo kết quả là bước quan trọng để đảm bảo rằng dự án đạt được mục tiêu ban đầu, đáp ứng yêu cầu khách hàng và tạo cơ sở cho việc cải tiến quy trình trong tương lai. Trong mục này, chúng ta sẽ áp dụng những kiến thức đã học vào dự án *xây dựng website đặt hàng trực tuyến cho cửa hàng bán lẻ*.

9.3.1. Đánh giá và báo cáo kết quả

Đánh giá kết quả là giai đoạn quan trọng nhằm xác định mức độ thành công của dự án so với các mục tiêu đã đặt ra. Trong dự án mẫu, cần dựa vào các tiêu chí rõ ràng để đánh giá, bao gồm tiến độ, chất lượng, chi phí, và mức độ hài lòng của khách hàng. Tiến độ được đánh giá dựa trên việc hoàn thành các giai đoạn đúng hạn và khả năng giải quyết các thách thức phát sinh. Chất lượng sản phẩm được kiểm tra thông qua hiệu suất website, khả năng hoạt động ổn định của các tính năng chính như giỏ hàng và thanh toán. Để đánh giá chi phí, nhóm cần so sánh ngân sách thực tế với dự toán ban đầu, đảm bảo rằng không vượt quá giới hạn. Cuối cùng, sự hài lòng của khách hàng được đo lường qua phản hồi thực tế sau khi sử dụng sản phẩm, bao gồm giao diện thân thiện và tính tiện lợi trong thao tác.

Báo cáo kết quả dự án cần được trình bày một cách chuyên nghiệp và đầy đủ. Báo cáo này không chỉ dành cho khách hàng mà còn là tài liệu tham khảo quan trọng cho nhóm dự án trong tương lai. Một báo cáo mẫu bao gồm các phần: mô tả dự án, kết quả đạt được, các thách thức đã xử lý, và bài học kinh nghiệm. Ví dụ, trong phần mô tả dự án, nhóm nên tóm tắt các mục tiêu và phạm vi ban đầu, đồng thời nhấn mạnh các tính năng nổi bật đã hoàn thành. Kết quả đạt được cần đi kèm với dữ liệu cụ thể như số lượng đơn hàng xử lý thành công hoặc thời gian tải trang trung bình. Phần thách thức cần liệt kê các vấn đề đã phát sinh (như lỗi API thanh toán) cùng cách giải quyết. Bằng cách tổ chức nội dung báo cáo rõ ràng và sử dụng công cụ hỗ trợ trực quan như biểu đồ tiến độ, nhóm có thể thuyết phục khách hàng và củng cố uy tín cho các dự án sau.

Phần sau đây, trình bày cụ thể các công việc cần thực hiện nhằm đánh giá và báo cáo kết quả trong quản lý dự án *xây dựng website đặt hàng trực tuyến cho một cửa hàng bán lẻ*.

9.3.1.1. Tiêu chí đánh giá dự án

Để đảm bảo một dự án đạt được mục tiêu và thành công như mong đợi, cần dựa trên một loạt các tiêu chí đánh giá rõ ràng. Những tiêu chí này cung cấp một thước đo khách quan về hiệu suất của nhóm dự án cũng như giúp xác định các điểm cần cải thiện để nâng cao chất lượng và sự hài lòng của khách hàng.

- **Tiến độ:** Đây là tiêu chí quan trọng hàng đầu trong việc đánh giá thành công của dự án. Một dự án hoàn thành đúng tiến độ chứng tỏ sự quản lý tốt về thời gian, nguồn lực và công việc. Trong dự án mẫu "Xây dựng Website Đặt Hàng Trực Tuyến", tiến độ được đánh giá dựa trên việc các giai đoạn như phân tích, thiết kế, phát triển và kiểm thử có hoàn thành theo lịch trình đã lập hay không. Nếu có chậm trễ, nhóm cần xác định nguyên nhân, chẳng hạn như vấn đề kỹ thuật hoặc thay đổi yêu cầu, và đưa ra các biện pháp khắc phục để đảm bảo không ảnh hưởng đến các giai đoạn sau.

- **Chất lượng:** Đánh giá chất lượng là yếu tố không thể thiếu, đặc biệt đối với các dự án công nghệ như phát triển phần mềm. Chất lượng được xác định thông qua việc website hoạt động ổn định và các tính năng chính như giỏ hàng, thanh toán hoạt động đúng yêu cầu. Ví dụ, tính năng giỏ hàng phải đảm bảo người dùng có thể thêm sản phẩm, chỉnh sửa số lượng, và thực hiện thanh toán mà không gặp lỗi. Bên cạnh đó, kiểm thử hiệu suất cũng rất quan trọng để đảm bảo rằng website không bị gián đoạn khi có nhiều người dùng truy cập đồng thời. Những lỗi nhỏ trong giai đoạn này nếu không được xử lý có thể làm giảm đáng kể sự tin tưởng và trải nghiệm của khách hàng.
- **Chi phí:** Một dự án thành công phải tuân thủ ngân sách đã lập ngay từ đầu. Dự toán ngân sách bao gồm các chi phí cố định như hosting, domain, công cụ hỗ trợ, và chi phí biến đổi liên quan đến nhân lực hoặc các thay đổi phát sinh. Ví dụ, nếu trong quá trình kiểm thử, khách hàng yêu cầu bổ sung thêm tính năng, nhóm cần đánh giá ngay tác động của thay đổi này đến ngân sách và thương lượng lại với khách hàng để đảm bảo sự cân đối giữa chi phí và giá trị mang lại. Việc kiểm soát chặt chẽ các khoản chi giúp nhóm tránh lãng phí tài nguyên và đảm bảo hiệu quả tài chính của dự án.
- **Sự hài lòng của khách hàng:** Đây là tiêu chí quyết định sự thành công cuối cùng của dự án, vì nó phản ánh mức độ mà sản phẩm đáp ứng được mong đợi của khách hàng. Giao diện website cần được thiết kế thân thiện với người dùng, dễ dàng thao tác trên cả máy tính và điện thoại di động. Chức năng phải đáp ứng đúng kỳ vọng, chẳng hạn như tốc độ tải trang nhanh, tính bảo mật cao, và trải nghiệm thanh toán mượt mà. Phản hồi tích cực từ khách hàng không chỉ cho thấy sự thành công của dự án hiện tại mà còn tạo tiền đề cho các cơ hội hợp tác trong tương lai.

9.3.1.2. Phương pháp đánh giá

Thu thập dữ liệu

- **Kiểm thử hệ thống:**
 - Sử dụng các công cụ kiểm thử như Selenium để đảm bảo không có lỗi nghiêm trọng.
 - Chạy thử các kịch bản người dùng, ví dụ: thêm sản phẩm vào giỏ hàng và thực hiện thanh toán.
- **Phản hồi từ khách hàng:**
 - Yêu cầu khách hàng sử dụng thử và ghi nhận ý kiến.

Đánh giá tổng thể

- Tóm tắt các vấn đề phát sinh trong từng giai đoạn của dự án và cách xử lý.

- Đánh giá hiệu quả quản lý rủi ro và tối ưu hóa tiến trình.

9.3.1.3 Báo cáo kết quả

Nội dung báo cáo

Một báo cáo dự án cần bao gồm các phần sau:

1. **Mô tả dự án:** Tóm tắt mục tiêu, phạm vi, và các bước thực hiện.
2. **Kết quả đạt được:**
 - Các tính năng chính của website đã được hoàn thành.
 - Đáp ứng yêu cầu khách hàng về hiệu suất và giao diện.
3. **Vấn đề gặp phải:** Liệt kê các thách thức đã xảy ra và cách giải quyết.
4. **Phân tích chi phí và tiến độ:** So sánh với kế hoạch ban đầu.
5. **Phản hồi khách hàng:** Đánh giá mức độ hài lòng.

Trình bày báo cáo

- **Hình thức:**
 - Sử dụng biểu đồ và bảng để trình bày tiến độ và chi phí.
 - Thêm ảnh chụp màn hình các tính năng chính của website.
- **Đối tượng trình bày:**
 - Báo cáo nội bộ cho nhóm phát triển.
 - Báo cáo cho khách hàng và các bên liên quan.

Ví dụ: trong thực tế từ dự án xây dựng website đặt hàng trực tuyến cho cửa hàng bán lẻ thì phần đánh giá và báo cáo có thể bao gồm các nội dung:

- **Kết quả đạt được:** Website hoạt động ổn định, khách hàng đánh giá cao giao diện và tính năng đặt hàng trực tuyến.
- **Thách thức:** Tích hợp thanh toán gặp vấn đề kỹ thuật, nhưng đã được xử lý qua việc cập nhật API.

9.3.2. Cải tiến quy trình cho các dự án thực tế

Cải tiến quy trình từ các bài học kinh nghiệm là bước đi cần thiết để nâng cao hiệu quả quản lý dự án trong tương lai. Trong dự án mẫu, nhóm có thể tổng hợp các sai sót và thành công để rút ra bài học giá trị. Ví dụ, nếu nhóm phát hiện việc lập kế hoạch ban đầu chưa tính đủ thời gian cho tích hợp thanh toán, điều này có thể dẫn đến việc điều chỉnh

lại cách dự toán thời gian trong các dự án sau. Cải tiến có thể bao gồm việc áp dụng các công cụ mới như Jira để quản lý nhiệm vụ chi tiết hơn, hoặc tăng thời gian kiểm thử hồi quy để giảm thiểu lỗi phát sinh. Đồng thời, nhóm nên thực hiện đánh giá toàn diện về quy trình giao tiếp nội bộ để đảm bảo rằng mọi vấn đề được xử lý kịp thời và không gây ảnh hưởng đến tiến độ chung.

Việc cải tiến quy trình không chỉ giới hạn ở kỹ thuật mà còn liên quan đến việc nâng cao kỹ năng và sự hợp tác trong nhóm. Nhóm dự án nên tổ chức các buổi đào tạo nội bộ để củng cố kỹ năng sử dụng công cụ hiện đại như kiểm thử tự động và tích hợp API. Hơn nữa, tinh thần làm việc nhóm cần được thúc đẩy thông qua các hoạt động đánh giá định kỳ và ghi nhận đóng góp cá nhân. Ví dụ, nhóm có thể thiết lập hệ thống bảng điểm hiệu suất (performance scorecard) để theo dõi mức độ hoàn thành công việc của từng thành viên, từ đó phát hiện và khắc phục các điểm yếu. Cải tiến quy trình không chỉ giúp tăng năng suất mà còn xây dựng một đội ngũ chuyên nghiệp, sẵn sàng đối mặt với những dự án phức tạp và yêu cầu cao hơn trong tương lai.

Phần sau đây, trình bày cụ thể các công việc cần thực hiện nhằm cải tiến quy trình cho các dự án thực tế sau khi thực hiện dự án *xây dựng website đặt hàng trực tuyến cho một cửa hàng bán lẻ*.

9.3.2.1. Rút ra bài học kinh nghiệm

Tổng kết sai sót:

Một trong những sai sót điển hình của dự án là không ước tính chính xác thời gian cho việc tích hợp thanh toán. Đây là một nhiệm vụ phức tạp, đòi hỏi sự đồng bộ giữa hệ thống backend của website và API thanh toán bên thứ ba. Sai sót này dẫn đến việc kéo dài tiến độ, làm ảnh hưởng đến giai đoạn kiểm thử và khiến các thành viên khác trong nhóm phải điều chỉnh lại lịch trình làm việc. Hậu quả là nhóm phải dồn thời gian kiểm thử trong khoảng thời gian ngắn, dễ dẫn đến việc bỏ sót lỗi hoặc không kiểm tra toàn diện hệ thống. Việc này cho thấy tầm quan trọng của việc đánh giá độ phức tạp của từng giai đoạn và phân bổ thời gian hợp lý.

Đề xuất cải tiến

Để khắc phục và tránh lặp lại sai sót, nhóm nên tăng thời gian cho các giai đoạn phức tạp trong kế hoạch dự án. Điều này yêu cầu việc lập kế hoạch chi tiết hơn, chẳng hạn như đánh giá kỹ mức độ phức tạp của từng nhiệm vụ và tham khảo ý kiến từ các thành viên có kinh nghiệm. Đồng thời, sử dụng các công cụ kiểm tra tích hợp tự động như Postman sẽ giúp giảm lỗi trong quá trình tích hợp, đảm bảo rằng hệ thống hoạt động ổn định trước khi chuyển sang giai đoạn kiểm thử toàn diện. Các công cụ này không chỉ phát hiện sớm các vấn đề mà còn tiết kiệm thời gian kiểm thử thủ công, giúp nhóm tập trung vào việc hoàn thiện sản phẩm.

9.3.2.2. Cải tiến quy trình

Áp dụng vào kế hoạch

Một trong những cải tiến quan trọng là thực hiện các bước lập kế hoạch chi tiết hơn dựa trên bài học kinh nghiệm từ dự án mẫu. Điều này bao gồm việc xác định rõ các giai đoạn phức tạp, dự toán thời gian và nguồn lực cụ thể cho từng nhiệm vụ, và tạo ra các biểu đồ tiến độ (như Gantt Chart) để dễ dàng theo dõi. Chẳng hạn, đối với dự án "Xây dựng Website Đặt Hàng Trực Tuyến", việc tích hợp thanh toán đã được xác định là một giai đoạn phức tạp trong dự án mẫu. Trong các dự án tương lai, nhóm cần lập kế hoạch chi tiết hơn cho nhiệm vụ này, bao gồm kiểm tra tính tương thích của API và dự phòng thời gian sửa lỗi. Ngoài ra, việc tích hợp các công cụ quản lý hiện đại như Jira sẽ giúp nhóm làm việc hiệu quả hơn. Công cụ này không chỉ hỗ trợ quản lý nhiệm vụ mà còn cung cấp các tính năng theo dõi tiến độ, quản lý lỗi và phân tích hiệu suất, giúp các thành viên dễ dàng phối hợp và giải quyết vấn đề.

Đào tạo đội ngũ

Nâng cao năng lực của đội ngũ là yếu tố quan trọng để đảm bảo sự thành công của các dự án tương lai. Một đề xuất cải tiến là tổ chức các buổi đào tạo nội bộ nhằm cải thiện kỹ năng quản lý rủi ro cho nhóm. Điều này bao gồm việc hướng dẫn cách xác định, phân tích và lập kế hoạch ứng phó với các rủi ro tiềm ẩn, từ đó giảm thiểu các tác động tiêu cực đến dự án. Bên cạnh đó, đào tạo sử dụng các công cụ hiện đại như kiểm thử tự động (Selenium) và theo dõi tiến độ (Trello hoặc Microsoft Project) cũng là một phần không thể thiếu. Những công cụ này giúp nhóm tiết kiệm thời gian, phát hiện sớm các vấn đề kỹ thuật, và cải thiện chất lượng sản phẩm. Chẳng hạn, sử dụng Selenium để kiểm thử tự động các tính năng giỏ hàng và thanh toán không chỉ giảm lỗi mà còn tăng tốc độ hoàn thiện sản phẩm.

9.3.2.3. Đề xuất cho dự án tương lai

Việc đưa ra các đề xuất cải tiến được đúc rút kinh nghiệm từ dự án hiện tại sẽ đóng vai trò định hướng cho các dự án tương lai. Những đề xuất này giúp tối ưu hóa quy trình quản lý, cải thiện hiệu suất làm việc nhóm, và đảm bảo chất lượng sản phẩm đạt được những tiêu chuẩn cao hơn.

Áp dụng quy trình đánh giá định kỳ trong từng giai đoạn:

Một trong những hạn chế của các dự án thường gặp là chỉ tiến hành đánh giá tổng thể khi dự án đã kết thúc, dẫn đến việc bỏ sót các vấn đề quan trọng trong quá trình triển khai. Để khắc phục, nhóm dự án nên thực hiện các đợt đánh giá định kỳ sau từng giai đoạn, ví dụ: sau giai đoạn thiết kế, phát triển, và kiểm thử. Quy trình này không chỉ giúp phát hiện sớm các vấn đề mà còn cho phép điều chỉnh kế hoạch kịp thời nếu cần thiết. Chẳng hạn, trong dự án "Xây dựng Website Đặt Hàng Trực Tuyến", một đánh giá định kỳ trong giai đoạn phát triển có thể giúp nhóm nhận diện các vấn đề về tương thích API thanh toán, từ

đó sửa lỗi trước khi chuyển sang giai đoạn kiểm thử. Việc áp dụng đánh giá định kỳ sẽ đảm bảo rằng tiến độ dự án không bị ảnh hưởng nghiêm trọng bởi các lỗi phát sinh muộn.

Thiết lập bảng điểm hiệu suất (scorecard) để theo dõi chất lượng từng thành viên:

Hiệu suất làm việc của từng thành viên trong nhóm có ảnh hưởng lớn đến sự thành công của dự án. Để đánh giá và cải thiện hiệu quả làm việc, nhóm dự án nên thiết lập một hệ thống bảng điểm hiệu suất (scorecard). Hệ thống này có thể bao gồm các tiêu chí cụ thể như số lượng nhiệm vụ hoàn thành đúng hạn, chất lượng công việc (ít lỗi, đáp ứng yêu cầu), và mức độ tham gia vào các hoạt động nhóm (họp định kỳ, đóng góp ý tưởng). Ví dụ, trong dự án mẫu, nếu một thành viên thường xuyên hoàn thành nhiệm vụ chậm hoặc phải sửa lỗi nhiều, bảng điểm hiệu suất sẽ cung cấp dữ liệu để nhận diện vấn đề và đưa ra kế hoạch hỗ trợ hoặc đào tạo bổ sung. Ngược lại, những thành viên có hiệu suất cao có thể được ghi nhận và khuyến khích, tạo động lực làm việc cho cả nhóm.

Những đề xuất trên đây giúp nâng cao chất lượng dự án và xây dựng một môi trường làm việc minh bạch và hiệu quả hơn. Áp dụng đánh giá định kỳ giúp nhóm quản lý chặt chẽ tiến độ và chất lượng trong từng giai đoạn, trong khi bảng điểm hiệu suất cung cấp thông tin cụ thể để cải thiện năng suất và tinh thần làm việc của đội ngũ. Đây là các bước tiến cần thiết để đảm bảo rằng các dự án tương lai có thể đáp ứng và vượt qua kỳ vọng của khách hàng.

Câu hỏi ôn tập cuối chương

1. Mục tiêu chính của việc lập kế hoạch cho dự án là gì, và các bước quan trọng trong quá trình này là gì?
2. Trong giai đoạn xác định phạm vi dự án, những yếu tố nào cần được cân nhắc để đảm bảo tính khả thi của dự án?
3. Các yêu cầu chức năng và phi chức năng khác nhau như thế nào? Hãy lấy ví dụ từ dự án mẫu đã triển khai.
4. Khi nào cần thực hiện đánh giá rủi ro, và làm thế nào để chuẩn bị kế hoạch ứng phó với rủi ro?
5. Những nguyên tắc nào cần tuân theo để phân công nhiệm vụ một cách hiệu quả trong một nhóm dự án?
6. Một báo cáo kết quả dự án cần bao gồm những nội dung gì để thuyết phục khách hàng và các bên liên quan?
7. Làm thế nào để cải tiến quy trình làm việc từ những bài học kinh nghiệm rút ra trong quá trình thực hiện dự án?
8. Khi phát hiện tiến độ dự án bị chậm trễ, những bước nào cần thực hiện để đưa dự án trở lại đúng tiến độ mà không ảnh hưởng đến chất lượng?

Tài liệu tham khảo

- [1]. Ngô Trung Việt. “Phương pháp quản lý dự án công nghệ thông tin”. Nhà xuất bản Giáo dục Việt Nam, 2018.
- [2]. Phạm Ngọc Hùng, Trần Hoàng Việt, “Giáo trình quản lý dự án phần mềm”, Nhà xuất bản Đại học Quốc gia Hà Nội, 2018.
- [3] Project Management Institute (PMI). “A Guide to the Project Management Body of Knowledge (PMBOK® Guide)”. Seventh Edition. Project Management Institute, 2021.
- [4]. Sommerville, Ian. "Software Engineering". 10th Edition. Pearson, 2015.
- [5] Thư Viện Học Liệu Mở Việt Nam: <https://voer.edu.vn/>